
1.1 Introducción al lenguaje Java

1.1 Introducción al lenguaje Java

1. Historia
2. Bytecode, JVM, JRE, JDK
3. Especificaciones oficiales del Java SE (Standard Edition)
4. Versiones de Java
5. Entornos de desarrollo integrado (IDE): Eclipse
6. Sentencias
7. Expresiones
8. Bloques
9. CamelCase
10. Import
11. Variables
 - 11.1 Declaración de variables
 - 11.2 Inicialización de variables
 - 11.3 Ámbito de vida de las variables
 - 11.4 Palabra clave var
12. Constantes
13. Tipos de datos primitivos
 - 13.1 Enteros
 - 13.2 Números decimales
 - 13.3 Booleanos
 - 13.4 Caracteres
14. Prioridad entre operadores
15. Operadores
 - 15.1 Operador asignación
 - 15.2 Operadores aritméticos
 - 15.3 Operadores aritméticos incrementales
 - 15.4 Operadores aritméticos combinados
 - 15.5 Operadores relacionales
 - 15.6 Operadores lógicos o booleanos
 - 15.7 Operador ternario condicional
 - 15.8 Operador concatenación de cadenas
 - 15.9 Operadores a nivel de bits
16. Evaluación de cortocircuito

1. Historia

Java es un lenguaje de programación de propósito general, concurrente, orientado a objetos, que fue diseñado específicamente para que los desarrolladores de aplicaciones escribieran el programa una vez y lo ejecutaran en cualquier dispositivo, lo que quiere decir que el código que es ejecutado en una plataforma no tiene que ser recompilado para correr en otra.

El lenguaje de programación Java fue originalmente desarrollado por James Gosling, de Sun Microsystems (constituida en 1982 y posteriormente adquirida el 27 de enero de 2010 por la compañía Oracle). Su sintaxis deriva en gran medida de C y C++, pero tiene menos utilidades de bajo nivel que cualquiera de ellos.

El lenguaje Java se creó con cinco objetivos principales:

1. Debería usar el paradigma de la programación orientada a objetos.
2. Debería permitir la ejecución de un mismo programa en múltiples sistemas operativos.
3. Debería incluir por defecto soporte para trabajo en red.
4. Debería diseñarse para ejecutar código en sistemas remotos de forma segura.
5. Debería ser fácil de usar y tomar lo mejor de otros lenguajes orientados a objetos, como C++.

2. Bytecode, JVM, JRE, JDK

En el mundo de la programación siempre se ha hablado de lenguajes compilados y de lenguajes interpretados. El resultado del proceso de compilación (en realidad de compilación y enlazado) es un archivo ejecutable. Un **archivo ejecutable** es un programa que se puede lanzar directamente en un sistema operativo. La ventaja es que los programas ejecutables no necesitan compilarse de nuevo, son programas terminados. El problema es que el formato y la forma en que los sistemas operativos manejan estos archivos son diferentes, es decir, un archivo ejecutable en Linux no sería compatible con Windows ni con macOS.

En Java el código no se traduce a código ejecutable. En Java el proceso se conoce como precompilación y sirve para producir un archivo (de extensión **class**) que contiene código que no es directamente ejecutable (no es código Java). Es un código intermedio llamado **bytecode**. Al no ser ejecutable, el archivo class no puede ejecutarse directamente con un doble clic en el sistema. El bytecode tiene que ser interpretado (es decir, traducido línea a línea) por una aplicación conocida como la máquina virtual de Java (**JVM**).

JRE es el Java Runtime Environment o, en español, el Entorno de Ejecución de Java. Contiene a la JVM y otras herramientas que permiten la ejecución de las aplicaciones Java. La gran ventaja es que el entorno de ejecución de Java se fabrica para todas las plataformas; lo que significa que un archivo class se puede ejecutar en cualquier ordenador o máquina que incorpore el JRE. Sólo hay una pega, si programamos utilizando por ejemplo la versión 10 de Java, el ordenador en el que queramos ejecutar el programa deberá incorporar el JRE al menos de la versión 10.

A la forma de producir código final de Java se la llama JIT (**Just In Time**, justo en el momento) ya que el código ejecutable se produce sólo en el instante de ejecución del programa. Es decir, no hay en ningún momento código ejecutable.

JRE no posee compiladores ni herramientas para desarrollar las aplicaciones Java, solo posee las herramientas para ejecutarlas. **JDK** es el Java Development Kit o, en español, Herramientas de Desarrollo de Java. Sirve para construir programas usando el lenguaje de programación Java. Trae herramientas útiles como el compilador (javac), el debugger, herramientas de evaluación de rendimiento de aplicaciones, etc. Una instalación de JDK ya contiene un JRE dentro de las carpetas.

Para programar en Java, el primer paso que tiene que realizar el alumno es instalarse el JDK de la última versión de Java. La descarga la puede efectuar desde la página web de Oracle <https://www.oracle.com/technetwork/java/javase/downloads/index.html> en la pestaña *Downloads*. Buscar la última versión y descargar el JDK del Sistema Operativo con el que el alumno va a trabajar. Pero si al instalar el JDK ya existe una instalación del JRE en el ordenador de una versión anterior, el JDK no actualizará el JRE a la última versión. En este caso, hay que desinstalar primero el JRE y ya después instalar el JDK de la última versión.

3. Especificaciones oficiales del Java SE (Standard Edition)

La página de Oracle proporciona las especificaciones oficiales del Java SE (Standard Edition), incluyendo tanto el lenguaje de programación Java como la Máquina Virtual Java (JVM). Cada versión del Java SE tiene su propia especificación detallada en formatos HTML y PDF. Estas especificaciones incluyen las características del lenguaje, mejoras en la JVM y nuevas funcionalidades de cada versión.

Puedes consultar los detalles específicos de cada versión en <https://docs.oracle.com/javase/specs/>.

4. Versiones de Java

<https://javaalmanac.io/> es una herramienta que permite explorar las diferentes versiones de Java y comparar los cambios que se han realizado entre ellas, es decir, recopila información sobre la historia de Java principalmente desde un punto de vista técnico. En <https://javaalmanac.io/features/> podemos encontrar ejemplos de código.

Las versiones con status **EOL** (End of Life), son versiones que han llegado al final de su ciclo de vida y ya no reciben soporte oficial, actualizaciones de seguridad ni correcciones de errores. Las versiones con status **LTS** (Long-Term Support), son versiones que reciben soporte oficial, actualizaciones de seguridad y correcciones de errores importantes durante varios años.

5. Entornos de desarrollo integrado (IDE): Eclipse

El código en Java se puede escribir en cualquier editor de texto, y para compilar el código en bytecodes, sólo hace falta descargar la versión del JDK deseada. Sin embargo, la escritura y compilación de programas hecha de esta forma es un poco incómoda. Por ello numerosas empresas fabrican sus propios entornos de edición, algunos incluyen el compilador y otras utilizan el propio JDK de Java.

Un **IDE (integrated development environment)** es un entorno de programación que consiste básicamente en un editor de código, un compilador y un depurador.

Algunas ventajas que ofrecen son:

- Facilidades para escribir código: coloreado de las palabras clave, autocorrección al escribir, abreviaturas,...
- Facilidades de depuración, para probar el programa.
- Facilidad de configuración del sistema.
- Facilidades para organizar los archivos de código.
- Facilidad para exportar e importar proyectos.

Algunos IDEs para programar en Java son *Eclipse*, *Netbeans* e *IntelliJ IDEA*.

En esta asignatura utilizaremos como IDE el Eclipse, que el alumno puede descargar de la página web <https://www.eclipse.org/downloads/eclipse-packages/>. Buscar la versión más reciente del **Eclipse IDE for Java Developers** del Sistema Operativo con el que el alumno va a trabajar.

A continuación, hay que definir en el Eclipse, la versión del compilador de Java en *Windows* -> *Preferences* -> *Java* -> *Compiler* de la versión del JRE que se desea utilizar. También hay que marcar dicha versión por defecto en *Windows* -> *Preferences* -> *Java* -> *Installed JREs*.

6. Sentencias

Una sentencia es la unidad mínima de ejecución de un programa. Un programa se compone de conjunto de sentencias que acaban resolviendo un problema. Al final de cada una de las sentencias encontraremos un punto y coma (;).

Veamos algunos ejemplos de sentencias en java:

- Sentencias de declaración: `int x;`
- Invocaciones o llamadas a métodos de tipo void:
`System.out.println("Bienvenidos a Programación");`
- Sentencias de control de flujo: alteran el flujo de ejecución para tomar decisiones o repetir sentencias.

7. Expresiones

Una expresión es una combinación de operadores y operandos que se evalúa generándose un único resultado de un tipo determinado.

La diferencia entre las sentencias y los expresiones es que las expresiones devuelven un valor y las sentencias no devuelven nada.

8. Bloques

Un bloque es un conjunto de sentencias las cuales están delimitadas por llaves:

```
{  
    sentencias  
}
```

9. CamelCase

CamelCase es un estilo de escritura que se aplica a frases o palabras compuestas. Consiste en juntar palabras poniendo la inicial de cada palabra en mayúsculas. Ejemplo: PrimerPrograma. El nombre se debe a que las mayúsculas a lo largo de una palabra en CamelCase se asemejan a las jorobas de un camello.

Existen dos tipos de CamelCase:

- UpperCamelCase, cuando la primera letra de cada una de las palabras es mayúscula.
Ejemplo: *EjemploDeUpperCamelCase*.
- lowerCamelCase, igual que la anterior con la excepción de que la primera letra es minúscula.
Ejemplo: *ejemploDeLowerCamelCase*.

La notación CamelCase es utilizado por muchos lenguajes de programación, entre ellos Java.

10. Import

En cualquier lenguaje de programación existen librerías que contienen código ya escrito que nos facilita la creación de programas. En el caso de Java no se llaman librerías, sino **paquetes**. Los paquetes son una especie de carpetas que contienen clases y más paquetes. Cuando se instala el kit de desarrollo de Java, además de los programas necesarios para compilar y ejecutar código Java, se incluyen miles de clases dentro de cientos de paquetes ya listos que facilitan la generación

de programas. Algunos paquetes sirven para utilizar funciones matemáticas, funciones de lectura y escritura, comunicación en red, programación de gráficos,...

`java.lang` es el paquete básico por lo que todas sus clases se importan por defecto. Dentro de este paquete están gran parte de las clases más utilizadas dentro de las aplicaciones o programas creados con tecnología Java. Por ejemplo la clase `System` está dentro del paquete `java.lang` y posee el método `out.println` que necesitamos para escribir fácilmente por pantalla. Para utilizar la clase `System`, no es necesario poner el correspondiente `import java.lang`. Si quisiéramos utilizar la clase `Arrays` que proporciona utilidades para los arrays, necesitaríamos incluir una instrucción que permita incorporar el código de `Arrays` cuando compilemos nuestro trabajo. Para eso sirve la instrucción **import**.

La sintaxis de esta instrucción es `import paquete.subpaquete.subsubpaquete...clase`. Esta instrucción se coloca arriba del todo en el código. Para la clase `Arrays` sería `import java.util.Arrays`; . Esto significa importar en el código la clase `Arrays` que se encuentra dentro del paquete `util` que, a su vez, está dentro del gran paquete llamado `java`.

También se puede utilizar el asterisco de esta forma: `import java.util.*`; . Esto significa que se va a incluir en el código todas las clases que están dentro del paquete `util` de `java`.

En realidad no es obligatorio incluir la palabra `import` para utilizar una clase dentro de un paquete, pero sin ella deberemos escribir el nombre completo de la clase. Es decir, no podríamos utilizar el nombre `Arrays`, sino `java.util.Arrays`.

El `import` puede ir acompañado también por la palabra clave **static**, utilizado para importar los miembros estáticos de una clase o interfaz. Esto se llama **Static Import**. Al usar *Static Import*, es posible hacer referencia a miembros estáticos directamente por sus nombres, sin tener que calificarlos con el nombre de su clase. Esto simplifica y acorta la sintaxis requerida para usar un miembro estático.

Ejemplo: `import static java.lang.Math.*`

De esa forma, para utilizar por ejemplo la función `pow` de la clase `Math`, sería suficiente con llamarla `pow` en lugar de `Math.pow`.

11. Variables

Las variables son contenedores que sirven para almacenar los datos que utiliza un programa. Dicho más sencillamente, son nombres que asociamos a determinados datos. La realidad es que cada variable ocupa un espacio en la memoria RAM del ordenador para almacenar el dato al que se refiere. Es decir, cuando utilizamos el nombre de la variable realmente estamos haciendo referencia a un dato que está en memoria.

Las variables tienen un nombre (un **identificador**) que debe cumplir lo siguiente:

- Se escribe en minúscula.
- Si se juntan palabras, se debe utilizar la notación lowerCamelCase. Ejemplo: `myFirstVariable`.
- No deben comenzar con los caracteres guion bajo (`_`) o el signo de dólar (`$`), aunque ambos se admiten. Ejemplo: `my_first_variable`.
- Se admiten los números pero no como primer carácter.
- Deben ser cortos pero significativos. La elección de un nombre de variable debe ser mnemónico, es decir, diseñado para indicar al observador casual la intención de su uso. Por ejemplo, si queremos usar una variable para almacenar una edad, la llamaremos *edad*.

- Se deben evitar los nombres de variables de un solo carácter excepto para las variables temporales "usar y tirar". Los nombres comunes de las variables temporales son i, j, k, m, y n para enteros; c, d, y e para los caracteres.

11.1 Declaración de variables

Antes de poder utilizar una variable, esta se debe declarar de la siguiente manera: `tipo`

`nombrevariable;`

Donde *tipo* es el tipo de datos que almacenará la variable (texto, números enteros,...) y

nombrevariable es el identificador de la variable. Ejemplos:

```
int days; // days es un número entero, sin decimales
boolean exit; //exit sólo puede ser verdadera o falsa
```

Java es un lenguaje muy estricto al utilizar tipos de datos. Variables de datos distintos son incompatibles. Algunos autores hablan de lenguaje **fuertemente tipado** o incluso lenguaje muy tipificado. Se debe a una traducción muy directa del inglés strongly typed referida a los lenguajes que, como Java, son muy rígidos en el uso de tipos.

El caso contrario sería el lenguaje C en el que jamás se comprueban de manera estricta los tipos de datos.

Parte de la seguridad y robustez de las que hace gala Java se deben a esta característica.

Por convención de código, todas las declaraciones de variables se ponen al principio.

11.2 Inicialización de variables

En Java se utiliza el operador asignación = para inicializar una variable, es decir, para darle un valor inicial.

La inicialización se puede realizar:

- En la misma línea de código que la declaración:

```
int x=7;
```

- En cualquier otro momento, pero siempre después de haberla declarado:

```
int x;
...
x=7;
```

Se puede declarar más de una variable a la vez del mismo tipo en la misma línea si las separamos con comas:

```
int days, year, weeks;
```

Incluso se pueden también inicializar:

```
int days=365, year=2019, weeks;
```

También se puede utilizar una expresión para asignar un valor a una variable:

```
int x;  
...  
x=7;  
...  
x=x*2; //Utilización de una expresión en la asignación de la variable
```

Incluso se puede utilizar una expresión en la misma inicialización:

```
int a=13, b=18;  
int c=a+b; //es válido, c vale 31
```

11.3 Ámbito de vida de las variables

Toda variable tiene un ámbito de vida. Esto es la parte del código en la que una variable se puede utilizar, que es en el bloque donde se ha declarado. De hecho las variables tienen un ciclo de vida:

1. En la declaración se reserva el espacio necesario para que se puedan comenzar a utilizar (digamos que se avisa de su futura existencia)
2. Se la asigna su primer valor (la variable nace)
3. Se la utiliza en diversas sentencias. Cuando finaliza el bloque en el que fue declarada, la variable muere. Es decir, se libera el espacio que ocupa esa variable en memoria.
4. Una vez que la variable ha sido eliminada, no se puede utilizar. Dicho de otro modo, no se puede utilizar una variable más allá del bloque en el que ha sido definida. Ejemplo:

```
{//Se utiliza { para indicar el comienzo del bloque de código  
    int x=9;  
}//Se utiliza } para indicar el fin del bloque de código  
int y=x; //error, ya no existe x
```

11.4 Palabra clave var

Java 10 introdujo el uso de la palabra clave `var` que permite declarar variables locales sin especificar explícitamente el tipo, ya que el compilador lo infiere (deduce) automáticamente:

```
var count = 10; // El compilador infiere que count es de tipo int
```

12. Constantes

Una constante es un valor que no puede ser modificado durante la ejecución de un programa, únicamente puede ser leído.

La forma de declarar constantes es la misma que la de las variables pero hay que anteponer la palabra **final** que es la que indica que estamos declarando una constante:

```
final double PI=3.141591;  
PI=4; //Error, no podemos cambiar el valor de PI
```

Los nombres de las constantes se deben escribir en mayúsculas. Pueden contener también guiones bajos. Incluso pueden contener dígitos pero no como primer carácter.

Ejemplos:

```
final int MIN1 = 1;
final int MAX_PARTICIPANTS = 10;
```

Cuando un mismo valor se utilice en varias partes del código, entonces hay que declararlo como una constante ya que si en algún momento de la vida de la aplicación, ese valor varía, solamente hay que cambiar el valor de la constante y no estar cambiándolo en todos los sitios del código donde aparezca.

13. Tipos de datos primitivos

Se llaman **tipos primitivos** a los tipos de datos originales de un lenguaje de programación, esto es, aquellos que nos proporciona el lenguaje. Java posee los siguientes:

Tipo de variable	Bytes que ocupa	Rango de valores
boolean	1	true, false
char	2	Caracteres en Unicode
byte	1	-128 a 127
short	2	-32.768 a 32.767
int	4	-2.147.483.648 a 2.147.483.647
long	8	$-9 \cdot 10^{18}$ a $9 \cdot 10^{18}$
float	4	$-3,4 \cdot 10^{38}$ a $3,4 \cdot 10^{38}$
double	8	$-1,79 \cdot 10^{308}$ a $1,79 \cdot 10^{308}$

13.1 Enteros

Los tipos **byte**, **short**, **int** y **long** sirven para almacenar datos enteros. Los enteros son números sin decimales.

Un **literal** es un elemento de programa que representa directamente un valor:

```
int number=16; //16 es un literal
```

Los literales se pueden expresar de varias maneras:

- En decimal, es como se representan por defecto: `16`
- En binario, anteponiendo 0b: `0b10000`
- En octal, anteponiendo 0: `020`
- En hexadecimal, anteponiendo 0x: `0x10`

Por defecto, un literal entero es de tipo int. Si se le coloca detrás la letra **L**, entonces el literal será de tipo long.

```
int number=16; //16 es un literal entero
long number_long=16L; //16L es un literal de tipo long
```

No se acepta en general asignar variables de distinto tipo pero existen excepciones. Por ejemplo, sí se pueden asignar valores de variables enteras a variables enteras de un tipo superior (por ejemplo asignar un valor int a una variable long). Pero al revés no se puede:


```
int i=12;
byte b=i; //Error de compilación, posible pérdida de precisión
```

La solución es hacer un **casting**. Esta operación permite convertir valores de un tipo a otro. Se usa así:

```
int i=12;
byte b=(byte) i; //El casting evita el error
```

Hay que tener en cuenta en estos castings que si el valor asignado sobrepasa el rango del elemento, el valor convertido no tendrá ningún sentido ya que no puede almacenar todos los bits necesarios para representar ese número:

```
int i=1200;
byte b=(byte) i; //El valor de b no tiene sentido
```

Si lo que asignamos a la variable es un literal, java hace una conversión implícita siempre y cuando el literal esté dentro del rango permitido para dicho tipo. Por ejemplo, el siguiente código no da error porque 127 está dentro del rango de los byte aunque el literal sea por defecto int:

```
byte b=127;
```

Sin embargo, el siguiente código sí da error porque 128 sobrepasa el rango de los tipos *byte*:

```
byte b=128; //Error de incompatibilidad de tipos: no puede convertir de int a byte
```

En el siguiente ejemplo, utilizamos `System.out.println` para escribir en pantalla el valor de las variables:

```
package tema1_1_IntroduccionALenguajeJava;

public class Integers {

    public void show() {

        int i;
        long l;
        byte b;
        short s;

        i = 16; // 16 decimal
        System.out.println(i); //16
        i = 020; // 20 octal=16 decimal
        System.out.println(i); //16
        i = 0x10; // 10 hexadecimal=16 decimal
        System.out.println(i); //16
        i = 0b10000; // 10000 binario=16 decimal
        System.out.println(i); //16

        l = 6985742369L; // Si se le quita la L da error
        System.out.println(l); //6985742369
    }
}
```

```

    b = 127; // No da error porque está dentro del rango de los byte aunque
    el literal sea por defecto int
    System.out.println(b); //127
    s = 32767; // No da error porque está dentro del rango de los short
    aunque el literal sea por defecto int
    System.out.println(s); //32767

    i = 1200;
    System.out.println(i); //1200
    b = (byte) i;
    System.out.println(b); // -80 El valor de b no tiene sentido

    // A partir de java7, se pueden usar guiones para facilitar la lectura al
    programador:
    System.out.println("Número: " + 1_000_000); //Número: 1000000

}

public static void main(String[] args) {

    new Integers().show();

}

}

```

13.2 Números decimales

Los decimales se almacenan en los tipos **float** y **double**. Los decimales no son almacenados de forma exacta por eso siempre hay un posible error y se habla de precisión. Es mucho más preciso el tipo double que el tipo float.

Para asignar valores literales a una variable decimal, hay que tener en cuenta que el separador decimal es el punto y no la coma: `x=2.75;`

A un valor **literal** (como 1.5 por ejemplo), se le puede indicar con una **f** al final del número que es float (1.5f por ejemplo) o una **d** para indicar que es double. Si no se indica nada, un número literal siempre se entiende que es double, por lo que al usar tipos float hay que convertir los literales:

```

double d=3.49; //El literal 3.49 por defecto es double
float f=3.49f; //El literal 3.49 se tiene que convertir a float
float f = 2.75; //Error de incompatibilidad de tipos: no puede convertir de double
a float

```

Lógicamente no podemos asignar valores decimales a tipos de datos enteros:

```

int x=9.5; //Error de incompatibilidad de tipos: no puede convertir de double a
int
//Podemos mediante un casting pero perdemos los decimales, es decir, x valdrá 9:
int x=(int) 9.5;

```

El caso contrario sin embargo sí se puede hacer:

```
int x=9;
double y=x; //Correcto
```

La razón es que los tipos decimales son más grandes que los enteros, por lo que no hay problema de pérdida de valores.

13.3 Booleanos

Los valores booleanos o lógicos se almacenan en el tipo **boolean**. Sirven para indicar si algo es verdadero (**true**) o falso (**false**).

```
boolean b=true;
boolean c=false;
```

Por otro lado, a diferencia del lenguaje C, no se puede en Java asignar números a una variable booleana (en C, el valor false se asocia al número 0, y cualquier valor distinto de cero se asocia a true).

Tampoco tiene sentido intentar asignar valores de otros tipos de datos a variables booleanas mediante casting:

```
boolean b=(boolean) 9; //No tiene sentido
```

13.4 Caracteres

Los valores de tipo carácter sirven para almacenar símbolos de escritura. En Java se puede almacenar cualquier código Unicode en el tipo **char**.

Los **literales** carácter van entre comillas simples, como por ejemplo: 'a'.

En programación, secuencias de escape es el conjunto de caracteres que en el código es interpretado con algún fin. En Java, la barra invertida `\` se denomina **carácter de escape**, el cual indica que el carácter puesto a continuación será convertido en carácter especial o, si ya es especial, dejará de ser especial. Por ejemplo, el carácter `n` no es especial pero con la `\` delante se convierte en especial ya que `\n` se interpreta como un salto de línea. La `\` es un carácter especial pero con otra `\` delante deja de ser especial y simplemente es una barra invertida.

En la siguiente tabla tenemos algunas secuencias de escape en Java:

Carácter	Significado
<code>\t</code>	Tabulador
<code>\n</code>	Salto de línea
<code>\\</code>	Barra invertida
<code>\udddd</code>	Representa el carácter Unicode cuyo código es representado por dddd en hexadecimal

La descripción completa del estándar Unicode está disponible en la página web <https://unicode.org/>. En el enlace *Code Charts* encontraremos las tablas de caracteres en hexadecimal. Los caracteres básicos del español los encontraremos en *Latin → Basic Latin (ASCII)* y los caracteres especiales del español como por ejemplo, las vocales acentuadas y la ñ, en *Latin1 → Supplement*.

Para saber el código de los caracteres en decimal, podemos acceder al siguiente enlace: <https://unicode-table.com/es/>. Los caracteres imprimibles son del 32 al 126 y del 161 al 255.

Otras páginas también interesantes de caracteres Unicode son <https://symbl.cc/es/unicode-table/> y <https://www.compart.com/en/unicode/>.

Para insertar en el código caracteres no disponibles en el teclado, se hace de manera diferente según el Sistema Operativo:

- Linux: *Ctrl+Shift* y luego se pulsa *u*(para indicar que es Unicode) y el código Unicode en hexadecimal.
- Windows: *Alt* y el código Unicode en hexadecimal.
- macOS: *Ctrl+Command+Espacio* para abrir el Visor de caracteres. Se busca el carácter y se hace clic para insertarlo.

```
package tema1_1_IntroduccionALenguajeJava;

public class Characters {

    public void show() {

        char character;

        character = 'C'; // Los literales carácter van entre comillas simples
        System.out.println(character);//C
        character = 67; // El código Unicode de la C es el 67. Esta línea hace lo
        mismo que character='C'
        System.out.println(character);//C
        character = '\u0043'; // El código Unicode de la C en hexadecimal es el
        0043. Esta línea hace lo mismo que character='C'
        System.out.println(character);//C
        character = '\n'; // Carácter especial Nueva línea
        System.out.print(character);//Se produce un salto de línea
        character = '\''; // Carácter especial Comillas simples
        System.out.println(character);//'
        character = '\"'; // Carácter especial Dobles comillas
        System.out.println(character);//"
        character = '\"'; // Carácter especial Dobles comillas se puede utilizar
        sin el carácter de escape en un literal carácter
        System.out.println(character);//"
        character = '\\'; // Carácter especial Barra inclinada
        System.out.println(character);//\
        character = 9752; // Código decimal del carácter trébol
        System.out.println(character);//♣
        character = '\u2618';// Código hexadecimal del carácter trébol
        System.out.println(character);//♣
        character = '♣';// Carácter trébol
        System.out.println(character);//♣

    }

    public static void main(String[] args) {

        new Characters().show();
    }
}
```

```
}  
  
}
```

Si necesitamos almacenar más de un carácter, entonces debemos usar otro tipo de datos que nos permite manejar cadenas de caracteres: **String**.

En Java, las cadenas no se modelan como un dato de tipo primitivo, sino a través de la clase String. El texto es uno de los tipos de datos más importantes y por ello Java lo trata de manera especial. Para Java, las cadenas de texto son objetos especiales. Los textos deben manejarse creando objetos de tipo String.

Los **literales** cadena se escriben entre comillas dobles: `"Esto es un literal cadena"`.

Ejemplo:

```
String s="Estamos aprendiendo a programar";
```

En java existe también la **cadena vacía o nula** (`""`), es decir, una cadena sin ningún carácter.

Ejemplo: `String s=""`; A la variable s se le está asignando la cadena vacía o nula.

14. Prioridad entre operadores

A veces hay expresiones con operadores que resultan confusas. Por ejemplo:

```
resultado=8+4/2;
```

Es difícil saber el resultado. ¿Cuál es? ¿seis o diez? La respuesta es 10 y la razón es que el operador de división siempre precede en el orden de ejecución al de la suma. Es decir, siempre se ejecuta antes la división que la suma.

Siempre se pueden usar paréntesis para forzar el orden deseado:

```
resultado=(8+4)/2;
```

Ahora no hay duda, el resultado es seis.

¿Cómo podemos saber en qué orden se van a ejecutar los operadores en una expresión en Java? Pues se ejecutan en función de una prioridad, es decir, primero se ejecuta el que tenga más prioridad. La siguiente tabla muestra todos los operadores Java ordenados de mayor a menor prioridad. La primera línea de la tabla contiene los operadores de mayor prioridad y la última los de menor prioridad. Los operadores que aparecen en la misma línea tienen la misma prioridad. Cuando una expresión tenga dos operadores con la misma prioridad, la expresión se evalúa según su asociatividad.

Nivel	Operador	Descripcion	Asociatividad
1	[] . ()	acceso elementos array acceso miembros objetos paréntesis	de izquierda a derecha
2	++ --	unario post-incremento unario post-decremento	no asociativos

Nivel	Operador	Descripcion	Asociatividad
3	++ -- + - ! ~	unario pre-incremento unario pre-decremento unario más unario menos unario lógico NOT unario NOT a nivel de bits	de derecha a izquierda
4	() new	cast creación objetos	de derecha a izquierda
5	* / %	multiplicación división módulo	de izquierda a derecha
6	+ - +	suma resta concatenación cadenas	de izquierda a derecha
7	<< >> >>>	desplazamientos a nivel de bits	de izquierda a derecha
8	< <= > >= instanceof	relacionales	no asociativos
9	= = !=	igual distinto	de izquierda a derecha
10	&	AND a nivel de bits	de izquierda a derecha
11	^	XOR a nivel de bits	de izquierda a derecha
12		OR a nivel de bits	de izquierda a derecha
13	&&	AND	de izquierda a derecha
14		OR	de izquierda a derecha
15	?:	ternario condicional	de derecha a izquierda

Nivel	Operador	Descripcion	Asociatividad
16	= += -= *= /= %= &= ^= = <<= >>= >>>=	asignaciones	de derecha a izquierda

Por ejemplo: `resultado = 9 / 3 * 3`; En este caso, la multiplicación y la división tienen la misma prioridad y su asociatividad es de izquierda a derecha por lo que se realiza primero la operación que esté más a la izquierda, que en este caso es la división. El resultado por lo tanto es nueve. Si se desea que se haga primero la multiplicación, habría que utilizar un paréntesis:

`resultado = 9 / (3 * 3)`; En este caso, el resultado sería 1.

Otro ejemplo: `x = y = z = 17`; Como la asociatividad de la asignación es de derecha a izquierda, primero se asigna el valor 17 a `z`, luego a `y` y por último a `x`. Esto se puede realizar porque el operador de asignación devuelve el valor asignado.

Algunos operadores son no asociativos, por ejemplo, la expresión `x <= y <= z` es inválida ya que el valor devuelto por estos operadores es de un tipo diferente (booleano) al de los operandos que necesita (numérico o carácter). En este caso, podríamos realizarla en Java de la siguiente manera:

`x <= y && y <= z`.

15. Operadores

Un operador lleva a cabo operaciones sobre uno (**operador unario**), dos (**operador binario**) o tres (**operador ternario**) datos u operandos de tipo primitivo devolviendo un valor determinado también de un tipo primitivo. El tipo de valor devuelto tras la evaluación depende del operador y del tipo de los operandos. Por ejemplo, los operadores aritméticos trabajan con operandos numéricos, llevan a cabo operaciones aritméticas básicas y devuelven el valor numérico correspondiente. Los operadores se pueden clasificar en distintos grupos según se muestra en los siguientes apartados.

15.1 Operador asignación

El operador asignación `=` es un operador binario que asigna el valor del término de la derecha al operando de la izquierda. El operando de la izquierda es una variable. El término de la derecha es una expresión de un tipo de dato compatible.

Operador	Descripción	Ejemplo de expresión	Resultado del ejemplo
=	Operador asignación	<code>n = 4</code>	<code>n</code> vale 4

No debe confundirse el operador asignación (=) con el operador relacional de igualdad (==) que se verá más adelante. Además Java dispone de otros operadores que combinan la asignación con otras operaciones (operadores aritméticos combinados).

15.2 Operadores aritméticos

El lenguaje de programación Java tiene varios operadores aritméticos para los datos numéricos enteros y decimales.

Operador	Descripción	Ejemplo de expresión	Resultado del ejemplo
-	Operador unario de cambio de signo	-4	-4
+	Suma	2.5 + 7.1	9.6
-	Resta	235.6 - 103.5	132.1
*	Multiplicación	1.2 * 1.1	1.32
/	División	0.050 / 0.2 7 / 2	0.25 3
%	Módulo	20 % 7 14.5 % 2	6 0.5

El resultado exacto depende de los tipos de operandos involucrados. Es conveniente tener en cuenta las siguientes peculiaridades:

- El resultado de una expresión se convierte al tipo más general según el siguiente orden de generalidad:
byte → short → int → long → float → double
Teniendo esto en cuenta, tenemos que:
 - El resultado es de tipo long si, al menos, uno de los operandos es de tipo long y ninguno es decimal.
 - El resultado es de tipo int si ninguno de los operandos es de tipo long ni decimal.
 - El resultado es de tipo double si, al menos, uno de los operandos es de tipo double.
 - El resultado es de tipo float si, al menos, uno de los operandos es de tipo float y ninguno es double.
- Con los números enteros, si se divide entre cero, se genera la excepción *ArithmeticException*. Pero si se realiza la división entre cero con decimales, el resultado es infinito (`Infinity`).
- El resultado de una expresión inválida, por ejemplo, dividir infinito por infinito, no genera una excepción ni un error de ejecución: es un valor Not a Number (`NaN`).

```
package tema1_1_IntroduccionALenguajeJava;  
  
public class ArithmeticOperators {  
  
    public void show() {
```



```

    int int1 = 100, int2 = 0;
    double dec1 = 20.36, dec2 = 0;

    System.out.println(int1 / int2); //Genera ArithmeticException
    System.out.println(dec1 / dec2); //Infinity
    System.out.println(dec1 % dec2); //NaN

}

public static void main(String[] args) {

    new ArithmeticOperators().show();

}

}

```

Hay que tener en cuenta que el resultado de estos operadores varía notablemente si usamos enteros o si usamos números decimales. Por ejemplo:

```

double result1, d1=14, d2=5;
int result2, i1=14, i2=5;
result1= d1 / d2; //result1=2.8
result2= i1 / i2; //result2=2

```

Es más incluso:

```

double result;
int i1=7, i2=2;
result=i1/i2; //result=3.0
result=(double)(i1/i2); //result=3.0
result=(double)i1/i2; //result=3.5

```

El operador del módulo (%) sirve para calcular el resto de una división tanto entera como decimal.

```

int remainder, i1=14, i2=5;
remainder = i1 % i2; //remainder=4

```

En los decimales, el resto se calcula asumiendo que la división produce un resultado entero.

```

double remainder, d1=7.5, d2=2;
remainder=d1 % d2; //remainder=1.5

```

15.3 Operadores aritméticos incrementales

Los operadores aritméticos incrementales son operadores unarios (un único operando). El operando puede ser numérico o de tipo char y el resultado es del mismo tipo que el operando.

Operador	Descripción	Ejemplo de expresión	Resultado del ejemplo
++	Incremento	4++	5

Operador	Descripción	Ejemplo de expresión	Resultado del ejemplo
--	Decremento	4 --	3

En el caso de los caracteres, el incremento/decremento se realiza a su código Unicode. Es decir, si una variable char tiene el valor 'C', su código Unicode es 67. Si se incrementa, su código Unicode pasa a valer 68 que corresponde al valor 'D'.

Estos operadores pueden emplearse de dos formas dependiendo de su posición con respecto al operando:

- si el operador está detrás del operando, primero se utiliza la variable y luego se incrementa/decrementa su valor:
 - Post-incremento: a++
 - Post-decremento: a--
- si el operador está delante del operando, primero se incrementa/decrementa el valor de la variable y luego se utiliza.
 - Pre-incremento: ++a
 - Pre-decremento: --a

```
package tema1_1_IntroduccionALenguajeJava;

public class IncrementalArithmeticOperators {

    public void show() {

        int integer1, integer2;
        char character1, character2;

        character1 = 'C';//Unicode 67
        character1++;
        System.out.println(character1);//Al incrementarse vale 'D', Unicode 68

        /*
         * También se pueden utilizar los caracteres con los operadores
         * aritméticos, pero entonces hace falta usar casting:
         */
        character2 = (char) (character1 + 6);
        System.out.println(character2);//character2 vale 'J', Unicode 74
        integer1 = character2 + 2;
        System.out.println(integer1);//integer1 vale 76
        character2++;
        System.out.println(character2);//character2 vale 'K', Unicode 75
        integer1 = character2;
        System.out.println(integer1);//integer1 vale 75

        integer1 = 5;
        integer2 = integer1++;
        System.out.println(integer1); //integer1 vale 6
        System.out.println(integer2); //integer2 vale 5
        integer1 = 5;
        integer2 = ++integer1;
        System.out.println(integer1); //integer1 vale 6
    }
}
```

```

        System.out.println(integer2); //integer2 vale 6
    }

    public static void main(String[] args) {

        new IncrementalArithmeticOperators().show();

    }

}

```

15.4 Operadores aritméticos combinados

Combinan un operador aritmético con el operador asignación. Como en el caso de los operadores aritméticos, pueden tener operandos numéricos enteros o decimales y el tipo específico del resultado numérico dependerá del tipo de éstos.

Operador	Descripción	Ejemplo de expresión	Resultado del ejemplo
+=	Suma combinada	a+=b	a=a+b
- =	Resta combinada	a - =b	a=a - b
=	Multiplicación combinada	a=b	a=a*b
/=	División combinada	a/=b	a=a/b
%=	Resto combinado	a%=b	a=a%b

También se pueden utilizar con caracteres:

```

char character='a';
character+=2; //character vale 'c'

```

15.5 Operadores relacionales

Realizan comparaciones entre datos compatibles de tipos primitivos (numéricos y carácter) obteniendo siempre un resultado booleano. También se pueden emplear con operandos booleanos pero solamente pueden utilizar los operadores de igualdad y desigualdad.

Operador	Descripción	Ejemplo de expresión	Resultado del ejemplo
==	Igual que	7 == 38	false
!=	Distinto que	'a' != 'k'	true
<	Menor que	'G' < 'B'	false
>	Mayor que	'b' > 'a'	true
<=	Menor o igual que	7.5 <= 7.38	false

Operador	Descripción	Ejemplo de expresión	Resultado del ejemplo
>=	Mayor o igual que	38 >= 7	true

15.6 Operadores lógicos o booleanos

Las puertas lógicas son circuitos electrónicos capaces de realizar operaciones lógicas básicas:

- **Puerta NOT:** la salida es la inversa de la entrada. Se corresponde con la siguiente tabla de verdad:

A (entrada)	S (salida)
0	1
1	0

- **Puerta AND:** la señal de salida se activa solo cuando se activan todas las señales de entrada. Equivale al producto lógico $S = A \cdot B$ y se corresponde con la siguiente tabla de verdad:

A (entrada1)	B (entrada 2)	S (salida)
0	0	0
0	1	0
1	0	0
1	1	1

- **Puerta OR:** la salida se activa cuando cualquiera de las entradas está activada. Equivale a la suma lógica $S = A + B$ y se corresponde con la siguiente tabla de verdad:

A (entrada1)	B (entrada2)	S (salida)
0	0	0
0	1	1
1	0	1
1	1	1

Los operadores lógicos o booleanos en Java se basan en estas puertas lógicas realizando operaciones sobre datos booleanos y obteniendo como resultado un valor booleano:

Operador	Descripción Operador - Puerta lógica correspondiente	Ejemplo de expresión	Resultado del ejemplo
!	Negación - Puerta NOT	! false !(5 == 5)	true false
	Suma lógica - Puerta OR	true false (5 != 5) (5 < 4)	true false
&&	Producto lógico - Puerta AND	false && true (5 == 5) && (4 < 5)	false true

Ejemplos de uso de operadores lógicos o booleanos:

```
boolean adult, younger;
int age = 21;
adult = age >= 18; //adult será true
younger = !adult; //younger será false
```

```
boolean drivingLicense=true;
int age=20;
boolean canDrive= (age>=18) && drivingLicense;
/*Si la edad es de al menos 18 años y tiene carnet de conducir,
entonces puede conducir*/
```

```
boolean snow =true, rain=false, hail=false;
boolean badweather= snow || rain || hail;
//Si nieva o llueve o graniza, hace mal tiempo
```

Cuando el operador de negación se encuentra delante de un paréntesis, hay que tener en cuenta lo siguiente:

- `!(A && B) = !A || !B`
- `!(A || B) = !A && !B`

15.7 Operador ternario condicional

El operador ternario condicional permite devolver valores en función de una expresión lógica. Su sintaxis es la siguiente:

```
expresionLogica ? expresion_1 : expresion2
```

Si el resultado de evaluar la expresión lógica es verdadero, devuelve el valor de la primera expresión, y en caso contrario, devuelve el valor de la segunda expresión.

Operador	Descripción	Ejemplo de expresión	Resultado del ejemplo
?:	Operador condicional	a = 4; b = a == 4 ? a+5 : 6-a; b = a > 4 ? a*7 : a+8;	b vale 9 b vale 12

Ejemplo:

```
int vble1 = 5;
int vble2 = 4;
int mayor;
mayor = (vble1 > vble2)?vble1:vble2;//mayor contiene el mayor de los dos, es
decir, 5
```

El operador ternario condicional da error si no se utiliza, es decir, hay que asignárselo a alguien o meterlo por ejemplo en un `System.out.println`. No pasa lo mismo con las funciones.

15.8 Operador concatenación de cadenas

El operador concatenación `+` es un operador binario que devuelve una cadena resultado de concatenar las dos cadenas que actúan como operandos. Si solo uno de los operandos es de tipo cadena, el otro operando se convierte implícitamente en tipo cadena.

Operador	Descripción	Ejemplo de expresión	Resultado del ejemplo
+	Operador concatenación	"Hola" + "Juan" "Hola" + 5	"HolaJuan" "Hola5"

15.9 Operadores a nivel de bits

Manipulan los bits de los números.

Operador	Descripción
&	AND
	OR
~	NOT
^	XOR
>>	Desplazamiento a la derecha
<<	Desplazamiento a la izquierda
>>>	Desplazamiento a la derecha con relleno de ceros
<<<	Desplazamiento a la izquierda con relleno de ceros

16. Evaluación de cortocircuito

La evaluación de cortocircuito denota la semántica de algunos operadores booleanos en algunos lenguajes de programación en los cuales si con la evaluación de la primera expresión ya se conoce el resultado, ya no se evalúan el resto de expresiones. En Java, se utiliza la evaluación de cortocircuito tanto en el producto lógico (AND) como en la suma lógica (OR).

Por ejemplo, veamos la siguiente expresión que utiliza operadores `OR`:

```
8 <= 13 || 12 < 9 || 5 > 1
```

Se evalúa la primera expresión `8 <= 13` dando *true*. Como el resultado va a ser *true* independientemente del resultado de la segunda y tercera expresión, entonces no se evalúan ni la segunda `12 < 9` ni la tercera expresión `5 > 1`, sino que solamente se evalúa la primera, dando como resultado *true*.

En el caso de utilizar operadores `OR`, es conveniente colocar la condición más propensa a ser verdadera en el término de la izquierda.

Lo mismo ocurre con el operador `AND`:

```
12 < 9 && 5 > 1 && 8 <= 13
```

Se evalúa la primera expresión `12 < 9` dando *false*. Como el resultado va a ser *false* independientemente del resultado de la segunda y tercera expresión, entonces no se evalúan ni la segunda `5 > 1` ni la tercera expresión `8 <= 13`, sino que solamente se evalúa la primera, dando como resultado *false*.

En el caso de utilizar operadores `AND`, es conveniente situar la condición más propensa a ser falsa en el término de la izquierda.

Estas técnicas reducen el tiempo de ejecución del programa y ayudan al programador a evitar ciertos errores. Por ejemplo, `5/b==2`, si *b* es una variable de tipo entero y su valor es 0, se genera la excepción *ArithmeticException*. Para evitar este problema, el programador puede hacer lo siguiente: `b!=0 && 5/b==2`. Si *b* contiene el valor 0, `b!=0` dará *false*, entonces `5/b==2` no se evalúa, evitando así la generación de la excepción.

1.2 Iniciación a la Programación Orientada a Objetos

1.2 Iniciación a la Programación Orientada a Objetos

1. Subrutinas
2. Funciones
3. Introducción a la Programación Orientada a Objetos
4. Creación de objetos
5. Acceso a los atributos y métodos del objeto
6. Sobrecarga de métodos
7. Métodos estáticos y dinámicos
8. Api de Java
9. La clase Math
10. La clase String
11. Wrappers
 - 11.1 Autoboxing
 - 11.2 Unboxing
 - 11.3 El método estático valueOf
12. Encadenamiento de llamadas a métodos
13. Números aleatorios

1. Subrutinas

Una subrutina o subprograma, como idea general, se presenta como un subalgoritmo que forma parte del algoritmo principal, el cual permite resolver una tarea específica.

Podemos distinguir tres términos que poseen diferencias:

1. **Función:** conjunto de instrucciones que devuelven un resultado.
2. **Procedimiento:** conjunto de instrucciones que se ejecutan sin devolver ningún resultado.
3. **Método:** función o procedimiento que pertenece a un objeto.

2. Funciones

Una función se construye de la siguiente manera:

```
modificador_acceso tipo_resultado nombre_función (tipo_parámetro
nombre_parámetro, ... ) {
    instrucciones
    return expresión;
}
```

- **modificador_acceso:** es la visibilidad que posee la función (Lo veremos más adelante. De momento, lo utilizaremos como public).
- **tipo_resultado:** es el tipo del resultado que devuelve la función.

- **nombre_función:** es el nombre que identifica a la función. Utiliza la notación lowerCamelCase. Ejemplo: imprimirResultadoDecimal.
- **tipo_parámetro nombre_parámetro, ...:** puede ocurrir que la función necesite ciertos valores para efectuar la misión para la que ha sido creada. Por ejemplo, la función suma necesitaría los valores que tiene que sumar. En este caso, se deben indicar cada uno de dichos valores con sus tipos correspondientes. A estos valores se les conoce como parámetros de la función. Si la función no necesita parámetros, entonces solamente se ponen los paréntesis: `nombre_funcion( )`

A todo esto se le conoce como la **firma** (signature) de la función:

```
modificador_acceso tipo_resultado nombre_función (tipo_parámetro
nombre_parámetro, ... )
```

- **instrucciones:** instrucciones que conforman el algoritmo de la función, para que realice la misión para la que ha sido creada.
- **return expresión;** : el return es el que nos devuelve el resultado, por lo tanto la expresión que acompaña al return tiene que devolver un valor correspondiente al *tipo_resultado* indicado en la firma de la función.

Ejemplo de función con dos parámetros:

```
public int add(int sum1, int sum2) {
    return sum1 + sum2;
}
```

Puede ser también que haya más de un return. En ese caso, el flujo de ejecución abandona la función en cuanto ejecute el primer return.

En el caso de que estemos definiendo un **procedimiento**, no tendremos *return* ya que no devuelve ningún resultado y el tipo_resultado es *void*. Como por ejemplo `System.out.println`, que escribe en pantalla lo que recibe por parámetro pero no devuelve nada.

Una función o procedimiento nos permite que reutilicemos un algoritmo ya que se puede utilizar cuando nos haga falta. Para ello, solamente tendremos que **llamar a la función o al procedimiento** por su nombre y pasarle los parámetros en el mismo orden que se han definido y pertenecientes al mismo tipo de dato. En la llamada, dichos parámetros se llaman **argumentos**.

```
package tema1_2_IniciacionPOO;

public class Functions {

    public void show() {

        boolean isAnEvenNumber;
        int result;

        isAnEvenNumber = isEven(5); //Se llama a la función isEven con un valor de
5 en el argumento
        showBoolean(isAnEvenNumber); //Se llama al procedimiento showBoolean con
la variable isAnEvenNumber como argumento
        isAnEvenNumber = isEven(4); //Se llama a la función isEven con un valor de
4 en el argumento
    }
}
```

```

        showBoolean(isAnEvenNumber); //Se llama al procedimiento showBoolean con
la variable isAnEvenNumber como argumento

        result = add(5, 2); //Se llama a la función add con los valores 5 y 2 en
los argumentos
        showInt(result); //Se llama al procedimiento showInt con la variable
result como argumento

    }

    public boolean isEven(int n) { //Función que devuelve true si n es par

        boolean result;
        result = n % 2 == 0;
        return result;

    }

    public int add(int sum1, int sum2) { //Función que suma sum1 y sum2

        return sum1 + sum2;

    }

    public void showBoolean(boolean b) { //Procedimiento que muestra por consola
el valor de b

        System.out.println(b);

    }

    public void showInt(int n) { //Procedimiento que muestra por consola el valor
de n

        System.out.println(n);

    }

    public static void main(String[] args) {

        new Functions().show();

    }

}

```

3. Introducción a la Programación Orientada a Objetos

La Programación Orientada a Objetos (**POO**) es una técnica de programar aplicaciones basada en una serie de objetos independientes que se comunican entre sí.

A Java se le considera un lenguaje orientado a objetos ya que siempre que se crea un programa en Java, por simple que sea, se necesita declarar una clase, y el concepto de clase pertenece a la programación orientada a objetos.

Un **objeto** es un elemento del programa que integra sus propios datos y su propio funcionamiento. Es decir, un objeto está formado por datos (atributos o propiedades) y por las funciones que es capaz de realizar el objeto (métodos). Esta forma de programar se asemeja más al pensamiento humano. La cuestión es detectar adecuadamente los objetos necesarios para una aplicación. De hecho hay que detectar las distintas clases de objetos.

Una **clase** es lo que define a un tipo de objeto. Al definir una clase lo que se hace es indicar como funciona un determinado tipo de objeto. Luego, a partir de la clase, podremos crear objetos de esa clase, es decir, la clase es como un molde a partir del cual se crean los objetos que pertenecen a ella. Realmente la programación orientada a objetos es una programación orientada a clases. Es decir, lo que necesitamos programar es como funcionan las clases de objetos.

Por ejemplo, una clase podría ser la clase *Coche*. Cuando se defina esta clase, indicaremos los atributos o propiedades (como el color, modelo, marca, velocidad máxima,...) y los métodos (arrancar, parar, repostar, acelerar, frenar...). Todos los coches, es decir, todos los objetos de la clase *Coche*, tendrán esas propiedades y esos métodos. Para explicar la diferencia entre clase y objeto:

- la clase *Coche* representa a todos los coches.
- un coche concreto es un objeto, es decir, un ejemplar de una clase es un objeto. También se le llama a los objetos *instancias* de la clase. Este término procede del inglés, *instance*, que realmente significa ejemplar.

Por ejemplo, si quisiéramos crear el juego del parchís en Java, una clase sería la casilla, otra las fichas, otra el dado, etc. En el caso de la casilla, se definiría la clase para indicar su funcionamiento y sus propiedades, y luego se crearían tantos objetos casilla como casillas tenga el juego. Lo mismo ocurriría con las fichas, la clase ficha definiría las propiedades de la ficha (color y posición por ejemplo) y su funcionamiento mediante sus métodos (por ejemplo un método sería mover, otro llegar a la meta, etc), luego se crearían tantos objetos ficha como fichas tenga el juego.

4. Creación de objetos

Una vez definida la clase, ya se pueden crear objetos de la misma. Para crear un objeto, hay que declarar una variable cuyo tipo será la propia clase.

Si por ejemplo definiéramos una clase llamada *Vehicle* para modelar vehículos, para crear un objeto tendríamos que declarar una variable de tipo *Vehicle*:

```
vehicle car; //car es una variable de tipo vehicle
```

Una vez definida la variable, se le crea el objeto llamando a un método que se llama constructor. Un constructor es un método que se invoca cuando se crea un objeto y que sirve para inicializar los atributos del objeto y para realizar las acciones pertinentes que requiera el mismo para ser creado. El constructor tiene el mismo nombre que la clase y para invocarlo se utiliza el operador **new**.

```
car = new vehicle(); //vehicle() es un método constructor
```

También se puede hacer todo en la misma línea:

```
vehicle car = new vehicle();
```

5. Acceso a los atributos y métodos del objeto

Una vez creado el objeto, se puede acceder a sus **atributos** de la siguiente manera:

```
objeto.atributo.
```

```
car.wheelCount = 4; //Se le asigna 4 al atributo número de ruedas de la variable car
```

Los **métodos** se utilizan de la misma forma que los atributos, a excepción de que los métodos poseen siempre paréntesis ya que son funciones que pertenecen a un objeto:

```
objeto.método(argumentos).
```

```
car.accelerate(30);  
/*El coche incrementa su velocidad en 30. Es decir, si iba a 90km/h, después de ejecutar el método el coche va a 120km/h */
```

6. Sobrecarga de métodos

Java admite sobrecargar los métodos, es decir, crear distintas variantes del mismo método con el mismo nombre pero que se diferencien en el orden, tipo o número de los parámetros.

Por ejemplo, tenemos el método para sumar `add(int x, double y)`:

- Sí podríamos definir el método `add(double x, int y)` porque varía el orden de los parámetros.

Otro ejemplo donde tenemos el método `add(int x, int y)`:

- No podríamos definir otro método `add(int a, int b)` porque no varía el tipo ni el número de parámetros.
- Sí podríamos definir `add(int a)` y `add(int a, int b, int c)` porque el número de parámetros varía.
- También podríamos definir `add(int x, double y)` porque aunque no varíe el número de parámetros, sí varía uno de los tipos.

7. Métodos estáticos y dinámicos

A los métodos asociados a los objetos se les conoce como métodos dinámicos. Pero puede ocurrir que tengamos métodos que no estén asociados a ningún objeto, por ejemplo, métodos de utilidad general. Dichos métodos se les conoce como métodos estáticos y se definen con la palabra **static**. Al no estar asociados a ningún objeto, se utilizan con el nombre de la clase:

```
Clase.metodoEstático(argumentos).
```

 Ejemplos de llamadas a métodos dinámicos y estáticos:

- Llamada a método dinámico: `car.accelerate(30);` Esto es: `objeto.nombreDelMetodo();`
- Llamada a método estático: `Math.pow(2, 3);` Es decir, `NombreDeLaClase.nombreDelMetodo();`

8. Api de Java

La **API de Java** es una interfaz de programación de aplicaciones (**API**, por sus siglas del inglés: Application Programming Interface) provista por los creadores del lenguaje de programación **Java**, que da a los programadores los medios para desarrollar aplicaciones **Java**.

Al instalar Java (el paquete JDK) en nuestro ordenador, además del compilador y la máquina virtual de Java se instalan bastantes más elementos. Entre ellos, una cantidad muy importante de clases que ofrece la multinacional desarrolladora de Java y que están a disposición de todos los programadores listas para ser usadas. Estas clases junto a otros elementos forman lo que se denomina API de Java.

Los paquetes donde se encuentran dichas clases los podemos encontrar en <https://docs.oracle.com> → Java → Java SE Documentation → Java SE Technical Documentation → JDK de la versión deseada → Specifications → API Documentation → Module: java.base.

Otra manera de acceder rápido es poniendo en un buscador de Internet `Api Java version` `clase`, como por ejemplo: `Api Java 12 Math`.

9. La clase Math

La clase Math contiene los métodos para realizar operaciones matemáticas básicas, como potencias, logaritmos, raíces cuadradas y funciones trigonométricas.

Si observamos esta clase en la API, todos los métodos tienen al principio la palabra *static*, ya que son métodos estáticos porque son funciones de utilidad que no se utilizan asociadas a un objeto. Después de la palabra *static* nos encontramos con el tipo del resultado que devuelve el método. Y a continuación, nos encontramos con el nombre del método y sus parámetros. Ejemplo: `static double abs(double a)`

```
package tema1_2_IniciacionPOO;

public class MathClass {

    public void show() {

        System.out.println(Math.abs(-3.2));
        System.out.println(Math.pow(2, 3));
        System.out.println(Math.sqrt(16));
        System.out.println(Math.min(20, 5));

    }

    public static void main(String[] args) {

        new MathClass().show();

    }

}
```

Obsérvese en la API la sobrecarga de los métodos *abs* y *min*.

10. La clase String

El texto es uno de los tipos de datos más importantes y por ello Java lo trata de manera especial. Para Java, las cadenas de texto son objetos especiales. Los textos deben manejarse creando objetos de tipo **String**.

Las cadenas se pueden inicializar de dos maneras:

- Usando el operador asignación: `String s="hola";`
- Usando el constructor: `String s=new String("hola");`

Los **literales** cadena se escriben entre comillas dobles: `"Esto es un literal cadena"`.

En java existe también la **cadena vacía**, es decir, una cadena sin ningún carácter.

Ejemplo: `String s=""`; A la variable s se le está asignando la cadena vacía.

Como vimos en el [Tema 1.1 Introducción al lenguaje Java](#), el operador concatenación `+` es un operador binario que devuelve una cadena resultado de concatenar las dos cadenas que actúan como operandos. Si sólo uno de los operandos es de tipo cadena, el otro operando se convierte implícitamente en tipo cadena.

Obsérvese en la API el método **valueOf**: es estático y está sobrecargado. Sirve para obtener la representación String de un valor u objeto.

```
package tema1_2_IniciacionPOO;

public class StringClass1 {

    public void show() {

        int i = 100;
        String string1, string2, string3, string4;

        string1 = "Esto es un literal cadena"; //Se le da un valor inicial a la
cadena con el operador de asignación =
        System.out.println(string1);
        System.out.println(string1 + " al cual le hemos concatenado este literal
cadena"); //Se concatena otra cadena con el operador +

        string2 = "hola";
        string3 = " que tal";
        string4 = string2 + string3;
        System.out.println(string4);

        System.out.println(i + 100); //Suma de enteros
        System.out.println(String.valueOf(i) + 100); //Concatenación de cadenas

    }

    public static void main(String[] args) {

        new StringClass1().show();

    }

}
```

```
}
```

Otros métodos de las cadenas muy útiles son:

- **charAt**: devuelve el carácter de la cadena del especificado índice. Dicho índice empieza en cero, es decir, con el cero se obtiene el primer carácter de la cadena.
- **length**: devuelve la longitud de la cadena.
- **equals**: compara si dos cadenas son iguales.

```
package tema1_2_IniciacionPOO;

public class StringClass2 {

    public void show() {

        String string = "hola";
        System.out.println(string.charAt(0)); //h
        System.out.println(string.charAt(1)); //o
        System.out.println(string.charAt(2)); //l
        System.out.println(string.charAt(3)); //a
        System.out.println(string.length()); //4
        System.out.println(string.equals("hola")); //true
        System.out.println(string.equals("adiós")); //false

        //También se le pueden aplicar métodos a un literal cadena:
        System.out.println("hola".equals("hola")); //true
        System.out.println("adios".equals("hola")); //false

    }

    public static void main(String[] args) {

        new StringClass2().show();

    }

}
```

11. Wrappers

En ocasiones es muy conveniente poder tratar los datos primitivos (int, boolean, etc.) como objetos. Pero los datos primitivos no son objetos. Para resolver esta situación, la API de Java incorpora las clases envoltorio (**wrapper class**), que no son más que dotar a los datos primitivos con un envoltorio que permita tratarlos como objetos. Las clases envoltorio proporcionan métodos de utilidad para la manipulación de datos primitivos (conversiones de / hacia datos primitivos, conversiones a String, etc).

La siguiente tabla muestra los tipos primitivos y sus wrappers asociados:

Tipo primitivo	Wrapper asociado
byte	Byte

Tipo primitivo	Wrapper asociado
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

La clase `Number` es padre de todas las clases de números, es decir, de todas las clases de la tabla anterior excepto `Character` y `Boolean`.

Como ocurre con las cadenas, los wrappers también se pueden inicializar con el operador asignación:

```
Integer integer1 = 5;
```

Todos estos wrappers los encontraremos en la API de Java. Por ejemplo, si observamos en la API la clase `Integer`, podemos ver la siguiente firma de método: `static int parseInt(String s)` que convierte la cadena pasada por parámetro a entero.

```
package tema1_2_IniciacionPOO;

public class IntegerClass {

    public void show() {

        Integer integer1, integer2;
        int i;
        integer1 = 5;
        System.out.println(integer1);
        i = Integer.parseInt("7");//Convierte la cadena a int. Método estático
        por lo que se utiliza con Integer
        System.out.println(i);
        integer2 = Integer.valueOf(i);//Convierte el int a Integer. También es
        estático
        System.out.println(integer2);
        i = integer1.intValue();//Convierte el Integer a int. Método dinámico por
        lo que se utiliza con el objeto
        System.out.println(i);

    }

    public static void main(String[] args) {

        new IntegerClass().show();

    }
}
```



```
}
```

En Java, los conceptos de **autoboxing** y **unboxing** se refieren a la conversión automática entre los tipos de datos primitivos (como `int`, `double`, etc.) y sus correspondientes clases envolventes (como `Integer`, `Double`, etc.). Estas clases envolventes, o *wrappers*, permiten que los tipos primitivos sean tratados como objetos, lo que es necesario en ciertas situaciones, como cuando se trabaja con colecciones de clases genéricas (`ArrayList`, `HashMap`, etc.), que no pueden contener tipos primitivos directamente.

11.1 Autoboxing

Autoboxing es el proceso automático mediante el cual Java convierte un tipo primitivo en su correspondiente tipo de clase envolvente. Esto ocurre, por ejemplo, cuando intentas asignar un valor primitivo a una variable que espera un objeto de la clase envolvente.

Ejemplo:

```
int num = 10;
Integer obj = num; // Aquí se realiza autoboxing de int a Integer
```

11.2 Unboxing

Unboxing es el proceso inverso, donde un objeto de una clase envolvente se convierte automáticamente en su tipo primitivo correspondiente.

```
Integer obj = 20;
int num = obj; // Aquí se realiza unboxing de Integer a int
```

En este caso, el objeto `obj` de tipo `Integer` se convierte automáticamente en un valor primitivo de tipo `int` mediante el proceso de unboxing.

11.3 El método estático `valueOf`

En versiones anteriores de Java, los objetos de las clases envolventes (wrappers) como *Integer*, *Double*, *Boolean*, etc., se podían crear utilizando constructores estándar. Sin embargo, a partir de Java 5, se introdujo el método de fábrica estático `valueOf` como una alternativa preferida para crear instancias de estos tipos envolventes.

Los métodos `valueOf` implementan un patrón conocido como *interning*, que permite reutilizar instancias previamente creadas en lugar de crear un nuevo objeto cada vez. Esto mejora la eficiencia, especialmente cuando se trabajan con valores pequeños o comunes, reduciendo la sobrecarga de creación de objetos y el consumo de memoria. Esto es particularmente útil para tipos como *Boolean*, donde solo existen dos posibles valores (*true* y *false*).

`Boolean.valueOf(true)` siempre devuelve la misma instancia de `Boolean.TRUE`, mientras que un constructor `new Boolean(true)` crearía un nuevo objeto cada vez.

Ejemplo:

```
Boolean x = Boolean.valueOf(true);
Boolean y = Boolean.valueOf(true);
System.out.println(x == y); // True, ambos referencian el mismo objeto
```

Veamos un ejemplo comparativo entre el uso de un constructor y el método `valueOf`:

Uso del Constructor:

```
Integer x = new Integer(10);
Integer y = new Integer(10);
System.out.println(x == y); // False, se crean dos objetos distintos
```

Uso de `valueOf`:

```
Integer x = Integer.valueOf(10);
Integer y = Integer.valueOf(10);
System.out.println(x == y); // True, se reutiliza el mismo objeto
```

La sustitución de los constructores por métodos de fábrica estáticos como `valueOf` es parte de una tendencia en Java hacia la optimización de recursos y la eficiencia en la gestión de memoria. Estos métodos proporcionan beneficios significativos en términos de rendimiento y claridad, y son una parte fundamental del manejo moderno de objetos en Java, especialmente en lo que respecta a los tipos envolventes. Aunque los constructores aún están disponibles, su uso no es recomendable en la mayoría de los casos debido a las ventajas que ofrece `valueOf`.

12. Encadenamiento de llamadas a métodos

Se emplea cuando invocamos a un método de un objeto que nos devuelve como resultado otro objeto al que podemos volver a invocar otro método y así encadenar varias operaciones.

```
package tema1_2_IniciacionPOO;

public class CallsToMethods {

    public void showCallsToMethods() {

        Boolean b;
        String string;

        string = "EntornosDeDesarrollo";
        System.out.println(string.substring(10).toUpperCase()); //DESARROLLO

        b = Boolean.TRUE;
        System.out.println(b.toString().charAt(2)); //u

    }

    public static void main(String[] args) {

        new CallsToMethods().showCallsToMethods();

    }

}
```

En este ejemplo, el método `substring(10)` está devolviendo una subcadena de la cadena string a partir del carácter 10 empezando en 0, es decir, "Desarrollo", al que se le invoca luego el método `toUpperCase` devolviendo como resultado la cadena "DESARROLLO".

Y el método `toString()` de la variable `b` de tipo Boolean está devolviendo la cadena "true" a la que se le encadena el método `charAt(2)` devolviendo el carácter 'u'.

13. Números aleatorios

En java disponemos de la clase **Random** para generar números aleatorios. La clase dispone de dos constructores, uno sin parámetros y otro con un parámetro llamado semilla (**seed**). Aunque no podemos predecir que números se generarán con una semilla particular, podemos sin embargo, duplicar una serie de números aleatorios usando la misma semilla. Es decir, cada vez que creamos un objeto de la clase Random con la misma semilla obtendremos la misma secuencia de números aleatorios. Podemos cambiar la semilla de los números aleatorios en cualquier momento utilizando el método `setSeed`.

Algunos métodos de la clase Random para generar números aleatorios:

- `nextInt()`: genera un número aleatorio entero de tipo `int`
- `nextInt(int n)`: genera un número aleatorio entero de tipo `int` entre 0(incluido) y `n`(excluido)
- `nextLong()`: genera un número aleatorio entero de tipo `long`
- `nextFloat()`: genera un número aleatorio de tipo `float` entre 0.0(incluido) y 1.0(excluido)
- `nextDouble()`: genera un número aleatorio de tipo `double` entre 0.0(incluido) y 1.0(excluido)
- `nextBoolean()`: genera un booleano aleatorio

Para generar números aleatorios enteros comprendidos entre un rango de dos números enteros *min* y *max* ambos incluidos, hay que utilizar la siguiente fórmula:

```
nextInt(max - min + 1) + min
```

Por ejemplo, si queremos generar un número aleatorio entre 5 y 10: `nextInt(10-5+1)+5` → `nextInt(6)+5`

En el siguiente ejemplo, se puede observar en la ejecución que utilizando el Random con semilla siempre se generan los mismos números:

```
package tema1_2_IniciacionPOO;

import java.util.Random;

public class RandomClass {

    public void show() {

        Random random = new Random(); // Sin semilla
        Random randomSeed = new Random(3816); // Con semilla. Siempre se generan
        los mismos números

        System.out.println(random.nextBoolean());
        System.out.println(random.nextInt());
        System.out.println(random.nextLong());
        System.out.println(random.nextFloat());
        System.out.println(random.nextDouble());
    }
}
```

```
        System.out.println(random.nextInt(6) + 5); // Genera un número aleatorio  
entre 5 y 10
```

```
        System.out.println(randomSeed.nextBoolean());  
        System.out.println(randomSeed.nextInt());  
        System.out.println(randomSeed.nextLong());  
        System.out.println(randomSeed.nextFloat());  
        System.out.println(randomSeed.nextDouble());  
        System.out.println(randomSeed.nextInt(6) + 5);
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        new RandomClass().show();
```

```
    }
```

```
}
```

1.3 Escritura en pantalla

1.3 Escritura en pantalla

1. Introducción
2. El método `printf`
 - 2.1 Número de decimales
 - 2.2 Números con signo
 - 2.3 Mostrar más de un valor
 - 2.4 Especificar un ancho de campo
 - 2.5 Alineación a la izquierda
3. El método `format` de la clase `String`
4. Caracteres de bloque
5. Colores
 - 5.1 Códigos de escape ANSI
 - 5.2 Modo de color RGB

1. Introducción

La salida por pantalla en Java se hace con el objeto `System.out`. Este objeto es una instancia de la clase **`PrintStream`** del paquete `java.lang`. Si miramos la API de `PrintStream` obtendremos la variedad de métodos para mostrar datos por pantalla, algunos de estos son:

- *`print`* y *`println`*: muestran los datos por pantalla. Están sobrecargados para varios tipos de datos. El *`println`* inserta un salto de línea después de mostrar el dato.
- *`printf`*: escribe una cadena de texto utilizando formato.

En *`print`* y *`println`*, cuando queramos escribir un mensaje y el valor de una variable, debemos utilizar el operador de **concatenación** de cadenas (+), por ejemplo:

```
System.out.println("Bienvenido, " + nombre);
```

Escribe el mensaje de "Bienvenido, Carlos", si el valor de la variable *`nombre`* es Carlos.

En *`print`* y *`println`*, todas las variables que se muestran se consideran como cadenas de texto sin formato, por ejemplo, no sería posible indicar que escriba un número decimal con dos cifras decimales. Para ello se utiliza *`printf`*.

2. El método `printf`

```
printf(String format, Object... args)
```

En `String format` se coloca la cadena a la cual se le quiere dar formato. Utiliza unos códigos de conversión para indicar del contenido a mostrar de qué tipo es. Estos códigos se caracterizan porque llevan delante el símbolo %, algunos de ellos son:

- `%c`: carácter.
- `%s`: cadena de texto.
- `%d`: entero.

- `%f`: número decimal.
- `%b`: booleanos

También se pueden usar las secuencias de escape que van precedidas de `\`, como por ejemplo `\n` y `\t`, incluso el salto de línea `\n` también se puede especificar como `%n`.

En `object... args` se colocan los valores correspondientes. El primero corresponde al primer `%`, el segundo al segundo `%` y así sucesivamente. Dichos valores tienen que ser expresiones que devuelvan resultados con tipos compatibles a los `%` que correspondan.

Ejemplos:

```
system.out.printf("El carácter es %c", 'a');
```

Nos mostraría por pantalla: `El carácter es a`

```
int integer = 10;
system.out.printf("%d", integer);
```

Salida por pantalla: `10`

Si queremos mostrar el símbolo `%`, se utiliza otro `%` delante:

```
system.out.printf("El 20%% de %d es %d\n", 200, 200*20/100);
```

Salida por pantalla: `El 20% de 200 es 40`

2.1 Número de decimales

Para `%f` podemos especificar el número de decimales escribiendo `.n` entre `%` y `f`, siendo `n` el número de decimales:

```
system.out.printf("%.2f", 12.3698);
```

La salida por pantalla es `12,37` ya que se realiza un redondeo para mostrar los decimales indicados.

2.2 Números con signo

Para mostrar números con signo se utiliza el símbolo `+` entre `%` y `d` si es un número entero o entre `%` y `f` si es un número decimal.

```
int integer = 10;
system.out.printf("%+d", integer);
```

Salida por pantalla: `+10`.

```
int integer = -10;
system.out.printf("%+d", integer);
```

Salida por pantalla: `-10`.

```
double decimal = 3.968;
System.out.printf("%+.2f", decimal);
```

Salida por pantalla: +3,97

```
double decimal = -3.968;
System.out.printf("%+f", decimal);
```

Salida por pantalla: -3,968000

2.3 Mostrar más de un valor

Utilizaremos tantos % como valores vamos a formatear. Después de la primera coma, se van poniendo los valores separados por comas, el primero corresponde al primer %, el segundo al segundo % y así sucesivamente.

```
double decimal = 1.25036;
int integer = 10;
System.out.printf("decimal = %.2f integer = %d", decimal, integer);
```

Salida por pantalla: decimal = 1,25 integer = 10

También podemos cambiar el orden por defecto de correspondencia entre los valores y los % con el símbolo \$. Ejemplo: %2\$d significa que al número entero(%d) se le va a asignar el segundo valor(2\$).

```
System.out.printf("decimal1 = %2$.2f integer = %1$d decimal2 = %2$+.1f", integer, decimal);
```

Salida por pantalla: decimal1 = 1,25 integer = 10 decimal2 = +1,3

Ejemplo: mostrar el número 123.4567 y su cuadrado ambos con dos decimales:

```
double decimal = 123.4567;
System.out.printf("El cuadrado de %.2f es %.2f", decimal, decimal*decimal);
```

Salida por pantalla: El cuadrado de 123,46 es 15241,56

2.4 Especificar un ancho de campo

printf permite también mostrar valores con un ancho de campo determinado. Por ejemplo, si queremos mostrar un número entero en un ancho de campo de 10 caracteres, escribimos 10 entre % y d:

```
int integer = 1234;
System.out.printf("Ancho de 10 caracteres con un entero:%10d", integer);
```

Salida por pantalla:

Ancho de 10 caracteres con un entero: 1234

Otro ejemplo con números decimales: mostrar un número decimal con dos decimales, con signo y en un ancho de campo de 10 caracteres:

```
double decimal = 1.25036;
System.out.printf("Ancho de 10 caracteres con un número decimal: %+10.2f", decimal);
```

Salida por pantalla:

```
Ancho de 10 caracteres con un número decimal:    +1,25
```

En el ancho de 10 caracteres, se cuentan además de las cifras del número, la coma decimal y el signo si lo lleva. En este caso, el número ocupa un espacio de 5 caracteres (3 cifras, la coma y el signo), por lo tanto, se añaden 5 espacios en blanco al principio para completar el tamaño de 10.

Para completar el ancho de caracteres con ceros en lugar de con espacios, se coloca un 0 delante del ancho de caracteres:

```
double decimal = 1.25036;
System.out.printf("Ancho de caracteres relleno con ceros: %+010.2f", decimal);
```

Salida por pantalla:

```
Ancho de caracteres relleno con ceros: +000001,25
```

Veamos un ejemplo con cadenas: mostrar la cadena "Manolo" con un ancho de 10 caracteres:

```
System.out.printf("Ancho de caracteres con cadenas: %10s", "Manolo");
```

Salida por pantalla:

```
Ancho de caracteres con cadenas:    Manolo
```

2.5 Alineación a la izquierda

Con el signo `-` se indica alineación a la izquierda.

Ejemplo: mostrar un decimal con un ancho de 9 caracteres, con tres decimales y alineado a la izquierda:

```
double decimal=58.965874f;
System.out.printf("decimal=%-9.3fQue ocupe 9 caracteres, con tres decimales y alineado a la izquierda", decimal);
```

Salida por pantalla:

```
decimal=58,966    Que ocupe 9 caracteres, con tres decimales y alineado a la izquierda
```

Ejemplo con cadenas: mostrar la cadena "Manolo" con un ancho de 10 caracteres y alineada a la izquierda:


```
System.out.printf("%-10s:Alineación a la izquierda con cadenas", "Manolo");
```

Salida por pantalla:

```
Manolo      :Alineación a la izquierda con cadenas
```

He aquí todos los ejemplos de *printf* vistos en este apartado:

```
package tema1_3_EscrituraEnPantalla;

public class Printf {

    public void show() {

        double decimal;
        int integer;

        System.out.printf("El carácter es %c\n", 'a');// El carácter es a
        integer = 10;
        System.out.printf("%d\n", integer);// 10
        System.out.printf("El 20%% de %d es %d\n", 200, 200 * 20 / 100);// El 20%
de 200 es 40
        System.out.printf("%.2f\n", 12.3698);// 12,37
        decimal = 1.25036;
        System.out.printf("%.3f\n", decimal);// 1,250
        System.out.printf("%+d\n", integer);// +10
        integer = -10;
        System.out.printf("%+d\n", integer);// -10
        decimal = 3.968;
        System.out.printf("%+.2f\n", decimal);// +3,97
        decimal = -3.968;
        System.out.printf("%+f\n", decimal);// -3,968000
        decimal = 1.25036;
        integer = 10;
        System.out.printf("decimal = %.2f integer = %d\n", decimal, integer);//
decimal = 1,25 integer = 10
        System.out.printf("decimal1 = %2$.2f integer = %1$d decimal2 =
%2$+.1f\n", integer, decimal);
        // decimal1 = 1,25, integer = 10, decimal2 = +1,3
        decimal = 123.4567;
        System.out.printf("El cuadrado de %.2f es %.2f\n", decimal, decimal *
decimal);
        integer = 1234;
        System.out.printf("Ancho de 10 caracteres con un entero:%10d\n",
integer);
        decimal = 1.25036;
        System.out.printf("Ancho de 10 caracteres con un decimal:%+10.2f\n",
decimal);
        decimal = 1.25036;
        System.out.printf("Ancho de caracteres relleno con ceros:%+010.2f\n",
decimal);
        System.out.printf("Ancho de caracteres con cadenas:%10s\n", "Manolo");
        decimal = 58.965874f;
```

```

        System.out.printf("decimal=%-9.3fQue ocupe 9 caracteres, con tres
decimales y alineado a la izquierda\n",
            decimal);
        System.out.printf("%-10s:Alineación a la izquierda con cadenas\n",
"Manolo");
    }

    public static void main(String[] args) {

        new Printf().show();

    }
}

```

3. El método format de la clase String

En la clase String se encuentra el método estático `static String format(String format, Object... args)` que funciona como *printf* pero la diferencia es que este método devuelve la cadena formateada:

```

package tema1_3_EscrituraEnPantalla;

public class StringFormat {

    public void show() {

        double decimal=3.8974523;
        String result;

        result = String.format("El número con dos decimales es %.2f", decimal);
        System.out.println(result);

    }

    public static void main(String[] args) {

        new StringFormat().show();

    }
}

```

La cadena *result* contiene la cadena `El número con dos decimales es 3,90`.

4. Caracteres de bloque

El carácter `█` es uno de varios caracteres de bloque disponibles en Unicode. Estos caracteres son útiles para crear gráficos simples o para resaltar texto en la consola. Aquí hay una lista de algunos caracteres de bloque en Unicode que se pueden utilizar:

```

package tema1_3_EscrituraEnPantalla.colores;

```

```

public class BlockCharacter {

    public void show() {

        char blockCharacter='\u2586';
        String hexadecimalBlockCharacter,hexadecimalBlockCharacterFormatted;

        for (int i = 0; i < 14; i++) {
            hexadecimalBlockCharacter=Integer.toHexString((int) blockCharacter);

            hexadecimalBlockCharacterFormatted=String.format("'\\u%s'",hexadecimalBlockCharac
            ter.toUpperCase());
            System.out.printf("%s: %c\n", hexadecimalBlockCharacterFormatted,
            blockCharacter);
            blockCharacter++;
        }

    }

    public static void main(String[] args) {

        new BlockCharacter().show();

    }

}

```

Salida por consola:

```

'\u2586': ■
'\u2587': ■
'\u2588': ■
'\u2589': ■
'\u258A': ■
'\u258B': ■
'\u258C': ▏
'\u258D': ▏
'\u258E': ▏
'\u258F': ▏
'\u2590': ▏
'\u2591': ░
'\u2592': ▒
'\u2593': ▓

```

5. Colores

5.1 Códigos de escape ANSI

Las secuencias de escape ANSI permiten enviar información de control a la consola para cambiar los atributos del texto representado. Solo debemos anteponer a la cadena que queremos mostrar en color el código de escape ANSI referente al color.

Cuando aplicamos un color a la salida por consola, el resto de salida por consola seguirá saliendo de dicho color hasta que se especifique otro color de salida o finalicemos el texto con algunos de los siguientes códigos:

- **RESET_TEXT:** "\u001B[39m": restablece el color del texto al valor predeterminado en la consola.
- **RESET:** "\u001B[0m": restablece el color del texto y del fondo al valor predeterminado en la consola.

```
package tema1_3_EscrituraEnPantalla.colores;

public final class Colors {

    public static final String BLACK = "\u001B[30m";
    public static final String RED = "\u001B[31m";
    public static final String GREEN = "\u001B[32m";
    public static final String YELLOW = "\u001B[33m";
    public static final String BLUE = "\u001B[34m";
    public static final String PURPLE = "\u001B[35m";
    public static final String CYAN = "\u001B[36m";
    public static final String WHITE = "\u001B[37m";

    public static final String BLACK_BRIGHT = "\u001B[90m";
    public static final String RED_BRIGHT = "\u001B[91m";
    public static final String GREEN_BRIGHT = "\u001B[92m";
    public static final String YELLOW_BRIGHT = "\u001B[93m";
    public static final String BLUE_BRIGHT = "\u001B[94m";
    public static final String PURPLE_BRIGHT = "\u001B[95m";
    public static final String CYAN_BRIGHT = "\u001B[96m";
    public static final String WHITE_BRIGHT = "\u001B[97m";

    public static final String BLACK_BACKGROUND = "\u001B[40m";
    public static final String RED_BACKGROUND = "\u001B[41m";
    public static final String GREEN_BACKGROUND = "\u001B[42m";
    public static final String YELLOW_BACKGROUND = "\u001B[43m";
    public static final String BLUE_BACKGROUND = "\u001B[44m";
    public static final String PURPLE_BACKGROUND = "\u001B[45m";
    public static final String CYAN_BACKGROUND = "\u001B[46m";
    public static final String WHITE_BACKGROUND = "\u001B[47m";

    public static final String BOLD = "\u001B[1m"; // Negrita
    public static final String UNDERLINE = "\u001B[4m"; // Subrayado
    public static final String REVERSED = "\u001B[7m"; // Invierte los colores del
    texto y del fondo
    public static final String RESET = "\u001B[0m"; // Restablece el color del
    texto y del fondo al valor predeterminado en la consola
    public static final String RESET_TEXT = "\u001B[39m"; // Restablece el color
    del texto al valor predeterminado en la consola

}
```

```
package tema1_3_EscrituraEnPantalla.colores;

import static tema1_4_EscrituraEnPantalla.colores.Colors.BOLD;
```

```

import static tema1_4_EscrituraEnPantalla.colores.Colors.CYAN;
import static tema1_4_EscrituraEnPantalla.colores.Colors.GREEN;
import static tema1_4_EscrituraEnPantalla.colores.Colors.GREEN_BACKGROUND;
import static tema1_4_EscrituraEnPantalla.colores.Colors.PURPLE_BACKGROUND;
import static tema1_4_EscrituraEnPantalla.colores.Colors.RED;
import static tema1_4_EscrituraEnPantalla.colores.Colors.RED_BACKGROUND;
import static tema1_4_EscrituraEnPantalla.colores.Colors.RESET;
import static tema1_4_EscrituraEnPantalla.colores.Colors.RESET_TEXT;
import static tema1_4_EscrituraEnPantalla.colores.Colors.REVERSED;
import static tema1_4_EscrituraEnPantalla.colores.Colors.UNDERLINE;
import static tema1_4_EscrituraEnPantalla.colores.Colors.WHITE_BACKGROUND;
import static tema1_4_EscrituraEnPantalla.colores.Colors.YELLOW;

public class ColorsUse {

    public void show() {

        String[] colorNames = {
            "Negro", "Rojo", "Verde", "Amarillo", "Azul", "Magenta", "Cian",
            "Blanco",
            "Negro brillante", "Rojo brillante", "Verde brillante", "Amarillo
            brillante",
            "Azul brillante", "Magenta brillante", "Cian brillante", "Blanco
            brillante"};

        //El primer RESET sirve por si ejecutamos más de una vez
        System.out.printf("%s%s\n", RESET, RED, "Este texto es de color rojo");
        System.out.printf("%s\n", RESET_TEXT, "Volvemos al color del texto por
        defecto");
        System.out.printf("%s\n", GREEN, "...y ahora es verde");
        System.out.printf("%s\n", PURPLE_BACKGROUND, "Fondo morado");
        System.out.printf("%s\n", CYAN, WHITE_BACKGROUND, "Fondo blanco con
        texto celeste");
        System.out.printf("%s\n", CYAN, WHITE_BACKGROUND, BOLD, "Fondo
        blanco con texto celeste en negrita");
        System.out.printf("%s\n", CYAN, WHITE_BACKGROUND, UNDERLINE, "Fondo
        blanco con texto celeste subrayado");
        System.out.printf("%s\n", RESET_TEXT,
            "Volvemos al color del texto por defecto y se mantiene el fondo
            como estaba");
        System.out.printf("%s%c\n", YELLOW, RED_BACKGROUND, (char) 9733);//
        Estrella
        System.out.printf("%s\n", YELLOW, GREEN_BACKGROUND, "Fondo verde con
        texto amarillo");
        System.out.printf("%s\n", REVERSED, "Fondo amarillo con texto verde
        usando REVERSED");
        System.out.printf("%s\n\n", RESET, "Volvemos al color del texto y del
        fondo por defecto");

        for (int i = 0; i < 8; i++) {
            System.out.printf("\u001B[3dmEste es el color
            %s\n", i, colorNames[i], RESET);
        }

        for (int i = 0; i < 8; i++) {

```



```

        sequence = String.format("%s%s%d%s", ESCAPED_RGB_MODE,
"0;", j, ";0m");
        System.out.printf("%-22s: %s%s%d%s%c\n", sequence,
RGB_MODE, "0;", j, ";0m",blockCharacter);
    }
}
case 2 -> {
    System.out.printf("%sAzul:\n",RESET_TEXT);
    for (int j = 0; j <= 255; j++) {
        sequence = String.format("%s%s%d%s", ESCAPED_RGB_MODE,
"0;0;", j, "m");
        System.out.printf("%-22s: %s%s%d%s%c\n", sequence,
RGB_MODE, "0;0;", j, "m",blockCharacter);
    }
}
default -> {
    System.out.println("Color erróneo");
}
};
}

}

public static void main(String[] args) {

    new RGBColors().show();

}

}

```

Se pueden mezclar y generar otros colores. Por ejemplo, mezclando el rojo y el verde se produce como resultado el amarillo. Dejando el azul a 0, se genera un amarillo puro. Si el azul se pone por ejemplo en 153, entonces el amarillo resultante es más claro:

```

package tema1_3_EscrituraEnPantalla.colores;

import static tema1_4_EscrituraEnPantalla.colores.Colors.RESET_TEXT;

public class YellowColor {

    public void show() {

        final String RGB_MODE = "\u001B[38;2;";
        final String ESCAPED_RGB_MODE = "\\u001B[38;2;";
        final char blockCharacter='\u2588';
        String sequence = String.format("%s%s", ESCAPED_RGB_MODE, "255;255;0m");
        System.out.printf("%sAmarillo puro: %-22s: %s%s%c\n", RESET_TEXT,
sequence, RGB_MODE, "255;255;0m",blockCharacter);
        sequence = String.format("%s%s", ESCAPED_RGB_MODE, "255;255;153m");
        System.out.printf("%sAmarillo más clarito: %-22s: %s%s%c\n", RESET_TEXT,
sequence, RGB_MODE, "255;255;153m",blockCharacter);

    }

}

```

```

    public static void main(String[] args) {

        new YellowColor().show();

    }

}

```

Se puede generar cualquier color mezclando los tres:

```

package tema1_3_EscrituraEnPantalla.colores;

import static tema1_4_EscrituraEnPantalla.colores.Colors.RESET_TEXT;

public class RGBMixingTheThreeColours {

    public void show() {

        mixingTheThreeColours(139,69,19);//Marrón
        mixingTheThreeColours(128, 128, 128);//Gris
        mixingTheThreeColours(255, 165, 0);//Naranja
        mixingTheThreeColours(255, 192, 203);//Rosa
        mixingTheThreeColours(255, 0, 255);//Fucsia
        mixingTheThreeColours(64, 224, 208);//Turquesa

    }

    public void mixingTheThreeColours(int red,int green,int blue) {

        final String RGB_MODE = "\u001B[38;2;";
        final String ESCAPED_RGB_MODE = "\\u001B[38;2;";
        final char blockCharacter='\u2588';
        String sequence;
        sequence = String.format("%s%d;%d;%dm", ESCAPED_RGB_MODE, red, green,
blue);
        System.out.printf("%s%-24s: %s%d;%d;%dm%c\n", RESET_TEXT, sequence,
RGB_MODE, red, green, blue, blockCharacter);

    }

    public static void main(String[] args) {

        new RGBMixingTheThreeColours().show();

    }

}

```

1.4 Lectura por teclado

1.4 Lectura por teclado

1. La clase `System`
2. La clase `Scanner`
3. Buffer de datos
4. Ejemplo de `Scanner` y `printf`

1. La clase `System`

Las aplicaciones pueden necesitar acceder a los recursos del sistema, como por ejemplo, a los dispositivos de entrada/salida estándar para recoger datos desde el teclado o mostrar datos por pantalla. En Java, la entrada por teclado y la salida de información por pantalla se hace mediante la clase `System` del paquete `java.lang` de la biblioteca de clases de Java. Dicha clase contiene el atributo ***in*** para el teclado y el atributo ***out*** para la pantalla.

Dichos atributos son estáticos por lo que se accede a ellos con el nombre de la clase `System`:

`System.in` Entrada estándar: teclado

`System.out` Salida estándar: pantalla

2. La clase `Scanner`

El kit de desarrollo de Java, a partir de su versión 1.5, incorpora la clase `Scanner` del paquete `java.util` la cual posee métodos para leer valores de entrada que pueden venir de varias fuentes, como por ejemplo, de datos introducidos por teclado o datos almacenados en un archivo. Para leer datos desde el teclado, tenemos que pasarle el `System.in` al constructor del `Scanner`:

```
Scanner keyboard = new Scanner (System.in);
```

Veamos algunos métodos de esta clase:

- `nextByte`, `nextShort`, `nextInt` y `nextLong`: para leer datos de tipo entero.
- `nextFloat` y `nextDouble`: para leer números decimales.
- `next`: lee una palabra, es decir, hasta que encuentre un espacio.
- `nextLine`: lee una línea completa, es decir, hasta que encuentre un salto de línea (`\n`).

Por ejemplo, para leer un dato de tipo `int` sería:

```
int i = keyboard.nextInt ();
```

O bien esta otra instrucción para leer una línea completa:

```
String string = keyboard.nextLine();
```

He aquí un ejemplo de entrada de teclado con la clase `Scanner`:

```

package tema1_4_LecturaPorTeclado;

import java.util.Scanner;

public class ScannerClass {

    @SuppressWarnings("resource")
    public void show() {
        /*
         * La JVM (máquina virtual de Java) cierra el teclado cuando
         * la aplicación termina, por lo tanto, no hace falta que
         * lo cerremos. Para que no nos salga el warning,
         * añadimos @SuppressWarnings("resource")
         */
        Scanner keyboard = new Scanner(System.in);
        String string;
        int i;
        float f;
        boolean b;

        string = keyboard.nextLine();
        System.out.println(string);
        i = keyboard.nextInt();
        System.out.println(i);
        b = keyboard.nextBoolean();
        System.out.println(b);
        /*
         * El símbolo separador de decimales será la coma si nuestro
         * idioma por defecto del sistema operativo está configurado
         * en español:
         */
        f = keyboard.nextFloat();
        System.out.println(f);
    }

    public static void main(String[] args) {

        new ScannerClass().show();
    }
}

```

El símbolo separador de decimales será la coma si nuestro idioma por defecto del sistema operativo está configurado en español. Si deseamos que el separador decimal sea el punto en lugar de la coma, entonces tendremos que añadir `useLocale(Locale.US)` cuando creamos el Scanner:

```

package tema1_4_LecturaPorTeclado;

import java.util.Locale;
import java.util.Scanner;

```

```

public class ScannerClassLocale {

    @SuppressWarnings("resource")
    public void show() {

        Scanner keyboard = new Scanner(System.in).useLocale(Locale.US);
        float f;
        // Debido al useLocale(Locale.US), el símbolo separador de decimales será
        el .

        f = keyboard.nextFloat();
        System.out.println(f);

    }

    public static void main(String[] args) {

        new ScannerClassLocale().show();

    }

}

```

Veamos un ejemplo de lectura de palabras con *next()*:

```

package tema1_4_LecturaPorTeclado;

import java.util.Scanner;

public class Next1 {

    @SuppressWarnings("resource")
    public void show() {

        Scanner keyboard = new Scanner(System.in);
        String string1, string2, string3;

        System.out.println("Introduzca 3 palabras separadas por espacio: ");
        string1 = keyboard.next();
        string2 = keyboard.next();
        string3 = keyboard.next();
        System.out.println("Primera palabra: " + string1);
        System.out.println("Segunda palabra: " + string2);
        System.out.println("Tercera palabra: " + string3);

    }

    public static void main(String[] args) {

        new Next1().show();

    }

}

```

3. Buffer de datos

Un buffer de datos es un espacio de memoria donde se almacenan datos de manera temporal mientras son transferidos o procesados. Los sistemas de entrada de teclado poseen un buffer que almacena las teclas presionadas.

Cuando introducimos un dato por teclado, para indicar que hemos finalizado la introducción pulsamos la tecla Enter. Cuando se ejecuta cualquier *next* con la clase Scanner(excepto *nextLine*), se coge el dato correspondiente del buffer pero el Enter se queda guardado como un *\n* (salto de línea) en el buffer. Si a continuación se ejecuta un *nextLine*, lee hasta el salto de línea del buffer, es decir, lee cadena vacía y ya no le da opción al usuario de introducir la cadena. Una posible solución sería limpiar el buffer con un *nextLine* y luego realizar otro *nextLine* para pedirle la cadena al usuario.

```
package tema1_4_LecturaPorTeclado;

import java.util.Scanner;

public class NextLine {

    @SuppressWarnings("resource")
    public void show() {

        Scanner keyboard = new Scanner(System.in);
        String string;
        int number;
        boolean b;

        // El nextLine toma el salto de línea dejado por cualquier otro next que
        no lo consuma (nextBoolean,nextInt, next)
        System.out.println("Introduzca un número entero: ");
        number = keyboard.nextInt();
        System.out.println(number);
        System.out.println("Introduzca una cadena: ");
        string = keyboard.nextLine();
        System.out.println(string);
        System.out.println("Introduzca un boolean: ");
        b = keyboard.nextBoolean();
        System.out.println(b);
        System.out.println("Introduzca una cadena: ");
        string = keyboard.nextLine();
        System.out.println(string);
        System.out.println("Introduzca una cadena con next: ");
        string = keyboard.next();
        System.out.println(string);
        System.out.println("Introduzca una cadena con nextLine: ");
        string = keyboard.nextLine();
        System.out.println(string);
        // Solución: poner un nextLine que coja el salto de línea.A esto se le
        conoce como limpiar el buffer.
        System.out.println("Vamos a solucionar el problema.\nIntroduzca un número
        entero: ");
        number = keyboard.nextInt();
        System.out.println(number);
```

```

        System.out.println("Introduzca una cadena: ");
        keyboard.nextLine(); // Limpieza del buffer
        string = keyboard.nextLine();
        System.out.println(string);

    }

    public static void main(String[] args) {

        new NextLine().show();

    }

}

```

Otra solución sería indicarle al objeto *scanner* que como delimitador use el `\n` ya que por defecto es el carácter espacio. De esta forma, cuando utilicemos *next()*, leerá todos los caracteres hasta el siguiente `\n`. El único inconveniente de esto es que no podremos leer palabra a palabra.

```

package tema1_4_LecturaPorTeclado;

import java.util.Scanner;

public class Next2 {

    @SuppressWarnings("resource")
    public void show() {

        // Para Linux y Mac:
        //Scanner keyboard = new Scanner(System.in).useDelimiter("\\n");
        // Para Windows:
        Scanner keyboard = new Scanner(System.in).useDelimiter("\\r\\n");
        String string;
        int number;

        System.out.println("Introduzca un número entero: ");
        number = keyboard.nextInt();
        System.out.println(number);
        System.out.println("Introduzca una cadena: ");
        string = keyboard.next();
        System.out.println(string);
        /* Ahora Scanner está configurado para usar un delimitador personalizado.
        En este caso, como nextInt() lee hasta el delimitador que has definido
        (que es \r\n en windows o \n
        en Linux), la secuencia de nueva línea que queda no causa problemas
        porque se trata como el delimitador de entrada.
        Después de nextInt(), llamas a keyboard.next(). Este método lee el
        siguiente token en función del delimitador que has establecido, que es
        \r\n o \n, ya no es el espacio en blanco.
        Dado que has definido el delimitador personalizado, next() ignora
        completamente los espacios en blanco y lee la entrada correctamente,
        consumiendo la cadena hasta el próximo salto de línea.*/
    }
}

```

```

    public static void main(String[] args) {

        new Next2().show();

    }

}

```

4. Ejemplo de Scanner y printf

He aquí un ejemplo de entrada/salida de datos utilizando Scanner y printf:

```

package tema1_4_LecturaPorTeclado;

import java.util.Scanner;

public class InputOutput {

    @SuppressWarnings("resource")
    public void show() {

        Scanner keyboard = new Scanner(System.in);
        String name;
        int age;
        float salary;

        // Entrada de datos
        System.out.print("Nombre: ");
        name = keyboard.nextLine();
        System.out.print("Edad: ");
        age = keyboard.nextInt();
        System.out.print("Salario: ");
        salary = keyboard.nextFloat();

        // Salida de datos
        System.out.printf("\nBienvenido: %s\n", name);
        System.out.printf("Tienes: %d años\n", age);
        System.out.printf("Tu salario es: %.2f euros\n", salary);

    }

    public static void main(String[] args) {

        new InputOutput().show();

    }

}

```

10 Genéricos

10 Genéricos

1. Introducción
2. Clases genéricas
3. Métodos genéricos
4. Interfaces genéricas
5. Uso de varios genéricos
6. Tipos vinculados (o limitados)
7. Tipos comodín

1. Introducción

Desde su versión original 1.0, muchas nuevas características han sido añadidas a Java. Todas han mejorado y ampliado el alcance del lenguaje, pero una que ha tenido un impacto especialmente profundo y de gran alcance es el uso de genéricos porque sus efectos se sintieron en todo el lenguaje Java ya que añadieron un elemento de sintaxis completamente nuevo y causaron cambios en muchas de las clases y métodos de la API principal. Los genéricos de Java están basados en los famosos *templates* de C++ y se introdujeron en la versión 5 de Java.

En su esencia, el término **genérico** significa **tipo parametrizado**, es decir, el tipo de datos sobre el que se opera se especifica como parámetro. Muchos algoritmos son lógicamente los mismos, independientemente del tipo de datos a los que se apliquen. Por ejemplo, un algoritmo de ordenación es el mismo si está ordenando elementos de tipo entero, decimales o cadena. Con los genéricos, se puede definir un algoritmo una vez, independientemente del tipo de datos y luego aplicar ese algoritmo a una amplia variedad de tipos de datos sin ningún esfuerzo adicional.

Lo que se use en un genérico debe ser un objeto, por lo tanto, los genéricos no funcionan con datos primitivos. Para resolver esta situación, la API de Java incorpora las clases envoltorio (wrapper class), que no son más que dotar a los datos primitivos con un envoltorio que permita tratarlos como objetos.

Los genéricos permiten crear clases, métodos e interfaces en los que los tipos de datos sobre los que actúan son un parámetro más, de manera que pueden adaptarse de forma automática para emplear dichos tipos.

2. Clases genéricas

Cuando una clase tiene algún parámetro de tipo se denomina **clase genérica**. Al crear objetos de esa clase, se debe indicar el tipo real por el que se va a sustituir el genérico. A partir de ese momento, donde se utilice el genérico, se sustituirá por el tipo real con el que se ha creado el objeto.

Veamos un ejemplo de clase genérica:

```
package tema10_Genericos;
```



```

public class GenericClass<T> { //Clase genérica de tipo T

    private T attribute; //Declaramos un atributo de tipo T

    public GenericClass(T attribute) {
        this.attribute = attribute;
    }

    public T getAttribute() {
        return attribute;
    }

    public void show() {
        System.out.printf("El tipo de T es %s",
attribute.getClass().getSimpleName());
        System.out.printf(" ---> valor del atributo: %s\n", getAttribute());
    }
}

```

```

package tema10_Genericos;

public class MainGenericClass {

    public void show() {

        //T es un parámetro de tipo que se sustituye por un tipo real al crear un
objeto de la clase
        GenericClass<Integer> genInteger; //Se sustituye T por Integer
        GenericClass<Double> genDouble; //Se sustituye T por Double
        GenericClass<String> genString; //Se sustituye T por String
        GenericClass<Vehicle> genVehicle; //Se sustituye T por Vehicle

        genInteger = new GenericClass<Integer>(80);
        genDouble = new GenericClass<Double>(78.6);
        genString = new GenericClass<String>("Clases genéricas");
        genVehicle = new GenericClass<Vehicle>(new Vehicle(4, "azul"));

        genInteger.show();
        genDouble.show();
        genString.show();
        genVehicle.show();
    }

    public static void main(String[] args) {

        new MainGenericClass().show();
    }
}

```

Salida por consola:

```
El tipo de T es Integer ---> Valor del atributo: 80
El tipo de T es Double ---> Valor del atributo: 78.6
El tipo de T es String ---> Valor del atributo: Clases genéricas
El tipo de T es Vehicle ---> Valor del atributo: vehicle [wheelCount=4,
speed=0.0, colour=azul]
```

Podemos observar que la clase tiene un atributo de tipo `T`, pero no se permiten variables estáticas de tipo `T`.

Una de las ventajas del uso de genéricos es que el compilador realiza comprobaciones de tipos y no permite asignar a un objeto la referencia de otro con distinta traducción del tipo de genérico:

```
genInteger = genDouble;//Error de compilación
```

El compilador informa con el siguiente mensaje de error: No puede convertir de `GenericClass<Double>` a `GenericClass<Integer>`. Si ambos hubieran sido del mismo tipo, sí se podría haber realizado la asignación.

A partir de Java 7, cuando instanciamos un objeto de una clase genérica, no es necesario especificar el tipo con el que queremos trabajar siempre y cuando el compilador pueda inferirlo (adivinarlo) a partir del contexto. Por lo tanto, el siguiente código:

```
genInteger = new GenericClass<Integer>(80);
genDouble = new GenericClass<Double>(78.6);
genString = new GenericClass<String>("Clases genéricas");
genVehicle = new GenericClass<Vehicle>(new vehicle(4, "azul"));
```

también se puede realizar de la siguiente manera:

```
genInteger = new GenericClass<>(80);/*Como la variable a la que se asigna es
GenericClass<Integer> es capaz de inferir que queremos crear un
GenericClass<Integer> al hacer el new */
genDouble = new GenericClass<>(78.6);
genString = new GenericClass<>("Clases genéricas");
genVehicle = new GenericClass<>(new vehicle(4, "azul"));
```

A esto se le conoce como **operador diamante**, dado que al quitar el tipo de la instanciación nos queda `<>` (que presenta una forma de diamante).

3. Métodos genéricos

Los métodos de una clase genérica pueden usar el parámetro del tipo de una clase y por tanto son genéricos de forma automática. Sin embargo, también podemos declarar métodos genéricos dentro de clases no genéricas, tanto estáticos como dinámicos, que reciban uno o más parámetros genéricos.

Los genéricos en un método se declaran en la firma del método delante del tipo de devolución.

Veamos un ejemplo de un método genérico:

```
package tema10_Genericos;

public class GenericMethod {
```

```

    public <T> String showType(T parameter) { //El genérico <T> se declara delante
    del tipo de devolución String

        return parameter.getClass().getSimpleName();

    }

    public void show() {

        System.out.printf("El tipo del parámetro es %s\n", showType(80));
        System.out.printf("El tipo del parámetro es %s\n", showType(78.6));
        System.out.printf("El tipo del parámetro es %s\n", showType("Métodos
genéricos"));
        System.out.printf("El tipo del parámetro es %s\n", showType(new
Vehicle(4, "azul")));

    }

    public static void main(String[] args) {

        new GenericMethod().show();

    }

}

```

Salida por consola:

```

El tipo del parámetro es Integer
El tipo del parámetro es Double
El tipo del parámetro es String
El tipo del parámetro es Vehicle

```

4. Interfaces genéricas

Las interfaces también pueden contener parámetros de tipo, denominándose interfaces genéricas. Las clases que implementan la interfaz, tienen dos posibilidades:

- Implementar la interface traduciendo el genérico por un tipo concreto.
- Implementar la interface de manera genérica convirtiéndose en una clase genérica.

Veamos ejemplos de ambos casos:

Ejemplo de implementación de la interface traduciendo el genérico por un tipo concreto:

```
package tema10_Genericos;

public interface GenericInterface<T> {

    T first(T[] array);

    T last(T[] array);

}
```

```
package tema10_Genericos;

public class MyClass implements GenericInterface<Vehicle> {

    @Override
    public Vehicle first(Vehicle[] array) {
        return array[0];
    }

    @Override
    public Vehicle last(Vehicle[] array) {
        return array[array.length - 1];
    }

    public void show() {

        MyClass myClass = new MyClass();
        Vehicle[] array = new Vehicle[3];
        array[0] = new Vehicle(4, "azul");
        array[1] = new Vehicle(4, "blanco");
        array[2] = new Vehicle(2, "rojo");
        System.out.printf("Primer vehículo del array: %s\n",
myClass.first(array));
        System.out.printf("Último vehículo del array: %s", myClass.last(array));

    }

    public static void main(String[] args) {

        new MyClass().show();

    }

}
```

Salida por consola:

```
Primer vehículo del array: Vehicle [wheelCount=4, speed=0.0, colour=azul]
Último vehículo del array: Vehicle [wheelCount=2, speed=0.0, colour=rojo]
```

Ejemplo de implementación de la interface de manera genérica convirtiéndose en una clase genérica:

```

package tema10_Genericos;

public class GenericClass2<T> implements GenericInterface<T> {

    @Override
    public T first(T[] array) {
        return array[0];
    }

    @Override
    public T last(T[] array) {
        return array[array.length - 1];
    }

}

```

```

package tema10_Genericos;

public class MainGenericClass2 {

    public void show() {

        vehicle[] array = new vehicle[3];
        array[0] = new vehicle(4, "azul");
        array[1] = new vehicle(4, "blanco");
        array[2] = new vehicle(2, "rojo");
        GenericClass2<vehicle> genVehicle = new GenericClass2<>();
        System.out.printf("Primer vehículo del array: %s\n",
genVehicle.first(array));
        System.out.printf("Último vehículo del array: %s",
genVehicle.last(array));

    }

    public static void main(String[] args) {

        new MainGenericClass2().show();

    }

}

```

Salida por consola:

```

Primer vehículo del array: vehicle [wheelCount=4, speed=0.0, colour=azul]
Último vehículo del array: vehicle [wheelCount=2, speed=0.0, colour=rojo]

```

5. Uso de varios genéricos

Se pueden utilizar varios genéricos indicándolos separados por comas dentro del operador diamante.

```

package tema10_Genericos;

```

```

public class GenericClass3<T, V> {

    private T attribute1;
    private V attribute2;

    public GenericClass3(T attribute1, V attribute2) {
        this.attribute1 = attribute1;
        this.attribute2 = attribute2;
    }

    public T getAttribute1() {
        return attribute1;
    }

    public V getAttribute2() {
        return attribute2;
    }

    public void showAttributes() {
        System.out.printf("El tipo de T es %s",
attribute1.getClass().getSimpleName());
        System.out.printf(" ---> valor del atributo: %s\n", getAttribute1());
        System.out.printf("El tipo de V es %s",
attribute2.getClass().getSimpleName());
        System.out.printf(" ---> valor del atributo: %s\n\n", getAttribute2());

    }

}

```

```

package tema10_Genericos;

public class MainGenericClass3 {

    public void show() {

        GenericClass3<Integer, Integer> variousGenerics1 = new GenericClass3<>
(80, 56);
        GenericClass3<Integer, Double> variousGenerics2 = new GenericClass3<>(5,
78.6);
        GenericClass3<String, Vehicle> variousGenerics3 = new GenericClass3<>
("Clases genéricas",
            new Vehicle(4, "azul"));
        variousGenerics1.showAttributes();
        variousGenerics2.showAttributes();
        variousGenerics3.showAttributes();

    }

    public static void main(String[] args) {

        new MainGenericClass3().show();

    }

}

```

```
}
```

Salida por consola:

```
El tipo de T es Integer ---> Valor del atributo: 80
El tipo de V es Integer ---> Valor del atributo: 56

El tipo de T es Integer ---> Valor del atributo: 5
El tipo de V es Double ---> Valor del atributo: 78.6

El tipo de T es String ---> Valor del atributo: Clases genéricas
El tipo de V es Vehicle ---> Valor del atributo: Vehicle [wheelCount=4,
speed=0.0, colour=azul]
```

El uso de genéricos puede crear situaciones de ambigüedad, sobretodo en casos de sobrecarga de métodos:

```
class Gen<T, V> {
    /* Estos dos métodos se sobrecargan pero dado que T y V pueden
       ser del mismo tipo, se generarían dos métodos iguales por
       lo que el compilador genera un error y este código no compila */
    void method(T ob) {}

    void method(V ob) {}
}
```

6. Tipos vinculados (o limitados)

Java ofrece los tipos vinculados que permiten, al especificar un parámetro de tipo, crear un vínculo superior que declare la superclase de la que deben derivarse todos los argumentos de tipo. Para ello usamos la cláusula *extends* al especificar los parámetros de tipo: `<T extends superclass>`. Esto especifica que `T` solo se puede reemplazar por *superclass* o por subclases de *superclass*.

La cláusula *extends* también puede referirse a interfaces: `<T extends interface>`. En este caso, `T` solo se puede reemplazar por una clase que implemente dicha interfaz.

Los tipos numéricos heredan de la clase abstracta *Number*, por lo tanto, podemos limitar los parámetros de tipo a únicamente tipos numéricos: `<T extends Number>`. Esto nos permitiría utilizar métodos de la clase *Number*:

```
package tema10_Genericos;

public class GenNumeric<T extends Number> { //Limitamos T a tipos numéricos

    private T attribute;

    public GenNumeric(T attribute) {
        this.attribute = attribute;
    }

    public T getAttribute() {
```

```

        return attribute;
    }

    double fraction() {
        return attribute.doubleValue() - attribute.intValue(); //Se pueden
        emplear métodos de la clase Number
    }
}

```

```

package tema10_Genericos;

public class MainGenNumeric {

    public void show() {

        GenNumeric<Integer> genInteger;
        GenNumeric<Double> genDouble;

        genInteger = new GenNumeric<Integer>(80);
        genDouble = new GenNumeric<Double>(78.63);

        System.out.printf("La parte decimal de %s es %.2f\n",
            genInteger.getAttribute(), genInteger.fraction());
        System.out.printf("La parte decimal de %s es %.2f\n",
            genDouble.getAttribute(), genDouble.fraction());

    }

    public static void main(String[] args) {

        new MainGenNumeric().show();

    }

}

```

Salida por consola:

```

La parte decimal de 80 es 0,00
La parte decimal de 78.63 es 0,63

```

Si intentamos utilizar una clase que no sea descendiente de *Number*, nos da un error de compilación:

```

GenNumeric<String> genString; //Error de compilación

```

Los tipos vinculados resultan especialmente útiles para garantizar que un parámetro sea compatible con otro:


```
class Pair<T, V extends T> { //V debe tener el mismo tipo que T o ser una
    subclase de T
    ....
}
```

V debe tener el mismo tipo que T o ser una subclase de T. Veamos ejemplos a la hora de crear objetos de la clase *Pair*:

```
Pair<Integer, Integer> pair1 = new Pair<Integer, Integer>(); //Correcto
Pair<Number, Integer> pair2 = new Pair<Number, Integer>(); //Correcto, Integer es
una subclase de Number
Pair<Integer, String> pair3 = new Pair<Integer, String>(); //Incorrecto, String
no es una subclase de Integer
```

7. Tipos comodín

El signo de interrogación `?` se conoce como comodín (wildcard) y representa un tipo desconocido. Tiene varios usos, uno de ellos es que es necesario utilizarlo cuando mezclamos objetos de la misma clase pero con distinta traducción del genérico. Veremos más usos de los tipos comodín en el [Tema 11. Colecciones](#).

Veamos un ejemplo sin tipo comodín para ver dónde nos podría hacer falta:

```
package tema10_Genericos;

import java.util.Arrays;

public class GenericClass4<T> {

    private T[] array;

    GenericClass4(T[] array) {
        this.array = array;
    }

    public void showArray() {
        System.out.println(Arrays.toString(array));
    }

    public boolean equalsSize(GenericClass4<T> gen) {
        return array.length == gen.array.length;
    }

}
```

```
package tema10_Genericos;

public class MainGenericClass4 {

    public void show() {

        Integer[] array1 = { 8, 7, 9 };
        Integer[] array2 = { 3, 5, 12 };
    }

}
```

```

        GenericClass4<Integer> gen1 = new GenericClass4<>(array1);
        GenericClass4<Integer> gen2 = new GenericClass4<>(array2);
        gen1.showArray();
        gen2.showArray();
        System.out.printf("%s tienen el mismo tamaño", gen1.equalsSize(gen2) ?
        "Sí" : "No");

    }

    public static void main(String[] args) {

        new MainGenericClass4().show();

    }

}

```

Salida por consola:

```

[8, 7, 9]
[3, 5, 12]
Sí tienen el mismo tamaño

```

El método `equalsSize` recibe un objeto por parámetro y devuelve si ambos objetos tienen el mismo tamaño del array. En este caso funciona porque tienen la misma traducción del genérico (Integer). Pero ¿y si le pasamos por ejemplo uno de Double?:

```

package tema10_Genericos;

public class Main {

    public void show() {

        Integer[] array1 = { 8, 7, 9 };
        Double[] array2 = { 3.6, 5.4, 12.42 };
        GenericClass4<Integer> gen1 = new GenericClass4<>(array1);
        GenericClass4<Double> gen2 = new GenericClass4<>(array2);
        gen1.showArray();
        gen2.showArray();
        System.out.printf("%s tienen el mismo tamaño", gen1.equalsSize(gen2) ?
        "Sí" : "No");

    }

    public static void main(String[] args) {

        new Main().show();

    }

}

```

Nos da un error de compilación porque en el objeto `gen1` se ha traducido el genérico por `Integer` y le estamos pasando como argumento `gen2` que se ha traducido por `Double`. Entonces, en este caso, como lo que queremos en el método `equalsSize` es simplemente comparar el tamaño de dos arrays y no nos importa que los arrays sean de distinto tipo, lo que podemos hacer es utilizar el tipo comodín `?` en el parámetro de `equalsSize` para indicar que nos tienen que pasar un objeto de un tipo pero que no tiene por qué necesariamente coincidir con `T`:

```
package tema10_Genericos;

import java.util.Arrays;

public class GenericClass5<T> {

    private T[] array;

    GenericClass5(T[] array) {
        this.array = array;
    }

    public void showArray() {
        System.out.println(Arrays.toString(array));
    }

    public boolean equalsSize(GenericClass5<?> gen) { //Tipo comodín
        return array.length == gen.array.length;
    }

}
```

```
package tema10_Genericos;

public class MainGenericClass5 {

    public void show() {

        Integer[] array1 = { 8, 7, 9 };
        Double[] array2 = { 3.6, 5.4, 12.42 };
        GenericClass5<Integer> gen1 = new GenericClass5<>(array1);
        GenericClass5<Double> gen2 = new GenericClass5<>(array2);
        gen1.showArray();
        gen2.showArray();
        System.out.printf("%s tienen el mismo tamaño", gen1.equalsSize(gen2) ?
        "Sí" : "No");

    }

    public static void main(String[] args) {

        new MainGenericClass5().show();

    }

}
```

Salida por consola:

```
[8, 7, 9]
```

```
[3.6, 5.4, 12.42]
```

```
Sí tienen el mismo tamaño
```

11 Colecciones

11 Colecciones

1. Introducción
2. Interfaz Collection
3. Interfaz List
4. Clase ArrayList
5. Invarianza, covarianza y contravarianza
6. Iteradores
7. Métodos equals y hashCode
8. Colecciones sin duplicados
 - 8.1 Clase HashSet
 - 8.2 Clase LinkedHashSet
 - 8.3 Clase EnumSet
9. Mapas
 - 9.1 Clase HashMap
 - 9.2 Clase LinkedHashMap
 - 9.3 Ejemplo de uso de un mapa en un enum
 - 9.4 Clase EnumMap
10. Árboles
 - 10.1 Clase TreeSet
 - 10.2 Clase TreeMap
11. Pilas y colas
 - 11.1 Clase ArrayDeque
12. Clase Collections

1. Introducción

Una colección representa un grupo de objetos. Estos objetos son conocidos como elementos. Cuando queremos trabajar con un conjunto de elementos, necesitamos un almacén donde poder guardarlos.

Las colecciones son estructuras de datos con la peculiaridad de que son estructuras dinámicas. Esto quiere decir que pueden aumentar o disminuir su tamaño dependiendo de los elementos que almacenan, lo que supone una mejora respecto a las estructuras de datos estáticas cuyo tamaño se define en su creación y no se puede alterar en tiempo de ejecución, como por ejemplo, los arrays.

El API de Java nos proporciona en el paquete `java.util` el framework de las colecciones, que nos permite utilizar diferentes estructuras de datos para almacenar y recuperar objetos de cualquier clase. Java tiene desde la versión 2, todo un juego de clases e interfaces para guardar colecciones de objetos donde todas las entidades conceptuales están representadas por interfaces y las clases se usan para proveer implementaciones de esas interfaces. Estas clases e interfaces están estructuradas en una jerarquía.

Antes de que aparecieran los genéricos, en las colecciones se almacenaba cualquier objeto que heredara de `Object`. Pero esto presenta ciertos inconvenientes:

- Podríamos tener una colección con objetos completamente distintos, lo que puede dar lugar a problemas ya que en todo momento deberíamos saber qué tipo de objeto y en qué posición de la colección se encuentra el elemento con el que queremos trabajar, de otro modo podríamos tener incongruencias en el código o incluso hacer saltar una excepción.
- Otro inconveniente es que tendríamos que hacer continuos castings para poder trabajar con los elementos de la colección, lo cual resulta tedioso y poco productivo.

Veamos un ejemplo para obtener la suma de los valores almacenados en una lista. La clase `ArrayList` es una colección de Java para listas de elementos, con el método `add` se añaden elementos a la lista y con `get(int index)` se obtiene un elemento de la posición `index`:

```
int total = 0;
ArrayList numbers = new ArrayList();//Creación de la lista
numbers.add(1);//Se añade el elemento 1 a la lista utilizando el método add
numbers.add(2);
numbers.add(3);
for (int i = 0; i < numbers.size(); i++) {
    // Nos vemos a obligados a hacer casting, dado que numbers.get(i) retorna un
    Object:
    total += (int) numbers.get(i);
}
System.out.printf("Total: %d\n", total);
```

Como vemos en el ejemplo anterior, nos vemos obligados a hacer explícitamente un casting cuando obtenemos un elemento de la lista, dado que la lista internamente trabaja con elementos de la clase `Object`. No hay ningún contrato que permita a la clase `ArrayList` saber con qué tipo de datos queremos que trabaje.

Además, es posible añadir elementos de distinto tipo a la misma lista, con el agravante de que más adelante cuando se intente acceder al elemento y se haga un casting sobre él se producirá un error en tiempo de ejecución. Así, si modificamos el ejemplo anterior de la siguiente manera:

```
int total = 0;
ArrayList numbers = new ArrayList();
numbers.add(1);
numbers.add(2);
numbers.add("Antonio");
for (int i = 0; i < numbers.size(); i++) {
    /* Esta línea lanza una excepción en tiempo de ejecución cuando
       se trata de convertir a entero el elemento "Antonio" */
    total += (int) numbers.get(i);
}
System.out.printf("Total: %d\n", total);
```

Así pues, para resolver este problema, a partir de la versión 5 de Java se empezaron a utilizar los **genéricos**. Los genéricos nos permiten establecer un tipo con el que vamos a trabajar en esa colección, de esa manera podemos evitar los problemas mencionados anteriormente.

Por ejemplo, la clase `ArrayList` es una colección de Java para listas de elementos. Si observamos en la API esta clase, se define como `ArrayList<E>`. Esto significa que es una clase genérica donde `E` es el tipo genérico. Cuando se instancie un objeto de esta clase, hay que indicar el tipo concreto que se va a utilizar:

```
ArrayList<String> list = new ArrayList<>();
```

Cualquier método que se ejecute sobre la variable *list* se hará traduciendo la letra genérica *E* por *String* en el código del método.

Para mantener la compatibilidad con las versiones anteriores a Java 5, podemos trabajar con colecciones sin tipo pero se recomienda **siempre utilizar las colecciones parametrizadas**, por ejemplo siempre utilizar `ArrayList<E>` y nunca `ArrayList`.

Así el ejemplo anterior podríamos modificarlo de la siguiente manera:

```
int total = 0;
ArrayList<Integer> numbers = new ArrayList<>();
numbers.add(1);
numbers.add(2);
/* La siguiente línea da un error de compilación, dado que el compilador
   detecta que estamos intentando añadir una cadena a una lista de
   enteros: */
numbers.add("Antonio");
for (int i = 0; i < numbers.size(); i++) {
    /* Ya no es necesario hacer un cast explícito, dado que el compilador
       lo hará internamente por nosotros, al haberle informado de que
       queríamos trabajar con una lista de enteros: */
    total += numbers.get(i);
}
System.out.printf("Total: %d\n", total);
```

Como vemos en el ejemplo anterior, gracias a la información que le suministramos al compilador sobre el tipo de lista con el que queremos trabajar, en este caso `Integer`, el compilador es capaz de detectar en tiempo de compilación que no debería ser posible añadir una cadena a la lista, y además nos evita tener que hacer explícitamente el casting a entero cuando obtenemos los elementos de la lista, porque ya lo puede hacer él internamente por nosotros.

2. Interfaz Collection

Es la interfaz raíz de la jerarquía de las colecciones. Java no proporciona ninguna implementación directa de esta interfaz, sino que proporciona implementaciones de subinterfaces más específicas como *Set* y *List*. Esta interfaz se utiliza normalmente para pasar colecciones y manipularlas cuando se desea la máxima generalidad.

Veamos algunos de los métodos que podemos observar en la API:

- `boolean add(E e)`: añade el elemento *e* a la colección.
- `boolean addAll(Collection<? extends E> c)`: añade todos los elementos de la colección *c*.
- `void clear()`: elimina todos los elementos de la colección.
- `boolean contains(Object o)`: comprueba si el elemento *o* está en la colección.
- `boolean containsAll(Collection<?> c)`: comprueba si todos los elementos de *c* están en la colección.
- `boolean isEmpty()`: comprueba si la colección está vacía.
- `boolean remove(Object o)`: elimina de la colección el elemento *o*.

- `boolean removeAll(Collection<?> c)`: elimina de la colección todos los elementos de *c*.
- `boolean retainAll(Collection<?> c)`: elimina de la colección todos los elementos exceptos los que están en *c*, es decir, obtiene la intersección.
- `int size()`: devuelve el tamaño de la colección.

3. Interfaz List

Esta interfaz es una subinterfaz o interfaz hija de *Collection*, por lo tanto, tiene todos los métodos de *Collection* y además añade los suyos propios.

Este tipo de colección se refiere a listas en las que los elementos de la colección tienen un orden, es decir, existe una secuencia de elementos. Cada elemento tiene un índice o posición. El primer elemento ocupa la posición 0.

La interfaz *List* sí admite elementos duplicados.

Veamos algunos de los métodos que podemos observar en la API:

- `void add(int index, E element)`: inserta el elemento *E* en la posición *index*.
- `boolean add(E e)`: añade el elemento *e* al final de la lista.
- `boolean addAll(int index, Collection<? extends E> c)`: inserta todos los elementos de *c* en la posición *index*.
- `boolean addAll(Collection<? extends E> c)`: añade todos los elementos de *c* al final de la lista.
- `E get(int index)`: devuelve el elemento de la posición *index*.
- `int indexOf(Object o)`: devuelve el índice de la primera ocurrencia del elemento *o* en la lista, o -1 si la lista no contiene el elemento.
- `int lastIndexOf(Object o)`: devuelve el índice de la última ocurrencia del elemento *o* en la lista, o -1 si la lista no contiene el elemento.
- `E remove(int index)`: elimina el elemento que se encuentra en la posición *index*.
- `E set(int index, E element)`: reemplaza el elemento que se encuentra en *index* por el elemento *element*.
- `List<E> subList(int fromIndex, int toIndex)`: devuelve la sublista comprendida entre las posiciones *fromIndex* incluida y *toIndex* excluida.
- Métodos estáticos `of`: devuelven listas no modificables formadas por los elementos recibidos por parámetro. Está sobrecargado varias veces para admitir desde 0 a 10 elementos incluso hay una versión para recibir un *vararg*.

Existen varios tipos de implementaciones realizadas dentro de la plataforma Java para la interfaz *List*, como por ejemplo, *ArrayList*.

4. Clase ArrayList

Se basa en un array redimensionable que aumenta su tamaño según crece la colección de elementos. Lo bueno es que el tiempo de acceso a un elemento en particular es ínfimo. Lo malo es que si queremos eliminar un elemento del principio o del medio, la clase debe mover todos los que le siguen a la posición anterior, para tapar el agujero que deja el elemento removido. Esto hace que sacar elementos del medio o del principio sea costoso.

ArrayList mantiene el orden de inserción, es decir, si recorremos la colección se nos mostrará en el mismo orden en que insertamos los objetos.

Veamos un ejemplo de declaración e inicialización:

```
ArrayList<Integer> list = new ArrayList<Integer>();
```

A partir de java7, no es necesario indicar el genérico en la inicialización:

```
ArrayList<Integer> list = new ArrayList<>();
```

Para hacer el código más genérico, se puede definir la variable de tipo interfaz, ya que dicho código podría funcionar con cualquier clase que implemente la interfaz, simplemente habría que cambiar el `new`:

```
List<Integer> list = new ArrayList<>();
```

Veamos un ejemplo de *ArrayList* donde utiliza métodos tanto de *Collection* como de *List*:

```
package tema11_Colecciones;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class ShowArrayList {

    public void show() {

        List<Integer> list1 = new ArrayList<>();
        List<Integer> list2;
        List<Integer> list3;
        List<Integer> list4;

        list1.add(1);
        list1.add(2); //Se añaden los elementos al final de la lista
        list1.add(6);
        list1.add(2, 5); //Se añade el 5 en la posición 2
        for (Integer i : list1) { //Recorremos la lista con un bucle for-each: 1
2 5 6
            System.out.printf(" %d ", i);
        }
        System.out.println();
        list2 = List.of(3,4);
        list1.addAll(2, list2); //Se inserta list2 en la posición 2 de list1
        for (Integer i : list1) { // 1 2 3 4 5 6
            System.out.printf(" %d ", i);
        }
        System.out.println();
        list3 = List.of(7,8);
        list1.addAll(list3); //Se inserta list3 al final de list1
        for (Integer i : list1) { // 1 2 3 4 5 6 7 8
            System.out.printf(" %d ", i);
        }
    }
}
```

```

    }
    System.out.printf("\nEl elemento de la posición 3 es: %d",
list1.get(3));//4
    System.out.printf("\nEl elemento 4 se encuentra en la posición %d",
list1.indexOf(4));//3
    list1.add(4);//Se añade un 4 al final de list1
    System.out.printf("\nLa posición del 4 en list1 por el final es: %d\n",
list1.lastIndexOf(4));//8
    list1.remove(8);//Se elimina el elemento de la posición 8, que es el
último 4 insertado
    for (Integer i : list1) {// 1 2 3 4 5 6 7 8
        System.out.printf(" %d ", i);
    }
    list1.set(6, 8);//Se reemplaza el elemento que se encuentra en la
posición 6 por un 8
    System.out.println();
    for (Integer i : list1) {// 1 2 3 4 5 6 8 8
        System.out.printf(" %d ", i);
    }
    System.out.printf("\nLa sublista comprendida entre las posiciones 2 y 5
es: ");
    for (Integer i : list1.subList(2, 6)) {// 3 4 5 6
        System.out.printf(" %d ", i);
    }
    System.out.printf("\nEl 4 %s se encuentra en list1", list1.contains(4) ?
"sí" : "no");//sí
    System.out.printf("\nEl 9 %s se encuentra en list1", list1.contains(9) ?
"sí" : "no");//no

    //list1: 1 2 3 4 5 6 8 8
    //list2: 3 4
    //list3: 7 8
    System.out.printf("\nTodos los elementos de list2 %s se encuentran en
list1",
        list1.containsAll(list2) ? "sí" : "no");//sí
    System.out.printf("\nTodos los elementos de list3 %s se encuentran en
list1\n",
        list1.containsAll(list3) ? "sí" : "no");//no
    list1.removeAll(list3);//Se eliminan de list1 todos los elementos de
list3, es decir, el 8
    for (Integer i : list1) {// 1 2 3 4 5 6
        System.out.printf(" %d ", i);
    }
    System.out.println();
    list1.retainAll(list2);//Intersección entre list1 y list2
    for (Integer i : list1) {// 3 4
        System.out.printf(" %d ", i);
    }
    System.out.printf("\nEl tamaño de list1 es: %d", list1.size());//2
    System.out.printf("\nlist1 %s está vacía", list1.isEmpty() ? "sí" :
"no");//no
    list1.clear();//Elimina todos los elementos de list1
    System.out.printf("\nlist1 %s está vacía", list1.isEmpty() ? "sí" :
"no");//sí
    list4 = Arrays.asList(1, 2, 3, 4);//Devuelve una lista a partir de un
varargs

```

```

        System.out.print("\nlist4 creada con Arrays.asList: ");
        for (Integer i : list4) { // 1 2 3 4
            System.out.printf(" %d ", i);
        }

        public static void main(String[] args) {

            new ShowArrayList().show();

        }
    }
}

```

Salida por consola:

```

1 2 5 6
1 2 3 4 5 6
1 2 3 4 5 6 7 8
El elemento de la posición 3 es: 4
El elemento 4 se encuentra en la posición 3
La posición del 4 en list1 por el final es: 8
1 2 3 4 5 6 7 8
1 2 3 4 5 6 8 8
La sublista comprendida entre las posiciones 2 y 5 es: 3 4 5 6
El 4 sí se encuentra en list1
El 9 no se encuentra en list1
Todos los elementos de list2 sí se encuentran en list1
Todos los elementos de list3 no se encuentran en list1
1 2 3 4 5 6
3 4
El tamaño de list1 es: 2
list1 no está vacía
list1 sí está vacía
list4 creada con Arrays.asList: 1 2 3 4

```

Fijémonos en cómo se ha inicializado *list4*:

```
list4 = Arrays.asList(1, 2, 3, 4);
```

El método `Arrays.asList` proporciona una forma de crear una lista de tamaño fijo a partir del array especificado. Este método actúa como puente entre las APIs basadas en arrays y en colecciones. Además, en la interfaz `Collection` existe el método `toArray()` que devuelve un array con todos los elementos de la colección.

5. Invarianza, covarianza y contravarianza

Para poder entender bien los conceptos de **invarianza**, **covarianza** y **contravarianza** en los genéricos, empezaremos por estudiar dichos conceptos en los arrays.

En Java **los arrays son covariantes**, lo que básicamente significa que, dadas las reglas de subtipado de Java, un array de tipo `T[]` puede contener elementos de tipo `T` o cualquier subtipo de `T`. Así, el siguiente ejemplo es perfectamente válido:

```

Number[] numbers = new Number[3];
numbers[0] = new Integer(10);
numbers[1] = new Double(3.14);
numbers[2] = new Byte((byte) 0);
for (Number number: numbers) {
    System.out.println(number);
}

```

Pero no solo eso, las reglas de subtipado de Java también establecen que un array `S[]` es un subtipo del array `T[]` si `S` es un subtipo de `T`. Así el siguiente código es totalmente válido:

```

Integer[] ints = {1, 2, 3, 4};
Number[] numbers = ints;
for (Number number : numbers) {
    System.out.println(number);
}

```

porque según las reglas de subtipado de Java un array `Integer[]` es un subtipo de un array `Number[]`, dado que `Integer` es un subtipo de `Number`.

Pero esta regla de subtipado de arrays nos lleva a una pregunta interesante: si podemos hacer referencia a un `Integer[]` desde una variable `Number[]` ¿podremos almacenar en él algún número que no sea un entero? Hagamos la prueba en el ejemplo anterior:

```

Integer[] ints = {1, 2, 3, 4};
Number[] numbers = ints;
numbers[0] = 3.14; //Error de ejecución: ArrayStoreException

```

Esta última línea compilaría sin error, pero si ejecutamos este código, se produciría una excepción `ArrayStoreException` porque estamos tratando de almacenar un valor de tipo `Double` en un `Integer[]`. Si lo pensamos bien, el hecho de que estemos accediendo a `Integer[]` a través de una referencia `Number[]` es irrelevante aquí, lo que importa es que el array es un `Integer[]`.

Esto significa que, en cierta manera, podemos engañar al compilador, pero no podemos engañar al sistema de tipos en tiempo de ejecución. Y no podemos hacerlo porque los arrays son lo que se conoce como tipos **reifiable** (literalmente, que se convierte en una cosa concreta, que no es abstracto). Esto significa que en tiempo de ejecución Java sabe que este array fue instanciado como un `Integer[]` al que, en este caso, simplemente se está accediendo a él a través de una referencia de tipo `Number[]`. Así que, como podemos ver, una cosa es el tipo real del objeto, y otra cosa distinta es el tipo de referencia que usamos para acceder a él.

Sin embargo, los genéricos de Java representan un gran reto en este sentido. Cuando se diseñaron para Java 5, se tuvo que hacer teniendo en cuenta que las colecciones de datos, como por ejemplo `List` (sin tipo) ya existían en Java, y que por tanto ya existía gran cantidad de código que hacía uso de listas de elementos de tipo `Object`. El problema es que no se podía permitir que al introducir los genéricos en las colecciones (como en `List<T>`) todo el código existente que usara colecciones dejara de compilar, es decir, no fuera compatible. Por este motivo, se tomó la decisión de que la información sobre el tipo que se le suministra a una clase genérica cuando se construye una instancia de ella solo se usara en tiempo de compilación para indicar cómo se debe crear el objeto correctamente, pero NO estuviera disponible en tiempo de ejecución. A este

proceso de le denomina borrado de tipos, en inglés **type erasure**, e implica básicamente que en ejecución una `List<Integer>` ya simplemente se ve como una `List`.

Y el problema asociado al borrado de tipos en los genéricos es que como en tiempo de ejecución no hay información del tipo con el que se creó la lista, no habría forma de asegurar, en tiempo de ejecución, que el valor que estamos intentando almacenar en una lista es del tipo con el que se creó. Los tipos genéricos son no verificables, ya que en tiempo de ejecución no podemos determinar la verdadera naturaleza del tipo genérico. Así, teóricamente, no habría manera de lanzar una excepción en tiempo de ejecución en el siguiente ejemplo, a diferencia de como vimos antes usando arrays:

```
List<Integer> ints = new ArrayList<>();
ints.add(1);
ints.add(2);
List<Number> numbers = ints;
numbers.add(3.14); /* Esta línea NO puede producir un error en tiempo de ejecución
porque la información sobre el tipo con el que se creó la lista numbers ha sido
eliminada. */
```

En tiempo de ejecución el sistema no tiene manera de evitarnos ejecutar la última línea, dado que no puede determinar que la lista referenciada por `numbers` en realidad es una lista de enteros, por lo que podríamos insertar cualquier objeto.

La solución a este grave problema es usar la información que disponemos en tiempo de compilación, de manera que el compilador nos impida hacer referencia a una lista de enteros desde una variable de tipo lista de números. Es decir, que la solución de Java es producir un error de compilación al intentar hacer la asignación:

```
List<Integer> ints = new ArrayList<>();
ints.add(1);
ints.add(2);
List<Number> numbers = ints; //Error de compilación.
numbers.add(3.14);
```

De esta manera, si no podemos engañar al compilador, tampoco podremos engañar al sistema en tiempo de ejecución, ya que no podremos llegar a ejecutar nuestro código.

¿Y esta solución qué implicación tiene? Que a diferencia de lo que sucede con los arrays, **en Java las clases con tipos genéricos son, por defecto, invariantes**, por lo que, por ejemplo, si el tipo `U` es un subtipo del tipo `T`, `List<U>` NO es un subtipo de `List<T>`, como ha quedado demostrado en el código anterior al producir un error de compilación.

Evidentemente esta característica de los tipos genéricos en Java tendría un impacto muy negativo en el polimorfismo. Consideremos el método que calcula el sumatorio de los elementos contenidos en el array:

```
long sum(Number[] numbers) {
    long summation = 0;
    for(Number number : numbers) {
        summation += number.longValue();
    }
    return summation;
}
```

Podremos llamar a dicho método de las siguientes maneras sin que se produzca ningún error de compilación ni en tiempo de ejecución:

```
Integer[] myInts = {1,2,3,4,5};
Long[] myLongs = {1L, 2L, 3L, 4L, 5L};
Double[] myDoubles = {1.0, 2.0, 3.0, 4.0, 5.0};
System.out.println(sum(myInts));
System.out.println(sum(myLongs));
System.out.println(sum(myDoubles));
```

Sin embargo, si intentamos implementar el mismo método pero para que reciba una lista en vez de un array:

```
long sum(List<Number> numbers) {
    long summation = 0;
    for(Number number : numbers) {
        summation += number.longValue();
    }
    return summation;
}
```

Al tratar de usarlo podemos obtener errores en tiempo de compilación, como en los siguientes casos:

```
List<Integer> myInts = List.of(1,2,3,4,5);
List<Long> myLongs = List.of(1L, 2L, 3L, 4L, 5L);
List<Double> myDoubles = List.of(1.0, 2.0, 3.0, 4.0, 5.0);
System.out.println(sum(myInts)); //Error de compilación
System.out.println(sum(myLongs)); //Error de compilación
System.out.println(sum(myDoubles)); //Error de compilación
```

¿Por qué se producen dichos errores de compilación? Porque, como acabamos de decir, los tipos genéricos NO son covariantes por defecto, sino invariantes, por lo que `List<Integer>` NO es un subtipo de `List<Number>` aunque `Integer` sea un subtipo de `Number`, y lo mismo con los casos de `Long` y `Double` del código anterior.

Evidentemente, desde el punto de vista del polimorfismo esto es inaceptable. Para solucionarlo, Java introdujo una nueva sintaxis para definir tipos parametrizados conocida como tipos comodín `?` (*bounded wildcard type*), que permite al desarrollador definir explícitamente la covarianza o la contravarianza de un genérico.

Empecemos con la **covarianza**. Así, en vez de usar un tipo `T` como argumento de tipo de una estructura genérica, podemos usar `<? extends T>` para indicar que podremos leer elementos de tipo `T` o de cualquier subclase de `T`, pero que no podremos insertar en la estructura ningún valor. Además, `T` puede corresponder a una interfaz y no a una clase.

Por ejemplo, este código es perfectamente válido, dado que solo leemos desde la referencia `numbers`:

```
List<Integer> ints = new ArrayList<>();
ints.add(1);
ints.add(2);
List<? extends Number> numbers = ints;
for (Number number : numbers) {
    System.out.println(number);
}
```

La lectura está permitida porque en la definición de la lista hemos asegurado que el tipo de la lista real que referencia `numbers` debe ser forzosamente de alguna clase que extiende de `Number`, por lo que al acceder a un elemento de la lista internamente se puede hacer un casting al tipo `Number` con la seguridad de que la conversión no fallará.

Sin embargo, no nos está permitido escribir nada a través de dicha referencia:

```
List<Integer> ints = new ArrayList<>();
ints.add(1);
ints.add(2);
List<? extends Number> numbers = ints;
for (Number number : numbers) {
    System.out.println(number);
}
ints.add(25);
numbers.add(17); //Error de compilación, no se permite escribir a través de una
referencia covariante
```

Y no se nos permite escribir porque entonces podríamos almacenar en la lista de enteros reverenciada por `numbers` cualquier valor que no fuera un entero.

Así, el método de un ejemplo anterior que sumaba todos los elementos de una lista quedaría de la siguiente manera:

```
long sum(List<? extends Number> numbers) {
    long summation = 0;
    for (Number number : numbers) {
        summation += number.longValue();
    }
    return summation;
}
```

y podríamos llamarla de las siguientes maneras sin que se produjera un error de compilación:

```
List<Integer> myInts = List.of(1,2,3,4,5);
List<Long> myLongs = List.of(1L, 2L, 3L, 4L, 5L);
List<Double> myDoubles = List.of(1.0, 2.0, 3.0, 4.0, 5.0);
System.out.println(sum(myInts));
System.out.println(sum(myLongs));
System.out.println(sum(myDoubles));
```

Veamos ahora la **contravarianza**. La contravarianza es el caso contrario a la covarianza, se utiliza para escribir elementos en una estructura genérica. Para definir la contravarianza usaremos `<? super T>` como indicador del tipo. Esto quiere decir que el tipo real es `T` o cualquier tipo que sea una superclase de `T` (en último caso `Object`). Así, el siguiente código es válido:

```
List<Integer> ints = new ArrayList<>();
ints.add(1);
ints.add(2);
List<? super Integer> numbers = ints;
numbers.add(25);
numbers.add(25.4); //Error de compilación
```

La lectura hacia el tipo T no es segura, porque no tenemos forma de saber exactamente el tipo del objeto que se pretende leer a través de la referencia, ya que puede ser de cualquier clase T o superclase de T.

```
List<Integer> ints = new ArrayList<>();
ints.add(1);
ints.add(2);
List<? super Integer> numbers = ints;
numbers.add(25);
numbers.add(25.4); //Error de compilación
Integer valor1 = ints.get(0);
Object valor2 = numbers.get(0); //Se puede asegurar que el primer valor de la
lista es un Object ya que es superclase de todos los tipos
Integer valor3 = numbers.get(0); //Error de compilación: no se puede asegurar que
el primer elemento de la lista sea un entero
```

Como podemos ver, si el compilador nos permitiera compilar esta última línea, podríamos obtener un `ClassCastException` en tiempo de ejecución al no poder asegurar en tiempo de ejecución el tipo del primer elemento de la lista. Así que, una vez más, el compilador no corre el riesgo de permitir esta operación insegura y la rechaza inmediatamente produciendo un error de compilación.

Así, a modo de resumen, podríamos decir que para los parámetros de un método se recomienda usar siempre la covarianza cuando solo pretendemos sacar valores genéricos de una estructura, la contravarianza cuando solo pretendemos poner valores genéricos en una estructura, y solo usar la invarianza (no usar el carácter comodín) cuando pretendemos hacer ambas cosas. Si seguimos esta regla aumentaremos la flexibilidad de nuestro código.

Sin embargo, no se recomienda usar el carácter comodín para los tipos de retorno de los métodos, ya que estaremos obligando al código cliente a usar tipo comodín.

Veamos un ejemplo en que se usa tanto covarianza como contravarianza. Consiste en un método que permite copiar los elementos de una lista de números, llamada *origen*, en otra lista llamada *destino*, y ambas listas son recibidas como parámetro. En este caso, para la lista *origen* usaremos covarianza, ya que solo vamos a leer de ella, y para la lista *destino* usaremos contravarianza, ya que solo vamos a escribir en ella:

```
public void copy(List<? extends Number> source, List<? super Number> target) {
    for(Number number : source) {
        target.add(number);
    }
}
```

De esta manera podremos realizar sin error las siguientes llamadas:


```
List<Integer> myInts = List.of(1, 2, 3, 4);
List<Double> myDoubles = List.of(3.14, 6.28);
List<Number> myObjs = new ArrayList<>();
copy(myInts, myObjs);
copy(myDoubles, myObjs);
```

6. Iteradores

En diseño de software, el patrón de diseño *Iterador* (en inglés, *Iterator*) define una interfaz que declara los métodos necesarios para acceder secuencialmente a un grupo de objetos de una colección.

Este patrón debe ser utilizado cuando se requiera una forma estándar de recorrer una colección, es decir, cuando no sea necesario que un cliente sepa el tipo de colección que está recorriendo.

La interfaz `Iterable<T>` contine el método `iterator()` que devuelve una instancia de alguna clase que implemente la interfaz `Iterator<T>`:

- `Iterator<T> iterator()` : devuelve un iterador al comienzo de la colección.

La interfaz `Iterator<E>` permite el acceso secuencial a los elementos de una colección y realizar recorridos sobre la colección. Los métodos de `Iterator<E>` son:

- `boolean hasNext()` : comprueba si hay siguiente elemento.
- `E next()` : devuelve el siguiente elemento y mueve el iterador.
- `void remove()` : se invoca después de `next()` para eliminar el último elemento leído.

La interfaz `Collection<E>` es una subinterfaz o interfaz hija de `Iterable<E>`. La clase `ArrayList<E>` implementa `Collection<E>` y por lo tanto también `Iterable<E>`, así que dispone del método `iterator()`. Veamos un ejemplo de recorrido de un `ArrayList<E>` utilizando iteradores:

```
package tema11_Colecciones;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class ShowIterator {

    public void show() {

        Iterator<String> it;
        List<String> list = new ArrayList<>();

        list.add("Juan");
        list.add("Antonio");
        list.add("Jaime");
        list.add("Vicente");

        it = list.iterator();
        while (it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
```

```

    }

    public static void main(String[] args) {

        new ShowIterator().show();

    }

}

```

Salida por consola:

```

Juan
Antonio
Jaime
Vicente

```

Si no hay siguiente, `next()` lanza en ejecución una excepción `NoSuchElementException`:

```

package tema11_Colecciones;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class NextException {

    public void show() {

        Iterator<String> it;
        List<String> list = new ArrayList<>();

        list.add("Juan");
        it = list.iterator();
        System.out.println(it.next());
        System.out.println(it.next()); //Se lanza en ejecución una excepción
        NoSuchElementException

    }

    public static void main(String[] args) {

        new NextException().show();

    }

}

```

Tal y como vemos en el ejemplo anterior, hay que comprobar si hay siguiente con un `hasNext()` para que el `next()` no lance la excepción.

No se puede modificar la colección dentro del bucle for-each porque se lanza en ejecución la excepción `ConcurrentModificationException`, ya que estamos recorriendo y modificando la lista a la vez:

```
package tem11_colecciones;

import java.util.ArrayList;
import java.util.List;

public class ModifyInsideForEach {

    public void show() {

        List<String> list = new ArrayList<>();

        list.add("Juan");
        list.add("Antonio");
        list.add("Jaime");
        list.add("Vicente");
        for (String s : list) { //Se lanza en ejecución la excepción
            ConcurrentModificationException
                System.out.printf(" %s ", s);
                if (s.equals("Antonio")) {
                    list.remove("Antonio");
                }
            }
        }

    public static void main(String[] args) {

        new ModifyInsideForEach().show();

    }

}
```

Para solucionarlo, podemos utilizar el método `remove()` de `Iterator<E>`. Si se modifica una colección mientras se recorre, los iteradores quedan invalidados, a excepción del método `remove()` de la interfaz `Iterator<E>`. El método `remove()` permite eliminar elementos de la colección siendo la única forma adecuada para eliminar elementos durante la iteración:

```
package tem11_colecciones;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class RemoveIterator {

    public void show() {

        String s;
        Iterator<String> it;
```

```

        List<String> list = new ArrayList<>();

        list.add("Juan");
        list.add("Antonio");
        list.add("Jaime");
        list.add("Vicente");

        it = list.iterator();
        while (it.hasNext()) {
            s = it.next();
            if (s.equals("Antonio")) {
                it.remove();
            } else {
                System.out.println(s);
            }
        }
    }

    public static void main(String[] args) {

        new RemoveIterator().show();

    }
}

```

Salida por consola:

```

Juan
Jaime
Vicente

```

Solo puede haber una invocación a `remove()` por cada invocación a `next()`. Si no se cumple, se lanza en ejecución una excepción `IllegalStateException`. Por ejemplo, imaginemos que tenemos una lista de personas y queremos eliminar a *Antonio* y a la persona que venga detrás. Si cuando encontramos a *Antonio*, hacemos dos `remove()` seguidos, entonces salta la excepción:

```

package tem11_Colecciones;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class RemoveNext1 {

    public void show() {

        String s;
        Iterator<String> it;
        List<String> list = new ArrayList<>();

        list.add("Juan");
        list.add("Antonio");
    }
}

```

```

        list.add("Jaime");
        list.add("Vicente");

        it = list.iterator();
        while (it.hasNext()) {
            s = it.next();
            if (s.equals("Antonio")) {
                it.remove();
                it.remove();//Se lanza en ejecución la excepción
IllegalStateException
            } else {
                System.out.println(s);
            }
        }

    }

    public static void main(String[] args) {

        new RemoveNext1().show();

    }

}

```

Tendríamos que hacer otro `next()` para el que venga detrás de *Antonio*:

```

package tema11_Colecciones;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class RemoveNext2 {

    public void show() {

        String s;
        Iterator<String> it;
        List<String> list = new ArrayList<>();

        list.add("Juan");
        list.add("Antonio");
        list.add("Jaime");
        list.add("Vicente");

        it = list.iterator();
        while (it.hasNext()) {
            s = it.next();
            if (s.equals("Antonio")) {
                it.remove();
                if(it.hasNext()) {
                    it.next();
                    it.remove();
                }
            }
        }
    }
}

```

```

        } else {
            System.out.println(s);
        }
    }

}

public static void main(String[] args) {

    new RemoveNext2().show();

}

}

```

Salida por consola:

```

Juan
Vicente

```

La interfaz `ListIterator<E>` es una subinterfaz o interfaz hija de `Iterator<E>`. Es un iterador para listas que permite al programador recorrer la lista hacia adelante y hacia atrás, modificar la lista durante la iteración y obtener la posición actual del iterador en la lista.

Hereda los métodos de `Iterator<E>` y además aporta otros métodos nuevos:

- `void add(E e)`: inserta el elemento en la lista antes del elemento que sería devuelto por `next()`, si lo hubiera, y después del elemento que sería devuelto por `previous()`, si lo hubiera. Una llamada posterior a `next()` no se vería afectada y una llamada posterior a `previous()` devolvería el nuevo elemento.
- `boolean hasPrevious()`: comprueba si hay un elemento anterior.
- `int nextIndex()`: devuelve el índice del elemento que sería devuelto por una llamada a `next()`. El índice del primer elemento es 0.
- `E previous()`: devuelve el elemento anterior de la lista y mueve la posición del cursor hacia atrás.
- `int previousIndex()`: devuelve el índice del elemento que sería devuelto por una llamada a `previous()`.
- `void set(E e)`: sustituye el último elemento devuelto por `next()` o `previous()` por el elemento `e`.

En la interfaz `List<E>`, hay dos métodos para crear este iterador:

- `ListIterator<E> listIterator()`: se coloca antes del primer elemento para que al hacer el primer `next()` se devuelva el primer elemento.
- `ListIterator<E> listIterator(int index)`: se coloca antes del elemento que se encuentra en la posición `index` para que al hacer un `next()` se devuelva dicho elemento. Para recorrer la lista al revés, hay que crearlo con el tamaño de la lista para que el primer `previous()` devuelva el último.

```

package tem11_colecciones;

```

```

import java.util.ArrayList;
import java.util.List;
import java.util.ListIterator;

public class ShowListIterator {

    public void show() {

        ListIterator<Integer> it;
        List<Integer> list = new ArrayList<>();

        list.add(1);
        list.add(2);
        list.add(4);
        list.add(6);

        //Recorrido de la lista hacia atrás:
        it = list.listIterator(list.size());
        while (it.hasPrevious()) {
            System.out.printf(" %d ", it.previous()); // 6 4 2 1
        }

        it = list.listIterator(1); //Se coloca antes del elemento que se encuentra
        en la posición 1
        System.out.printf("\n %d ", it.next()); //2
        System.out.printf("\n %d ", it.nextIndex()); //2
        System.out.printf("\n %d ", it.previousIndex()); //1
        it.add(3); //Una llamada posterior a next() no se ve afectada
        System.out.printf("\n %d ", it.next()); //4
        it.add(5);
        System.out.printf("\n %d ", it.previous()); //5 Una llamada posterior a
        previous() devuelve el nuevo elemento
        System.out.printf("\n %d \n", it.next()); //5
        it.set(7); //Sustituye el último elemento devuelto por next() por 7
        for (Integer i : list) {
            System.out.printf(" %d ", i); //1 2 3 4 7 6
        }

    }

    public static void main(String[] args) {

        new ShowListIterator().show();

    }

}

```

7. Métodos equals y hashCode

Vimos en el [Tema 7 Herencia 6 El método toString](#) que la clase *Object* proporciona un cierto número de métodos de utilidad general que pueden utilizar todos los objetos ya que los heredan. Pero normalmente hay que sobrescribirlos para que funcionen adecuadamente adaptándolos a la clase correspondiente. Esto se hace con la idea de que todas las clases utilicen el mismo nombre y prototipo de método para hacer operaciones comunes. Como por ejemplo, `equals()` que

comprueba si dos elementos son iguales y `hashCode()` que devuelve un número entero que identifica al objeto cuando se guarda en algunas estructuras de datos.

Sin embargo, debemos tener en cuenta que al sobrescribir todos estos métodos debemos seguir cumpliendo con el comportamiento que se espera de ellos, ya que son usados internamente por muchas clases del propio lenguaje. De lo contrario, las clases que dependen de ello, como `HashMap` y `HashSet` dejarían de funcionar correctamente.

En la implementación de `equals()` de la clase `Object`, dos instancias de clase son iguales si están situadas en la misma posición de memoria.

Debemos tener en cuenta que cuando un programador usa el método `equals()` sobre un objeto pasándole como argumento otro objeto, lo que pretende es descubrir si ambos objetos son equivalentes lógicamente (representan el mismo "valor"), no si están almacenados en la misma posición de memoria.

No sobrescribir el método `equals()` en una determinada clase es la opción recomendada en los siguientes casos:

- Cuando cada instancia de la clase es intrínsecamente única, lo cual es cierto para clases como `Scanner` que representan entidades activas en lugar de valores.
- Cuando se considera que no hay necesidad de que la clase provea una prueba de "equivalencia lógica".
- Cuando una superclase de la clase ya lo ha sobrescrito y el comportamiento de la superclase es apropiado para las subclases.
- Cuando la clase es privada o friendly, y estamos completamente seguros de que su método `equals()` nunca será invocado, ni explícita ni implícitamente.
- Cuando la clase controla las instancias que pueden ser creadas para asegurarse que no pueden existir dos objetos que representen el mismo "valor", como por ejemplo con las clases `enum` o las clases que siguen el patrón de diseño *singleton*. Para estas clases, la equivalencia lógica es igual que la igualdad de identidad, ya que no puede existir más de una instancia para un "valor". Dado que esta es la comprobación que realiza la implementación de `equals()` en la clase `Object`, no tiene sentido sobrescribirlo.

Si nuestra clase no se encuentra en ninguno de los casos anteriores, es muy recomendable que sobrescribamos el método `equals()`. Un ejemplo muy característico es cuando la clase corresponda a una entidad que represente un valor (*value class*).

Como hemos comentado, cuando sobrescribamos el método `equals()` debemos seguir cumpliendo el comportamiento que el sistema espera de él, que incluye las siguientes propiedades:

- Reflexiva: para todo objeto *x* distinto de *null* se debe cumplir que `x.equals(x)` sea *true*.
- Simétrica: para todo par de objetos *x* e *y* distintos de *null* se debe cumplir que `x.equals(y)` solo debe retornar *true* si `y.equals(x)` retorna *true*.
- Transitiva: para todo trío de objetos *x*, *y*, *z* distintos de *null* se debe cumplir que si `x.equals(y)` retorna *true* y `y.equals(z)` retorna *true* entonces `x.equals(z)` debe retornar *true*.
- Consistente: para todo par de objetos *x* e *y* distintos de *null* se debe cumplir que `x.equals(y)` siempre retorne el mismo valor si no hemos cambiado los atributos que se usan para comparar en alguno de los dos objetos.

- Para todo objeto *x* distinto de *null* se debe cumplir que `x.equals(null)` debe retornar *false*.

Así, para sobrescribir el método `equals()` cumpliendo con las propiedades anteriores se recomienda seguir los siguientes pasos:

1. Usar el operador `==` para comprobar si el argumento corresponde a otra referencia al mismo objeto, en cuyo caso retornar *true*.
2. Usar el operador `instanceof` para comprobar si el objeto recibido como argumento no es de la misma clase, en cuyo caso retornar *false*. También nos sirve para comprobar si dicho argumento es *null*, ya que en este caso *instanceof* retorna *false*.
3. Hacer casting del objeto recibido como argumento convirtiéndolo a la clase correspondiente. Dado que hemos hecho antes *instanceof*, el casting siempre tendrá éxito.
4. Para cada atributo significativo de la clase, comprobar que dicho atributo en el objeto argumento es equivalente al atributo en el objeto *this*. Si no tenemos éxito en alguno de ellos, retornar *false*. Si todas las comprobaciones se han hecho con éxito, retornar *true*. Para realizar la comprobación de cada atributo usar:
 - El operador `==` para valores primitivos que no sean *float* ni *double*.
 - Para valores *float* usar `Float.compare(value1, value2)` y para valores *double* usar `Double.compare(value1, value2)`.
 - Para valores correspondientes a objetos llamar a `equals()` recursivamente. Si es válido que dichos objetos contengan *null*, entonces debemos usar `Objects.equals(object1, object2)` para que no se produzca la excepción `NullPointerException`.
 - Para los valores correspondientes a arrays, compara uno a uno los elementos significativos del array. Si todos los elementos son significativos, usa alguna de las versiones del método estático `Arrays.equals()`.

Veamos un ejemplo:

```
public final class PhoneNumber {  
  
    private final short areaCode, prefix, lineNum;  
  
    @Override  
    public boolean equals(Object o) {  
        PhoneNumber pn;  
        if (o == this) return true;  
        if (!(o instanceof PhoneNumber)) return false;  
        pn = (PhoneNumber) o;  
        return pn.lineNum == lineNum && pn.prefix == prefix  
            && pn.areaCode == areaCode;  
    }  
  
    // ...  
}
```

Una aspecto muy importante es que no debemos cambiar el tipo del objeto recibido como argumento, que siempre debe ser `Object`, o no estaremos sobrescribiendo el método `equals()`, sino sobrecargándolo, lo que puede producir falsos positivos. El compilador no se quejará si no usamos la anotación `@Override` (por eso siempre se recomienda usarla). Por ejemplo, nunca hagas esto:

```
// NUNCA HAGAS ESTO. EL TIPO DE o DEBE SER Object
public boolean equals(MyClass o) {
    // ...
}
```

Escribir a mano los métodos `equals()` y `hashCode()` es bastante tedioso. Para facilitarnos esta tarea tenemos librerías como la librería `AutoValue` de Google que lo genera automáticamente para nosotros con tan solo usar una determinada anotación. Otra opción es dejar que el IDE nos genere dichos métodos, aunque esto tiene el inconveniente de que no se generan automáticamente de nuevo conforme añadimos atributos a nuestra clase, por lo que debemos tener cuidado, algo que sí hace `AutoValue`. En todo caso, es mejor usar el IDE que hacerlo nosotros a mano, ya que el humano es más propenso a los errores. En el Eclipse se encuentra en Menú *Source* → *Generate hashCode() and equals()*.

Un detalle muy importante que no debemos olvidar es que **si en una clase sobrescribimos el método `equals()` debemos obligatoriamente sobrescribir también el método `hashCode()`** o de lo contrario no se estará cumpliendo con el comportamiento esperado de este último, lo que impedirá que los objetos de dichas clases funcionen correctamente en colecciones como `HashMap` y `HashSet`.

La implementación por defecto de `hashCode()` de la clase `Object` devuelve una representación numérica de la dirección de memoria en la que se encuentra ubicado el objeto. Tenemos que anularlo si lo que queremos es que `hashCode()` devuelva un número entero que identifique al objeto cuando se guarde en algunas estructuras de datos.

El comportamiento que se espera de `hashCode()` es el siguiente:

- Debe ser consistente, es decir, que repetidas llamadas al método `hashCode()` deben retornar el mismo valor, siempre y cuando no se haya modificado ninguno de los atributos usados para las comparaciones.
- Si dos objetos son equivalentes, es decir, si `x.equals(y)` retorna *true*, entonces `x.hashCode()` e `y.hashCode()` deben retornar el mismo valor entero.
- Si dos objetos no son equivalentes, es decir si `x.equals(y)` retorna *false*, no es estrictamente necesario, aunque sí recomendable, que `x.hashCode()` e `y.hashCode()` retornen valores diferentes, de manera que se mejore el rendimiento de las tablas *hash*. El algoritmo de la función *hash* debe distribuir una colección de instancias de un tamaño considerable de forma uniforme entre todos los valores enteros.

La implementación característica al sobrescribir el método `hashCode()` en la clase `PhoneNumber` es la siguiente, usando los atributos *areaCode*, *prefix* y *lineNum*:

```
public final class PhoneNumber {

    private final short areaCode, prefix, lineNum;

    @Override
    public boolean equals(Object o) {
        PhoneNumber pn;
        if (o == this){
            return true;
        }
        if (!(o instanceof PhoneNumber)){
```

```

        return false;
    }
    pn = (PhoneNumber) o;
    return pn.lineNum == lineNum && pn.prefix == prefix
        && pn.areaCode == areaCode;
}

@Override
public int hashCode() {
    int result = Short.hashCode(areaCode);
    result = 31 * result + Short.hashCode(prefix);
    result = 31 * result + Short.hashCode(lineNum);
    return result;
}

// ...
}

```

Además de escribir nosotros a mano el código del método `hashCode()`, podemos usar la implementación proporcionada por algunas librerías, como Guava o AutoValue. Adicionalmente los IDEs nos proporcionan asistentes para la generación de este método, seleccionando los atributos que queremos usar. En el Eclipse se encuentra en Menú *Source* → *Generate hashCode() and equals()*.

Si una clase es inmutable y el coste de calcular el valor *hash* es significativo, podría considerar almacenar cacheado el código *hash* en el propio objeto, en lugar de recalcularlo cada vez que se solicite. Si cree que la mayoría de los objetos de esta clase se usarán como claves *hash*, entonces debería calcular el código *hash* cuando se cree la instancia. De lo contrario, podría elegir calcular perezosamente el código *hash* la primera vez que se invoque al método `hash()`.

Dos consideraciones finales: **no excluir atributos significativos del cálculo del valor hash**, así se logrará un mejor rendimiento. En segundo lugar, **no proporcionar a los clientes de la clase demasiada información acerca de cómo se calcula el valor hash**, de esta manera el código cliente no podrá depender de cómo se calcula, permitiéndonos modificar la implementación del método en el futuro sin afectar a los clientes.

8. Colecciones sin duplicados

La interfaz `Set`, que hereda de *Collection*, permite implementar listas de elementos sin duplicados, es decir, modela la abstracción matemática de los conjuntos.

Existen varios tipos de implementaciones realizadas dentro de la plataforma Java:

- **HashSet**: esta implementación almacena los elementos en una tabla *hash*. Es la implementación con mejor rendimiento de todas pero no garantiza ningún orden a la hora de realizar iteraciones.
- **LinkedHashSet**: esta implementación almacena los elementos en función del orden de inserción. Es un poco más costosa que *HashSet*.
- **TreeSet**: esta implementación utiliza una estructura de árbol para ordenar los elementos. Es bastante más lenta que *HashSet*. La veremos más adelante en el apartado de los árboles.

8.1 Clase HashSet

Implementa la interfaz anterior. Es la clase más utilizada para implementar listas sin duplicados. Esta clase permite el elemento nulo. No garantiza ningún orden a la hora de realizar iteraciones.

Utiliza internamente una tabla de tipo *hash*:

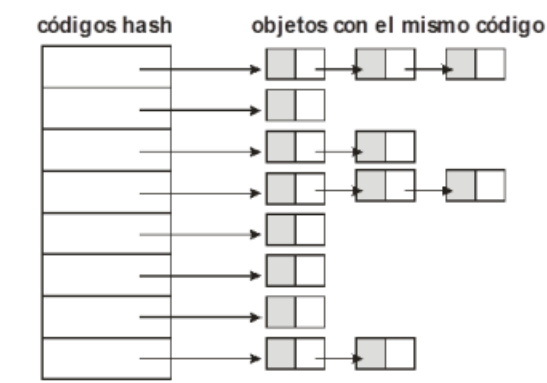
Cajón 0	Cajón 1	Cajón 2	Cajón 3	Cajón 4	Cajón 5	Cajón 6	Cajón ...	Cajón N
------------	------------	------------	------------	------------	------------	------------	--------------	------------

Hashtable

Al querer guardar un objeto en esta estructura, se llama al método `hashCode()` el cual devuelve un número entero que la estructura usará para decidir en qué cajón guardará este dato. Para recuperar el objeto se llama al método `hashCode()` para determinar de qué cajón debo recuperar el objeto. El objetivo de guardar los datos de esta forma y de llamar al método es lograr almacenar y recuperar información en tiempo constante (lo cual no ocurre siempre, pero se acerca). El que no suceda esto depende, casi siempre, del valor que devuelva el método `hashCode()` para cada objeto.

Supongamos que guardamos 3 objetos en esta estructura y el método `hashCode()` de los 3 devuelve 0, esto quiere decir que los 3 objetos se guardarán en el cajón 0. Cuando se necesite recuperar un objeto, hay que recorrer los objetos del cajón 0 para determinar cuál es el que se quiere recuperar. Por lo tanto, este método `hashCode()` no es útil ya que lo que se pretende al guardar los elementos es que éstos queden dispersos de forma uniforme en toda la estructura quedando la menor cantidad de cajones vacíos y que no haya cajones donde se guarden muchos más elementos que en otros.

Si dos objetos tienen el mismo `hashCode()`, ambos objetos se guardarán en el mismo cajón. La estructura usa entonces el método `equals()` dentro de ese cajón para determinar cuál corresponde con el solicitado y para eso depende de que el programador haya sobrescrito el método, de lo contrario no garantiza un resultado correcto.



Los objetos *HashSet* se construyen con un tamaño inicial de tabla (el tamaño del array) y un factor de carga que indica cuándo se debe redimensionar el array. Es decir, si se creó un array de 100 elementos y la carga se estableció al 80%, cuando se hayan rellenado 80 valores, se redimensiona el array. Por defecto, el tamaño del array se toma con 16 y el factor de carga con 0,75 (75%). No obstante, se puede construir una lista *HashSet* indicando ambos parámetros.

Esta implementación proporciona tiempos constantes en las operaciones básicas siempre y cuando la función *hash* disperse de forma correcta los elementos dentro de la tabla *hash*. Es importante definir el tamaño inicial de la tabla ya que este tamaño marcará el rendimiento de esta implementación.

Veamos un ejemplo de `HashSet` con la clase `Vehicle`. Los atributos significativos a tener en cuenta para el `equals()` y el `hashCode()` son *wheelCount* y *colour*. La velocidad (*speed*) no se incluye ya que si comparamos el mismo coche pero con velocidades distintas, en realidad, no deja de ser el mismo coche. El `equals()` y el `hashCode()` están generados con el Eclipse:

```
package temall_Colecciones.sinDuplicados;

public class Vehicle {

    private int wheelCount;
    private double speed;
    private String colour;

    public Vehicle(int wheelCount, String colour) {
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }

    @Override
    public String toString() {
        return String.format("Número de ruedas: %d, color: %s, velocidad: %.2f",
            wheelCount, colour, speed);
    }

    @Override
```

```

public int hashCode() {
    final int prime = 31;
    int result = 1;
    result = prime * result + ((colour == null) ? 0 : colour.hashCode());
    result = prime * result + wheelCount;
    return result;
}

@Override
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if (obj == null)
        return false;
    if (!(obj instanceof Vehicle))
        return false;
    Vehicle other = (Vehicle) obj;
    if (colour == null) {
        if (other.colour != null)
            return false;
    } else if (!colour.equals(other.colour))
        return false;
    if (wheelCount != other.wheelCount)
        return false;
    return true;
}
}

```

```

package temall_Colecciones.sinDuplicados;

import java.util.HashSet;
import java.util.Set;

public class ShowHashSet {

    public void show() {

        Set<Vehicle> set = new HashSet<>();
        set.add(new Vehicle(4, "azul"));
        set.add(new Vehicle(2, "rojo"));
        set.add(new Vehicle(4, "azul"));
        set.add(new Vehicle(2, "rojo"));
        set.add(new Vehicle(4, "verde"));
        for (Vehicle v : set) {
            System.out.println(v); //Se llama al toString del objeto
        }

    }

    public static void main(String[] args) {

        new ShowHashSet().show();

    }
}

```

```
}
```

Salida por consola:

```
Número de ruedas: 4, color: verde, velocidad: 0,00  
Número de ruedas: 4, color: azul, velocidad: 0,00  
Número de ruedas: 2, color: rojo, velocidad: 0,00
```

8.2 Clase LinkedHashSet

Almacena los elementos en función del orden de inserción. Es un poco más costosa que *HashSet*.

```
package tema11_Colecciones.sinDuplicados;  
  
import java.util.LinkedHashSet;  
import java.util.Set;  
  
public class ShowLinkedHashSet {  
  
    public void show() {  
  
        Set<Vehicle> set = new LinkedHashSet<>();  
        set.add(new Vehicle(4, "azul"));  
        set.add(new Vehicle(2, "rojo"));  
        set.add(new Vehicle(4, "azul"));  
        set.add(new Vehicle(2, "rojo"));  
        set.add(new Vehicle(4, "verde"));  
        for (Vehicle v : set) {  
            System.out.println(v); //Se llama al toString del objeto  
        }  
  
    }  
  
    public static void main(String[] args) {  
  
        new ShowLinkedHashSet().show();  
  
    }  
  
}
```

Salida por consola:

```
Número de ruedas: 4, color: azul, velocidad: 0,00  
Número de ruedas: 2, color: rojo, velocidad: 0,00  
Número de ruedas: 4, color: verde, velocidad: 0,00
```

8.3 Clase EnumSet

Es una implementación de conjuntos de alto rendimiento de tipos enumerados. Requiere que las constantes de enumeración colocadas en él pertenezcan al mismo tipo de enumeración. Veamos algunos de sus métodos:

- `static <E extends Enum<E>> EnumSet<E> allOf(Class<E> elementType)`: crea un conjunto de enumeraciones que contiene todos los valores del tipo de enumeración especificado.
- `static <E extends Enum<E>> EnumSet<E> complementOf(EnumSet<E> s)`: crea un conjunto de enumeraciones con el mismo tipo de elemento que el conjunto de enumeraciones especificado, conteniendo inicialmente todos los elementos de este tipo que no están contenidos en el conjunto especificado.
- `static <E extends Enum<E>> EnumSet<E> copyOf(Collection<E> c)`: crea un conjunto de enumeraciones a partir de una colección.
- `static <E extends Enum<E>> EnumSet<E> copyOf(EnumSet<E> s)`: crea un conjunto de enumeraciones a partir de otro.
- `static <E extends Enum<E>> EnumSet<E> noneOf(Class<E> elementType)`: crea un conjunto de enumeraciones vacío con el tipo de elemento especificado.
- `static <E extends Enum<E>> EnumSet<E> of(E e)`: crea un conjunto de enumeraciones que contiene el elemento especificado. Este método tiene varias sobrecargas para admitir más elementos.
- `static <E extends Enum<E>> EnumSet<E> range(E from, E to)`: crea un conjunto de enumeraciones que contiene inicialmente todos los elementos del rango definido por los dos elementos especificados.

```
package tem11_Colecciones.enumSet;

public enum Operation {

    PLUS('+'), MINUS('-'), TIMES('*'), DIVIDE('/');

    private final char symbol;

    private Operation(char symbol) {
        this.symbol = symbol;
    }

    public char getSymbol() {
        return symbol;
    }

}
```

```
package tem11_Colecciones.enumSet;

import java.util.EnumSet;

public class ShowEnumSet {
```



```

public void show() {

    EnumSet<Operation> allOperations1, allOperations2, operations1,
operations2, operations3, operations4;

    allOperations1 = EnumSet.allOf(Operation.class);
    System.out.printf("allOf: %s", allOperations1);//allOf: [PLUS, MINUS,
TIMES, DIVIDE]

    allOperations2 = EnumSet.copyOf(allOperations1);
    System.out.printf("\ncopyOf: %s", allOperations2);//copyOf: [PLUS, MINUS,
TIMES, DIVIDE]

    operations1 = EnumSet.noneOf(Operation.class);
    operations1.add(Operation.PLUS);
    operations1.add(Operation.MINUS);
    System.out.printf("\nnoneOf y add: %s", operations1);//noneOf y add:
[PLUS, MINUS]

    operations2 = EnumSet.complementOf(operations1);
    System.out.printf("\ncomplementOf: %s", operations2);//complementOf:
[TIMES, DIVIDE]

    operations3 = EnumSet.of(Operation.DIVIDE, Operation.MINUS);
    System.out.printf("\nof: %s", operations3);//of: [MINUS, DIVIDE]

    operations4 = EnumSet.range(Operation.MINUS, Operation.DIVIDE);
    System.out.printf("\nrange: %s\n", operations4);//range: [MINUS, TIMES,
DIVIDE]
    System.out.println(operations4.contains(Operation.PLUS));//false
    System.out.println(operations4.contains(Operation.MINUS));//true

}

public static void main(String[] args) {

    new ShowEnumSet().show();

}

}

```

Salida por consola:

```

allOf: [PLUS, MINUS, TIMES, DIVIDE]
copyOf: [PLUS, MINUS, TIMES, DIVIDE]
noneOf y add: [PLUS, MINUS]
complementOf: [TIMES, DIVIDE]
of: [MINUS, DIVIDE]
range: [MINUS, TIMES, DIVIDE]
false
true

```

9. Mapas

Las colecciones de tipo Set tienen el inconveniente de tener que almacenar una copia exacta del elemento a buscar. Sin embargo, en la práctica es habitual que haya datos que se consideran clave, es decir, que identifican a cada objeto (el dni de las personas por ejemplo) de tal manera que se buscan los datos en base a esa clave y por otro lado se almacenan el resto de los datos. Los mapas permiten definir colecciones de elementos que poseen pares de datos clave-valor. Esto se utiliza para localizar valores en función de la clave que poseen. Son muy interesantes y rápidos. Los mapas también son conocidos como diccionarios.

La interfaz `Map<K,V>` es la raíz de todas las clases que implementan mapas. Hasta la versión 5, los mapas eran colecciones de pares clave-valor donde tanto la clave como el valor eran de tipo `Object`. Desde la versión 5, esta interfaz tiene dos genéricos: `K` para el tipo de datos de la clave y `V` para el tipo de los valores. Esta estructura de datos nos permite obtener el objeto `V` muy rápidamente a partir de su clave `K`.

Esta interfaz no hereda de `Collection` por lo que no tiene los métodos vistos anteriormente. La razón es que la obtención, búsqueda y borrado de elementos se hace de manera muy distinta.

Las claves no se pueden repetir por lo que se implementan con una tabla hash para que no haya duplicados. Por lo tanto, la clase que se utilice como clave tiene que sobrescribir los métodos `equals()` y `hashCode()`.

Veamos algunos métodos de esta interfaz:

- `boolean containsKey(Object key)`: devuelve *true* si el mapa contiene dicha clave.
- `boolean containsValue(Object value)`: devuelve *true* si el mapa contiene dicho valor.
- `V get(Object key)`: devuelve el valor asociado a la clave o *null* si no existe esa clave en el mapa.
- `V getOrDefault(Object key, V defaultValue)`: devuelve el valor asociado a la clave o *defaultValue* si no existe esa clave en el mapa.
- `V put(K key, V value)`: añade un par clave-valor al mapa. Si ya había un valor para esa clave, se reemplaza. Devuelve el valor que tenía antes dicha clave o *null* si la clave no estaba en el mapa.
- `V putIfAbsent(K key, V value)`: si la clave especificada no está ya asociada a un valor o está asignada a *null*, se le asocia al valor dado y devuelve *null*, en caso contrario, devuelve el valor previamente asociado con la clave.
- `void putAll(Map<? extends K,? extends V> m)`: añade los pares clave-valor del mapa *m*.
- `V remove(Object key)`: elimina la clave y su valor asociado, el cual se devuelve. Si no existe dicha clave, devuelve *null*.

Existen varios tipos de implementaciones realizadas dentro de la plataforma Java:

- **HashMap**: esta implementación almacena las claves en una tabla *hash*. Es la implementación con mejor rendimiento de todas pero no garantiza ningún orden a la hora de realizar iteraciones.
- **LinkedHashMap**: esta implementación almacena las claves en función del orden de inserción. Es un poco más costosa que *HashMap*.
- **TreeMap**: esta implementación utiliza una estructura de árbol para ordenar las claves. Es bastante más lenta que *HashMap*. La veremos más adelante en el apartado de los árboles.

9.1 Clase HashMap

Esta implementación proporciona tiempos constantes en las operaciones básicas siempre y cuando la función *hash* disperse de forma correcta las claves dentro de la tabla *hash*. Es importante definir el tamaño inicial de la tabla ya que este tamaño marcará el rendimiento de esta implementación. No garantiza ningún orden a la hora de recorrer el mapa.

Veamos un ejemplo de un mapa de vehículos donde la clave es la matrícula. Añadimos la matrícula como atributo por lo que hay que generar de nuevo los métodos `toString()`, `hashCode()` y `equals()`:

```
package tem11_colecciones.mapas;

public class Vehicle {

    private String registration; //Atributo para almacenar la matrícula del coche
    private int wheelCount;
    private double speed;
    private String colour;

    public Vehicle(String registration, int wheelCount, String colour) {
        this.registration = registration;
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public String getRegistration() { //get de la matrícula
        return registration;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }
}
```

```

    /*Como se ha añadido el nuevo atributo para la matrícula,
    hay que generar de nuevo los métodos toString(), hashCode()
    y equals(): */

    @Override
    public String toString() {
        return String.format("Matrícula: %s Número de ruedas: %d, color: %s,
    velocidad: %.2f", registration, wheelCount, colour, speed);
    }

    @Override
    public int hashCode() {
        final int prime = 31;
        int result = 1;
        result = prime * result + ((colour == null) ? 0 : colour.hashCode());
        result = prime * result + ((registration == null) ? 0 :
    registration.hashCode());
        result = prime * result + wheelCount;
        return result;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (!(obj instanceof Vehicle))
            return false;
        Vehicle other = (Vehicle) obj;
        if (colour == null) {
            if (other.colour != null)
                return false;
        } else if (!colour.equals(other.colour))
            return false;
        if (registration == null) {
            if (other.registration != null)
                return false;
        } else if (!registration.equals(other.registration))
            return false;
        if (wheelCount != other.wheelCount)
            return false;
        return true;
    }
}

```

```

package tem11_Colecciones.mapas;

import java.util.HashMap;
import java.util.Map;

public class ShowHashMap {

    public void show() {

```

```

Map<String, Vehicle> map = new HashMap<>();
Map<String, Vehicle> map2 = new HashMap<>();
Vehicle vehicles[] = new Vehicle[6];
vehicles[0] = new Vehicle("9685KMX", 4, "azul");
vehicles[1] = new Vehicle("1235GTR", 2, "rojo");
vehicles[2] = new Vehicle("7314QWE", 4, "verde");
vehicles[3] = new Vehicle("5930POI", 2, "negro");
vehicles[4] = new Vehicle("1705UBG", 4, "blanco");
vehicles[5] = new Vehicle("3495JZA", 2, "naranja");
for (int i = 0; i < vehicles.length; i++) {
    map.put(vehicles[i].getRegistration(), vehicles[i]);
}

System.out.println(map.containsKey("1005SAW")); //false
System.out.println(map.containsKey("1705UBG")); //true
System.out.println(map.containsValue(new Vehicle("5930POI", 4,
"negro"))); //false
System.out.println(map.containsValue(new Vehicle("5930POI", 2,
"negro"))); //true
System.out.println(map.get("4554ASD")); //null
System.out.println(map.get("1705UBG")); //Matrícula: 1705UBG Número de
ruedas: 4, color: blanco, velocidad: 0,00
System.out.println(map.getOrDefault("8080SAS", new Vehicle("4554ASD", 4,
"negro"))); //Matrícula: 4554ASD Número de ruedas: 4, color: negro, velocidad:
0,00
System.out.println(map.getOrDefault("1705UBG", new Vehicle("4554ASD", 4,
"negro"))); //Matrícula: 1705UBG Número de ruedas: 4, color: blanco, velocidad:
0,00
System.out.println(map.put("6320LPL", new Vehicle("6320LPL", 2,
"verde"))); //null
System.out.println(map.put("6320LPL", new Vehicle("6320LPL", 4,
"beis"))); //Matrícula: 6320LPL Número de ruedas: 2, color: verde, velocidad: 0,00
System.out.println(map.putIfAbsent("4687RTB", new Vehicle("4687RTB", 2,
"blanco"))); //null
System.out.println(map.putIfAbsent("4687RTB", new Vehicle("4687RTB", 4,
"naranja"))); //Matrícula: 4687RTB Número de ruedas: 2, color: blanco, velocidad:
0,00
System.out.println(map.remove("1234ABC")); //null
System.out.println(map.remove("4687RTB")); //Matrícula: 4687RTB Número de
ruedas: 2, color: blanco, velocidad: 0,00

System.out.printf("El mapa tiene %d vehículos", map.size());
map2.put("7410HJH", new Vehicle("7410HJH", 4, "rojo"));
map2.put("8520FGF", new Vehicle("8520FGF", 2, "verde"));
map.putAll(map2); //añade a map los pares clave-valor del mapa map2
System.out.printf("\nDespués de añadirle map2, el mapa tiene %d
vehículos", map.size());
}

public static void main(String[] args) {

    new ShowHashMap().show();

}

```

```
}
```

Salida por consola:

```
false
true
false
true
null
Matrícula: 1705UBG Número de ruedas: 4, color: blanco, velocidad: 0,00
Matrícula: 4554ASD Número de ruedas: 4, color: negro, velocidad: 0,00
Matrícula: 1705UBG Número de ruedas: 4, color: blanco, velocidad: 0,00
null
Matrícula: 6320LPL Número de ruedas: 2, color: verde, velocidad: 0,00
null
Matrícula: 4687RTB Número de ruedas: 2, color: blanco, velocidad: 0,00
null
Matrícula: 4687RTB Número de ruedas: 2, color: blanco, velocidad: 0,00
El mapa tiene 7 vehículos
Después de añadirle map2, el mapa tiene 9 vehículos
```

Veamos las distintas maneras de recorrer un mapa:

- `Set<K> keySet()` : devuelve un conjunto con todas las claves. Como entre las claves no puede haber elementos duplicados , las claves forman un conjunto (*Set*).
- `Collection<V> values()` : devuelve una colección con todos los valores. Los valores sí pueden estar duplicados, por lo tanto, este método devuelve un *Collection*.
- `Set<Map.Entry<K,V>> entrySet()` : devuelve un conjunto de objetos *Map.Entry<K, V>*. Los pares de elementos (también llamados entradas) de los que está compuesto un *Map<K, V>* son de un tipo que viene implementado por la interfaz *Map.Entry<K, V>*. La interfaz *Map.Entry<K, V>* se define de forma interna a la interfaz *Map<K, V>* y representa un objeto de par clave-valor, es decir, mediante esta interfaz podemos trabajar con una entrada del mapa. Veamos algunos métodos de la interfaz *Map.Entry<K, V>*:
 - `K getKey()` : retorna la clave.
 - `V getValue()` : retorna el valor.
 - `V setValue(V value)` : reemplaza el valor por *value* y devuelve el valor anterior.

```
package tema11_Colecciones.mapas;

import java.util.HashMap;
import java.util.Iterator;
import java.util.Map;
import java.util.Set;

public class TraverseHashMap {

    public void show() {

        Map<String, Vehicle> map = new HashMap<>();
        Vehicle vehicles[] = new Vehicle[6];
```

```

vehicles[0] = new Vehicle("9685KMX", 4, "azul");
vehicles[1] = new Vehicle("1235GTR", 2, "rojo");
vehicles[2] = new Vehicle("7314QWE", 4, "verde");
vehicles[3] = new Vehicle("5930POI", 2, "negro");
vehicles[4] = new Vehicle("1705UBG", 4, "blanco");
vehicles[5] = new Vehicle("3495JZA", 2, "naranja");
for (int i = 0; i < vehicles.length; i++) {
    map.put(vehicles[i].getRegistration(), vehicles[i]);
}

System.out.println("Claves del mapa:\n");
for (String s : map.keySet()) { //keySet() devuelve un conjunto con todas
las claves
    System.out.println(s);
}

System.out.println("\nValores del mapa:\n");
for (Vehicle v : map.values()) { //values() devuelve una colección con
todos los vehículos
    System.out.println(v);
}

System.out.println("\nPares clave-valor del mapa usando un foreach:\n");
for (Map.Entry<String, Vehicle> entry : map.entrySet()) {
    System.out.printf("Matrícula -> %s Vehículo -> %s\n", entry.getKey(),
entry.getValue());
}

System.out.println("\nPares clave-valor del mapa usando iteradores:\n");
Set<Map.Entry<String, Vehicle>> entrySet = map.entrySet();
Iterator<Map.Entry<String, Vehicle>> it = entrySet.iterator();
Map.Entry<String, Vehicle> entry;
while (it.hasNext()) {
    entry = it.next();
    System.out.printf("Matrícula -> %s Vehículo -> %s\n", entry.getKey(),
entry.getValue());
}

}

public static void main(String[] args) {

    new TraverseHashMap().show();

}

}

```

Salida por consola:

Claves del mapa:

```

3495JZA
1705UBG
1235GTR

```

```
7314QWE
9685KMX
5930POI
```

valores del mapa:

```
Matrícula: 3495JZA Número de ruedas: 2, color: naranja, velocidad: 0,00
Matrícula: 1705UBG Número de ruedas: 4, color: blanco, velocidad: 0,00
Matrícula: 1235GTR Número de ruedas: 2, color: rojo, velocidad: 0,00
Matrícula: 7314QWE Número de ruedas: 4, color: verde, velocidad: 0,00
Matrícula: 9685KMX Número de ruedas: 4, color: azul, velocidad: 0,00
Matrícula: 5930POI Número de ruedas: 2, color: negro, velocidad: 0,00
```

Pares clave-valor del mapa usando un foreach:

```
Matrícula -> 3495JZA Vehículo -> Matrícula: 3495JZA Número de ruedas: 2, color:
naranja, velocidad: 0,00
Matrícula -> 1705UBG Vehículo -> Matrícula: 1705UBG Número de ruedas: 4, color:
blanco, velocidad: 0,00
Matrícula -> 1235GTR Vehículo -> Matrícula: 1235GTR Número de ruedas: 2, color:
rojo, velocidad: 0,00
Matrícula -> 7314QWE Vehículo -> Matrícula: 7314QWE Número de ruedas: 4, color:
verde, velocidad: 0,00
Matrícula -> 9685KMX Vehículo -> Matrícula: 9685KMX Número de ruedas: 4, color:
azul, velocidad: 0,00
Matrícula -> 5930POI Vehículo -> Matrícula: 5930POI Número de ruedas: 2, color:
negro, velocidad: 0,00
```

Pares clave-valor del mapa usando iteradores:

```
Matrícula -> 3495JZA Vehículo -> Matrícula: 3495JZA Número de ruedas: 2, color:
naranja, velocidad: 0,00
Matrícula -> 1705UBG Vehículo -> Matrícula: 1705UBG Número de ruedas: 4, color:
blanco, velocidad: 0,00
Matrícula -> 1235GTR Vehículo -> Matrícula: 1235GTR Número de ruedas: 2, color:
rojo, velocidad: 0,00
Matrícula -> 7314QWE Vehículo -> Matrícula: 7314QWE Número de ruedas: 4, color:
verde, velocidad: 0,00
Matrícula -> 9685KMX Vehículo -> Matrícula: 9685KMX Número de ruedas: 4, color:
azul, velocidad: 0,00
Matrícula -> 5930POI Vehículo -> Matrícula: 5930POI Número de ruedas: 2, color:
negro, velocidad: 0,00
```

9.2 Clase LinkedHashMap

Almacena las claves en función del orden de inserción. Es un poco más costosa que *HashMap*.

```
package tema11_Colecciones.mapas;

import java.util.LinkedHashMap;
import java.util.Map;

public class ShowLinkedHashMap {

    public void show() {
```



```

Map<String, Vehicle> map = new LinkedHashMap<>();
Vehicle vehicles[] = new Vehicle[6];
vehicles[0] = new Vehicle("9685KMX", 4, "azul");
vehicles[1] = new Vehicle("1235GTR", 2, "rojo");
vehicles[2] = new Vehicle("7314QWE", 4, "verde");
vehicles[3] = new Vehicle("5930POI", 2, "negro");
vehicles[4] = new Vehicle("1705UBG", 4, "blanco");
vehicles[5] = new Vehicle("3495JZA", 2, "naranja");
for (int i = 0; i < vehicles.length; i++) {
    map.put(vehicles[i].getRegistration(), vehicles[i]);
}

for (Map.Entry<String, Vehicle> entry : map.entrySet()) {
    System.out.printf("Matrícula -> %s Vehículo -> %s\n", entry.getKey(),
entry.getValue());
}

}

public static void main(String[] args) {

    new ShowLinkedHashMap().show();

}

}

```

Salida por consola:

```

Matrícula -> 9685KMX Vehículo -> Vehicle [registration=9685KMX, wheelCount=4,
speed=0.0, colour=azul]
Matrícula -> 1235GTR Vehículo -> Vehicle [registration=1235GTR, wheelCount=2,
speed=0.0, colour=rojo]
Matrícula -> 7314QWE Vehículo -> Vehicle [registration=7314QWE, wheelCount=4,
speed=0.0, colour=verde]
Matrícula -> 5930POI Vehículo -> Vehicle [registration=5930POI, wheelCount=2,
speed=0.0, colour=negro]
Matrícula -> 1705UBG Vehículo -> Vehicle [registration=1705UBG, wheelCount=4,
speed=0.0, colour=blanco]
Matrícula -> 3495JZA Vehículo -> Vehicle [registration=3495JZA, wheelCount=2,
speed=0.0, colour=naranja]

```

Podemos observar que los datos se muestran en el mismo orden en el que se insertaron.

9.3 Ejemplo de uso de un mapa en un enum

Si hemos sobrescrito el método `toString()` o las instancias del *enum* tienen alguna forma adicional de referirnos a ellas, tiene bastante sentido que creamos un método estático parecido a `valueOf()`, pero que reciba dicha forma adicional de referirnos a las instancias.

```

package tema11_Colecciones.mapas;

import java.util.Map;

```

```

public enum Operation {

    PLUS('+') {
        @Override
        public double apply(double x, double y) {
            return x + y;
        }
    },
    MINUS('-') {
        @Override
        public double apply(double x, double y) {
            return x - y;
        }
    },
    TIMES('*') {
        @Override
        public double apply(double x, double y) {
            return x * y;
        }
    },
    DIVIDE('/') {
        @Override
        public double apply(double x, double y) {
            return x / y;
        }
    };

    private final char symbol;

    private static final Map<Character, Operation> symbolToOperation =
Map.of(Operation.PLUS.getSymbol(), Operation.PLUS,
        Operation.MINUS.getSymbol(), Operation.MINUS,
        Operation.TIMES.getSymbol(), Operation.TIMES,
        Operation.DIVIDE.getSymbol(), Operation.DIVIDE);

    Operation(char symbol) {
        this.symbol = symbol;
    }

    public char getSymbol() {
        return symbol;
    }

    public abstract double apply(double x, double y);

    public static Operation fromSymbol(char symbol) {
        return symbolToOperation.get(symbol);
    }

}

```

```

package temall_colecciones.mapas;

public class ExampleUseMapEnum {

```

```

public void show() {

    Operation operation;
    operation = operation.fromSymbol('+');//operation se asigna con la
    instancia correspondiente al símbolo +
    System.out.printf("La variable operation es de tipo enum %s y su símbolo
    es %c", operation,
        operation.getSymbol());

}

public static void main(String[] args) {

    new ExampleUseMapEnum().show();

}

}

```

Salida por consola:

```
La variable operation es de tipo enum PLUS y su símbolo es +
```

Como vemos en el código anterior, creamos un mapa estático que relaciona cada símbolo con cada instancia, de manera que podamos obtener la instancia adecuada a partir del símbolo. Debemos tener en cuenta que no está permitido que los constructores de las instancias de un *enum* accedan a los atributos estáticos del *enum*, con la excepción de las constantes de las instancias, dado que los atributos estáticos aún no han sido inicializados cuando se están ejecutando los constructores de las instancias. En los constructores de las instancias tampoco se puede acceder a otras instancias del *enum*.

9.4 Clase EnumMap

Es una implementación de mapa muy eficiente donde las claves son elementos de una enumeración.

```

package tem11_Colecciones.enumMap;

public enum Operation {

    PLUS('+'), MINUS('-'), TIMES('*'), DIVIDE('/');

    private final char symbol;

    Operation(char symbol) {
        this.symbol = symbol;
    }

    public char getSymbol() {
        return symbol;
    }
}

```

```
}
```

```
package temal1_Colecciones.enumMap;

import java.util.EnumMap;
import java.util.Map;

public class ShowEnumMap {

    public void show() {

        EnumMap<Operation, String> operationsMap = new EnumMap<>
(Operation.class);
        operationsMap.put(Operation.PLUS, "Esta operación se utiliza para
sumar");
        operationsMap.put(Operation.MINUS, "Esta operación se utiliza para
restar");
        operationsMap.put(Operation.TIMES, "Esta operación se utiliza para
multiplicar");
        operationsMap.put(Operation.DIVIDE, "Esta operación se utiliza para
dividir");
        for (Map.Entry<Operation, String> entry : operationsMap.entrySet()) {
            System.out.printf("%-6s: %s\n", entry.getKey(), entry.getValue());
        }

    }

    public static void main(String[] args) {

        new ShowEnumMap().show();

    }

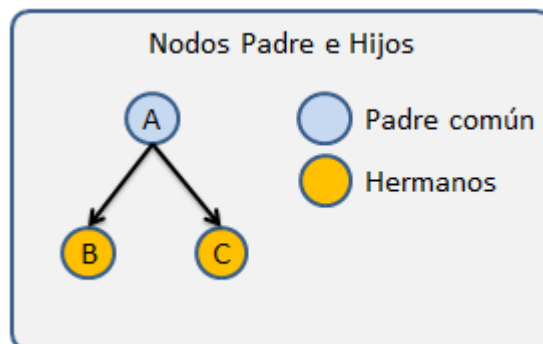
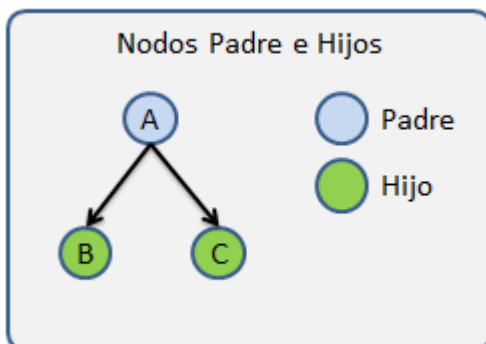
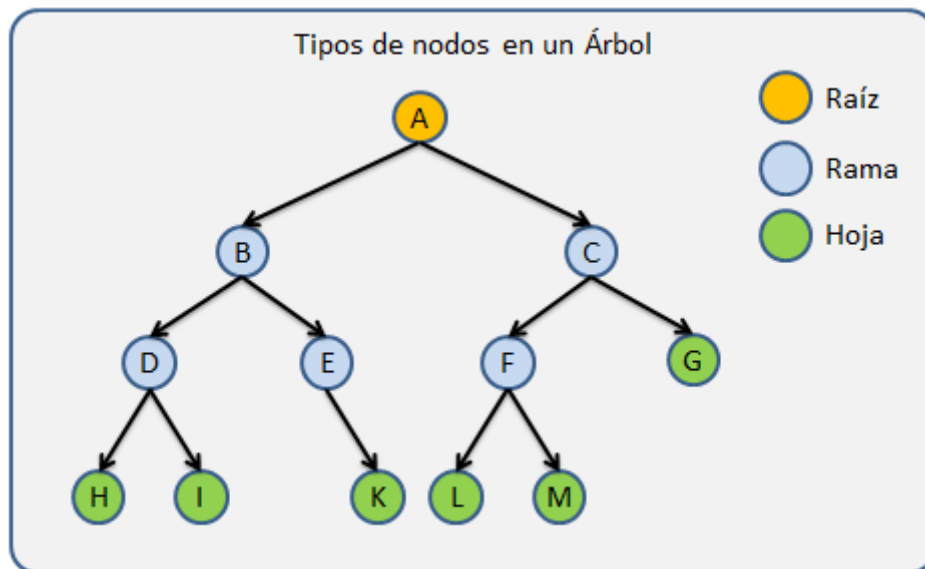
}
```

Salida por consola:

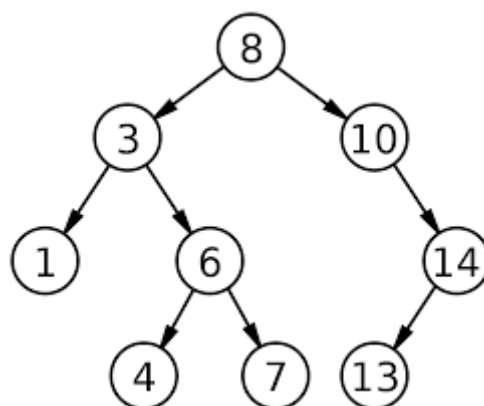
```
PLUS   : Esta operación se utiliza para sumar
MINUS  : Esta operación se utiliza para restar
TIMES  : Esta operación se utiliza para multiplicar
DIVIDE : Esta operación se utiliza para dividir
```

10. Árboles

Los árboles se caracterizan por almacenar sus nodos en forma jerárquica y no en forma lineal como las listas.



Un árbol es una estructura en la que los datos se organizan en nodos. Los **árboles binarios** son aquellos en los que un nodo solamente puede tener dos hijos como máximo. Se utilizan para ordenar datos, de tal manera que a la izquierda se colocan los valores menores y a la derecha los valores mayores. Un árbol binario se puede recorrer de varias formas, siendo el recorrido **inorden** el que muestra los datos ordenados: subárbol izquierdo, raíz, subárbol derecho. Veamos un ejemplo:



Nos posicionamos en el 8. Mientras tenga subárbol izquierdo vamos avanzando hasta llegar al 1. Como es una hoja, la mostramos: el **1**. Luego, mostramos la raíz: el **3**. Después, continuamos con el subárbol derecho. Nos encontramos con el 6 pero tiene subárbol izquierdo, así que avanzamos hasta el 4. Como es una hoja, lo mostramos: el **4**. Luego, continuamos con la raíz: el **6**. Después, continuamos con el subárbol derecho, avanzando hasta el 7. Como es hoja, lo mostramos: el **7**. Ya hemos tratado todo el subárbol izquierdo del 8, que es la raíz. Ahora mostramos la raíz: el **8**, y a continuación comenzamos con el subárbol derecho del 8. Y así sucesivamente...El resultado final es: 1, 3, 4, 6, 7, 8, 10, 13 y 14, es decir, los elementos ordenados de menor a mayor.

Si queremos introducir un nuevo nodo en el árbol, hay que tener cuidado de no romper la estructura ni el orden del árbol. Hay que tener en cuenta que cada nodo no puede tener más de dos hijos, por esta razón si un nodo ya tiene 2 hijos, el nuevo nodo nunca se podrá insertar como su hijo. Con esta restricción nos aseguramos mantener la estructura del árbol, pero aún nos falta mantener el orden. Para localizar el lugar adecuado del árbol donde insertar el nuevo nodo se realizan comparaciones entre los nodos del árbol y el elemento a insertar. El primer nodo que se compara es el nodo raíz, si el nuevo nodo es menor que el raíz, la búsqueda prosigue por el nodo izquierdo de éste. Si el nuevo nodo fuese mayor, la búsqueda seguiría por el hijo derecho. Y así sucesivamente hasta llegar a un nodo que no tenga hijo en la rama por la que la búsqueda debería seguir. En este caso, el nuevo nodo se inserta en ese hueco, como su nuevo hijo.

Por ejemplo, queremos insertar el elemento 9. Lo primero es comparar el nuevo elemento con el nodo raíz. Como $9 > 8$, entonces la búsqueda prosigue por el lado derecho. Ahora el nuevo nodo se compara con el elemento 10. En este caso $9 < 10$, por lo que hay que continuar la búsqueda por la rama izquierda. Como la rama izquierda de 10 no tiene ningún nodo, se inserta en ese lugar el nuevo nodo.

La interfaz `SortedSet<E>` es la encargada de definir esta estructura. Esta interfaz es hija de `Set<E>`, que a su vez es hija de `Collection<E>`, que a su vez es hija de `Iterable<E>`. Por lo tanto, tiene los métodos de todas y además añade sus propios métodos:

- `E first()`: devuelve el elemento más pequeño.
- `E last()`: devuelve el elemento más grande.
- `SortedSet<E> headSet(E toElement)`: devuelve un *SortedSet* que contendrá todos los elementos menores que *toElement*.
- `SortedSet<E> tailSet(E fromElement)`: devuelve un *SortedSet* que contendrá todos los elementos mayores que *fromElement*.
- `SortedSet<E> subSet(E fromElement, E toElement)`: devuelve un *SortedSet* que contendrá los elementos que van desde *fromElement* incluido hasta *toElement* excluido.

Pero ¿cómo ordenamos elementos como por ejemplo vehículos? Para ello, Java nos proporciona dos interfaces: `Comparable<T>` y `Comparator<T>`. La diferencia entre ambas es que *Comparable* se implementa desde la propia clase que se quiere ordenar y *Comparator* no.

10.1 Clase TreeSet

La clase `TreeSet<E>` es la que se utiliza prioritariamente para trabajar con árboles ordenados ya que implementa la interfaz `SortedSet<E>`.

Los objetos a incluir en un *TreeSet* deben implementar *Comparable* o bien crear el árbol con un constructor que reciba un *Comparator*.

La interfaz `Comparable` contiene un único método, el método `compareTo`, que recibe un objeto de la misma clase y que debe realizar una comparación entre ambos objetos, retornando un valor entero negativo, cero o positivo, dependiendo de si el objeto sobre el que se ejecuta es, respectivamente, menor, igual o mayor que el objeto recibido.

La definición de la interfaz es la siguiente:

```
public interface Comparable<T> {  
    int compareTo(T o);  
}
```

Por ejemplo, si quisiéramos crear un árbol para ordenar vehículos, lo primero que tendríamos que hacer es que la clase *Vehicle* implemente la interfaz *Comparable* y que el método *compareTo* ordene por el atributo que deseemos. Por ejemplo, vamos a ordenar vehículos alfabéticamente por el color. Como el color es de tipo *String*, debemos utilizar el *compareTo* de la clase *String*:

```
package tema11_Colecciones.arboles1;

public class Vehicle implements Comparable<Vehicle> { //La clase Vehicle
    implementa Comparable

    private String registration;
    private int wheelCount;
    private double speed;
    private String colour;

    public Vehicle(String registration, int wheelCount, String colour) {
        this.registration = registration;
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public String getRegistration() {
        return registration;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }

    @Override
    public String toString() {
        return String.format("Matrícula: %s Número de ruedas: %d, color: %s,
    velocidad: %.2f", registration, wheelCount, colour, speed);
    }
}
```

```

@Override
public int compareTo(Vehicle o) { //Tenemos que definir en este método el
criterio de ordenación
    return colour.compareTo(o.colour); //Utilizamos el compareTo de la clase
String
    }
}

```

```

package tem11_colecciones.arboles1;

import java.util.SortedSet;
import java.util.TreeSet;

public class TreeSet1 {

    public void show() {

        SortedSet<Vehicle> tree = new TreeSet<>();
        tree.add(new Vehicle("9685KMX", 4, "azul"));
        tree.add(new Vehicle("1235GTR", 2, "rojo"));
        tree.add(new Vehicle("7314QWE", 4, "verde"));
        tree.add(new Vehicle("5930POI", 2, "negro"));
        tree.add(new Vehicle("1705UBG", 4, "blanco"));
        tree.add(new Vehicle("3495JZA", 2, "naranja"));
        for (Vehicle v : tree) {
            System.out.println(v);
        }

    }

    public static void main(String[] args) {

        new TreeSet1().show();

    }

}

```

Salida por consola:

```

Matrícula: 9685KMX Número de ruedas: 4, color: azul, velocidad: 0,00
Matrícula: 1705UBG Número de ruedas: 4, color: blanco, velocidad: 0,00
Matrícula: 3495JZA Número de ruedas: 2, color: naranja, velocidad: 0,00
Matrícula: 5930POI Número de ruedas: 2, color: negro, velocidad: 0,00
Matrícula: 1235GTR Número de ruedas: 2, color: rojo, velocidad: 0,00
Matrícula: 7314QWE Número de ruedas: 4, color: verde, velocidad: 0,00

```

Pero ¿qué ocurriría si tuviéramos colores repetidos?:

```

package tem11_colecciones.arboles1;

import java.util.SortedSet;

```



```

import java.util.TreeSet;

public class TreeSet2 {

    public void show() {

        SortedSet<Vehicle> tree = new TreeSet<>();
        tree.add(new Vehicle("9685KMX", 4, "azul"));
        tree.add(new Vehicle("1235GTR", 2, "rojo"));
        tree.add(new Vehicle("7314QWE", 4, "verde"));
        tree.add(new Vehicle("5930POI", 2, "azul"));
        tree.add(new Vehicle("1705UBG", 4, "rojo"));
        tree.add(new Vehicle("3495JZA", 2, "verde"));
        for (Vehicle v : tree) {
            System.out.println(v);
        }

    }

    public static void main(String[] args) {

        new TreeSet2().show();

    }

}

```

Salida por consola:

```

Matrícula: 9685KMX Número de ruedas: 4, color: azul, velocidad: 0,00
Matrícula: 1235GTR Número de ruedas: 2, color: rojo, velocidad: 0,00
Matrícula: 7314QWE Número de ruedas: 4, color: verde, velocidad: 0,00

```

Los tres primeros vehículos se introducen en el árbol. Cuando se va a introducir el cuarto que es de color azul, el árbol lo va comparando con *compareTo* con los vehículos que ya existen en el árbol para encontrar la posición ordenada donde incluirlo. Pero cuando lo compara con el que es azul, el *compareTo* le devuelve 0, por lo que el árbol interpreta que ese objeto ya existe en el árbol, que es igual a otro, por lo tanto no lo incluye en el árbol. Lo mismo ocurre con el quinto y el sexto. Por lo tanto, lo que ocurriría es que los 3 últimos no se introducen en el árbol porque el *compareTo* devuelve 0 entre vehículos del mismo color, por lo que el árbol considera que son iguales. En estos casos, lo que se hace es que se incluye un segundo criterio de comparación: vamos a ordenar por el color, y en aquellos casos donde los vehículos tengan el mismo color, entonces vamos a ordenar por matrícula:

```

package tem11_Colecciones.arboles2;

public class Vehicle implements Comparable<Vehicle> {

    private String registration;
    private int wheelCount;
    private double speed;
    private String colour;

```

```

public Vehicle(String registration, int wheelCount, String colour) {
    this.registration = registration;
    this.wheelCount = wheelCount;
    this.colour = colour;
    speed = 0;
}

public int getWheelCount() {
    return wheelCount;
}

public double getSpeed() {
    return speed;
}

public String getColour() {
    return colour;
}

public void setColour(String colour) {
    this.colour = colour;
}

public String getRegistration() {
    return registration;
}

public void accelerate(double amount) {
    speed += amount;
}

public void brake(double amount) {
    speed -= amount;
}

@Override
public String toString() {
    return String.format("Matrícula: %s Número de ruedas: %d, color: %s, velocidad: %.2f", registration, wheelCount, colour, speed);
}

@Override
public int compareTo(Vehicle o) {
    int result = colour.compareTo(o.colour);
    if (result == 0) { //Introducimos un segundo criterio de ordenación
        result = registration.compareTo(o.registration);
    }
    return result;
}
}

```

```

package tem11_colecciones.arboles2;

import java.util.SortedSet;

```

```
import java.util.TreeSet;

public class ShowTreeSet {

    public void show() {

        SortedSet<Vehicle> tree = new TreeSet<>();
        tree.add(new Vehicle("9685KMX", 4, "azul"));
        tree.add(new Vehicle("1235GTR", 2, "rojo"));
        tree.add(new Vehicle("7314QWE", 4, "verde"));
        tree.add(new Vehicle("5930POI", 2, "azul"));
        tree.add(new Vehicle("1705UBG", 4, "rojo"));
        tree.add(new Vehicle("3495JZA", 2, "verde"));
        for (Vehicle v : tree) {
            System.out.println(v);
        }

    }

    public static void main(String[] args) {

        new ShowTreeSet().show();

    }

}
```

Salida por consola:

```
Matrícula: 5930POI Número de ruedas: 2, color: azul, velocidad: 0,00
Matrícula: 9685KMX Número de ruedas: 4, color: azul, velocidad: 0,00
Matrícula: 1235GTR Número de ruedas: 2, color: rojo, velocidad: 0,00
Matrícula: 1705UBG Número de ruedas: 4, color: rojo, velocidad: 0,00
Matrícula: 3495JZA Número de ruedas: 2, color: verde, velocidad: 0,00
Matrícula: 7314QWE Número de ruedas: 4, color: verde, velocidad: 0,00
```

Pero, ¿qué ocurriría si la clase *Vehicle* no implementase la interfaz *Comparable*? Pues que no contendría el método *compareTo*, entonces el árbol no tendría la información de cómo ordenar los vehículos. En este caso, se lanzaría una excepción `ClassCastException`:

```
package tema11_Colecciones.arboles3;

public class Vehicle { //No implementa Comparable por lo que no contiene el método compareTo

    private String registration;
    private int wheelCount;
    private double speed;
    private String colour;

    public Vehicle(String registration, int wheelCount, String colour) {
        this.registration = registration;
        this.wheelCount = wheelCount;
        this.colour = colour;
    }
}
```

```

        speed = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public String getRegistration() {
        return registration;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }

    @Override
    public String toString() {
        return String.format("Matrícula: %s Número de ruedas: %d, color: %s, velocidad: %.2f", registration, wheelCount, colour, speed);
    }
}

```

```

package tem11_Colecciones.arboles3;

import java.util.SortedSet;
import java.util.TreeSet;

public class ShowException {

    public void show() {

        SortedSet<Vehicle> tree = new TreeSet<>();
        tree.add(new Vehicle("9685KMX", 4, "azul"));
        tree.add(new Vehicle("1235GTR", 2, "rojo"));
        tree.add(new Vehicle("7314QWE", 4, "verde"));
        tree.add(new Vehicle("5930POI", 2, "negro"));
        tree.add(new Vehicle("1705UBG", 4, "blanco"));
        tree.add(new Vehicle("3495JZA", 2, "naranja"));
    }
}

```

```

        for (Vehicle v : tree) {
            System.out.println(v);
        }

    }

    public static void main(String[] args) {

        new ShowException().show();

    }

}

```

Salida por consola:

```

Exception in thread "main" java.lang.ClassCastException: class
tema11_Colecciones.arboles3.Vehicle cannot be cast to class java.lang.Comparable
(tema11_Colecciones.arboles3.Vehicle is in unnamed module of loader 'app';
java.lang.Comparable is in module java.base of loader 'bootstrap')
    at java.base/java.util.TreeMap.compare(TreeMap.java:1291)
    at java.base/java.util.TreeMap.put(TreeMap.java:536)
    at java.base/java.util.TreeSet.add(TreeSet.java:255)
    at tema11_Colecciones.arboles3.ShowException.show(ShowException.java:11)
    at tema11_Colecciones.arboles3.ShowException.main(ShowException.java:25)

```

Otra posibilidad es utilizar un objeto `Comparator<E>`. Esta es otra interfaz que define el método `compare` al que se le pasan los dos objetos a comparar y cuyo resultado es como el de `compareTo` (0 si son iguales, positivo si el primero es mayor y negativo si el segundo es mayor). Para definir un comparador de esta forma, hay que crear una clase que implemente esta interfaz y definir el método `compare`, después crear un objeto de ese tipo y usarlo en la construcción del árbol mediante un constructor que recibe un *Comparator*: `TreeSet(Comparator<? super E> comparator)`. En este caso, ésta será la forma prioritaria para ordenar la lista, por encima del método `compareTo` de la interfaz `Comparable`. Como ya dijimos anteriormente, la diferencia entre *Comparable* y *Comparator* es que *Comparable* se implementa desde la propia clase que se quiere ordenar y *Comparator* no, ya que *Comparator* se implementa desde otra clase distinta a la que se quiere ordenar:

```

package tema11_Colecciones.arboles3;

import java.util.Comparator;

public class VehicleComparator implements Comparator<Vehicle> {

    @Override
    public int compare(Vehicle o1, Vehicle o2) {
        int result = o1.getColour().compareTo(o2.getColour());
        if (result == 0) {
            result = o1.getRegistration().compareTo(o2.getRegistration());
        }
        return result;
    }

}

```

```
}
```

```
package temal1_Colecciones.arboles3;

import java.util.SortedSet;
import java.util.TreeSet;

public class ShowComparator {

    public void show() {

        SortedSet<Vehicle> tree = new TreeSet<>(new VehicleComparator());
        tree.add(new Vehicle("9685KMX", 4, "azul"));
        tree.add(new Vehicle("1235GTR", 2, "rojo"));
        tree.add(new Vehicle("7314QWE", 4, "verde"));
        tree.add(new Vehicle("5930POI", 2, "azul"));
        tree.add(new Vehicle("1705UBG", 4, "rojo"));
        tree.add(new Vehicle("3495JZA", 2, "verde"));
        for (Vehicle v : tree) {
            System.out.println(v);
        }

    }

    public static void main(String[] args) {

        new ShowComparator().show();

    }

}
```

Salida por consola:

```
Matrícula: 5930POI Número de ruedas: 2, color: azul, velocidad: 0,00
Matrícula: 9685KMX Número de ruedas: 4, color: azul, velocidad: 0,00
Matrícula: 1235GTR Número de ruedas: 2, color: rojo, velocidad: 0,00
Matrícula: 1705UBG Número de ruedas: 4, color: rojo, velocidad: 0,00
Matrícula: 3495JZA Número de ruedas: 2, color: verde, velocidad: 0,00
Matrícula: 7314QWE Número de ruedas: 4, color: verde, velocidad: 0,00
```

Pero si esta comparación solamente la vamos a utilizar una vez, tenemos que crear una clase solamente para un uso. Y si necesitamos ordenar los vehículos de varias maneras, tenemos que tener una clase por cada criterio de ordenación. En estos casos, podemos utilizar una clase inline anónima:

```
package temal1_Colecciones.arboles3;

import java.util.Comparator;
import java.util.SortedSet;
import java.util.TreeSet;

public class AnonymousComparator {
```

```

public void show() {

    SortedSet<Vehicle> tree = new TreeSet<>(new Comparator<Vehicle>() {

        @Override
        public int compare(Vehicle o1, Vehicle o2) {
            int result = o1.getColour().compareTo(o2.getColour());
            if (result == 0) {
                result =
o1.getRegistration().compareTo(o2.getRegistration());
            }
            return result;
        }

    });

    tree.add(new Vehicle("9685KMX", 4, "azul"));
    tree.add(new Vehicle("1235GTR", 2, "rojo"));
    tree.add(new Vehicle("7314QWE", 4, "verde"));
    tree.add(new Vehicle("5930POI", 2, "azul"));
    tree.add(new Vehicle("1705UBG", 4, "rojo"));
    tree.add(new Vehicle("3495JZA", 2, "verde"));
    for (Vehicle v : tree) {
        System.out.println(v);
    }

}

public static void main(String[] args) {

    new AnonymousComparator().show();

}

}

```

Salida por consola:

```

Matrícula: 5930POI Número de ruedas: 2, color: azul, velocidad: 0,00
Matrícula: 9685KMX Número de ruedas: 4, color: azul, velocidad: 0,00
Matrícula: 1235GTR Número de ruedas: 2, color: rojo, velocidad: 0,00
Matrícula: 1705UBG Número de ruedas: 4, color: rojo, velocidad: 0,00
Matrícula: 3495JZA Número de ruedas: 2, color: verde, velocidad: 0,00
Matrícula: 7314QWE Número de ruedas: 4, color: verde, velocidad: 0,00

```

Si quisiéramos ordenar de manera descendente, cambiamos el orden entre *o1* y *o2*, es decir, hacemos que sea *o2* el que ejecute el *compareTo*:

```

package tema11_Colecciones.arboles3;

import java.util.Comparator;
import java.util.SortedSet;
import java.util.TreeSet;

```

```

public class DescendingOrder {

    public void show() {

        SortedSet<Vehicle> tree = new TreeSet<>(new Comparator<Vehicle>() {

            /*
             * Si quisiéramos ordenar de manera descendente,
             * cambiamos el orden entre o1 y o2, es decir,
             * hacemos que sea o2 el que ejecute el compareTo
             */
            @Override
            public int compare(Vehicle o1, Vehicle o2) {
                int result = o2.getColour().compareTo(o1.getColour());
                if (result == 0) {
                    result =
o2.getRegistration().compareTo(o1.getRegistration());
                }
                return result;
            }
        });

        tree.add(new Vehicle("9685KMX", 4, "azul"));
        tree.add(new Vehicle("1235GTR", 2, "rojo"));
        tree.add(new Vehicle("7314QWE", 4, "verde"));
        tree.add(new Vehicle("5930POI", 2, "azul"));
        tree.add(new Vehicle("1705UBG", 4, "rojo"));
        tree.add(new Vehicle("3495JZA", 2, "verde"));
        for (Vehicle v : tree) {
            System.out.println(v);
        }

    }

    public static void main(String[] args) {

        new DescendingOrder().show();

    }

}

```

Salida por consola:

```

Matrícula: 7314QWE Número de ruedas: 4, color: verde, velocidad: 0,00
Matrícula: 3495JZA Número de ruedas: 2, color: verde, velocidad: 0,00
Matrícula: 1705UBG Número de ruedas: 4, color: rojo, velocidad: 0,00
Matrícula: 1235GTR Número de ruedas: 2, color: rojo, velocidad: 0,00
Matrícula: 9685KMX Número de ruedas: 4, color: azul, velocidad: 0,00
Matrícula: 5930POI Número de ruedas: 2, color: azul, velocidad: 0,00

```


Los comparadores de tipo `Comparator` permiten ordenar de diferentes formas, por eso en la práctica se utilizan mucho. Por ejemplo, el método `sort` de la clase `Arrays` también admite indicar un comparador para saber de qué forma deseamos ordenar el array.

Cuando creemos clases que representen valores que posean un determinado orden natural, como por ejemplo un orden alfabético, numérico o cronológico, deberemos hacer que dicha clase implemente la interfaz `Comparable`, permitiendo así que los objetos de dicha clase puedan trabajar con muchos algoritmos genéricos e implementaciones de colecciones que dependen de dicha interfaz.

La mayoría de las clases estándar que representan valores y de las clases `enums` incorporadas a Java, implementan la interfaz `Comparable`, como por ejemplo la clase `String`. Las clases que definamos nosotros que representen valores también deberían implementarla.

A la hora de realizar la implementación debemos respetar una serie de reglas:

- `x.compareTo(y) == -y.compareTo(x)` para todo valor de `x` e `y`.
- La relación es transitiva, es decir, que si `(x.compareTo(y) > 0 && y.compareTo(z) > 0)` entonces `x.compareTo(z) > 0`.
- Si `x.compareTo(y) == 0` entonces `x.compareTo(z) == y.compareTo(z)` para cualquier valor de `z`.
- Se recomienda que si `(x.compareTo(y) == 0)`, `x.equals(y)` sea true.

Si para comparar los objetos debemos comparar un atributo de un tipo primitivo, se recomienda usar los métodos estáticos de comparación `compare` de las clases boxed correspondientes, como `Long.compare()`, `Float.compare()`, etc., disponibles a partir de Java 7, en vez de usar los operadores `<` o `>`, ya que son menos verbosos y propensos al error:

```
package tem11_colecciones.arboles3;

import java.util.Comparator;
import java.util.SortedSet;
import java.util.TreeSet;

public class CompareBoxedClasses {

    public void show() {

        SortedSet<Vehicle> tree = new TreeSet<>(new Comparator<Vehicle>() {

            @Override
            public int compare(Vehicle o1, Vehicle o2) {
                int result = Integer.compare(o1.getWheelCount(),
                o2.getWheelCount());
                if (result == 0) {
                    result = Double.compare(o1.getSpeed(), o2.getSpeed());
                }
                return result;
            }

        });

        Vehicle vehicles[] = new Vehicle[6];
        vehicles[0] = new Vehicle("9685KMX", 4, "azul");
        vehicles[0].accelerate(100);
```

```

vehicles[1] = new Vehicle("1235GTR", 2, "rojo");
vehicles[1].accelerate(150);
vehicles[2] = new Vehicle("7314QWE", 4, "verde");
vehicles[2].accelerate(200);
vehicles[3] = new Vehicle("5930POI", 2, "negro");
vehicles[3].accelerate(80);
vehicles[4] = new Vehicle("1705UBG", 4, "blanco");
vehicles[4].accelerate(75);
vehicles[5] = new Vehicle("3495JZA", 2, "naranja");
vehicles[5].accelerate(170);

for (int i = 0; i < vehicles.length; i++) {
    tree.add(vehicles[i]);
}
for (Vehicle v : tree) {
    System.out.println(v);
}

}

public static void main(String[] args) {

    new CompareBoxedClasses().show();

}

}

```

Salida por consola:

```

Matrícula: 5930POI Número de ruedas: 2, color: negro, velocidad: 80,00
Matrícula: 1235GTR Número de ruedas: 2, color: rojo, velocidad: 150,00
Matrícula: 3495JZA Número de ruedas: 2, color: naranja, velocidad: 170,00
Matrícula: 1705UBG Número de ruedas: 4, color: blanco, velocidad: 75,00
Matrícula: 9685KMX Número de ruedas: 4, color: azul, velocidad: 100,00
Matrícula: 7314QWE Número de ruedas: 4, color: verde, velocidad: 200,00

```

10.2 Clase TreeMap

Esta implementación utiliza una estructura de árbol que permite que los elementos del mapa se ordenen en sentido ascendente según la clave, por lo tanto, la clase de las claves tiene que implementar la interfaz *Comparable* o bien indicar un objeto *Comparator* durante la creación del *TreeMap*.

TreeMap implementa la interfaz `SortedMap` que, a su vez, es heredera de `Map`, por lo que todo lo dicho sobre los mapas funciona con las colecciones de tipo *TreeMap*.

```

package tema11_colecciones.arboles3;

import java.util.Map;
import java.util.SortedMap;
import java.util.TreeMap;

public class ShowTreeMap {

```

```

public void show() {

    SortedMap<String, Vehicle> sortedMap = new TreeMap<>();
    Vehicle vehicles[] = new Vehicle[6];
    vehicles[0] = new Vehicle("9685KMX", 4, "azul");
    vehicles[1] = new Vehicle("1235GTR", 2, "rojo");
    vehicles[2] = new Vehicle("7314QWE", 4, "verde");
    vehicles[3] = new Vehicle("5930POI", 2, "negro");
    vehicles[4] = new Vehicle("1705UBG", 4, "blanco");
    vehicles[5] = new Vehicle("3495JZA", 2, "naranja");
    for (int i = 0; i < vehicles.length; i++) {
        sortedMap.put(vehicles[i].getRegistration(), vehicles[i]);
    }

    for (Map.Entry<String, Vehicle> entry : sortedMap.entrySet()) {
        System.out.printf("Matrícula -> %s Vehículo -> %s\n", entry.getKey(),
entry.getValue());
    }

}

public static void main(String[] args) {

    new ShowTreeMap().show();

}

}

```

Salida por consola:

```

Matrícula -> 1235GTR Vehículo -> Matrícula: 1235GTR Número de ruedas: 2, color:
rojo, velocidad: 0,00
Matrícula -> 1705UBG Vehículo -> Matrícula: 1705UBG Número de ruedas: 4, color:
blanco, velocidad: 0,00
Matrícula -> 3495JZA Vehículo -> Matrícula: 3495JZA Número de ruedas: 2, color:
naranja, velocidad: 0,00
Matrícula -> 5930POI Vehículo -> Matrícula: 5930POI Número de ruedas: 2, color:
negro, velocidad: 0,00
Matrícula -> 7314QWE Vehículo -> Matrícula: 7314QWE Número de ruedas: 4, color:
verde, velocidad: 0,00
Matrícula -> 9685KMX Vehículo -> Matrícula: 9685KMX Número de ruedas: 4, color:
azul, velocidad: 0,00

```

Como podemos observar, está ordenado ascendentemente por la matrícula ya que la matrícula es de tipo `String` que implementa la interfaz `Comparable`. Si la clave fuera una clase hecha por nosotros, tendríamos que hacer que implementara `Comparable` o bien indicar un `Comparator` en la creación del `TreeMap`.

11. Pilas y colas

Una cola es un tipo de dato que sigue el principio **FIFO** (*first in, first out*) que implica que el primer elemento en ser insertado en la cola es también el primero en ser eliminado de la misma.

En el mundo real podemos encontrar este ejemplo en las colas de un banco, la cadena de impresión de documentos, etc. En el caso de la cola en el banco, la primera persona en llegar es también la primera en irse (suponiendo una única ventanilla) y en los documentos a imprimir, la impresora imprime según el orden de llegada.

`Queue<E>` es una interfaz que hereda de *Collection* que proporciona operaciones para trabajar con una cola. Veamos alguna de ellas:

- `boolean add(E e)` : inserta el elemento al final de la cola.
- `E element()` : Devuelve, pero no elimina, el principio de la cola. Lanza la excepción `NoSuchElementException` si la cola está vacía.
- `E peek()` : Devuelve, pero no elimina, el principio de la cola. Devuelve *null* si la cola está vacía.
- `E poll()` : Devuelve y elimina el principio de la cola. Devuelve *null* si la cola está vacía.
- `E remove()` : Devuelve y elimina el principio de la cola. Lanza la excepción `NoSuchElementException` si la cola está vacía.

`Deque<E>` representa una cola de doble extremo, lo que significa que se puede insertar y eliminar elementos desde ambos extremos de la cola. El nombre *Deque* es una abreviatura de *Double Ended Queue*. Admite, por lo tanto, la implementación de la cola *FIFO* como la implementación de la pila *LIFO*, que implica que el último elemento que se ha insertado, es el primero en ser eliminado: **LIFO (last in, first out)**.

Deque hereda de *Queue*, por lo que tiene todos sus métodos y además añade los suyos propios. Veamos algunos de ellos:

- `void addFirst(E e)` : inserta el elemento al principio.
- `void addLast(E e)` : inserta el elemento al final.
- `E getFirst()` : Devuelve, pero no elimina, el primer elemento. Lanza la excepción `NoSuchElementException` si el deque está vacío.
- `E getLast()` : Devuelve, pero no elimina, el último elemento. Lanza la excepción `NoSuchElementException` si el deque está vacío.
- `E peekFirst()` : Devuelve, pero no elimina, el primer elemento. Devuelve *null* si el deque está vacío.
- `E peekLast()` : Devuelve, pero no elimina, el último elemento. Devuelve *null* si el deque está vacío.
- `E pollFirst()` : Devuelve y elimina el primer elemento. Devuelve *null* si el deque está vacío.
- `E pollLast()` : Devuelve y elimina el último elemento. Devuelve *null* si el deque está vacío.
- `E removeFirst()` : Devuelve y elimina el primer elemento. Lanza la excepción `NoSuchElementException` si el deque está vacío.
- `E removeLast()` : Devuelve y elimina el último elemento. Lanza la excepción `NoSuchElementException` si el deque está vacío.

11.1 Clase ArrayDeque

La clase `ArrayDeque<E>` implementa la interfaz *Deque* y por lo tanto, también *Queue*, ya que *Deque* hereda de *Queue*.

Veamos un ejemplo de uso de *ArrayDeque* utilizado como una cola:

```
package tema11_Colecciones.pilasYColas;

import java.util.ArrayDeque;
import java.util.Queue;

public class ShowQueue {

    public void show() {

        Queue<Vehicle> queue = new ArrayDeque<>();
        queue.add(new Vehicle("9685KMX", 4, "azul"));
        queue.add(new Vehicle("1235GTR", 2, "rojo"));
        queue.add(new Vehicle("7314QWE", 4, "verde"));
        System.out.println(queue.element()); //Devuelve pero no elimina: 9685KMX
        System.out.println(queue.peek()); //Devuelve pero no elimina: 9685KMX
        System.out.println(queue.poll()); //Devuelve y elimina: 9685KMX
        System.out.println(queue.remove()); //Devuelve y elimina: 1235GTR
        System.out.println(queue.remove()); //Devuelve y elimina el último, se
        queda la cola vacía: 7314QWE
        System.out.println(queue.peek()); //Devuelve null
        System.out.println(queue.poll()); //Devuelve null
        System.out.println(queue.element()); //Lanza NoSuchElementException porque
        la cola está vacía
        System.out.println(queue.remove()); //Lanza NoSuchElementException porque
        la cola está vacía

    }

    public static void main(String[] args) {

        new ShowQueue().show();

    }

}
```

Salida por consola:

```

Matrícula: 9685KMX Número de ruedas: 4, color: azul, velocidad: 0,00
Matrícula: 9685KMX Número de ruedas: 4, color: azul, velocidad: 0,00
Matrícula: 9685KMX Número de ruedas: 4, color: azul, velocidad: 0,00
Matrícula: 1235GTR Número de ruedas: 2, color: rojo, velocidad: 0,00
Matrícula: 7314QWE Número de ruedas: 4, color: verde, velocidad: 0,00
null
null
Exception in thread "main" java.util.NoSuchElementException
    at java.base/java.util.ArrayDeque.getFirst(ArrayDeque.java:401)
    at java.base/java.util.ArrayDeque.element(ArrayDeque.java:550)
    at tema11_Colecciones.pilasYColas.ShowQueue.show(ShowQueue.java:21)
    at tema11_Colecciones.pilasYColas.ShowQueue.main(ShowQueue.java:28)

```

Veamos otro ejemplo de uso de un *ArrayDeque* utilizado como una pila:

```

package tema11_Colecciones.pilasYColas;

import java.util.ArrayDeque;
import java.util.Deque;

public class ShowDeque {

    public void show() {

        Deque<Vehicle> deque = new ArrayDeque<>();
        deque.add(new Vehicle("9685KMX", 4, "azul"));
        deque.add(new Vehicle("1235GTR", 2, "rojo"));
        deque.add(new Vehicle("7314QWE", 4, "verde"));
        System.out.println(deque.getLast()); //Devuelve pero no elimina: 7314QWE
        System.out.println(deque.peekLast()); //Devuelve pero no elimina: 7314QWE
        System.out.println(deque.pollLast()); //Devuelve y elimina: 7314QWE
        System.out.println(deque.removeLast()); //Devuelve y elimina: 1235GTR
        System.out.println(deque.removeLast()); //Devuelve y elimina el último, se
        queda la pila vacía: 9685KMX
        System.out.println(deque.peekLast()); //Devuelve null
        System.out.println(deque.pollLast()); //Devuelve null
        System.out.println(deque.getLast()); //Lanza NoSuchElementException porque
        la pila está vacía
        System.out.println(deque.removeLast()); //Lanza NoSuchElementException
        porque la pila está vacía

    }

    public static void main(String[] args) {

        new ShowDeque().show();

    }

}

```

Salida por consola:

```
Matrícula: 7314QWE Número de ruedas: 4, color: verde, velocidad: 0,00
Matrícula: 7314QWE Número de ruedas: 4, color: verde, velocidad: 0,00
Matrícula: 7314QWE Número de ruedas: 4, color: verde, velocidad: 0,00
Matrícula: 1235GTR Número de ruedas: 2, color: rojo, velocidad: 0,00
Matrícula: 9685KMX Número de ruedas: 4, color: azul, velocidad: 0,00
null
null
Exception in thread "main" java.util.NoSuchElementException
    at java.base/java.util.ArrayDeque.getLast(ArrayDeque.java:412)
    at tema11_Colecciones.pilasYColas.ShowDeque.show(ShowDeque.java:21)
    at tema11_Colecciones.pilasYColas.ShowDeque.main(ShowDeque.java:28)
```

12. Clase Collections

La clase `Collections` contiene numerosos métodos estáticos para utilizar con todo tipo de colecciones, como por ejemplo, para añadir, buscar, copiar, reemplazar, ordenar, obtener el máximo o el mínimo, etc.

12 Programación Funcional

12 Programación Funcional

1. Introducción
2. Interfaces funcionales
3. Implementación de interfaces funcionales mediante clases inline anónimas
4. Interfaces funcionales puras predefinidas
5. Interfaces funcionales para tipos primitivos
6. Expresiones lambda
 - 6.1 Sintaxis de la expresión lambda
 - 6.2 Ámbito de una expresión lambda
 - 6.3 Limitaciones de las expresiones lambda
7. Referencias a método
8. Consumer y BiConsumer
9. Predicate y BiPredicate
10. Function
11. Comparator
12. Optional
 - 12.1 Construcción
 - 12.2 Igualdad
 - 12.3 Procesamiento
 - 12.4 Dónde no usar Optional

1. Introducción

La **programación imperativa** es uno de los paradigmas de programación de computadoras más utilizados. Bajo este paradigma, la programación se describe en términos del estado del programa y de sentencias que cambian dicho estado. Java es un lenguaje imperativo, lo que implica que un programa Java está compuesto por una secuencia de instrucciones, que son ejecutadas en el mismo orden en el que se escriben, de manera que al ejecutarla se produce cambios en el estado del programa.

Por su parte, la **programación funcional** es un paradigma de programación alternativo, en el que el resultado de un programa deriva de la aplicación de distintas funciones a la entrada, sin cambiar el estado interno del programa. En la programación funcional los bloques principales de construcción de nuestros programas son las funciones y no los objetos.

Al aplicar programación funcional se produce normalmente un código más corto y más sencillo de entender que aplicando programación imperativa, ya que es más fácil crear abstracciones a través de funciones que a través de interfaces.

Java siempre fue un lenguaje para programación imperativa y de hecho las funciones en Java NO son objetos, por lo que una función no puede pasarse directamente como argumento de otra función para que se ejecute su código. Sin embargo, gracias a las interfaces funcionales y a las clases inline anónimas podíamos superar esta limitación. Pero ¿qué es una interfaz funcional?

2. Interfaces funcionales

Una interfaz funcional es una interfaz que contiene **un único método abstracto**. Esto no quiere decir que no pueda contener otros métodos. De hecho, puede contener:

- Otros métodos `static` (Java 8+).
- Otros métodos `default` (Java 8+).
- Otros métodos `private` (Java 9+)
- Métodos que sobrescriban métodos de la clase `Object`.

A la hora de definir una interfaz funcional, Java 8 proporciona la anotación `@FunctionalInterface`, que informa al compilador de que dicha interfaz es funcional y por tanto tiene un único método abstracto. El objetivo de esta anotación es que se produzca un error de compilación si le añadimos un segundo método abstracto a la interfaz. El uso de esta anotación no se ha establecido como obligatoria para mantener la compatibilidad con el código ya existente, pero sí que está recomendada.

Una **interfaz funcional pura** es aquella en la que las clases que la implementan no almacenan ningún estado, como por ejemplo `Comparator`. Veamos el método `sort` de la interfaz `List<E>` que recibe un objeto de una clase que implementa la interfaz `Comparator`: `default void sort(Comparator<? super E> c)`. El método utiliza el `Comparator` para ordenar la lista llamando al método `compare` de dicho objeto cada vez que debe comparar dos objetos de la lista. Por lo tanto, debemos crear una clase que implemente `Comparator` para determinar cómo se comparan dos elementos:

```
package tema12_ProgramacionFuncional.comparator;

import java.util.Comparator;

public class ListOrder implements Comparator<Integer> {

    @Override
    public int compare(Integer o1, Integer o2) {
        return Integer.compare(o1, o2);
    }

}
```

Así, cuando queramos ordenar una lista de enteros haríamos:

```
package tema12_ProgramacionFuncional.comparator;

import java.util.Arrays;
import java.util.List;

public class Main{

    public void show() {
        List<Integer> list = Arrays.asList(3, 2, 6, 1, 5, 4);
        list.sort(new ListOrder());
        for (Integer i : list) { // 1 2 3 4 5 6
            System.out.printf(" %d ", i);
        }
    }
}
```

```

    }
}

public static void main(String[] args) {

    new Main().show();

}

}

```

3. Implementación de interfaces funcionales mediante clases inline anónimas

El problema del código anterior es que si esta ordenación se hace solamente en dicha ocasión, se ha creado la clase `ListOrder` para un único uso. En ese caso, es más conveniente utilizar una clase inline anónima:

```

package tema12_ProgramacionFuncional;

import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

public class InlineAnonymousClass{

    public void show() {

        List<Integer> list = Arrays.asList(3, 2, 6, 1, 5, 4);
        list.sort(new Comparator<Integer>() {
            @Override
            public int compare(Integer o1, Integer o2) {
                return Integer.compare(o1, o2);
            }
        });
        for (Integer i : list) { // 1 2 3 4 5 6
            System.out.printf(" %d ", i);
        }

    }

    public static void main(String[] args) {

        new InlineAnonymousClass().show();

    }

}

```

Lo que estamos haciendo es indicarle al método `sort()` el código que debe ejecutar para comparar dos objetos. Entonces, ¿no sería más fácil que al método `sort()` le pudiéramos pasar directamente el código que debe ejecutar? El problema es que en Java las funciones no son objetos, por lo que no pueden ser referenciadas mediante una variable o pasadas directamente como argumento.

En realidad, lo que nos interesa es poder establecer *tipos función*, es decir, tipos que representen una función que reciba unos determinados parámetros de algún tipo y que devuelva un valor de retorno de algún tipo. Si existieran los *tipos función*, podríamos definir variables o parámetros de dichos tipos. De hecho hay lenguajes de programación que tienen *tipos función*. En Java, debido a la necesidad de mantener la compatibilidad con versiones anteriores, no existe ninguna sintaxis especial para definir *tipos función* sino que se utilizan las interfaces funcionales para representarlos. Dado que una interfaz funcional solo puede tener un único método abstracto, la firma de dicho método puede ser usado como *tipo función*.

4. Interfaces funcionales puras predefinidas

Java incorpora, a partir de la versión 8, una serie de interfaces funcionales puras predefinidas en el paquete `java.util.function` para permitir la programación funcional en Java:

- `Function<T,R>`: su método abstracto es `R apply(T t)`.
- `UnaryOperator<T>`: es un caso específico de la interfaz funcional `Function<T,T>`, es decir, coinciden el tipo del argumento y el tipo de retorno, por lo que está parametrizada con un único tipo.
- `BiFunction<T,U,R>`: su método abstracto es `R apply(T t,U u)`.
- `BinaryOperator<T>`: es un caso específico de la interfaz funcional `BiFunction<T,T,T>` en el que coinciden el tipo de los dos argumentos recibidos por el método `apply` y el tipo de retorno del mismo. Es por tanto similar a `BiFunction<T,T,T>`. La interfaz funcional `BinaryOperator` está, por tanto, parametrizada con un único tipo.
- `Predicate<T>`: su método abstracto es `boolean test(T t)`.
- `BiPredicate<T,U>`: su método abstracto es `boolean test(T t,U u)`.
- `Consumer<T>`: su método abstracto es `void accept(T t)`.
- `BiConsumer<T,U>`: su método abstracto es `void accept(T t,U u)`.
- `Supplier<T>`: su método abstracto es `T get()`.

Veamos un ejemplo utilizando la interfaz funcional `BinaryOperator<T>`:

```
package tema12_ProgramacionFuncional.binaryOperator;

import java.util.function.BinaryOperator;

public class ShowBinaryOperator{

    public Integer calculate(Integer value1,Integer
value2,BinaryOperator<Integer> binaryOperation) {
        return binaryOperation.apply(value1,value2);
    }

}
```

```

package tema12_ProgramacionFuncional.binaryOperator;

import java.util.function.BinaryOperator;

public class Main{

    public void show() {
        ShowBinaryOperator binOper = new ShowBinaryOperator();
        system.out.printf("12 + 6 = %d\n", binOper.calculate(12, 6, new
BinaryOperator<Integer>() {

            @Override
            public Integer apply(Integer t, Integer u) {
                return t + u;
            }

        }));
        system.out.printf("12 - 6 = %d\n", binOper.calculate(12, 6, new
BinaryOperator<Integer>() {

            @Override
            public Integer apply(Integer t, Integer u) {
                return t - u;
            }

        }));
        system.out.printf("12 / 6 = %d\n", binOper.calculate(12, 6, new
BinaryOperator<Integer>() {

            @Override
            public Integer apply(Integer t, Integer u) {
                return t / u;
            }

        }));
        system.out.printf("12 * 6 = %d\n", binOper.calculate(12, 6, new
BinaryOperator<Integer>() {

            @Override
            public Integer apply(Integer t, Integer u) {
                return t * u;
            }

        }));
    }

    public static void main(String[] args) {

        new Main().show();

    }

}

```

Salida por consola:

```
12 + 6 = 18
12 - 6 = 6
12 / 6 = 2
12 * 6 = 72
```

5. Interfaces funcionales para tipos primitivos

Como no podemos usar la parametrización de clases e interfaces con los tipos primitivos, el paquete `java.util.function` define también una serie de interfaces funcionales similares a las explicadas anteriormente pero específicas para los tipos primitivos:

- Para el tipo primitivo **boolean**: `BooleanSupplier`
- Para el tipo primitivo **double**: `DoubleBinaryOperator`, `DoubleConsumer`, `DoubleFunction`, `DoublePredicate`, `DoubleSupplier`, `DoubleToIntFunction`, `DoubleToLongFunction`, `DoubleUnaryOperator`, `ToDoubleBiFunction`, `ToDoubleFunction`, `ObjDoubleConsumer`.
- Para el tipo primitivo **int**: `IntBinaryOperator`, `IntConsumer`, `IntFunction`, `IntPredicate`, `IntSupplier`, `IntToDoubleFunction`, `IntToLongFunction`, `IntUnaryOperator`, `ToIntBiFunction`, `ToIntFunction`, `ObjIntConsumer`.
- Para el tipo primitivo **long**: `LongBinaryOperator`, `LongConsumer`, `LongFunction`, `LongPredicate`, `LongSupplier`, `LongToDoubleFunction`, `LongToIntFunction`, `LongUnaryOperator`, `ToLongBiFunction`, `ToLongFunction`, `ObjLongConsumer`.

Además, la mayoría de las interfaces vistas hasta ahora incluyen métodos cuyo nombre incluye `ToTipo` que retornan objetos de interfaces funcionales para tipos primitivos.

6. Expresiones lambda

Con objeto de incorporar a Java funcionalidades propias de la programación funcional, Java 8 trajo consigo dos nuevas sintaxis para representar interfaces funcionales: expresiones lambda (*lambda expressions*) y referencias a método (*method references*).

Una **expresión lambda** es una nueva sintaxis con la que representar la implementación del método abstracto de interfaces funcionales. De esta manera, podemos escribir el código de apartados anteriores mediante una expresión lambda, haciéndolo mucho más legible.

El nuevo operador para las expresiones lambda se denomina **operador lambda** y tiene la forma de flecha `->`. Divide la expresión lambda en dos partes: la parte izquierda especifica los parámetros necesarios y la parte derecha contiene el cuerpo de la expresión. Este cuerpo puede estar compuesto por una única expresión o puede ser un bloque de código. Cuando es una única expresión se denomina **lambda de expresión** y cuando es un bloque de código se denomina **lambda de bloque**.

Debemos tener en cuenta que cuando se especifica una expresión *lambda*, no indicamos nada sobre la interfaz funcional a la queremos aplicarla, es decir, dependiendo de donde se esté usando la expresión *lambda*, el compilador deberá determinar si la firma de la expresión *lambda* coincide con la firma del método abstracto de la correspondiente interfaz funcional. Si la expresión *lambda* no incluye los tipos de los parámetros, el compilador tratará de inferirlos a partir de los tipos de los parámetros del método abstracto de la interfaz funcional.

El ejemplo anterior de la interfaz `BinaryOperator<Integer>`, si lo realizamos con una expresión lambda, resultaría de la siguiente manera:

```
package tema12_ProgramacionFuncional.binaryOperator;

public class Main2 {

    public void show() {
        ShowBinaryOperator binOper = new ShowBinaryOperator();
        System.out.printf("12 + 6 = %d\n", binOper.calculate(12, 6, (t, u) -> t +
u));
        System.out.printf("12 - 6 = %d\n", binOper.calculate(12, 6, (t, u) -> t -
u));
        System.out.printf("12 / 6 = %d\n", binOper.calculate(12, 6, (t, u) -> t /
u));
        System.out.printf("12 * 6 = %d\n", binOper.calculate(12, 6, (t, u) -> t *
u));
    }

    public static void main(String[] args) {

        new Main2().show();
    }
}
```

Salida por consola:

```
12 + 6 = 18
12 - 6 = 6
12 / 6 = 2
12 * 6 = 72
```

Veamos el ejemplo de ordenación de la lista hecho de las dos maneras, con una clase inline anónima y con una expresión lambda:

```
package tema12_ProgramacionFuncional;

import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

public class LambdaExpression {

    public void show() {

        List<Integer> list = Arrays.asList(3, 2, 6, 1, 5, 4);
        list.sort(new Comparator<Integer>() {//Clase inline anónima
            @Override
            public int compare(Integer o1, Integer o2) {
                return Integer.compare(o1, o2);
            }
        });
    }
}
```

```

    });
    for (Integer i : list) { // 1 2 3 4 5 6
        System.out.printf(" %d ", i);

    }
    System.out.println();
    list.sort((o1, o2) -> Integer.compare(o1, o2)); // Expresión lambda
    for (Integer i : list) { // 1 2 3 4 5 6
        System.out.printf(" %d ", i);

    }

}

public static void main(String[] args) {

    new LambdaExpression().show();

}

}

```

Aún así, debemos tener en cuenta que un objeto de una clase inline anónima y una expresión *lambda* no son lo mismo, porque una expresión *lambda* no crea ninguna clase adicional y además en una expresión *lambda* no se puede almacenar ningún estado, mientras que en una clase anónima inline sí. Un objeto de una clase anónima inline genera un archivo de clase independiente durante la compilación que aumenta el tamaño del archivo *jar*. Sin embargo, una expresión *lambda* se convierte en un simple método privado. En una *lambda*, *this* representa la clase actual desde la que se está usando la *lambda*. En el caso de una clase anónima, *this* representa ese objeto de la clase anónima en particular.

Como vemos, las expresiones *lambda* son muy útiles para simplificar el código, pero presentan un problema: para que podamos usar esta nueva sintaxis de expresión *lambda* es necesario que la interfaz contra la que la usemos sea una interfaz funcional, es decir, que sólo contenga un único método abstracto, porque si tuviera por ejemplo dos métodos, ¿el código proporcionado mediante la expresión *lambda* cuándo se debería ejecutar, cuando se ejecute uno o cuando se ejecute el otro? Por este motivo las expresiones *lambda* solo se pueden usar con interfaces funcionales.

Se puede almacenar una expresión *lambda* en una variable cuyo tipo corresponda a una interfaz funcional compatible con dicha *lambda*, es decir, cuya firma del método abstracto de la interfaz sea compatible con la expresión *lambda*. Por ejemplo:

```

Comparator<Integer> comparador = (o1, o2) -> Integer.compare(o1, o2);
BinaryOperator<Integer> operacionBinaria = (o1, o2) -> o1 + o2;

```

6.1 Sintaxis de la expresión lambda

Veamos la **sintaxis** de la expresión *lambda*:

```
(tipo param1, tipo param2, ...) -> {  
    cuerpoExpresionLambda  
    return valorRetorno  
}
```

Podemos omitir el tipo de dato de cada parámetro, siempre y cuando el compilador pueda inferirlos (deducirlos) a partir del contexto, es decir, a partir de los tipos de los parámetros del método abstracto de la interfaz funcional para la que se está usando.

El cuerpo de las expresiones *lambda* puede contener los mismos tipos de sentencias que cualquier otra función, como sentencias condicionales, iterativas o `try catch`.

Si se especifica un único parámetro y no se especifica el tipo de éste sino que es inferido, podemos omitir los paréntesis. Por ejemplo:

```
x -> x + x;
```

Si no se especifica ningún parámetro, es obligatorio poner los paréntesis. Por ejemplo:

```
() -> System.out.println("Hola mundo");
```

En el cuerpo podemos omitir las llaves si éste contiene una única línea de código. Si es una única expresión, ésta será evaluada y la expresión *lambda* retornará el valor obtenido.

Si el cuerpo contiene más de una sentencia y la expresión *lambda* debe retornar un valor, entonces debemos usar una sentencia `return valor`.

Si el cuerpo contiene una sentencia `return valor`, forzosamente debemos poner las llaves, incluso si el cuerpo contiene una única sentencia, ya que `return` no es una expresión.

Una expresión *lambda* se puede usar como argumento de un parámetro de tipo interfaz funcional y como valor de retorno de una función cuyo tipo de retorno sea una interfaz funcional. Sin embargo, habrá ocasiones donde debemos realizar un *cast* explícitamente para indicar la interfaz funcional a la que queremos aplicar una determinada expresión *lambda*.

6.2 Ámbito de una expresión lambda

Una expresión lambda puede acceder a las variables `static` definidas en el ámbito en el que la expresión *lambda* es usada. También puede acceder a las variables locales pero que sean eficazmente finales, es decir, variables cuyo valor no cambia una vez asignado. Estas variables no tienen necesariamente que estar definidas como `final`. Una expresión lambda también tiene acceso a `this`, lo que hace referencia a la instancia de invocación de la clase contenedora de la expresión lambda.

Si en el cuerpo de una expresión *lambda* con más de una sentencia definimos una variable local, debemos tener en cuenta que dicha variable tendrá como ámbito el correspondiente a donde se ha definido la expresión lambda, ya que la expresión *lambda* no define su propio ámbito independiente. Si ya existiera una variable con el mismo nombre en dicho ámbito se produciría un error de compilación. Por ejemplo:


```

int z = 2;
BinaryOperator<Integer> operacion = (x, y) -> {
    // ERROR: z ya está definida en el ámbito
    int z = 4;
    System.out.println(x + z);
};

```

Una expresión lambda puede generar una excepción. No obstante, si genera una excepción comprobada, esta tendrá que ser compatible con la excepción (o excepciones) indicadas en la cláusula `throws` del método abstracto de la interfaz funcional. Veamos un ejemplo:

```

package tema12_ProgramacionFuncional;

public interface FunctionalInterface {
    int ioAction() throws Exception;
}

```

```

package tema12_ProgramacionFuncional;

import java.util.Scanner;

public class LambdaException {

    @SuppressWarnings("resource")
    public void show() {

        FunctionalInterface fi = () -> {
            Scanner keyboard = new Scanner(System.in);
            int num = keyboard.nextInt();
            return num;
        };
        try {
            System.out.printf("Introduce un número:");
            System.out.println(method(fi));
        } catch (Exception e) {
            System.err.println("Error en la lectura");
        }

    }

    public int method(FunctionalInterface fi) throws Exception {
        return fi.ioAction();
    }

    public static void main(String[] args) {

        new LambdaException().show();

    }

}

```

6.3 Limitaciones de las expresiones lambda

El código de una expresión *lambda* se convierte en el código del método abstracto de la interfaz funcional que implementa. Por tanto, las expresiones *lambda* no sirven para sobrescribir la implementación por defecto de un método *default* de la interfaz, sino que debe tratarse de un método abstracto. De hecho si la interfaz solo tiene un método *default*, no será considerada una interfaz funcional. Si nos vemos en la obligación de sobrescribir un método *default* de una interfaz, entonces tendremos que usar una clase anónima *inline*.

Por otro lado, una expresión *lambda* no es consciente de qué interfaz funcional concreta está implementando, por lo que no puede llamar a su vez a métodos privados ni *default* de la interfaz.

Finalmente, las expresiones *lambda* no pueden usarse con clases abstractas que tengan un único método abstracto, solo se pueden usar con interfaces funcionales.

7. Referencias a método

Con objeto de incorporar a Java funcionalidades propias de la programación funcional, Java 8 trajo consigo dos nuevas sintaxis para representar interfaces funcionales: expresiones lambda (*lambda expressions*) y referencias a método (*method references*).

Una referencia a método (*method reference*) permite hacer referencia a un método sin ejecutarlo. Las posibles sintaxis son:

- El nombre de la clase y el método: `NombreClase::nombreMétodo`. Ejemplos:
 - `v -> Math.sqrt(v)` equivaldría a `Math::sqrt`: el parámetro del lambda se pasa como argumento en la llamada del método.
 - `(o1, o2) -> Integer.compare(o1, o2)` equivaldría a `Integer::compare`: los parámetros del lambda se pasan como argumento en la llamada del método.
 - `(cadena1, cadena2) -> cadena1.compareToIgnoreCase(cadena2)` equivaldría a `String::compareToIgnoreCase`: el primer parámetro de la expresión lambda es quien ejecuta el método y el resto de parámetros se pasan como argumentos en la llamada del método.
 - `persona -> persona.getNombre()` equivaldría a `Persona::getNombre`: el parámetro de la expresión lambda es quien ejecuta el método.
 - `() -> new TreeMap<>()` equivaldría a `TreeMap::new`: este tipo se conoce como referencia a constructor, que emplearemos cuando queramos que se llame al método constructor de una clase.
- Una referencia de un objeto y el método: `refObj::nombreMétodo`. Ejemplos:
 - `n -> System.out.println(n)` equivaldría a `System.out::println`: el parámetro del lambda se pasa como argumento en la llamada del método.
 - `empleado -> jefe.comparaSalarioCon(empleado)` equivaldría a `jefe::comparaSalarioCon`: un objeto ajeno a la expresión lambda es quien ejecuta el método y dicho método recibe como argumento el (o los) parámetro(s) de la expresión lambda.
 - `i -> new int[i]` equivaldría a `int[]::new`: se llama al constructor de un array.
- Sintaxis para métodos genéricos:
 - `NombreClase::<T>nombreMétodo`

- `refObj::<T>nombreMétodo`

8. Consumer y BiConsumer

- `Consumer<T>`: su método abstracto es `void accept(T t)`.
- `BiConsumer<T,U>`: su método abstracto es `void accept(T t,U u)`.

La interfaz `Consumer` es empleada por el método `forEach(Consumer<T> action)` de la interfaz `Iterable`, que ejecuta la acción indicada sobre cada elemento del iterable.

```
package tema12_ProgramacionFuncional;

import java.util.List;

public class InterfaceConsumer {

    public void show() {

        List<Integer> list = List.of(3, 2, 6, 1, 5, 4);
        list.forEach(System.out::println);

    }

    public static void main(String[] args) {

        new InterfaceConsumer().show();

    }

}
```

Salida por consola:

```
3
2
6
1
5
4
```

En este ejemplo, se ejecuta la acción de mostrar por consola una línea con cada elemento de la lista.

La interfaz funcional `BiConsumer` es similar a `Consumer` pero su método recibe dos argumentos, uno de tipo `T` y otro de tipo `U` y no retorna nada: `void accept(T t, U u)`

```
package tema12_ProgramacionFuncional;

import java.util.HashMap;
import java.util.Map;

public class InterfaceBiConsumer {

    public void show() {
```

```

    Map<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < 5; i++) {
        map.put(i, i * i);
    }
    map.forEach((k, v) -> System.out.printf("Clave:%d Valor:%d\n", k, v));

}

public static void main(String[] args) {

    new InterfaceBiConsumer().show();

}

}

```

Salida por consola:

```

Clave:0 Valor:0
Clave:1 Valor:1
Clave:2 Valor:4
Clave:3 Valor:9
Clave:4 Valor:16

```

La interfaz funcional `Consumer` posee un método *default* llamado `andThen(Consumer<T> after)` que llama al método `accept` del consumidor recibido después de haber llamado a su propio `accept`. Gracias a este método, podemos tener una serie de objetos `Consumer` predefinidos y encadenarlos de la forma que nos interese. Veamos un ejemplo:

```

package tema12_ProgramacionFuncional;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.function.Consumer;

public class ConsumerAndThen {

    public void show() {

        List<Integer> list = List.of(3, 2, 6, 1, 5, 4);
        List<Integer> listCopy = new ArrayList<>();
        Consumer<Integer> copy = listCopy::add;
        Consumer<Integer> show = System.out::println;
        list.forEach(copy.andThen(show));
        listCopy.forEach(show);

    }

    public static void main(String[] args) {

        new ConsumerAndThen().show();

    }
}

```

```
}  
  
}
```

Salida por consola:

```
3  
2  
6  
1  
5  
4  
3  
2  
6  
1  
5  
4
```

9. Predicate y BiPredicate

- `Predicate<T>`: su método abstracto es `boolean test(T t)`.
- `BiPredicate<T>`: su método abstracto es `boolean test(T t, U u)`.

Las interfaces funcionales `Predicate` y `BiPredicate` poseen métodos *default* que retornan un nuevo objeto que implementa la misma interfaz y que permiten componer predicados mediante operaciones lógicas, como `or(otherPredicate)`, `and(otherPredicate)` o `negate(otherPredicate)`. El orden en el que se ejecutarán serán el orden en el que aparecen en la composición, es decir, no existe una prioridad preestablecida como con los operadores lógicos.

Veamos un ejemplo utilizando el método `removeIf` de la interfaz `Collection` que recibe un *Predicate* por parámetro: `default boolean removeIf(Predicate<? super E> filter)`. Dicho método elimina de la colección aquellos elementos que cumplan el *Predicate*:

```
package tema12_ProgramacionFuncional;  
  
import java.util.ArrayList;  
import java.util.Arrays;  
import java.util.List;  
import java.util.function.Predicate;  
  
public class PredicateComposition {  
  
    public void show() {  
  
        List<Integer> list = new ArrayList<>(List.of(3, 2, 6, 1, 5, 4));  
        Predicate<Integer> esPar = n -> n % 2 == 0;  
        Predicate<Integer> mayorQue3 = n -> n > 3;  
        list.removeIf(esPar.and(mayorQue3));  
        list.forEach(System.out::println);  
  
    }  
}
```

```

    public static void main(String[] args) {

        new PredicateComposition().show();

    }

}

```

Salida por consola:

```

3
2
1
5

```

La interfaz `Predicate` tiene también un método estático factoría `Predicate.isEqual(Object o)` que retorna el predicado correspondiente a comprobar si un elemento es igual a otro objeto. Internamente simplemente se llamará al método `equals`:

```

package tema12_ProgramacionFuncional;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.function.Predicate;

public class PredicateIsEqual {

    public void show() {

        List<Integer> list = new ArrayList<>(List.of(5, 3, 2, 5, 6, 1, 5, 4));
        list.removeIf(Predicate.isEqual(5));
        list.forEach(System.out::println);

    }

    public static void main(String[] args) {

        new PredicateIsEqual().show();

    }

}

```

Salida por consola:

```

3
2
6
1
4

```

10. Function

`Function<T,R>`: su método abstracto es `R apply(T t)`.

Veamos un ejemplo de utilización en el método `transform` de las cadenas: el método toma una función `Function<String, String>` como argumento. Esta función se aplica a la cadena original y el resultado de la función se devuelve como una nueva cadena. Este método es útil para encadenar transformaciones o aplicar funciones más complejas a una cadena de texto:

```
package tema12_ProgramacionFuncional;

public class InterfaceFunction {

    public void show() {

        String text = "    Estoy aprendiendo a programar";
        String upperCaseText, replacedText, transformedText;

        // Usar transform para convertir el texto a mayúsculas
        upperCaseText = text.transform(String::toUpperCase);
        System.out.println(upperCaseText);

        // Usar transform para reemplazar "programar" por "probar"
        replacedText = text.transform(t -> t.replace("programar", "probar"));
        System.out.println(replacedText);

        // Usar transform para eliminar espacios y convertir a mayúsculas
        transformedText = text
                                .transform(t -> t.trim()) // Elimina los espacios en
blanco
                                .transform(t -> t.toUpperCase()); // Convierte a
mayúsculas
        System.out.println(transformedText);
    }

    public static void main(String[] args) {

        new InterfaceFunction().show();
    }
}
```

Veamos un ejemplo de utilización en el método `computeIfAbsent` de los mapas: `default V computeIfAbsent(K key, Function<? super K, ? extends V> mappingFunction)`: la función `mappingFunction` se ejecuta solo si la clave especificada no está presente en el mapa o si su valor asociado es `null`. Lo que hace la función es tomar la clave como entrada y devolver el valor que se va a asignar a la clave. En caso de que la clave ya exista en el mapa y su valor no sea `null`, la función no se ejecutará y el valor existente asociado con la clave se mantendrá sin cambios.

```
package tema12_ProgramacionFuncional;
```

```

import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

public class InterfaceFunction2 {

    public void show() {

        Function<Integer, Integer> elevarAlCuadrado = n -> n * n;
        Map<Integer, Integer> map = new HashMap<>();
        map.put(0, null); //se va a ejecutar la Function porque su valor asociado
        es null
        map.put(1, 0); //no se va a ejecutar la Function porque tiene un valor
        distinto de null
        for (int i = 0; i < 10; i++) {
            map.computeIfAbsent(i, elevarAlCuadrado);
        }
        map.forEach((k, v) -> System.out.printf("Clave:%d Valor:%d\n", k, v));

    }

    public static void main(String[] args) {

        new InterfaceFunction2().show();

    }

}

```

Salida por consola:

```

Clave:0 Valor:0
Clave:1 Valor:0
Clave:2 Valor:4
Clave:3 Valor:9
Clave:4 Valor:16
Clave:5 Valor:25
Clave:6 Valor:36
Clave:7 Valor:49
Clave:8 Valor:64
Clave:9 Valor:81

```

La interfaz funcional `Function` posee un método *default* llamado `andThen(afterFunction)` que permite que después de la ejecución del método `apply` de la *function* original, su resultado se pase como valor de entrada del método `apply` del objeto `Function` pasado como argumento, retornando el objeto `Function` correspondiente a la cadena de operaciones de transformación. Veamos un ejemplo:

```

package tema12_ProgramacionFuncional;

import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

```



```

public class FunctionAndThen {

    public void show() {

        Function<Integer, Integer> elevarAlCuadrado = n -> n * n;
        Function<Integer, String> aCadena = String::valueOf;
        Map<Integer, String> map = new HashMap<>();
        for (int i = 0; i < 10; i++) {
            map.computeIfAbsent(i, elevarAlCuadrado.andThen(aCadena));
        }
        map.forEach((k, v) -> System.out.printf("Clave:%d Valor:%s\n", k, v));

    }

    public static void main(String[] args) {

        new FunctionAndThen().show();

    }

}

```

Salida por consola:

```

Clave:0 Valor:0
Clave:1 Valor:1
Clave:2 Valor:4
Clave:3 Valor:9
Clave:4 Valor:16
Clave:5 Valor:25
Clave:6 Valor:36
Clave:7 Valor:49
Clave:8 Valor:64
Clave:9 Valor:81

```

Muy parecido al anterior es el método *default* `compose(beforeFunction)` que primero ejecuta el método `apply` del objeto `Function` recibido y después el método `apply` del objeto `Function` original, es decir, se ejecutan en orden inverso al de `andThen`. Gracias a estos dos métodos, podemos tener una serie de objetos `Function` predefinidos y encadenarlos de la forma que nos interese.

```

package tema12_ProgramacionFuncional;

import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

public class FunctionCompose {

    public void show() {

        Function<Integer, Integer> elevarAlCuadrado = n -> n * n;
        Function<Integer, String> aCadena = String::valueOf;
        Map<Integer, String> map = new HashMap<>();
    }
}

```

```

        for (int i = 0; i < 10; i++) {
            map.computeIfAbsent(i, aCadena.compose(elevarAlCuadrado));
        }
        map.forEach((k, v) -> System.out.printf("Clave:%d Valor:%s\n", k, v));
    }

    public static void main(String[] args) {

        new FunctionCompose().show();

    }
}

```

Salida por consola:

```

Clave:0 Valor:0
Clave:1 Valor:1
Clave:2 Valor:4
Clave:3 Valor:9
Clave:4 Valor:16
Clave:5 Valor:25
Clave:6 Valor:36
Clave:7 Valor:49
Clave:8 Valor:64
Clave:9 Valor:81

```

11. Comparator

La interfaz funcional `Comparator` (comparador) contiene el método abstracto `int compare(T o1, T o2)` que recibe dos valores de tipo `T` y retorna un entero que vale:

- 0 si o1 es igual a o2.
- Menor que 0 si o1 es menor que o2.
- Mayor que 0 si o1 es mayor que o2.

La interfaz `Comparator` ya existía en Java 7 y de hecho no se encuentra en el paquete `java.util.function`, sino directamente en `java.util`. Esta interfaz posee una serie de métodos estáticos factoría que retornan objetos `Comparator` correspondientes a los casos más habituales de comparación:

- `static <T extends Comparable<? super T>> Comparator<T> naturalOrder()`: devuelve un objeto `Comparator` para ordenar por el orden natural. Pero, ¿qué considera Java como orden natural? El orden indicado en la implementación de `Comparable`.

```

package tema12_ProgramacionFuncional.comparator;

import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

public class NaturalOrder {

```

```

public void show() {

    List<Integer> list = Arrays.asList(3, 2, 6, 1, 5, 4);
    list.sort(Comparator.naturalOrder());
    list.forEach(System.out::println);

}

public static void main(String[] args) {

    new NaturalOrder().show();

}

}

```

Salida por consola:

```

1
2
3
4
5
6

```

- `static <T extends Comparable<? super T>>Comparator<T> reverseOrder()`: ordena por el orden natural inverso.

```

package tema12_ProgramacionFuncional.comparator;

import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

public class ReverseOrder {

    public void show() {

        List<Integer> list = Arrays.asList(3, 2, 6, 1, 5, 4);
        list.sort(Comparator.reverseOrder());
        list.forEach(System.out::println);

    }

    public static void main(String[] args) {

        new ReverseOrder().show();

    }

}

```

Salida por consola:

```
6
5
4
3
2
1
```

- `static <T> Comparator<T> nullsFirst(Comparator<? super T> comparator)`: recibe un comparador y retorna un nuevo comparador según el cual los elementos `null` precederán a los que no lo sean, que serán ordenados atendiendo al comparador recibido. También tenemos `nullsLast`, similar al anterior pero los elementos que sean `null` se sitúan al final.

```
package tema12_ProgramacionFuncional.comparator;

import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

public class NullsFirst {

    public void show() {

        List<Integer> list = Arrays.asList(3, 2, null, 6, 1, 5, 4, null);
        list.sort(Comparator.nullsFirst(Comparator.naturalOrder()));
        System.out.println("nullsFirst:");
        list.forEach(System.out::println);
        list.sort(Comparator.nullsLast(Comparator.naturalOrder()));
        System.out.println("nullsLast:");
        list.forEach(System.out::println);

    }

    public static void main(String[] args) {

        new NullsFirst().show();

    }

}
```

Salida por consola:

```
nullsFirst:
null
null
1
2
3
4
5
6
nullsLast:
1
2
```

```
3
4
5
6
null
null
```

- `static <T, U extends Comparable<? super U>> Comparator<T> comparing(Function<? super T, ? extends U> keyExtractor)`: recibe una función que debe retornar el valor por el que debe comparar el comparador retornado. Este método es muy útil si queremos ordenar una lista de objetos por un determinado campo.

```
package tema12_ProgramacionFuncional.comparator;

public class Vehicle {

    private String registration;
    private int wheelCount;
    private double speed;
    private String colour;

    public Vehicle(String registration, int wheelCount, String colour) {
        this.registration = registration;
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public String getRegistration() {
        return registration;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }
}
```

```

@Override
public String toString() {
    return String.format("Matrícula: %s Número de ruedas: %d, color: %s,
    velocidad: %.2f", registration, wheelCount, colour, speed);
}
}

```

```

package tema12_ProgramacionFuncional.comparator;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class Comparing {

    public void show() {

        List<Vehicle> list = new ArrayList<>();
        list.add(new Vehicle("1705UBG", 4, "blanco"));
        list.add(new Vehicle("9685KMX", 4, "azul"));
        list.add(new Vehicle("1235GTR", 2, "rojo"));
        list.add(new Vehicle("7314QWE", 4, "verde"));
        list.add(new Vehicle("3495JZA", 2, "blanco"));
        list.add(new Vehicle("5930POI", 2, "negro"));
        list.sort(Comparator.comparing(Vehicle::getColour));
        list.forEach(System.out::println);

    }

    public static void main(String[] args) {

        new Comparing().show();

    }

}

```

Salida por consola:

```

Matrícula: 9685KMX Número de ruedas: 4, color: azul, velocidad: 0,00
Matrícula: 1705UBG Número de ruedas: 4, color: blanco, velocidad: 0,00
Matrícula: 3495JZA Número de ruedas: 2, color: blanco, velocidad: 0,00
Matrícula: 5930POI Número de ruedas: 2, color: negro, velocidad: 0,00
Matrícula: 1235GTR Número de ruedas: 2, color: rojo, velocidad: 0,00
Matrícula: 7314QWE Número de ruedas: 4, color: verde, velocidad: 0,00

```

Hay versiones de este método estático factoría para cuando el valor por el que se debe ordenar es de un tipo primitivo, como `comparingInt`, `comparingLong` o `comparingDouble`.

La interfaz funcional `Comparator` también incorpora una serie de métodos *default* que nos permiten encadenar comparadores:

- `default Comparator<T> reversed()` : sirve para obtener el orden inverso al del comparador original.

```
package tema12_ProgramacionFuncional.comparator;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class Reversed {

    public void show() {

        List<Vehicle> list = new ArrayList<>();
        list.add(new Vehicle("1705UBG", 4, "blanco"));
        list.add(new Vehicle("9685KMX", 4, "azul"));
        list.add(new Vehicle("1235GTR", 2, "rojo"));
        list.add(new Vehicle("7314QWE", 4, "verde"));
        list.add(new Vehicle("3495JZA", 2, "blanco"));
        list.add(new Vehicle("5930POI", 2, "negro"));
        list.sort(Comparator.comparing(Vehicle::getColour).reversed());
        list.forEach(System.out::println);

    }

    public static void main(String[] args) {

        new Reversed().show();

    }

}
```

Salida por consola:

```
Matrícula: 7314QWE Número de ruedas: 4, color: verde, velocidad: 0,00
Matrícula: 1235GTR Número de ruedas: 2, color: rojo, velocidad: 0,00
Matrícula: 5930POI Número de ruedas: 2, color: negro, velocidad: 0,00
Matrícula: 1705UBG Número de ruedas: 4, color: blanco, velocidad: 0,00
Matrícula: 3495JZA Número de ruedas: 2, color: blanco, velocidad: 0,00
Matrícula: 9685KMX Número de ruedas: 4, color: azul, velocidad: 0,00
```

- `default <U extends Comparable<? super U>> Comparator<T>`
`thenComparing(Function<? super T,? extends U> keyExtractor)` : recibe una función con la que indicar el valor por el que comparar si con el comparador original los elementos son iguales.

```
package tema12_ProgramacionFuncional.comparator;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
```

```

public class ThenComparingFunction {

    public void show() {

        List<Vehicle> list = new ArrayList<>();
        list.add(new Vehicle("1705UBG", 4, "blanco"));
        list.add(new Vehicle("9685KMX", 4, "azul"));
        list.add(new Vehicle("1235GTR", 2, "rojo"));
        list.add(new Vehicle("7314QWE", 4, "verde"));
        list.add(new Vehicle("3495JZA", 2, "blanco"));
        list.add(new Vehicle("5930POI", 2, "negro"));

        list.sort(Comparator.comparing(Vehicle::getColour).thenComparing(Vehicle::getWheelCount));
        list.forEach(System.out::println);

    }

    public static void main(String[] args) {

        new ThenComparingFunction().show();

    }

}

```

Salida por consola:

```

Matrícula: 9685KMX Número de ruedas: 4, color: azul, velocidad: 0,00
Matrícula: 3495JZA Número de ruedas: 2, color: blanco, velocidad: 0,00
Matrícula: 1705UBG Número de ruedas: 4, color: blanco, velocidad: 0,00
Matrícula: 5930POI Número de ruedas: 2, color: negro, velocidad: 0,00
Matrícula: 1235GTR Número de ruedas: 2, color: rojo, velocidad: 0,00
Matrícula: 7314QWE Número de ruedas: 4, color: verde, velocidad: 0,00

```

Existen versiones específicas de este método para cuando la función retorna un tipo primitivo, como `thenComparingInt`, `thenComparingLong` y `thenComparingDouble`.

12. Optional

La clase `Optional<T>` es un *wrapper* alrededor de un valor que puede estar presente o no. El objetivo de esta clase es servir básicamente como tipo de retorno de aquellos métodos que pueden retornar un valor o no.

Por ejemplo, supongamos que queremos realizar un método que retorne la posición en la que se encuentra un determinado elemento de una lista de enteros. La pregunta que inmediatamente nos haríamos como desarrolladores sería ¿qué debo hacer si el elemento a buscar, que ha sido pasado como argumento del método, no se encuentra en la lista? ¿Debo lanzar una excepción? No puedo retornar `false` porque el método retorna un entero. ¿Debo retornar un valor especial como por ejemplo `-1`? ¿Retorno como valor especial el valor `null`?

Tradicionalmente los desarrolladores han resuelto esta situación de distintas maneras y todas tienen sus inconvenientes. Por un lado, lanzar una excepción parece excesivo porque no es un error del programa y al cliente del método simplemente hay que informarle de alguna manera de que no se ha encontrado el elemento.

Por otra parte, retornar un valor especial tiene el inconveniente de que obliga a que el cliente conozca dicho valor especial y además debe acordarse de comprobar que el valor retornado no es el valor especial si quiere usarlo. En cierta manera estamos dándole al cliente la responsabilidad de la comprobación pero no estamos obligándole a ello. La consecuencia es que si el desarrollador del código cliente olvida realizar la comprobación, estará usando un valor no válido. Este hecho se convierte en más peligroso aún si el valor retornado es `null`, porque si olvida realizar la comprobación y más adelante en el código se trata de acceder a una propiedad del objeto retornado por el método, se produciría una excepción `NullPointerException`.

Entonces, ¿cuál es la solución? La solución propuesta por Java 8 es que el método no retorne directamente un objeto de la clase `T`, sino un `Optional<T>`. La ventaja de esta solución es que si el código cliente quiere acceder al objeto real debe forzosamente comprobar si el *Optional* tiene un valor presente o no, de manera que pueda extraerse.

Es muy importante resaltar que **no es posible tener un *Optional* cuyo valor contenido sea null**.

12.1 Construcción

La clase `Optional<T>` es una clase inmutable, por lo que no proporciona métodos *setter* que permitan cambiar el valor que contiene. Además, el proceso de construcción de un *Optional* no se realiza a través de un constructor, sino usando alguno de los métodos estáticos factoría que se indican a continuación:

- `static <T> Optional<T> empty()`: retorna un *Optional* que no contiene valor.
- `static <T> Optional<T> of(T value)`: retorna un *Optional* que contiene el valor pasado como argumento. Si tratamos de pasar el valor `null` a dicho método, se producirá una excepción.
- `static <T> Optional<T> ofNullable(T value)`: retorna un *Optional* que contiene el valor pasado como argumento. A diferencia del método anterior, si tratamos de pasar el valor `null`, el método retorna un *Optional* vacío.

Ejemplo:

```
package tema12_ProgramacionFuncional.optional;

import java.util.Optional;

public class Construction {

    public void show() {

        Optional<Integer> optional1 = Optional.empty(); // Crea un optional sin
valor.
        System.out.println(optional1);
        Optional<Integer> optional2 = Optional.of(1000); // Crea un optional con
valor 1000
        System.out.println(optional2);
    }
}
```

```

// Retorna un Optional vacío si el valor pasado como argumento es null:
Optional<Integer> optional3 =
Optional.ofNullable(methodWhichCanReturnNull(5));
System.out.println(optional3);
Optional<Integer> optional4 =
Optional.ofNullable(methodWhichCanReturnNull(2));
System.out.println(optional4);

Optional<Integer> optional5;
try {
    optional5 = Optional.of(null);
    System.out.println(optional5);
} catch (NullPointerException e) {
    System.err.println("Se ha lanzado la excepción
NullPointerException");
}

}

public Integer methodWhichCanReturnNull(int num) {

    if (num >= 5) {
        return num;
    } else {
        return null;
    }

}

public static void main(String[] args) {

    new Construction().show();

}

}

```

Salida por consola:

```

Optional.empty
Optional[1000]
Optional[5]
Optional.empty
Se ha lanzado la excepción NullPointerException

```

12.2 Igualdad

Podemos comparar los valores de dos *Optional* con el método `boolean equals(Object obj)`: compara los valores de los dos *Optional*. Dos *Optional* vacíos de distinto tipo son considerados iguales.

Ejemplo:

```

package tema12_ProgramacionFuncional.optional;

```

```
import java.util.Optional;

public class Equality {

    public void show() {

        Optional<Integer> optional1 = Optional.of(1000); //Crea un optional con
valor 1000
        Optional<Integer> optional2 = Optional.of(1000);
        Optional<Integer> optional3 = Optional.empty();
        Optional<String> optional4 = Optional.empty();

        System.out.println(optional1.equals(optional2)); //Comprueba si tienen el
mismo valor: true
        System.out.println(optional1 == optional2); //Comprueba si son el mismo
objeto: false
        System.out.println(optional3.equals(optional4)); //Dos Optional vacíos de
distinto tipo son considerados iguales: true
        System.out.println(optional3.equals(null)); //False

    }

    public static void main(String[] args) {

        new Equality().show();

    }

}
```

Salida por consola:

```
true
false
true
false
```

12.3 Procesamiento

Supongamos que queremos usar el método estático `Collections.max(collection)` que retorna el valor máximo contenido en una colección:

```
// Suponiendo que list es una colección de enteros.
Integer maximo = Collections.max(list);
```

Pero ¿qué ocurre si la colección está vacía? En este caso el método `max()` lanzará la excepción `NoSuchElementException`. ¿Cuál es el problema? Que el cliente debe mirar la documentación para enterarse de ello y capturar la excepción o de lo contrario se producirá un error en tiempo de ejecución. Está obligado a darse cuenta de que debe hacer:

```
try {
    Integer maximo = Collections.max(list);
    // ...
} catch (NoSuchElementException e) {
    // ...
}
```

¿Y si existiera una forma de indicar que el valor de retorno de una función es opcional? Pues bien, Java 8 introdujo para este problema la clase `Optional<T>`.

La clase `Optional<T>` es una clase parametrizada que representa la abstracción de un valor de retorno opcional. Así, podríamos codificar nuestro método anterior haciendo que retorne un `Optional<T>` en vez de lanzar una excepción en el caso de que la colección esté vacía:

```
public static <T extends Comparable<T>> Optional<T> max(Collection<? extends T> coll) {
    try {
        return Optional.of(Collections.max(coll));
    } catch (NoSuchElementException e) {
        return Optional.empty();
    }
}
```

En ese caso, cuando el cliente llame a este método estará obligado a tratar el hecho de que puede que no se retorne valor. No tiene que mirar la documentación ni recordar hacer ningún tipo de comprobación porque el tipo de retorno es `Optional<T>`, de manera que si de verdad quiere obtener el valor, va a tener que comprobar si el *Optional* tiene o no valor.

```
Optional<Integer> maximoOpt = max(list);
```

Veamos más métodos de *Optional*:

- `T get()`: si un valor está presente, devuelve el valor, de lo contrario lanza la excepción *NoSuchElementException*.
- `boolean isPresent()`: si un valor está presente, devuelve *true*, en caso contrario *false*.

```
package tema12_ProgramacionFuncional.optional;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.Collections;
import java.util.List;
import java.util.NoSuchElementException;
import java.util.Optional;

public class GetAndIsPresent {

    public void show() {

        List<Integer> list1 = List.of(3, 2, 6, 1, 5, 4);
        List<Integer> list2 = new ArrayList<>();
        Optional<Integer> maximumOptional1 = max(list1);
```

```

Optional<Integer> maximumOptional2 = max(list2);
Integer maximum1 = 0, maximum2;

if (maximumOptional1.isPresent()) {
    maximum1 = maximumOptional1.get();
}
System.out.println(maximumOptional1); //Optional[6]
System.out.println(maximum1); //6

maximum2 = maximumOptional2.isPresent() ? maximumOptional2.get() : 0;
System.out.println(maximumOptional2); //Optional.empty
System.out.println(maximum2); //0

}

public static <T extends Comparable<T>> Optional<T> max(Collection<?
extends T> coll) {
    try {
        return Optional.of(Collections.max(coll));
    } catch (NoSuchElementException e) {
        return Optional.empty();
    }
}

public static void main(String[] args) {

    new GetAndIsPresent().show();

}

}

```

- `T orElse(T other)`: si un valor está presente, devuelve el valor, de lo contrario devuelve el valor suministrado como argumento.
- `T orElseGet(Supplier<? extends T> supplier)`: si hay un valor, devuelve el valor, en caso contrario devuelve el resultado producido por la función suministradora.
- `T orElseThrow()`: si un valor está presente, devuelve el valor, de lo contrario lanza la excepción *NoSuchElementException*.
- `<X extends Throwable> T orElseThrow(Supplier<? extends X> exceptionsupplier) throws X`: si hay un valor, devuelve el valor, en caso contrario lanza una excepción producida por la función suministradora.

```

package tema12_ProgramacionFuncional.optional;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.Collections;
import java.util.List;
import java.util.NoSuchElementException;
import java.util.Optional;
import java.util.Random;

public class OrElse {

```

```

public void show() {

    List<Integer> list1 = List.of(3, 2, 6, 1, 5, 4);
    List<Integer> list2 = new ArrayList<>();
    Optional<Integer> maximumOptional1 = max(list1);
    Optional<Integer> maximumOptional2 = max(list2);
    Integer maximum1, maximum2;

    maximum1 = maximumOptional1.orElse(0);
    maximum2 = maximumOptional2.orElse(0);
    System.out.println(maximum1); //6
    System.out.println(maximum2); //0

    maximum1 = maximumOptional1.orElseGet(() -> obtainRandomNumber());
    maximum2 = maximumOptional2.orElseGet(() -> obtainRandomNumber());
    System.out.println(maximum1); //6
    System.out.println(maximum2); //Número aleatorio entre 1 y 10

    maximum1 = maximumOptional1.orElseThrow();
    System.out.println(maximum1); //6
    try {
        maximum2 = maximumOptional2.orElseThrow();
    } catch (NoSuchElementException e) {
        System.err.println("Valor no presente");
    }

    maximum1 = maximumOptional1.orElseThrow(IllegalStateException::new);
    System.out.println(maximum1); //6
    try {
        maximum2 =
maximumOptional2.orElseThrow(IllegalStateException::new);
    } catch (IllegalStateException e) {
        System.err.println("Valor no presente");
    }

}

public Integer obtainRandomNumber() {
    return new Random().nextInt(10) + 1; //Devuelve un número aleatorio
entre 1 y 10
}

public static <T extends Comparable<T>> Optional<T> max(Collection<?
extends T> coll) {
    try {
        return Optional.of(Collections.max(coll));
    } catch (NoSuchElementException e) {
        return Optional.empty();
    }
}

public static void main(String[] args) {

    new OrElse().show();
}

```

```

    }

}

```

- `Optional<T> or(Supplier<? extends Optional<? extends T>> supplier)`: se introdujo en Java 9 para encadenar *Optionals*. Si el *Optional* original contiene un valor, el método retorna un nuevo *Optional* con dicho valor. Si el *Optional* original no contiene valor, el método retornará el *Optional* producido por la función suministradora.
- `<U> Optional<U> map(Function<? super T,? extends U> mapper)`: sirve para transformar el valor contenido en un *Optional*. Si el *Optional* original está vacío, devuelve un nuevo *Optional* vacío y si tiene valor, devolverá un *Optional* que contendrá como valor el resultado de aplicar la función de transformación al valor contenido en el *Optional* original. Si dicha función devuelve un resultado nulo, entonces se devuelve un *Optional* vacío.
- `Optional<T> filter(Predicate<? super T> predicate)`: retorna un nuevo *Optional* que estará vacío si el *Optional* original estaba vacío o si no se cumple el predicado recibido como argumento (su método `test()` retorna `false`). Si el *Optional* original contenía un valor y dicho valor cumple con el predicado, el nuevo *Optional* retornado contendrá el valor del *Optional* original.

```

package tema12_ProgramacionFuncional.optional;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.Collections;
import java.util.List;
import java.util.NoSuchElementException;
import java.util.Optional;
import java.util.Random;

public class OrMapFilter {

    public void show() {

        List<Integer> list1 = List.of(3, 2, 6, 1, 5, 4);
        List<Integer> list2 = new ArrayList<>();
        Optional<Integer> maximumOptional1 = max(list1);
        Optional<Integer> maximumOptional2 = max(list2);
        Optional<Integer> maximumOptional3, maximumOptional4,
maximumOptional5;

        maximumOptional3 = maximumOptional1.or(() ->
Optional.of(ObtainRandomNumber()));
        maximumOptional4 = maximumOptional2.or(() ->
Optional.of(ObtainRandomNumber()));
        System.out.println(maximumOptional3); //Optional[6]
        System.out.println(maximumOptional4); //Optional de un número
aleatorio

        maximumOptional3 = maximumOptional1.map(n -> n * 2);
        maximumOptional4 = maximumOptional2.map(n -> n * 2);
        System.out.println(maximumOptional3); //Optional[12]
        System.out.println(maximumOptional4); //Optional.empty
    }
}

```

```

        maximumOptional3 = maximumOptional1.filter(n -> n % 2 == 0);
        maximumOptional4 = maximumOptional1.filter(n -> n % 2 != 0);
        maximumOptional5 = maximumOptional2.filter(n -> n % 2 == 0);
        System.out.println(maximumOptional3); //Optional[6]
        System.out.println(maximumOptional4); //Optional.empty
        System.out.println(maximumOptional5); //Optional.empty
    }

    public Integer obtainRandomNumber() {
        return new Random().nextInt(10) + 1; //Devuelve un número aleatorio
        entre 1 y 10
    }

    public static <T extends Comparable<T>> Optional<T> max(Collection<?
    extends T> coll) {
        try {
            return Optional.of(Collections.max(coll));
        } catch (NoSuchElementException e) {
            return Optional.empty();
        }
    }

    public static void main(String[] args) {

        new OrMapFilter().show();

    }
}

```

- `void ifPresent(Consumer<? super T> action)`: permite consumir (usar) directamente el valor contenido en un *Optional* si es que éste contiene un valor. Si el *Optional* está vacío, no hace nada.
- `void ifPresentOrElse(Consumer<? super T> action, Runnable emptyAction)`: se incorporó este método en Java 9. Ejecuta el consumidor pasado como argumento si el *Optional* posee un valor o ejecuta el *Runnable* pasado como argumento si el *Optional* no contiene ningún valor. Ejemplo:

```

package tema12_ProgramacionFuncional.optional;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.Collections;
import java.util.List;
import java.util.NoSuchElementException;
import java.util.Optional;

public class IfPresent {

    public void show() {

        List<Integer> list1 = List.of(3, 2, 6, 1, 5, 4);
    }
}

```



```

List<Integer> list2 = new ArrayList<>();
Optional<Integer> maximumOptional1 = max(list1);
Optional<Integer> maximumOptional2 = max(list2);
maximumOptional1.ifPresent(System.out::println);//6
maximumOptional2.ifPresent(System.out::println);//No hace nada

maximumOptional1.ifPresentOrElse(System.out::println,
    () -> System.out.println("No hay máximo porque la lista está
vacía"));//6
maximumOptional2.ifPresentOrElse(System.out::println,
    () -> System.out.println("No hay máximo porque la lista está
vacía"));//No hay máximo porque la lista está vacía

}

public static <T extends Comparable<T>> Optional<T> max(Collection<?
extends T> coll) {
    try {
        return Optional.of(Collections.max(coll));
    } catch (NoSuchElementException e) {
        return Optional.empty();
    }
}

public static void main(String[] args) {

    new IfPresent().show();

}

}

```

Desafortunadamente, **no podemos garantizar que la referencia al objeto *Optional* en sí no sea `null`**. De hecho, en todos los ejemplos anteriores estamos confiando en que el método `max()` nunca va a retornar un `null`. Otros lenguajes de programación más modernos, como Kotlin o Swift, incorporan el concepto de *Optional* al sistema de tipos, de manera que existirán dos tipos distintos, uno que incorpora la posibilidad de tener el valor `null` y otro tipo que no lo permite. Por ejemplo, en Kotlin existe el tipo `String`, que no puede contener el valor `null` y el tipo `String?`, que sí puede contenerlo. Al establecer el tipo de retorno de una función tendremos que decidir si el tipo es el que puede incluir `null` o el que no. La ventaja de esta técnica es que si el tipo de retorno del método es `String?`, el cliente deberá comprobar si realmente se ha retornado un valor o no, pero siempre podrá realizar la comprobación. El problema de Java es que al encapsular un valor en un objeto *Optional*, el propio objeto *Optional* podría ser `null` y deberíamos fiarnos de que eso nunca puede suceder o realizar la comprobación cada vez. Evidentemente, cualquier desarrollador que realiza un método que retorna un *Optional* debería asegurarse y documentar que dicho método nunca retornará `null`, sino siempre un *Optional*.

12.4 Dónde no usar Optional

Debemos tener en cuenta que la clase `Optional<T>` ha sido diseñada específicamente para ser usada como tipo de retorno de los métodos. No se recomienda su uso en los siguientes casos:

- No se recomienda usar *Optional* como tipo de los atributos de una clase.

- No se recomienda usar *Optional* como tipo de los parámetros de un método porque ensucian mucho el código y realmente no hacen que el parámetro sea opcional.
- No se recomienda usar *Optional* como tipo de una colección, como por ejemplo en una lista.

13 Streams

13 Streams

1. Introducción
2. Pipeline y modelo MapReduce
3. Creación de un stream a partir de una fuente de datos
4. Tratamiento individual de elementos
5. Filtrado
6. Ordenación
7. Transformación
8. Reducción pura
 - 8.1 Métodos estándar de reducción
 - 8.2 Método reduce
9. Operaciones terminales de consulta
10. Reducción mutable
 - 10.1 Recolectores a estructuras de datos clásicas
 - 10.2 Recolectores de operaciones de reducción básicas
 - 10.3 Recolectores de transformación
 - 10.4 Recolectores de agrupación
 - 10.5 Recolectores de particionado
 - 10.6 Combinación de dos recolectores

1. Introducción

Un *stream* (flujo) en Java es una secuencia de elementos que se pueden procesar (mapear, filtrar, transformar, reducir y recolectar), de forma secuencial o paralela, mediante una cadena de operaciones especificadas a través de expresiones lambda. Introducidos en Java 8, los *streams* permiten optimizar la forma de procesar grandes colecciones de datos.

La interfaz `Stream<T>` representa un flujo de elementos de tipo `T` aunque también se definen interfaces concretas para los tipos primitivos, como `IntStream`, `LongStream`, etc.

Un *stream* es una abstracción que representa un flujo de datos pero no una estructura de datos, ya que los elementos no son almacenados en el *stream*, sino tan solo procesados por él. De hecho, no se puede acceder individualmente a un determinado elemento del *stream*, sino que se define la fuente de datos origen del *stream* y la secuencia de operaciones que se deben aplicar sobre sus elementos, especificadas de forma funcional mediante expresiones *lambda*.

Más aún, la fuente de datos origen del *stream* no se ve afectada por las operaciones realizadas dentro del *stream*. Por ejemplo, si se filtran algunos elementos de datos del *stream*, no se eliminan realmente de la fuente de datos origen, simplemente se omiten en el *stream* a partir de ese momento y ya no se tienen en cuenta en la siguiente operación incluida en la secuencia de operaciones del *stream*. Por tanto, los datos con los que trabajamos no se ven afectados por el *stream*.

Los *streams* solo gestionan datos transitorios en memoria, lo que implica que si la aplicación falla dichos datos se perderán.

Un *stream* puede ser finito, es decir, tener un número finito de elementos, o infinito, si genera un número infinito de elementos. Algunas operaciones permiten restringir el número de elementos procesados, como `limit()` o `findFirst()`.

2. Pipeline y modelo MapReduce

Una vez hayamos creado un *stream* a partir de una fuente de datos, podemos ejecutar sobre él cero o más operaciones intermedias y, forzosamente, una operación final. A esta cadena de operaciones se le conoce como *pipeline*. Un *pipeline* tiene los siguientes elementos en el siguiente orden:

1. Una función generadora del *stream*.
2. Cero o más operaciones intermedias.
3. Una operación terminal.

Debemos tener en cuenta que cada operación intermedia del *pipeline* genera un nuevo *stream* resultante de aplicar la operación indicada al *stream* anterior de la cadena.

Veamos cómo se clasifican las operaciones que podemos llevar a cabo en un *pipeline*:

- **Operaciones intermedias** (*aggregate operations*): producen como resultado un nuevo *stream*. Se usan para transformar, filtrar y clasificar los elementos del *stream*. Pueden ser:
 - Operaciones sin estado: al aplicarlas, el procesamiento de un elemento del *stream* es independiente de cualquier otro elemento del mismo. Por ejemplo, la operación de filtrado es sin estado, ya que el filtro de cada elemento sólo depende de una condición, no de ningún otro elemento del *stream*.
 - Operaciones con estado: al aplicarlas, el procesamiento de un elemento del *stream* depende de algún otro elemento del mismo. Por ejemplo, la operación de ordenación es con estado, ya que para posicionar un elemento es necesario compararlo con el resto.
- **Operaciones terminales** (*terminal operations*): procesan todos los elementos del *stream* para generar un resultado o un efecto secundario. De hecho, no retornan un *stream*. Después de su ejecución, el *stream* original no puede ser usado de nuevo, produciendo una excepción si se intenta. De ahí que se denominen operaciones terminales. Por tanto, un determinado *stream* puede ser usado una sola vez. Si necesitamos **procesar** la misma fuente de datos, deberemos crear un nuevo *stream* con ella como origen.

Otra característica importantísima del *pipeline* es que es perezoso (*lazy*), lo que quiere decir que las operaciones intermedias sólo son ejecutadas cuando las requiere la operación terminal que se esté ejecutando.

Por defecto, los elementos de un *stream* son procesados secuencialmente de uno en uno en el mismo hilo de ejecución. Es lo se conoce como *stream* secuencial. Sin embargo, podemos convertir un *stream* secuencial en un *stream* paralelo con tan sólo llamar a su método `parallel()`. Los elementos de los *streams* paralelos son agrupados en conjuntos y se usa un grupo de hilos de ejecución, denominado *common fork-join pool*, para procesar estos conjuntos de elementos en hilos de ejecución independientes.

Debemos tener en cuenta que las operaciones intermedias con estado no utilizarán todas las posibilidades de paralelismo existentes, dada su naturaleza en lo relativo a la dependencia entre elementos.

En Java, los *streams* utilizan el modelo **MapReduce**, que es un modelo de programación utilizado para procesar conjuntos de datos muy grandes y que ha sido adoptado por la programación funcional. Este modelo se basa en los siguientes tipos de operaciones:

- **Transformación (*map*):** filtra o crea copias modificadas de los elementos originales. Todas las operaciones intermedias de los *streams* corresponden a operaciones de transformación.
- **Reducción (*reduce*):** genera un resultado resumen de todos los elementos, por ejemplo, la suma o la media aritmética. Las operaciones terminales de la clase `Stream` corresponden a operaciones de reducción. De hecho, la clase `Stream` implementa dos operaciones de reducción diferentes:
 - **De reducción pura:** implementada en las diferentes versiones del método `reduce`, que procesa un flujo de elementos para obtener un único valor.
 - **De reducción mutable:** implementada en las diferentes versiones del método `collect`, que procesa un flujo de elementos para generar una estructura de datos mutable, como por ejemplo, una colección.

3. Creación de un stream a partir de una fuente de datos

Java permite muchas maneras de crear un *stream*, dependiendo de la fuente de datos origen deseada. Veamos algunas de estas fuentes:

- **Colección:** se ejecuta el método `stream()` sobre una colección para crear un *stream* que tenga como fuente de datos origen dicha colección. También tenemos disponible el método `parallelStream()` para que los elementos sean procesados en modo paralelo.

Podemos crear un *stream* a partir de cualquier interfaz que extienda de `Collection`, como `List`, `Set` o `Queue` y cualquiera de las clases que implementen dichas interfaces. Por ejemplo, desde una lista:

```
List<Integer> list = Arrays.asList(1, 2, 3, 4);
Stream<Integer> stream = list.stream();
```

o desde el `Set` correspondiente a las entradas de un `Map`:

```
Map<Integer, String> map = new HashMap<>();
Stream<Map.Entry<Integer, String>> stream = map.entrySet().stream();
```

- **Array:** el método estático `Arrays.stream(array)` recibe un array, que actuará como fuente de datos origen del *stream*. Ejemplo:

```
Integer array[] = {1, 2, 3, 4};
Stream<Integer> stream = Arrays.stream(array);
```

- **Conjunto predeterminado de elementos:** el método estático `Stream.of()` recibe un número variable de elementos que actuarán como fuente de datos origen del *stream*. Ejemplo:

```
Stream<Integer> stream = Stream.of(1, 2, 3, 4);
```

- **Función suministradora de objetos** (interfaz funcional `Supplier`): el método estático `Stream.generate()` recibe un `Supplier`, es decir, una función suministradora de elementos, que actuará como fuente de datos origen infinita para el *stream*. Ejemplo:

```
Stream<Integer> stream = Stream.generate(new Random()::nextInt);
```

- **Un valor inicial y una función que obtiene el siguiente elemento a partir del anterior:** el método estático `Stream.iterate(seed, unaryOperator)` recibe un valor inicial y una función que recibe el elemento anterior y retorna el valor del nuevo elemento, que debe ser del mismo tipo. Se tratará de un *stream* infinito. Ejemplo:

```
Stream<Integer> stream = Stream.iterate(0, x -> x + 5);
```

Java 9 introdujo una nueva versión de este método `Stream.iterate(seed, predicate, unaryOperator)` que recibe un parámetro intermedio adicional correspondiente a un *predicate* que al dejar de cumplir hace que el *stream* no emita más valores, convirtiéndose en un *stream* finito. Ejemplo:

```
Stream<Integer> stream = Stream.iterate(0, x -> x < 100, x -> x + 5);
```

- **Detectando patrones en una cadena:** el método `splitAsStream(cadena)` de la clase `Pattern` permite dividir una cadena en base a un patrón y retornar un *stream* de subcadenas. Por ejemplo:

```
Stream<String> stream =  
    Pattern.compile(",").splitAsStream("Luis,Paco,Ricardo");
```

retorna un *stream* cuyos elementos serán Luis, Paco y Ricardo.

- **Generador de números aleatorios:** el método `ints(limiteInf, limiteSup)` de la clase `Random` retorna un *stream* (un `IntStream`) cuya fuente de datos origen es el generador de números aleatorios contenidos entre `limiteInf` y `limiteSup`. Es especialmente útil para hacer pruebas. Ejemplo:

```
IntStream stream = new Random().ints(1, 100);
```

- **Métodos estáticos de la interfaz `IntStream`:**

- `IntStream.range(start, end)`: retorna un `IntStream` ordenado cuyos elementos corresponden a los enteros que van desde `start` hasta `end - 1`, es decir, `end` está excluido. Ejemplo:

```
IntStream stream = IntStream.range(1, 8);
```

- `IntStream.rangeClosed(start, end)`: retorna un `IntStream` ordenado cuyos elementos corresponden a los enteros que van desde `start` hasta `end` incluido. Ejemplo:

```
IntStream stream = IntStream.rangeClosed(1, 8);
```

- **Método chars() de String:** retorna un `IntStream` cuyos elementos corresponden a los caracteres de la cadena. Ejemplo:

```
IntStream stream = "Programación".chars();
```

- **Un valor inicial:** Java 9 incorpora el método estático `Stream.ofNullable(T value)` que retorna un `Stream<T>` con el valor indicado o vacío si el valor proporcionado es `null`.
- **Un Optional:** Java 9 incorpora el método `stream()` que retorna un `Stream<T>` con un único valor correspondiente al valor contenido en el *optional* o un *stream* vacío si el *optional* no tiene valor presente.
- **Un stream vacío:** podemos crear un *stream* vacío mediante el método estático `empty()` de la interfaz `Stream<T>`.
- **Un stream builder:** podemos crear un *stream* a partir de un objeto `Stream.Builder<T>` al que podemos agregar elementos mediante el método `add(item)` y posteriormente usar el método `build()` del mismo para obtener el objeto `Stream<T>`.
- **La concatenación de dos streams:** El método estático `Stream.concat(stream1, stream2)` retorna un *stream* resultante de la concatenación de los dos *streams* recibidos. Ejemplo:

```
Stream<String> stream1 = Stream.of("Luis", "Paco", "Ricardo");  
Stream<String> stream2 = Stream.of("Ana", "Lidia", "Esther");  
Stream<String> stream = Stream.concat(stream1, stream2);
```

4. Tratamiento individual de elementos

En algunas ocasiones necesitamos realizar algún tratamiento sobre cada uno de los elementos del *stream*. En dicho caso debemos diferenciar entre operaciones intermedias y terminales. Las operaciones terminales no producirán ningún nuevo *stream* mientras que las operaciones intermedias sí que lo producirán.

- `void forEach(Consumer<? super T> action)`: operación terminal para tratar cada uno de los elementos del *stream*. Aplica la acción recibida en forma de *Consumer* a cada uno de los elementos del *stream*.
- `Stream<T> peek(Consumer<? super T> action)`: también aplica la acción recibida en forma de *Consumer* a cada uno de los elementos del *stream* pero retorna un nuevo *stream* con los mismos elementos que el original, por lo que se trata de una operación intermedia. Este método se utiliza para tratar individualmente cada uno de los elementos del *stream* sin tener por ello que terminar la cadena de operaciones.

5. Filtrado

Otra de las operaciones intermedias que se pueden realizar sobre un *stream* es el filtrado de sus elementos, es decir, la generación de un nuevo *stream* que sólo contenga algunos de los elementos del *stream* original. Java nos proporciona distintos métodos:

- `Stream<T> distinct()`: retorna un nuevo *stream* con los elementos del *stream* original, excepto aquellos que estuvieran repetidos. Para determinar que dos elementos son iguales se usará el método `equals()` del elemento. Ejemplo:

```
package tema13_streams;
```

```
import java.util.stream.Stream;

public class Distinct {

    public void show() {

        Stream.of(1, 3, 2, 3, 1)
            .distinct()
            .forEach(System.out::println);

    }

    public static void main(String[] args) {

        new Distinct().show();

    }

}
```

Salida por consola:

```
1
3
2
```

- `Stream<T> limit(long maxSize)`: retorna un nuevo *stream* con tan sólo *maxSize* elementos del *stream* original, atendiendo al orden intrínseco del mismo. Tiene un mal rendimiento en *streams* paralelos ordenados. Ejemplo:

```
package tema13_Streams;

import java.util.stream.Stream;

public class Limit {

    public void show() {

        Stream.of("Ricardo", "Luis", "Paco")
            .limit(2)
            .forEach(System.out::println);

    }

    public static void main(String[] args) {

        new Limit().show();

    }

}
```

Salida por consola:

Ricardo
Luis

- `Stream<T> filter(Predicate<? super T> predicate)`: retorna un nuevo *stream* que sólo incorpora los elementos del *stream* original que cumplan el predicado recibido. Ejemplo:

```
package tema13_streams;

import java.util.stream.Stream;

public class Filter {

    public void show() {

        Stream.of(9, 12, 15, 24, 37, 6)
            .filter(n -> n % 2 == 0)
            .forEach(System.out::println);

    }

    public static void main(String[] args) {

        new Filter().show();

    }

}
```

Salida por consola:

```
12
24
6
```

- `Stream<T> skip(long n)`: retorna un nuevo *stream* en el que no se incluyen los primeros *n* elementos del *stream* original pero sí se incluye el resto. No proporciona un buen rendimiento en *streams* paralelos ordenados. Ejemplo:

```
package tema13_streams;

import java.util.stream.Stream;

public class Skip {

    public void show() {

        Stream.of(9, 12, 15, 24, 37, 6)
            .skip(3)
            .forEach(System.out::println);

    }

    public static void main(String[] args) {
```

```

        new Skip().show();
    }
}

```

Salida por consola:

```

24
37
6

```

- `default Stream<T> dropWhile(Predicate<? super T> predicate)`: retorna un nuevo *stream* con el primer elemento que no cumpla el predicado y el resto de elementos, independientemente de si cumplen el predicado o no. Proporciona un mal rendimiento con *streams* paralelos ordenados. Ejemplo:

```

package tema13_Streams;

import java.util.stream.Stream;

public class DropWhile {

    public void show() {

        Stream.of(9, 13, 15, 24, 37, 6)
            .dropWhile(n -> n % 2 != 0)
            .forEach(System.out::println);

    }

    public static void main(String[] args) {

        new DropWhile().show();

    }

}

```

Salida por consola:

```

24
37
6

```

- `default Stream<T> takeWhile(Predicate<? super T> predicate)`: mientras los elementos cumplan el predicado se van incluyendo en el *stream*, pero en cuanto se encuentra un elemento que no cumple el predicado se dejan de incluir el resto de elementos, incluso aunque cumplan el predicado. Ejemplo:

```

package tema13_Streams;

import java.util.stream.Stream;

```

```

public class Takewhile {

    public void show() {

        Stream.of(9, 13, 15, 24, 37, 6)
            .takeWhile(n -> n % 2 != 0)
            .forEach(System.out::println);

    }

    public static void main(String[] args) {

        new Takewhile().show();

    }

}

```

Salida por consola:

```

9
13
15

```

6. Ordenación

Algunos *streams* son ordenados, es decir, que sus elementos poseen un determinado orden intrínseco significativo, conocido como *encounter order*. Por ejemplo, un *stream* cuya fuente de datos corresponda a una lista creará un *stream* ordenado, cuyo *encounter order* será el orden en el que los elementos están situados en la lista. Sin embargo, otros *streams* no son ordenados, en el sentido de que sus elementos no tienen un orden intrínseco significativo. Por ejemplo, un *stream* cuya fuente de datos sea un conjunto (*Set*) será un *stream* sin *encounter order*, ya que en un conjunto los elementos no tienen un orden preestablecido.

El hecho de que un *stream* sea ordenado o no dependerá del tipo de fuente de datos asociada y de las operaciones intermedias anteriores que hayamos realizado mediante las que se ha obtenido el *stream*.

Algunas operaciones trabajan por defecto en base a este *encounter order*, imponiendo una restricción acerca del orden en el que los elementos deben ser procesados, como por ejemplo las operaciones intermedias *limit* o *skip*.

Sin embargo, existen otras operaciones que no tienen en cuenta el *encounter order*, como por ejemplo *forEach*. Si se ejecuta sobre un *stream* paralelo, no hay ninguna garantía sobre en que orden se aplica la acción a los elementos. Si queremos que sí se tenga en cuenta el orden, entonces tendríamos que usar el método `void forEachOrdered(Consumer<? super T> action)`. Normalmente se usa encadenado después de llamar a un método de ordenación que habrá ordenado el *stream*. La ventaja de este método es que se garantiza que la acción se aplica a los elementos en el orden intrínseco del *stream*, incluso aunque éste se trate de un *stream* paralelo, aunque conlleve un peor rendimiento.

Al trabajar con *streams* secuenciales, el *encounter order* no afecta al rendimiento de la aplicación, pero si trabajamos con *streams* paralelos, el empleo del *encounter order* por parte de algunos operadores pueden afectar en gran medida al rendimiento general de la aplicación. Dependiendo de la operación de la que se trate, será necesario procesar a la vez más de un elemento del *stream* o incluso almacenar en un *buffer* intermedio una gran cantidad de datos. En estos casos podemos usar el método `unordered()` para generar un nuevo *stream* a partir del anterior, en el que no se tenga en cuenta el *encounter order*. Al ejecutar el método `unordered()`, tan solo se está creando un nuevo *stream* en el que se ha borrado el indicador de que el *encounter order* debe tenerse en cuenta. Normalmente, esta operación de desactivación del *encounter order* se realiza con el objetivo de mejorar el rendimiento en *streams* paralelos.

Por otra parte, si queremos obtener un *stream* ordenado a partir de otro desordenado o a partir de otro *stream* ordenado pero por un orden distinto, podemos usar el método `sorted()`, en cuyo caso los elementos del *stream* deben implementar la interfaz `Comparable` para determinar el orden en el que deben ser ordenados. Otra posibilidad es usar una versión sobrecargada de dicho método que recibe un objeto `Comparator` como argumento. Ejemplo:

```
package tema13_Streams;

import java.util.stream.Stream;

public class Sorted {

    public void show() {

        Stream.of("Ricardo", "Luis", "Paco")
            .sorted()
            .limit(2)
            .forEach(System.out::println);

    }

    public static void main(String[] args) {

        new Sorted().show();

    }

}
```

Salida por consola:

```
Luis
Paco
```

Ejemplo utilizando `Comparator`:

```
package tema13_Streams;

import java.util.Comparator;
import java.util.stream.Stream;

public class SortedReverseOrder {
```

```

public void show() {

    Stream.of("Ricardo", "Luis", "Paco")
        .sorted(Comparator.reverseOrder())
        .limit(2)
        .forEach(System.out::println);

}

public static void main(String[] args) {

    new SortedReverseOrder().show();

}

}

```

Salida por consola:

```

Ricardo
Paco

```

7. Transformación

Java nos proporciona distintos métodos:

- `<R> Stream<R> map(Function<? super T,? extends R> mapper)`: retorna un nuevo *stream* obtenido a partir de aplicar la función de transformación indicada a cada uno de los elementos del *stream* original. El tipo del *stream* resultante corresponderá al tipo de retorno de la función de transformación, que puede ser distinto al tipo del *stream* original, pero contendrá tantos elementos como éste. Ejemplo:

```

package tema13_Streams;

import java.util.stream.Stream;

public class Map {

    public void show() {

        Stream.of(20, 27, 31)
            .map(n -> "Número " + n)
            .forEach(System.out::println);

    }

    public static void main(String[] args) {

        new Map().show();

    }

}

```

Salida por consola:

```
Número 20  
Número 27  
Número 31
```

- Métodos que permiten obtener un *stream* de un tipo primitivo a partir de uno que no lo sea:
 - `DoubleStream mapToDouble(ToDoubleFunction<? super T> mapper)` : retorna un `DoubleStream` correspondiente de aplicar a cada elemento del *stream* original la función de conversión a *double* recibida. Ejemplo:

```
package tema13_Streams;  
  
import java.util.stream.Stream;  
  
public class MapToDouble {  
  
    public void show() {  
  
        Stream.of(20, 27, 31)  
            .mapToDouble(n -> n * 0.5)  
            .forEach(System.out::println);  
  
    }  
  
    public static void main(String[] args) {  
  
        new MapToDouble().show();  
  
    }  
  
}
```

Salida por consola:

```
10.0  
13.5  
15.5
```

- `IntStream mapToInt(ToIntFunction<? super T> mapper)` : retorna un `IntStream` correspondiente de aplicar a cada elemento del *stream* original la función de conversión a *int* recibida. Ejemplo:

```
package tema13_Streams;  
  
import java.util.stream.Stream;  
  
public class MapToInt {  
  
    public void show() {  
  
        Stream.of("Ricardo", "Luis Miguel", "Paco")  
            .mapToInt(n -> n.length())
```

```

        .forEach(System.out::println);

    }

    public static void main(String[] args) {

        new MapToInt().show();

    }

}

```

Salida por consola:

```

7
11
4

```

- `LongStream mapToLong(ToLongFunction<? super T> mapper)`: retorna un `LongStream` correspondiente de aplicar a cada elemento del *stream* original la función de conversión a *long* recibida.

```

package tema13_Streams;

import java.util.stream.Stream;

public class MapToLong {

    public void show() {

        Stream.of(55000, 60000, 72500)
            .mapToLong(n -> (long)n * n)
            .forEach(System.out::println);

    }

    public static void main(String[] args) {

        new MapToLong().show();

    }

}

```

Salida por consola:

```

3025000000
3600000000
5256250000

```

- `<R> Stream<R> flatMap(Function<? super T,? extends Stream<? extends R>> mapper)`: cuando una función de transformación retorna un *stream* y se aplica esta función con el método *map*, el *stream* resultante es un `Stream<Stream<Tipo>>`. En estos casos, es más óptimo obtener un único `Stream<Tipo>` que contuviera concatenados todos los

elementos de todos los *substreams*. A este proceso se le conoce como aplanado (*flat*) de *substreams*. Ejemplo:

```
package tema13_Streams;

import java.util.stream.IntStream;
import java.util.stream.Stream;

public class FlatMap {

    public void show() {

        Stream.of(1, 2, 3)
            .flatMap(n -> IntStream.rangeClosed(1, n).boxed())
            .forEach(System.out::println);

    }

    public static void main(String[] args) {

        new FlatMap().show();

    }

}
```

Salida por consola:

```
1
1
2
1
2
3
```

En el ejemplo, el método `rangeClosed` de `IntStream` devuelve un `IntStream` y con el método `boxed` lo pasa a `Stream<Integer>`. Se muestran los valores `1` (proveniente del primer *substream*), `1`, `2` (provenientes del segundo *substream*) y `1`, `2` y `3`, provenientes del tercer *substream*, en este orden.

Si nos interesara que el tipo del *stream* resultante fuera primitivo, podemos usar los métodos `flatMapToDouble`, `flatMapToInt` o `flatMapToLong`. Ejemplo:

```
package tema13_Streams;

import java.util.stream.IntStream;
import java.util.stream.Stream;

public class FlatMapToInt {

    public void show() {

        Stream.of(1, 2, 3)
            .flatMapToInt(n -> IntStream.rangeClosed(1, n))
```



```

        .forEach(System.out::println);

    }

    public static void main(String[] args) {

        new FlatMapToInt().show();

    }

}

```

Salida por consola:

```

1
1
2
1
2
3

```

El método `flatMapToInt` de la interfaz `Stream<T>` devuelve un `IntStream`.

8. Reducción pura

8.1 Métodos estándar de reducción

- `Long count()`: retorna el número de elementos de un *stream*. Ejemplo:

```

package tema13_Streams;

import java.util.stream.Stream;

public class Count {

    public void show() {

        long howManyAreEven;
        howManyAreEven = Stream.of(30, 23, 24, 57, 8, 15)
            .filter(n -> n % 2 == 0)
            .count();
        System.out.println(howManyAreEven); //3

    }

    public static void main(String[] args) {

        new Count().show();

    }

}

```

- Las interfaces `IntStream`, `LongStream` y `DoubleStream`, correspondientes a *streams* de elementos de tipos primitivos numéricos, disponen del método `sum()` para calcular la suma de los elementos del *stream*:

```
package tema13_streams;

import java.util.stream.IntStream;

public class Sum {

    public void show() {

        int sumEvenNumbers;
        sumEvenNumbers = IntStream.of(30, 23, 24, 57, 8, 15)
            .filter(n -> n % 2 == 0)
            .sum();
        System.out.println(sumEvenNumbers); //62

    }

    public static void main(String[] args) {

        new Sum().show();

    }

}
```

Si disponemos de un *stream* genérico de elementos y no de un *stream* de alguna de las interfaces mencionadas anteriormente, podemos usar alguno de los métodos `mapToInt()`, `mapToLong()`, `mapToDouble()`, `flatMapToInt()`, etc., para obtener un *stream* de un tipo específico. Ejemplo:

```
package tema13_streams;

import java.util.stream.Stream;

public class SumMapToInt {

    public void show() {

        int sumEvenNumbers;
        sumEvenNumbers = Stream.of(30, 23, 24, 57, 8, 15)
            .filter(n -> n % 2 == 0)
            .mapToInt(n -> n)
            .sum();
        System.out.println(sumEvenNumbers); //62

    }

    public static void main(String[] args) {

        new SumMapToInt().show();

    }

}
```

```

    }

}

```

- Estas interfaces de *streams* de elementos de tipo primitivo numérico también disponen de métodos para obtener el valor máximo, `max()`, el valor mínimo, `min()` y la media aritmética, `average()`, de los elementos numéricos del *stream*. Dado que el *stream* sobre el que se apliquen puede estar vacío, estos métodos retornan un *Optional*.

```

package tema13_Streams;

import java.util.OptionalDouble;
import java.util.OptionalInt;
import java.util.stream.IntStream;

public class MaxMinAverage {

    public void show() {

        OptionalInt minEvenNumbers, maxEvenNumbers;
        OptionalDouble averageEvenNumbers;

        minEvenNumbers = IntStream.of(30, 23, 24, 57, 8, 15)
            .filter(n -> n % 2 == 0)
            .min();
        minEvenNumbers.ifPresent(System.out::println);//8

        maxEvenNumbers = IntStream.of(30, 23, 24, 57, 8, 15)
            .filter(n -> n % 2 == 0)
            .max();
        maxEvenNumbers.ifPresent(System.out::println);//30

        averageEvenNumbers = IntStream.of(30, 23, 24, 57, 8, 15)
            .filter(n -> n % 2 == 0)
            .average();
        averageEvenNumbers.ifPresent(n -> System.out.printf("%.2f",
n));//20,67

    }

    public static void main(String[] args) {

        new MaxMinAverage().show();

    }

}

```

Si se trata de un *stream* genérico, tenemos disponibles métodos para calcular el máximo y el mínimo que reciben un *Comparator* para comparar los elementos del *stream* y así obtener en cada caso el valor mínimo o el máximo:

- `Optional<T> max(Comparator<? super T> comparator)`
- `Optional<T> min(Comparator<? super T> comparator)`

```

package tema13_Streams;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Optional;

public class MaxMinComparator {

    public void show() {

        Optional<Vehicle> minVehicle;
        Optional<Vehicle> maxVehicle;
        String registration;
        List<Vehicle> list = new ArrayList<>();
        List<Vehicle> listEmpty = new ArrayList<>();
        list.add(new Vehicle("1705UBG", 4, "blanco"));
        list.add(new Vehicle("1235GTR", 2, "rojo"));

        System.out.println("Vehículo con menor número de ruedas:");
        minVehicle = list.stream()
            .min(Comparator.comparingInt(Vehicle::getWheelCount));
        minVehicle.ifPresent(System.out::println);

        System.out.println("Vehículo con mayor matrícula alfabéticamente:");
        maxVehicle = list.stream()
            .max(Comparator.comparing(Vehicle::getRegistration));
        maxVehicle.ifPresent(System.out::println);

        registration = list.stream()
            .min(Comparator.comparingInt(Vehicle::getWheelCount))
            .map(Vehicle::getRegistration)
            .orElse("No encontrado");
        System.out.printf("Matrícula del vehículo con menor número de ruedas: %s\n", registration);

        registration = listEmpty.stream()
            .min(Comparator.comparingInt(Vehicle::getWheelCount))
            .map(Vehicle::getRegistration)
            .orElse("No encontrado");
        System.out.printf("Matrícula del vehículo con menor número de ruedas: %s", registration);

    }

    public static void main(String[] args) {

        new MaxMinComparator().show();

    }

}

```

Salida por consola:

```
Vehículo con menor número de ruedas:  
Vehicle [registration=1235GTR, wheelCount=2, speed=0.0, colour=rojo]  
Vehículo con mayor matrícula alfabéticamente:  
Vehicle [registration=1705UBG, wheelCount=4, speed=0.0, colour=blanco]  
Matrícula del vehículo con menor número de ruedas: 1235GTR  
Matrícula del vehículo con menor número de ruedas: No encontrado
```

8.2 Método reduce

¿Y si queremos realizar una operación de reducción distinta a las anteriores? Para casos más genéricos usaremos el método `Optional<T> reduce(BinaryOperator<T> accumulator)` que recibe una función acumuladora. La función acumuladora debe ser una función asociativa, es decir, da igual en qué orden se opere con los elementos porque siempre se va a obtener el mismo resultado. Se trata de una regla que se cumple en la suma y en la multiplicación. Por ejemplo, `1+2+3` da el mismo resultado que `2+3+1`.

En este caso, el proceso de reducción comienza cuando se obtiene el segundo elemento, ya que necesitamos al menos dos elementos para hacer la primera reducción. Por este motivo, el método retorna un `Optional`, dado que si el *stream* no tiene elementos suficientes no se puede realizar la reducción ni producir ningún valor. Ejemplo:

```
package tema13_Streams;  
  
import java.util.OptionalInt;  
import java.util.stream.IntStream;  
  
public class Reduce {  
  
    public void show() {  
  
        OptionalInt integerSum;  
  
        integerSum = IntStream.of(30, 23, 24, 57, 8, 15)  
            .reduce((subtotal, element) -> subtotal + element); //Con  
        lambda  
        integerSum.ifPresent(System.out::println); //157  
  
        integerSum = IntStream.of(30, 23, 24, 57, 8, 15)  
            .reduce(Integer::sum); //Con referencia a método  
        integerSum.ifPresent(System.out::println); //157  
  
        integerSum = IntStream.empty()  
            .reduce(Integer::sum);  
        System.out.println(integerSum); //OptionalInt.empty  
        integerSum.ifPresent(System.out::println); //No hace nada  
  
    }  
  
    public static void main(String[] args) {  
  
        new Reduce().show();  
  
    }  
}
```

```
}
```

Este método está sobrecargado para pasarle como primer parámetro un valor conocido como identidad (*identity*), que es usado como valor inicial de la operación de reducción y el resultado por defecto si el *stream* está vacío: `T reduce(T identity, BinaryOperator<T> accumulator)`.

Debemos elegir cuidadosamente el valor de *identity* atendiendo a la operación que se lleve a cabo en la función *binaryOperator*. El valor de *identity* debe ser una identidad para la función acumuladora, es decir, si se aplica la función acumuladora a cualquier elemento con la identidad, debe devolver el mismo elemento. Por ejemplo, si utilizamos como operación una suma, la identidad debe ser 0 ya que cualquier número al que le sumemos 0, nos devuelve el mismo número. Por el mismo razonamiento, si la operación es una multiplicación, la identidad debe ser 1.

```
package tema13_Streams;

import java.util.stream.IntStream;

public class ReduceIdentity {

    public void show() {

        Integer sum,mult;

        sum = IntStream.of(30, 23, 24, 57, 8, 15)
            .reduce(0, Integer::sum);
        System.out.println(sum);//157

        sum = IntStream.empty()
            .reduce(0, Integer::sum);
        System.out.println(sum);//0

        mult = IntStream.of(2,3,4)
            .reduce(1, (subtotal,element) -> subtotal * element);//Con lambda
        System.out.println(mult);//24

        mult = IntStream.of(2,3,4)
            .reduce(1, Math::multiplyExact);//Con referencia a método
        System.out.println(mult);//24

        mult = IntStream.empty()
            .reduce(1, Math::multiplyExact);
        System.out.println(mult);//1

    }

    public static void main(String[] args) {

        new ReduceIdentity().show();

    }

}
```

Tenemos disponible una tercera versión del método, `<U> U reduce(U identity, BiFunction<U,? super T,U> accumulator, BinaryOperator<U> combiner)`, especialmente útil para *streams* paralelos, que se usa cuando queremos que el método retorne un valor de tipo diferente al del *stream* original. La función *combiner* es necesaria para indicar cómo se deben combinar acumulaciones parciales realizadas en distintos hilos en *streams* paralelos.

```
package tema13_Streams;

import java.util.stream.Stream;

public class ReduceCombiner {

    public void show() {

        int count = Stream.of("Juan", "Pepe", "Luis", "Ricardo", "Laura")
            .reduce(0, (subtotal, element) -> {
                if (element.length() % 2 == 0) {
                    return subtotal + 1;
                }
                else {
                    return subtotal;
                }
            }, Integer::sum);

        System.out.printf("Cuántos nombres con número de caracteres pares: %d",
            count); //3

    }

    public static void main(String[] args) {

        new ReduceCombiner().show();

    }

}
```

Debemos tener en cuenta que la función `reduce()` en cualquiera de sus versiones respeta el orden del *stream* a la hora de combinar los cálculos intermedios.

9. Operaciones terminales de consulta

La clase `Stream` también proporciona una serie de métodos de consulta sobre los elementos de un *stream*, denominadas operaciones de cortocircuito (*short-circuit terminal operations*). Se llaman así porque se deja de procesar el resto de elementos si con los elementos que ya han sido procesados se puede determinar el resultado.

Tenemos un conjunto de métodos que permiten consultar, respectivamente, si todos, ninguno o algunos de los elementos del *stream* cumplen con un determinado predicado, retornando un valor booleano:

- `boolean allMatch(Predicate<? super T> predicate)`
- `boolean noneMatch(Predicate<? super T> predicate)`

- `boolean anyMatch(Predicate<? super T> predicate)`

```
package tema13_Streams;

import java.util.stream.IntStream;

public class Match {

    public void show() {

        boolean match;
        match = IntStream.of(1, 2, 4)
            .allMatch(n -> n % 2 == 0);
        System.out.println(match); //false

        match = IntStream.of(2, 4, 6)
            .allMatch(n -> n % 2 == 0);
        System.out.println(match); //true

        match = IntStream.of(1, 2, 4)
            .noneMatch(n -> n % 2 == 0);
        System.out.println(match); //false

        match = IntStream.of(1, 3, 5)
            .noneMatch(n -> n % 2 == 0);
        System.out.println(match); //true

        match = IntStream.of(1, 3, 5)
            .anyMatch(n -> n % 2 == 0);
        System.out.println(match); //false

        match = IntStream.of(2, 3, 5)
            .anyMatch(n -> n % 2 == 0);
        System.out.println(match); //true

    }

    public static void main(String[] args) {

        new Match().show();

    }

}
```

Por otro lado tenemos los métodos `findFirst()` y `findAny()`, que retornan un `Optional<T>` con, respectivamente, el primer elemento del *stream*, o algún elemento del *stream* (no está indicado cuál), si es que existe. Estos métodos no reciben ningún predicado con el que indicar la condición de búsqueda por lo que normalmente se usan después de haber ejecutado el método `filter` sobre el *stream*.

```
package tema13_Streams;

import java.util.Optional;
```



```

import java.util.stream.Stream;

public class Find {

    public void show() {

        Optional<Integer> find;
        find = Stream.of(1, 2, 4)
            .filter(n -> n % 2 == 0)
            .findFirst();
        find.ifPresent(System.out::println);//2

        find = Stream.of(6, 2, 4)
            .filter(n -> n % 2 == 0)
            .findAny();
        find.ifPresent(System.out::println);//6

    }

    public static void main(String[] args) {

        new Find().show();

    }

}

```

No se proporciona ningún método para obtener el último elemento de un *stream*. Sin embargo, podemos obtenerlo usando el método `skip` (siempre y cuando se trate de un *stream* finito):

```

package tema13_Streams;

import java.util.List;
import java.util.Optional;

public class LastElement {

    public void show() {

        List<Integer> list = List.of(30, 23, 24, 57, 8, 15);
        long count = list.stream().count();
        Optional<Integer> last = list.stream()
            .skip(count - 1)
            .findFirst();
        last.ifPresent(System.out::println);//15

    }

    public static void main(String[] args) {

        new LastElement().show();

    }

}

```

```
}
```

Otra manera de hacerlo es mediante una reducción en la que siempre nos quedemos con el segundo elemento:

```
package tema13_Streams;

import java.util.OptionalInt;
import java.util.stream.IntStream;

public class LastReduce {

    public void show() {

        OptionalInt last;

        last = IntStream.of(30, 23, 24, 57, 8, 15)
            .reduce((first, second) -> second);
        last.ifPresent(System.out::println);//15

    }

    public static void main(String[] args) {

        new LastReduce().show();

    }

}
```

Debemos tener en cuenta que si ejecutamos estos métodos sobre *streams* paralelos el resultado puede ser distinto entre distintas llamadas.

10. Reducción mutable

La operación de **recolección** es una operación terminal que permite crear una estructura de datos con los resultados del procesamiento de datos asociado a un *stream*. La operación de recolección también recibe el nombre de operación de reducción mutable. Para llevarla a cabo usaremos el método `<R, A> R collect(Collector<? super T,A,R> collector)`.

El método recibe un objeto de una clase que implemente la interfaz `Collector`. Aunque podemos crear nuestras propias clases que implementen dicha interfaz, en la mayoría de las ocasiones podremos utilizar alguno de los recolectores estándar proporcionados por Java a través de la clase auxiliar `Collectors`, que contiene métodos estáticos que retornan objetos `Collector` correspondientes a los recolectores más habituales. Todos estos métodos están diseñados para funcionar de manera óptima incluso con *streams* paralelos.

10.1 Recolectores a estructuras de datos clásicas

La clase `Collectors` tiene un conjunto de métodos estáticos que nos permiten recolectar los elementos de un *stream* y almacenarlos en una estructura de datos:

- `public static <T> Collector<T,?,List<T>> toList()` : retorna una lista con los elementos del stream. En Java 16, se ha incorporado también un método `toList()` a la interfaz `Stream<T>`.
- `public static <T> Collector<T,?,List<T>> toUnmodifiableList()` : retorna una lista inmutable de los elementos del *stream* en el orden en que son producidos (*encounter order*).
- `public static <T> Collector<T,?,Set<T>> toSet()` : retorna un *set* (conjunto) con los elementos del *stream*. Si se hay elementos duplicados en el *stream*, son ignorados.
- `public static <T> Collector<T,?,Set<T>> toUnmodifiableSet()` : retorna un *set* (conjunto) inmutable con los elementos del *stream*. Si hay elementos duplicados en el *stream*, son ignorados.
- `public static <T, C extends Collection<T>> Collector<T,?,C> toCollection(Supplier<C> collectionFactory)` : el problema de `toList()` y `toSet()` es que no podemos especificar la implementación concreta que queremos que se use. Por ejemplo, no podemos indicar que se use un `LinkedList`, en el caso de `toList()`, o un `TreeSet`, en el caso de `toSet()`. Para solucionar este problema, este método recibe un *supplier* que retorna la estructura de datos concreta en la que queremos que se recolecte el *stream*.

```
package tema13_Streams.collect;

import java.util.List;
import java.util.TreeSet;
import java.util.stream.Collectors;

public class CollectorsClassicDataStructures {

    public void show() {

        List<Integer> list = List.of(3, 6, 1, 2, 4, 5);
        list.stream()
            .filter(n -> n % 2 == 0)
            .collect(Collectors.toList())
            .forEach(System.out::println);

        list.stream()
            .filter(n -> n % 2 == 0)
            .collect(Collectors.toCollection(TreeSet::new))
            .forEach(System.out::println);

    }

    public static void main(String[] args) {

        new CollectorsClassicDataStructures().show();

    }

}
```

Salida por consola:

6
2
4
2
4
6

- `public static <T, K, U> Collector<T,?,Map<K,U>> toMap(Function<? super T,? extends K> keyMapper, Function<? super T,? extends U> valueMapper):` retorna un mapa obtenido de la siguiente manera:
 - Con el `Function` `keyMapper` obtiene la clave, de manera que su valor de retorno será usado como clave del elemento en el mapa resultante. Como el mapa resultante no puede tener claves repetidas, si retorna el mismo valor para dos elementos distintos del *stream*, se lanzará la excepción `IllegalStateException`.
 - Con el `Function` `valueMapper` se obtiene el valor del elemento en el mapa resultante.

```
package tema13_Streams.collect;

import java.util.Arrays;
import java.util.stream.Collectors;

public class CollectorsToMap {

    public void show() {

        Vehicle vehicles[] = new Vehicle[6];
        vehicles[0] = new Vehicle("9685KMX", 4, "azul");
        vehicles[1] = new Vehicle("1235GTR", 2, "rojo");
        vehicles[2] = new Vehicle("7314QWE", 4, "verde");
        vehicles[3] = new Vehicle("5930POI", 2, "negro");
        vehicles[4] = new Vehicle("1705UBG", 4, "blanco");
        vehicles[5] = new Vehicle("3495JZA", 2, "naranja");
        Arrays.stream(vehicles)
            .collect(Collectors.toMap(Vehicle::getRegistration, vehicle ->
vehicle))
            .forEach((k, v) -> System.out.printf("Clave:%s Valor:%s\n", k,
v));

    }

    public static void main(String[] args) {

        new CollectorsToMap().show();

    }

}
```

Salida por consola:

```
Clave:3495JZA valor:Vehicle [registration=3495JZA,
wheelCount=2,speed=0.0,colour=naranja]
Clave:1705UBG valor:Vehicle [registration=1705UBG,
wheelCount=4,speed=0.0,colour=blanco]
Clave:1235GTR valor:Vehicle [registration=1235GTR, wheelCount=2, speed=0.0,
colour=rojo]
Clave:7314QWE valor:Vehicle [registration=7314QWE, wheelCount=4, speed=0.0,
colour=verde]
Clave:9685KMX valor:Vehicle [registration=9685KMX, wheelCount=4, speed=0.0,
colour=azul]
Clave:5930POI valor:Vehicle [registration=5930POI, wheelCount=2, speed=0.0,
colour=negro]
```

Si nos fijamos en el `Function` `valueMapper (vehicle -> vehicle)`, también podemos usar el método estático `static <T> Function<T,T> identity()`, que es una función que siempre devuelve su argumento de entrada:

```
package tema13_streams.collect;

import java.util.Arrays;
import java.util.function.Function;
import java.util.stream.Collectors;

public class FunctionIdentity {

    public void show() {

        Vehicle vehicles[] = new Vehicle[6];
        vehicles[0] = new Vehicle("9685KMX", 4, "azul");
        vehicles[1] = new Vehicle("1235GTR", 2, "rojo");
        vehicles[2] = new Vehicle("7314QWE", 4, "verde");
        vehicles[3] = new Vehicle("5930POI", 2, "negro");
        vehicles[4] = new Vehicle("1705UBG", 4, "blanco");
        vehicles[5] = new Vehicle("3495JZA", 2, "naranja");
        Arrays.stream(vehicles)
            .collect(Collectors.toMap(Vehicle::getRegistration,
Function.identity()))
            .forEach((k, v) -> System.out.printf("Clave:%s valor:%s\n", k,
v));

    }

    public static void main(String[] args) {

        new FunctionIdentity().show();

    }

}
```

En el `Function` `keyMapper`, como el mapa resultante no puede tener claves repetidas, si retorna el mismo valor para dos elementos distintos del *stream*, se lanzará la excepción `IllegalStateException`:

```

package temal3_Streams.collect;

import java.util.Arrays;
import java.util.stream.Collectors;

public class RepeatedKeys {

    public void show() {

        Vehicle vehicles[] = new Vehicle[6];
        vehicles[0] = new Vehicle("9685KMX", 4, "azul");
        vehicles[1] = new Vehicle("1235GTR", 2, "rojo");
        vehicles[2] = new Vehicle("7314QWE", 4, "verde");
        vehicles[3] = new Vehicle("5930POI", 2, "rojo");
        vehicles[4] = new Vehicle("1705UBG", 4, "blanco");
        vehicles[5] = new Vehicle("3495JZA", 2, "azul");
        Arrays.stream(vehicles)
            .collect(Collectors.toMap(Vehicle::getColour,
Vehicle::getRegistration))
            .forEach((k, v) -> System.out.printf("Clave:%s valor:%s\n", k,
v));

    }

    public static void main(String[] args) {

        new RepeatedKeys().show();

    }

}

```

Tenemos disponible una segunda versión del método para que en lugar de lanzar una excepción, proporcionemos una función de combinación de elementos con la misma clave:

- `public static <T, K, U> Collector<T,?,Map<K,U>> toMap(Function<? super T,? extends K> keyMapper, Function<? super T,? extends U> valueMapper, BinaryOperator<U> mergeFunction)`: el tercer parámetro indica cómo se deben combinar dos elementos con la misma clave.

```

package temal3_Streams.collect;

import java.util.Arrays;
import java.util.stream.Collectors;

public class CombinationRepeatedKeys {

    public void show() {

        Vehicle vehicles[] = new Vehicle[6];
        vehicles[0] = new Vehicle("9685KMX", 4, "azul");
        vehicles[1] = new Vehicle("1235GTR", 2, "rojo");
        vehicles[2] = new Vehicle("7314QWE", 4, "verde");
        vehicles[3] = new Vehicle("5930POI", 2, "rojo");
        vehicles[4] = new Vehicle("1705UBG", 4, "blanco");

```

```

        vehicles[5] = new Vehicle("3495JZA", 2, "azul");
        Arrays.stream(vehicles)
                .collect(Collectors.toMap(Vehicle::getColour,
        Vehicle::getRegistration,
                (r1,r2) -> String.format("%s-%s", r1 ,r2)))
                .forEach((k, v) -> System.out.printf("Clave:%-7s Valor:%s\n",
        k, v));

    }

    public static void main(String[] args) {

        new CombinationRepeatedKeys().show();

    }

}

```

Salida por consola:

```

Clave:rojo    Valor:1235GTR-5930POI
Clave:blanco  Valor:1705UBG
Clave:verde   Valor:7314QWE
Clave:azul    Valor:9685KMX-3495JZA

```

Estas dos versiones del método `toMap` retornan por defecto un `HashMap`, por lo que existe una tercera versión para obtener una implementación distinta de la interfaz `Map`, como por ejemplo `LinkedHashMap` o `TreeMap`:

- `public static <T, K, U, M extends Map<K, U>> Collector<T,?,M> toMap(Function<? super T,? extends K> keyMapper, Function<? super T,? extends U> valueMapper, BinaryOperator<U> mergeFunction, Supplier<M> mapFactory):` recibe un `Supplier` como cuarto parámetro para indicar el tipo de mapa que se quiere obtener:

```

package tema13_streams.collect;

import java.util.Arrays;
import java.util.TreeMap;
import java.util.stream.Collectors;

public class CombinationRepeatedKeysSupplier {

    public void show() {

        Vehicle vehicles[] = new Vehicle[6];
        vehicles[0] = new Vehicle("9685KMX", 4, "azul");
        vehicles[1] = new Vehicle("1235GTR", 2, "rojo");
        vehicles[2] = new Vehicle("7314QWE", 4, "verde");
        vehicles[3] = new Vehicle("5930POI", 2, "rojo");
        vehicles[4] = new Vehicle("1705UBG", 4, "blanco");
        vehicles[5] = new Vehicle("3495JZA", 2, "azul");
        Arrays.stream(vehicles)
                .collect(Collectors.toMap(Vehicle::getColour,
        Vehicle::getRegistration,

```

```

        (r1,r2) -> String.format("%s-%s", r1 ,r2),
        TreeMap::new))
        .forEach((k, v) -> System.out.printf("Clave:%-7s Valor:%s\n",
k, v));

    }

    public static void main(String[] args) {

        new CombinationRepeatedKeysSupplier().show();

    }

}

```

Salida por consola:

```

Clave:azul      Valor:9685KMX-3495JZA
Clave:blanco   Valor:1705UBG
Clave:rojo     Valor:1235GTR-5930POI
Clave:verde    Valor:7314QWE

```

- Recolección de un *stream* hacia un **array**: no se realizará a través de ningún recolector ni del método `collect`, sino directamente a través del método `toArray()` de la clase `Stream`, que retorna un `Object[]`, es decir un *array* de elementos de la clase `Object`, debido a que los arrays no usan genéricos.

```

package tema13_streams.collect;

import java.util.ArrayList;
import java.util.List;

public class ToArray {

    public void show() {

        List<Vehicle> list = new ArrayList<>();
        list.add(new Vehicle("9685KMX", 4, "azul"));
        list.add(new Vehicle("1235GTR", 2, "rojo"));
        list.add(new Vehicle("7314QWE", 4, "verde"));
        list.add(new Vehicle("5930POI", 2, "negro"));
        list.add(new Vehicle("1705UBG", 4, "blanco"));
        list.add(new Vehicle("3495JZA", 2, "naranja"));
        Object vehiclesArray[] = list.stream()
                                    .toArray();

        for(Object v: vehiclesArray) {
            System.out.println(v);
        }

    }

    public static void main(String[] args) {

        new ToArray().show();

    }

}

```



```
}  
  
}
```

10.2 Recolectores de operaciones de reducción básicas

La clase `Collectors` dispone además de una serie métodos estáticos que retornan recolectores parecidos a las operaciones de reducción:

- `counting()`: Para obtener el **número de elementos**.

```
package tema13_Streams.collect;  
  
import java.util.stream.Collectors;  
import java.util.stream.Stream;  
  
public class CollectorsCounting {  
  
    public void show() {  
  
        Long howManyAreEven;  
        howManyAreEven = Stream.of(30, 23, 24, 57, 8, 15)  
                                .filter(n -> n % 2 == 0)  
                                .collect(Collectors.counting());  
        System.out.println(howManyAreEven); //3  
  
    }  
  
    public static void main(String[] args) {  
  
        new CollectorsCounting().show();  
  
    }  
  
}
```

- Para obtener la **suma** de los elementos que son convertidos al tipo indicado mediante la función suministrada:
 - `public static <T> Collector<T,?,Integer> summingInt(ToIntFunction<? super T> mapper)`
 - `public static <T> Collector<T,?,Long> summingLong(ToLongFunction<? super T> mapper)`
 - `public static <T> Collector<T,?,Double> summingDouble(ToDoubleFunction<? super T> mapper)`

```
package tema13_Streams.collect;  
  
import java.util.ArrayList;  
import java.util.List;  
import java.util.stream.Collectors;  
  
public class CollectorsSumming {
```

```

public void show() {

    List<Vehicle> list = new ArrayList<>();
    list.add(new Vehicle("9685KMX", 4, "azul"));
    list.add(new Vehicle("1235GTR", 2, "rojo"));
    list.add(new Vehicle("7314QWE", 4, "verde"));
    list.add(new Vehicle("5930POI", 2, "negro"));
    list.add(new Vehicle("1705UBG", 4, "blanco"));
    list.add(new Vehicle("3495JZA", 2, "naranja"));
    int sumWheels = list.stream()
        .collect(Collectors.summingInt(Vehicle::getWheelCount));
    System.out.println(sumWheels); //18

}

public static void main(String[] args) {

    new CollectorsSumming().show();

}

}

```

- Para obtener el valor **mínimo y el máximo** atendiendo a un comparador pasado como argumento:

- `public static <T> Collector<T,?,Optional<T>> minBy(Comparator<? super T> comparator)`
- `public static <T> Collector<T,?,Optional<T>> maxBy(Comparator<? super T> comparator)`

```

package tema13_Streams.collect;

import java.util.Comparator;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

public class CollectorsMinMax {

    public void show() {

        Optional<Integer> min,max;
        List<Integer> list = List.of(30, 23, 24, 57, 8, 15);
        min = list.stream()
            .collect(Collectors.minBy(Comparator.naturalOrder()));
        min.ifPresent(System.out::println); //8

        max = list.stream()
            .collect(Collectors.maxBy(Comparator.naturalOrder()));
        max.ifPresent(System.out::println); //57

    }

}

```

```

    public static void main(String[] args) {

        new CollectorsMinMax().show();

    }

}

```

- Para obtener la **media aritmética** de los valores (que son convertidos al tipo indicado mediante la función suministrada). Si el *stream* no tiene elementos retorna cero:

- `public static <T> Collector<T,?,Double> averagingInt(ToIntFunction<? super T> mapper)`
- `public static <T> Collector<T,?,Double> averagingLong(ToLongFunction<? super T> mapper)`
- `public static <T> Collector<T,?,Double> averagingDouble(ToDoubleFunction<? super T> mapper)`

```

package tema13_streams.collect;

import java.util.List;
import java.util.stream.Collectors;

public class CollectorsAveraging {

    public void show() {

        double average;
        List<Integer> list = List.of(30, 23, 24, 57, 8, 15);
        average = list.stream()
            .collect(Collectors.averagingInt(Integer::intValue));
        System.out.printf("%.2f", average); //26,17

    }

    public static void main(String[] args) {

        new CollectorsAveraging().show();

    }

}

```

- Para obtener todo lo anterior (número de elementos, suma, mínimo, máximo y la media) de una sola vez:
 - `public static <T> Collector<T,?,IntSummaryStatistics> summarizingInt(ToIntFunction<? super T> mapper)`
 - `public static <T> Collector<T,?,LongSummaryStatistics> summarizingLong(ToLongFunction<? super T> mapper)`
 - `public static <T> Collector<T,?,DoubleSummaryStatistics> summarizingDouble(ToDoubleFunction<? super T> mapper)`

```

package tema13_Streams.collect;

import java.util.IntSummaryStatistics;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class Summarizing {

    public void show() {

        IntSummaryStatistics oddStatistics;
        oddStatistics = Stream.of(30, 23, 24, 57, 8, 15)
            .filter(n -> n % 2 != 0)
            .collect(Collectors.summarizingInt(Integer::intValue));
        System.out.println(oddStatistics);

    }

    public static void main(String[] args) {

        new Summarizing().show();

    }

}

```

Salida por consola:

```
IntSummaryStatistics{count=3, sum=95, min=15, average=31,666667, max=57}
```

- Para recolectar con una operación de reducción distinta de las anteriores: `public static <T> Collector<T,?,Optional<T>> reducing(BinaryOperator<T> op)`

```

package tema13_Streams.collect;

import java.util.Optional;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class CollectorsReducing {

    public void show() {

        Optional<Integer> integerSum;

        integerSum = Stream.of(30, 23, 24, 57, 8, 15)
            .collect(Collectors.reducing(
                (subtotal, element) -> subtotal + element));
        //Con lambda
        integerSum.ifPresent(System.out::println); //157

        integerSum = Stream.of(30, 23, 24, 57, 8, 15)
            .collect(Collectors.reducing(
                Integer::sum)); //Con referencia a método
        integerSum.ifPresent(System.out::println); //157
    }

}

```

```

        integerSum = Stream.<Integer>empty()
            .collect(Collectors.reducing(
                Integer::sum)); //Optional.empty
        integerSum.ifPresent(System.out::println); //No hace nada
    }

    public static void main(String[] args) {

        new CollectorsReducing().show();

    }

}

```

También tenemos otra versión del método que recibe como primer parámetro el valor correspondiente a la identidad: `public static <T> Collector<T,?,T> reducing(T identity, BinaryOperator<T> op)`

```

package tema13_Streams.collect;

import java.util.stream.Collectors;
import java.util.stream.Stream;

public class CollectorsReducingIdentity {

    public void show() {

        Integer sum,mult;

        sum = Stream.of(30, 23, 24, 57, 8, 15)
            .collect(Collectors.reducing(0, Integer::sum));
        System.out.println(sum); //157

        sum = Stream.<Integer>empty()
            .collect(Collectors.reducing(0, Integer::sum));
        System.out.println(sum); //0

        mult = Stream.of(2,3,4)
            .collect(Collectors.reducing(1, Math::multiplyExact));
        System.out.println(mult); //24

        mult = Stream.<Integer>empty()
            .collect(Collectors.reducing(1, Math::multiplyExact));
        System.out.println(mult); //1

    }

    public static void main(String[] args) {

        new CollectorsReducingIdentity().show();

    }

}

```

```
}
```

Existe además una tercera versión del método que recibe como segundo parámetro una función de transformación que será ejecutada sobre cada elemento antes de realizar la recolección: `public static <T, U> Collector<T,?,U> reducing(U identity, Function<? super T,? extends U> mapper, BinaryOperator<U> op)`

```
package tema13_Streams.collect;

import java.util.stream.Collectors;
import java.util.stream.Stream;

public class CollectorsReducingCombiner {

    public void show() {

        int count = Stream.of("Juan", "Pepe", "Luis", "Ricardo", "Laura")
            .collect(Collectors.reducing(0, element -> {
                if(element.length() % 2 == 0) {
                    return 1;
                }
                else {
                    return 0;
                }
            }, Integer::sum));

        System.out.printf("Cuántos nombres con número de caracteres pares: %d", count); //3

    }

    public static void main(String[] args) {

        new CollectorsReducingCombiner().show();

    }

}
```

Como podemos apreciar, todos los recolectores vistos en este apartado son muy similares (casi iguales) a los métodos estándar de reducción que vimos en un apartado anterior. Entonces, ¿por qué existen estos recolectores? El motivo es que, como veremos más adelante, Java nos va a ofrecer la oportunidad de encadenar varios recolectores, de manera que usaremos los recolectores vistos en este apartado normalmente como acompañante de algún otro recolector. De hecho, **no se recomienda usar estos recolectores si no es en conjunción con otro recolector**. Si se va a emplear de forma aislada, es más óptimo emplear los métodos estándar de recolección que vimos en un apartado anterior.

10.3 Recolectores de transformación

- `public static Collector<CharSequence,?,String> joining()` : permite obtener un `String` correspondiente a la concatenación de los elementos del *stream*. Solo podremos usar este recolector sobre un *stream* de elementos de tipo `CharSequence`, por lo que es posible que antes hayamos tenido que aplicar una operación de transformación mediante el método `map` para obtener un *stream* adecuado. `CharSequence` es una interfaz que representa una secuencia de caracteres. Esta interfaz no impone la mutabilidad, por lo tanto, nos podemos encontrar con clases inmutables y mutables que implementen esta interfaz. Por ejemplo, `String` es inmutable y `StringBuilder` y `StringBuffer` son mutables.

```
package tema13_Streams.collect;  
  
import java.util.stream.Collectors;  
import java.util.stream.Stream;  
  
public class CollectorsJoining {  
  
    public void show() {  
  
        String result = Stream.of("Juan", "Pepe", "Luis", "Ricardo", "Laura")  
                                .collect(Collectors.joining());  
        System.out.println(result);  
  
    }  
  
    public static void main(String[] args) {  
  
        new CollectorsJoining().show();  
  
    }  
  
}
```

Salida por consola:

```
JuanPepeLuisRicardoLaura
```

Existe una segunda versión para concatenar los elementos de entrada separados por un delimitador que se pasa por parámetro: `public static Collector<CharSequence,?,String> joining(CharSequence delimiter)`.

```
package tema13_Streams.collect;  
  
import java.util.stream.Collectors;  
import java.util.stream.Stream;  
  
public class CollectorsJoiningDelimiter {  
  
    public void show() {  
  
        String result = Stream.of("Juan", "Pepe", "Luis", "Ricardo", "Laura")  
                                .collect(Collectors.joining(" - "));  
  
    }  
  
}
```

```

        System.out.println(result);

    }

    public static void main(String[] args) {

        new CollectorsJoiningDelimiter().show();

    }

}

```

Salida por consola:

```
Juan - Pepe - Luis - Ricardo - Laura
```

Existe una tercera versión del método que permite indicar el prefijo y el sufijo que queremos poner a la cadena resultante de la concatenación: `public static Collector<CharSequence,?,String> joining(CharSequence delimiter, CharSequence prefix, CharSequence suffix)`

```

package tema13_Streams.collect;

import java.util.stream.Collectors;
import java.util.stream.Stream;

public class CollectorsJoiningPrefixSuffix {

    public void show() {

        String result = Stream.of("Juan", "Pepe", "Luis", "Ricardo", "Laura")
            .collect(Collectors.joining(" - ", "Lista de nombres: ", "."));
        System.out.println(result);

    }

    public static void main(String[] args) {

        new CollectorsJoiningPrefixSuffix().show();

    }

}

```

Salida por consola:

```
Lista de nombres: Juan - Pepe - Luis - Ricardo - Laura.
```

- `public static <T, U, A, R> Collector<T,?,R> mapping(Function<? super T,? extends U> mapper, Collector<? super U,A,R> downstream)`: realiza alguna operación de transformación sobre los elementos justo antes de aplicarles un recolector que se pasa por parámetro.


```

package tema13_Streams.collect;

import java.util.Arrays;
import java.util.stream.Collectors;

public class CollectorsMapping {

    public void show() {

        Vehicle vehicles[] = new Vehicle[6];
        vehicles[0] = new Vehicle("9685KMX", 4, "azul");
        vehicles[1] = new Vehicle("1235GTR", 2, "rojo");
        vehicles[2] = new Vehicle("7314QWE", 4, "verde");
        vehicles[3] = new Vehicle("5930POI", 2, "negro");
        vehicles[4] = new Vehicle("1705UBG", 4, "blanco");
        vehicles[5] = new Vehicle("3495JZA", 2, "naranja");
        String result = Arrays.stream(vehicles)
            .collect(Collectors.mapping(Vehicle::getRegistration,
                                      Collectors.joining(", ")));
        System.out.println(result);

    }

    public static void main(String[] args) {

        new CollectorsMapping().show();

    }

}

```

Salida por consola:

```
9685KMX, 1235GTR, 7314QWE, 5930POI, 1705UBG, 3495JZA
```

10.4 Recolectores de agrupación

`public static <T, K> Collector<T,?, Map<K, List<T>>> groupingBy(Function<? super T, ? extends K> classifier):` permite obtener un mapa `Map<K, List<T>>` donde las claves son los valores resultantes de aplicar la función de clasificación a los elementos de entrada y los valores son listas que contienen los elementos de entrada que al aplicarles la función de clasificación se obtiene la clave correspondiente, es decir, todos aquellos elementos del *stream* original que al aplicarles la función clasificadora retornen el mismo valor, dicho valor será la clave y los elementos serán agrupados en la misma lista con dicha clave. Veamos un ejemplo: vamos a crear un mapa donde la clave será el número de ruedas del vehículo. El mapa tendrá dos entradas, una para los vehículos de 2 ruedas y otra para los vehículos de 4 ruedas. Cada clave tendrá una lista con los vehículos que correspondan con dicho número de ruedas:

```

package tema13_Streams.collect;

import java.util.Arrays;
import java.util.stream.Collectors;

```

```

public class CollectorsGroupingBy {

    public void show() {

        vehicle vehicles[] = new vehicle[6];
        vehicles[0] = new vehicle("9685KMX", 4, "azul");
        vehicles[1] = new vehicle("1235GTR", 2, "rojo");
        vehicles[2] = new vehicle("7314QWE", 4, "verde");
        vehicles[3] = new vehicle("5930POI", 2, "negro");
        vehicles[4] = new vehicle("1705UBG", 4, "blanco");
        vehicles[5] = new vehicle("3495JZA", 2, "naranja");
        Arrays.stream(vehicles)
            .collect(Collectors.groupingBy(vehicle::getWheelCount))
            .forEach((k, v) -> {
                System.out.printf("\nVehículos con %d ruedas:", k);
                v.forEach(vehicle -> System.out.printf("\n%s", vehicle));
            });

    }

    public static void main(String[] args) {

        new CollectorsGroupingBy().show();

    }

}

```

Salida por consola:

```

vehículos con 2 ruedas:
vehicle [registration=1235GTR, wheelCount=2, speed=0.0, colour=rojo]
vehicle [registration=5930POI, wheelCount=2, speed=0.0, colour=negro]
vehicle [registration=3495JZA, wheelCount=2, speed=0.0, colour=naranja]
vehículos con 4 ruedas:
vehicle [registration=9685KMX, wheelCount=4, speed=0.0, colour=azul]
vehicle [registration=7314QWE, wheelCount=4, speed=0.0, colour=verde]
vehicle [registration=1705UBG, wheelCount=4, speed=0.0, colour=blanco]

```

Sin embargo, será muy habitual que queramos realizar algún cálculo sobre la lista de elementos de cada grupo. Para hacernos más sencilla dicha tarea, tenemos disponible otra versión del método: `public static <T, K, A, D> Collector<T,?,Map<K,D>> groupingBy(Function<? super T,? extends K> classifier, Collector<? super T,A,D> downstream)`, que recibe un recolector que queremos que se le aplique a la lista de elementos de cada grupo. Para este cometido, podemos usar los recolectores de operaciones de reducción básicas que vimos en el apartado anterior. Si lo aplicamos al ejemplo anterior, podemos obtener cuántos vehículos hay con 2 ruedas y cuántos hay con 4 ruedas:

```

package tema13_Streams.collect;

import java.util.Arrays;
import java.util.stream.Collectors;

public class CollectorsGroupingByDownstream {

```

```

public void show() {

    Vehicle vehicles[] = new Vehicle[6];
    vehicles[0] = new Vehicle("7314QWE", 4, "verde");
    vehicles[1] = new Vehicle("9685KMX", 2, "azul");
    vehicles[2] = new Vehicle("1235GTR", 2, "rojo");
    vehicles[3] = new Vehicle("5930POI", 2, "negro");
    vehicles[4] = new Vehicle("1705UBG", 4, "blanco");
    vehicles[5] = new Vehicle("3495JZA", 2, "naranja");
    Arrays.stream(vehicles)
        .collect(Collectors.groupingBy(Vehicle::getWheelCount,
Collectors.counting()))
        .forEach((k, v) -> System.out.printf("Número de vehículos con %d
ruedas: %d\n", k, v));
}

public static void main(String[] args) {

    new CollectorsGroupingByDownstream().show();

}

}

```

Salida por consola:

```

Número de vehículos con 2 ruedas: 4
Número de vehículos con 4 ruedas: 2

```

Estas dos versiones del método `groupingBy` retornan por defecto un `HashMap`, por lo que existe una tercera versión para obtener una implementación distinta de la interfaz `Map`, como por ejemplo `LinkedHashMap` o `TreeMap`: `public static <T, K, D, A, M extends Map<K, D>> Collector<T,?,M> groupingBy(Function<? super T,? extends K> classifier, Supplier<M> mapFactory, Collector<? super T,A,D> downstream)`.

```

package tema13_Streams.collect;

import java.util.Arrays;
import java.util.LinkedHashMap;
import java.util.stream.Collectors;

public class CollectorsGroupingBySupplier {

    public void show() {

        Vehicle vehicles[] = new Vehicle[6];
        vehicles[0] = new Vehicle("7314QWE", 4, "verde");
        vehicles[1] = new Vehicle("9685KMX", 2, "azul");
        vehicles[2] = new Vehicle("1235GTR", 2, "rojo");
        vehicles[3] = new Vehicle("5930POI", 2, "negro");
        vehicles[4] = new Vehicle("1705UBG", 4, "blanco");
        vehicles[5] = new Vehicle("3495JZA", 2, "naranja");
        Arrays.stream(vehicles)

```

```

.collect(Collectors.groupingBy(Vehicle::getWheelCount, LinkedHashMap::new,
Collectors.counting()))
    .forEach((k, v) -> System.out.printf("Número de vehículos con %d
ruedas: %d\n", k, v));
}

public static void main(String[] args) {

    new CollectorsGroupingBySupplier().show();

}

}

```

Salida por consola:

```

Número de vehículos con 4 ruedas: 2
Número de vehículos con 2 ruedas: 4

```

¿Y si queremos realizar alguna operación de conversión sobre los elementos de la lista de cada grupo antes de aplicarle el recolector *downstream*? Podemos usar el método `Collectors.mapping` que ya conocemos:

```

package tema13_Streams.collect;

import java.util.Arrays;
import java.util.TreeMap;
import java.util.stream.Collectors;

public class CollectorsGroupingByMapping {

    public void show() {

        Vehicle vehicles[] = new Vehicle[6];
        vehicles[0] = new Vehicle("7314QWE", 4, "verde");
        vehicles[1] = new Vehicle("9685KMX", 2, "azul");
        vehicles[2] = new Vehicle("1235GTR", 2, "rojo");
        vehicles[3] = new Vehicle("5930POI", 2, "negro");
        vehicles[4] = new Vehicle("1705UBG", 4, "blanco");
        vehicles[5] = new Vehicle("3495JZA", 2, "naranja");
        Arrays.stream(vehicles)
            .collect(Collectors.groupingBy(Vehicle::getWheelCount,
TreeMap::new,

Collectors.mapping(Vehicle::getRegistration,

Collectors.joining(" - "))))
        .forEach((k, v) -> System.out.printf("Matrículas con %d ruedas:
%s\n", k, v));
    }

    public static void main(String[] args) {

```

```

        new CollectorsGroupingByMapping().show();
    }
}

```

Salida por consola:

```

Matrículas con 2 ruedas: 9685KMX - 1235GTR - 5930POI - 3495JZA
Matrículas con 4 ruedas: 7314QWE - 1705UBG

```

Si la función de transformación retorna un *stream*, podemos usar `public static <T, U, A, R> Collector<T,?,R> flatMapping(Function<? super T,? extends Stream<? extends U>> mapper, Collector<? super U,A,R> downstream)` disponible a partir de Java 9.

Por ejemplo, vamos a crear la clase *ModelVehicle* para registrar modelos de coches con un determinado número de ruedas y una lista de todos los colores en los que está disponible dicho modelo:

```

package tema13_Streams.modelVehicle;

import java.util.List;

public class ModelVehicle{

    private String model;
    private int wheelCount;
    private List<String> colours;

    public ModelVehicle(String model, int wheelCount, List<String> colours) {
        this.model = model;
        this.wheelCount = wheelCount;
        this.colours = colours;
    }

    public String getModel() {
        return model;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public List<String> getColours() {
        return colours;
    }

    @Override
    public String toString() {
        String result = String.format("Modelo: %s Número de ruedas: %d Colores: ", model, wheelCount);
        for(int i=0;i<colours.size();i++) {
            if(i==0) {
                result = result.concat(String.format("%s",colours.get(i)));
            }
        }
    }
}

```

```

        }
        else {
            result = result.concat(String.format(", %s", colours.get(i)));
        }
    }
    return result;
}
}
}

```

Si quisiéramos obtener de cada número de ruedas cuántos colores hay disponibles, podríamos hacer lo siguiente:

```

package tema13_Streams.modelVehicle;

import java.util.Arrays;
import java.util.List;
import java.util.TreeMap;
import java.util.stream.Collectors;

public class CollectorsGroupingByMapping {

    public void show() {

        ModelVehicle vehicles[] = new ModelVehicle[4];
        vehicles[0] = new ModelVehicle("Audi", 4,
List.of("azul", "rojo", "blanco"));
        vehicles[1] = new ModelVehicle("Ford", 4, List.of("naranja", "blanco"));
        vehicles[2] = new ModelVehicle("Audi", 2, List.of("negro", "verde"));
        vehicles[3] = new ModelVehicle("Ford", 2, List.of("negro", "blanco"));
        Arrays.stream(vehicles)
            .collect(Collectors.groupingBy(ModelVehicle::getWheelCount,
TreeMap::new,
                                collectors.mapping(v ->
v.getColours(),
collectors.toList()))))
            .forEach((k, v) -> System.out.printf("Número de ruedas:%d Colores:
%s\n", k, v));
    }

    public static void main(String[] args) {

        new CollectorsGroupingByMapping().show();

    }
}

```

Salida por consola:

```

Número de ruedas:2 Colores: [[negro, verde], [negro, blanco]]
Número de ruedas:4 Colores: [[azul, rojo, blanco], [naranja, blanco]]

```

Si nos fijamos en la salida de consola, salen las sublistas. Para quitarlas, podemos pasar estas listas a *streams* y luego utilizar el método `flatMap` para aplanarlas:

```
package tema13_Streams.modelVehicle;

import java.util.Arrays;
import java.util.List;
import java.util.TreeMap;
import java.util.stream.Collectors;

public class CollectorsGroupingByFlatMapping {

    public void show() {

        ModelVehicle vehicles[] = new ModelVehicle[4];
        vehicles[0] = new ModelVehicle("Audi", 4,
List.of("azul", "rojo", "blanco"));
        vehicles[1] = new ModelVehicle("Ford", 4, List.of("naranja", "blanco"));
        vehicles[2] = new ModelVehicle("Audi", 2, List.of("negro", "verde"));
        vehicles[3] = new ModelVehicle("Ford", 2, List.of("negro", "blanco"));
        Arrays.stream(vehicles)
            .collect(Collectors.groupingBy(ModelVehicle::getWheelCount,
TreeMap::new,
                                Collectors.flatMapping(v ->
v.getColours().stream(),
Collectors.joining("-"))))
            .forEach((k, v) -> System.out.printf("Número de ruedas:%d Colores:
%s\n", k, v));
    }

    public static void main(String[] args) {

        new CollectorsGroupingByFlatMapping().show();

    }

}
```

Salida por consola:

```
Número de ruedas:2 Colores: negro-verde-negro-blanco
Número de ruedas:4 Colores: azul-rojo-blanco-naranja-blanco
```

Ahora nos encontramos con colores repetidos. Para quitarlos, podemos hacer uso del método `public static <T, A, R, RR> Collector<T,A,RR> collectingAndThen(Collector<T,A,R> downstream, Function<R,RR> finisher)`. Este método recolecta y después se puede realizar un *Function* sobre el resultado de la recolección:

```
package tema13_Streams.modelVehicle;

import java.util.Arrays;
import java.util.List;
```

```

import java.util.TreeMap;
import java.util.stream.Collectors;

public class CollectorsGroupingByCollectingAndThen {

    public void show() {

        ModelVehicle vehicles[] = new ModelVehicle[4];
        vehicles[0] = new ModelVehicle("Audi", 4,
List.of("azul", "rojo", "blanco"));
        vehicles[1] = new ModelVehicle("Ford", 4, List.of("naranja", "blanco"));
        vehicles[2] = new ModelVehicle("Audi", 2, List.of("negro", "verde"));
        vehicles[3] = new ModelVehicle("Ford", 2, List.of("negro", "blanco"));
        Arrays.stream(vehicles)
                .collect(Collectors.groupingBy(ModelVehicle::getwheelCount,
TreeMap::new,

Collectors.collectingAndThen(Collectors.flatMapping(v ->
v.getColours().stream(), Collectors.toList()),

list ->

list.stream().distinct().collect(Collectors.joining("-")))))
                .forEach((k, v) -> System.out.printf("Número de ruedas:%d Colores:
%s\n", k, v));
    }

    public static void main(String[] args) {

        new CollectorsGroupingByCollectingAndThen().show();

    }

}

```

Salida por consola:

```

Número de ruedas:2 Colores: negro-verde-blanco
Número de ruedas:4 Colores: azul-rojo-blanco-naranja

```

En otras ocasiones, lo que queremos es filtrar los elementos de cada lista en base a algún criterio, antes de aplicarle el recolector *downstream*. Para ello, Java 9 incorporó el método `public static <T, A, R> Collector<T,?,R> filtering(Predicate<? super T> predicate, Collector<? super T,A,R> downstream)`:

```

package tema13_Streams.modelVehicle;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class CollectorsGroupingByFiltering {

    public void show() {

        List<ModelVehicle> vehicles = new ArrayList<>();
    }
}

```



```

        vehicles.add(new ModelVehicle("Audi", 4, List.of("azul","rojo")));
        vehicles.add(new ModelVehicle("Ford", 4, List.of("naranja","blanco")));
        vehicles.add(new ModelVehicle("Audi", 3, List.of("blanco","azul")));
        vehicles.add(new ModelVehicle("Ford", 3, List.of("naranja","verde")));
        vehicles.add(new ModelVehicle("Audi", 2, List.of("negro","verde")));
        vehicles.add(new ModelVehicle("Ford", 2, List.of("amarillo","blanco")));
        vehicles.stream()
                .collect(Collectors.groupingBy(ModelVehicle::getModel,
                                                Collectors.filtering(v ->
v.getWheelCount() > 2,

Collectors.toList()))))
                .forEach((k, v) -> System.out.printf("Modelo:%s Vehículos con más
de 2 ruedas: %s\n", k, v));
    }

    public static void main(String[] args) {

        new CollectorsGroupingByFiltering().show();

    }

}

```

Salida por consola:

```

Modelo:Audi Vehículos con más de 2 ruedas: [Modelo: Audi Número de ruedas: 4
Colores: azul, rojo, Modelo: Audi Número de ruedas: 3 Colores: blanco, azul]
Modelo:Ford Vehículos con más de 2 ruedas: [Modelo: Ford Número de ruedas: 4
Colores: naranja, blanco, Modelo: Ford Número de ruedas: 3 Colores: naranja,
verde]

```

Se pueden crear varios niveles de agrupamiento, aplicando `groupingBy` como recolector *downstream* de los elementos de cada grupo:

```

package tema13_Streams.modelVehicle;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class CollectorsVariousLevelsOfGroupingBy {

    public void show() {

        List<ModelVehicle> vehicles = new ArrayList<>();
        vehicles.add(new ModelVehicle("Audi", 4, List.of("azul","rojo")));
        vehicles.add(new ModelVehicle("Ford", 4,
List.of("naranja","blanco","verde")));
        vehicles.add(new ModelVehicle("Seat", 4, List.of("amarillo","verde")));
        vehicles.add(new ModelVehicle("Audi", 2, List.of("negro")));
        vehicles.add(new ModelVehicle("Ford", 2, List.of("rojo","blanco")));
        vehicles.add(new ModelVehicle("Seat", 2, List.of("amarillo","morado")));
        vehicles.stream()

```

```

        .collect(Collectors.groupingBy(ModelVehicle::getModel,
Collectors.groupingBy(ModelVehicle::getWheelCount,
Collectors.flatMapping(v -> v.getColours().stream(),
Collectors.counting()))))
        .forEach((k, v) -> {
            v.forEach((k2, v2) -> {
                System.out.printf("Modelo %s con %d ruedas está disponible en
%d %s\n",k,k2,v2,v2==1?"color":"colores");
            });
        });
    }

    public static void main(String[] args) {

        new CollectorsVariousLevelsOfGroupingBy().show();

    }
}

```

Salida por consola:

```

Modelo Seat con 2 ruedas está disponible en 2 colores
Modelo Seat con 4 ruedas está disponible en 2 colores
Modelo Audi con 2 ruedas está disponible en 1 color
Modelo Audi con 4 ruedas está disponible en 2 colores
Modelo Ford con 2 ruedas está disponible en 2 colores
Modelo Ford con 4 ruedas está disponible en 3 colores

```

10.5 Recolectores de particionado

`public static <T> Collector<T,?,Map<Boolean,List<T>>> partitioningBy(Predicate<? super T> predicate)`: aplica el predicado proporcionado a cada uno de los elementos del *stream* y crea dos grupos, uno con los que cumplen el predicado y otro con los que no lo cumplen:

```

package tema13_Streams.modelVehicle;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class CollectorsPartitioningBy {

    public void show() {

        List<ModelVehicle> vehicles = new ArrayList<>();
        vehicles.add(new ModelVehicle("Audi", 4, List.of("azul","rojo")));
        vehicles.add(new ModelVehicle("Ford", 4, List.of("naranja","blanco")));
        vehicles.add(new ModelVehicle("Audi", 2, List.of("negro","verde")));
        vehicles.add(new ModelVehicle("Ford", 2, List.of("amarillo","blanco")));
        vehicles.stream()
    }
}

```

```

        .collect(Collectors.partitioningBy(v -> v.getWheelCount() == 4))
        .forEach((k, v) -> {
            System.out.printf("%s\n", k?"Vehículos de 4
ruedas:":"Vehículos que no son de 4 ruedas:");
            v.forEach(System.out::println);
        });
    }

    public static void main(String[] args) {

        new CollectorsPartitioningBy().show();

    }
}

```

Salida por consola:

```

Vehículos que no son de 4 ruedas:
Modelo: Audi Número de ruedas: 2 Colores: negro, verde
Modelo: Ford Número de ruedas: 2 Colores: amarillo, blanco
Vehículos de 4 ruedas:
Modelo: Audi Número de ruedas: 4 Colores: azul, rojo
Modelo: Ford Número de ruedas: 4 Colores: naranja, blanco

```

Existe otra versión del método que recibe como segundo argumento un recolector para que sea ejecutado sobre la lista de cada grupo:

```

package tema13_Streams.modelVehicle;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class CollectorsPartitioningByDownstream {

    public void show() {

        List<ModelVehicle> vehicles = new ArrayList<>();
        vehicles.add(new ModelVehicle("Audi", 4, List.of("azul","rojo")));
        vehicles.add(new ModelVehicle("Ford", 4,
List.of("naranja","blanco","verde")));
        vehicles.add(new ModelVehicle("Seat", 4, List.of("amarillo","verde")));
        vehicles.add(new ModelVehicle("Audi", 2, List.of("negro")));
        vehicles.add(new ModelVehicle("Ford", 2, List.of("rojo","blanco")));
        vehicles.stream()
            .collect(Collectors.partitioningBy(v -> v.getWheelCount() == 4,
Collectors.counting()))
            .forEach((k, v) -> System.out.printf("%s hay en %d modelos\n",
k?"Vehículos de 4 ruedas:":"Vehículos que no son de 4 ruedas:",v));
    }

    public static void main(String[] args) {

```

```

        new CollectorsPartitioningByDownstream().show();

    }

}

```

Salida por consola:

```

vehículos que no son de 4 ruedas: hay en 2 modelos
vehículos de 4 ruedas: hay en 3 modelos

```

10.6 Combinación de dos recolectores

`public static <T, R1, R2, R> Collector<T,?,R> teeing(Collector<? super T,?,R1> downstream1, Collector<? super T,?,R2> downstream2, BiFunction<? super R1,? super R2,R> merger)`: ejecuta ambos recolectores sobre los elementos y después ejecuta sobre los resultados la *BiFunction* proporcionada, que combinará ambos resultados, de manera que la combinación será el producto final de la recolección:

```

package tema13_Streams.modelVehicle;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.stream.Collectors;

public class CollectorsTeeing {

    public void show() {

        Result result;
        List<ModelVehicle> vehicles = new ArrayList<>();
        vehicles.add(new ModelVehicle("Audi", 4, List.of("azul","rojo")));
        vehicles.add(new ModelVehicle("Ford", 4,
List.of("naranja","blanco","verde")));
        vehicles.add(new ModelVehicle("Seat", 4, List.of("amarillo","verde")));
        vehicles.add(new ModelVehicle("Audi", 2, List.of("negro")));
        vehicles.add(new ModelVehicle("Ford", 2, List.of("rojo","blanco")));
        vehicles.add(new ModelVehicle("Seat", 2, List.of("amarillo","morado")));
        result = vehicles.stream()
            .collect(Collectors.teeing(Collectors.minBy(
                Comparator.comparing((ModelVehicle v) ->
                    v.getColours().size()),Collectors.maxBy(
                Comparator.comparing((ModelVehicle v) ->
                    v.getColours().size()),Result::new));

        result.getMin().ifPresent(v -> System.out.printf("El vehículo que tiene
el mínimo número de colores es %s", v));
        result.getMax().ifPresent(v -> System.out.printf("\nEl vehículo que tiene
el máximo número de colores es %s", v));

    }
}

```

```

    public static void main(String[] args) {

        new CollectorsTeeing().show();

    }

}

```

siendo la clase `Result`:

```

package tema13_streams.modelVehicle;

import java.util.Optional;

public class Result{

    private Optional<ModelVehicle> min;
    private Optional<ModelVehicle> max;

    public Result(Optional<ModelVehicle> min,Optional<ModelVehicle> max) {
        this.min = min;
        this.max = max;
    }

    public Optional<ModelVehicle> getMin() {
        return min;
    }

    public Optional<ModelVehicle> getMax() {
        return max;
    }

}

```

Salida por consola:

```

El vehículo que tiene el mínimo número de colores es Modelo: Audi Número de
ruedas: 2 Colores: negro
El vehículo que tiene el máximo número de colores es Modelo: Ford Número de
ruedas: 4 Colores: naranja, blanco, verde

```

Este método es de la versión 12 de Java. En versiones anteriores, sería necesario operar dos veces sobre el *stream*, almacenar los resultados intermedios en variables temporales y después combinar las variables temporales.

2.1 Sentencias condicionales

2.1 Sentencias condicionales

- 1. Introducción
- 2. Secuencia
- 3. Sentencias condicionales
 - 3.1 Condicional simple
 - 3.2 Condicional doble
 - 3.3 Condicional múltiple
 - 3.3.1 Anidación
 - 3.3.2 Switch

1. Introducción

Los programas se construyen para procesar datos, manipulándolos de formas diferentes dependiendo de los valores que tengan. Los lenguajes de programación deben proveer estructuras que les permitan a los programadores controlar el flujo de ejecución de un programa dependiendo de los datos que procesan. Para ello, se incluyen las sentencias de control de flujo, que alteran el flujo de ejecución para tomar decisiones o repetir sentencias.

La programación estructurada es un paradigma de programación orientado a mejorar la claridad, calidad y tiempo de desarrollo de un programa de computadora recurriendo únicamente a subrutinas y tres estructuras básicas: secuencia, sentencias condicionales y bucles.

Una subrutina o subprograma, como idea general, se presenta como un subalgoritmo que forma parte del algoritmo principal, el cual permite resolver una tarea específica. Por ejemplo, los métodos en Java.

2. Secuencia

La ejecución secuencial es el más básico de los mecanismos de control de flujo y consiste en la ejecución de instrucciones en el orden en que se encuentran en el código fuente del programa.

Hasta ahora las instrucciones que hemos visto, son instrucciones que se ejecutan secuencialmente; es decir, podemos saber lo que hace el programa leyendo las líneas de izquierda a derecha y de arriba abajo.

3. Sentencias condicionales

La sentencia condicional consiste en ejecutar instrucciones diferentes dependiendo del resultado de evaluar una expresión lógica. Una **expresión lógica** es cualquier tipo de expresión que devuelva un resultado booleano (true o false). Las expresiones lógicas se construyen por medio de variables booleanas o bien a través de los operadores relacionales (==, >, <, ...) y/o lógicos (&&, |, !).

3.1 Condicional simple

Se ejecutan una serie de instrucciones en el caso de que la expresión lógica sea verdadera:

```
if(expresiónLógica){  
    instrucción1  
    instrucción2  
    ....  
}
```

Ejemplo:

```
package tema2_1_SentenciasCondicionales;  
  
import java.util.Scanner;  
  
public class If {  
  
    @SuppressWarnings("resource")  
    public void show() {  
  
        Scanner keyboard = new Scanner(System.in);  
        int age;  
        String name;  
  
        System.out.print("Introduce tu nombre: ");  
        name = keyboard.nextLine();  
        System.out.print("Introduce tu edad: ");  
        age = keyboard.nextInt();  
  
        if (age >= 18) {  
            System.out.printf("Bienvenido/a %s\n", name);  
            System.out.println("Eres mayor de edad");  
        }  
  
    }  
  
    public static void main(String[] args) {  
  
        new If().show();  
  
    }  
  
}
```

3.2 Condicional doble

Es igual que la anterior, solo que se añade un apartado **else** que contiene instrucciones que se ejecutarán si la expresión evaluada por el if es falsa.

```

if(expresiónLógica){
    instrucciones    //se ejecutan si la expresión lógica es verdadera
    ....
}
else{
    instrucciones    //se ejecutan si la expresión lógica es falsa
    ...
}

```

Ejemplo:

```

package tema2_1_SentenciasCondicionales;

import java.util.Scanner;

public class IfElse {

    @SuppressWarnings("resource")
    public void show() {

        Scanner keyboard = new Scanner(System.in);
        int age;
        String name;

        System.out.print("Introduce tu nombre: ");
        name = keyboard.nextLine();
        System.out.print("Introduce tu edad: ");
        age = keyboard.nextInt();

        if (age >= 18) {
            System.out.printf("Bienvenido/a %s\n", name);
            System.out.println("Eres mayor de edad");
        } else {
            System.out.printf("Bienvenido/a %s\n", name);
            System.out.println("Eres menor de edad");
        }

    }

    public static void main(String[] args) {

        new IfElse().show();

    }

}

```


3.3 Condicional múltiple

3.3.1 Anidación

Dentro de una sentencia *if* se puede colocar otra sentencia *if*. A esto se le llama anidación y permite crear programas donde se valoren expresiones complejas. La nueva sentencia puede ir tanto en la parte *if* como en la parte *else*.

Las anidaciones se utilizan muchísimo al programar. Solo hay que tener en cuenta que siempre se debe cerrar primero el último *if* que se abrió. Es muy importante también tabular el código correctamente para que las anidaciones sean legibles.

Ejemplo:

```
if (x==1) {  
    instrucciones  
    ...  
}  
else {  
    if(x==2) {  
        instrucciones  
        ...  
    }  
    else {  
        if(x==3) {  
            instrucciones  
            ...  
        }  
    }  
}
```

Una forma más legible de escribir ese mismo código dando lugar a la llamada estructura **if-else-if** sería:

```
if (x==1) {  
    instrucciones  
    ...  
}  
else if (x==2) {  
    instrucciones  
    ...  
}  
else if (x==3) {  
    instrucciones  
    ...  
}
```

Se van comprobando las condiciones en orden y cuando una condición se cumpla, se ejecutan sus instrucciones y después se sale de la estructura *if-else-if* ya que esta estructura se utiliza cuando solamente se cumple una de las condiciones. Por eso, no estaría bien hacer lo siguiente:

```

if (x==1){    //Forma incorrecta de programar
    instrucciones
    ...
}
if (x==2){
    instrucciones
    ...
}
if (x==3){
    instrucciones
    ...
}

```

El motivo de que no sea adecuado es porque se pierde tiempo en comprobar todas las condiciones. Por ejemplo, si x vale 1, se ejecutan sus instrucciones correspondientes y luego se comprobaría si x vale 2, si x vale 3, etc. cuando no se va a cumplir ninguna más ya que si x vale 1 no puede valer ni 2 ni 3.

Ejemplo:

```

package tema2_1_SentenciasCondicionales;

import java.util.Scanner;

public class IfElseIf {

    @SuppressWarnings("resource")
    public void show() {

        Scanner keyboard = new Scanner(System.in);
        int age;

        System.out.print("Introduce tu edad: ");
        age = keyboard.nextInt();

        if (age >= 18) {
            System.out.println("Eres mayor de edad");
        } else if (age >= 16 && age < 18) {
            System.out.println("Eres menor de edad pero tienes ciertos privilegios");
        } else if (age >= 14 && age < 16) {
            System.out.println("Eres menor de edad y además no tienes privilegios");
        } else {
            System.out.println("Eres menor de edad");
        }

    }

    public static void main(String[] args) {

        new IfElseIf().show();

    }
}

```

```
}
```

3.3.2 Switch

```
switch(expresión) {  
    case valor1 :  
        instrucciones  
        break; // opcional  
  
    case valor2 :  
        instrucciones  
        break; // opcional  
    ....  
    ....  
    default : // opcional  
        instrucciones  
        break; // opcional  
}
```

El cuerpo de una sentencia switch se conoce como **bloque switch**.

Esta sentencia evalúa una expresión y cada **case** contiene un posible valor del resultado de dicha expresión; si efectivamente el resultado equivale a ese valor, se ejecutan las instrucciones de ese *case* y de los siguientes.

La instrucción **break** se utiliza para salir del *switch*. De tal modo que si queremos que para un determinado valor se ejecuten las instrucciones de un apartado *case* y solo las de ese apartado, entonces habrá que finalizar ese *case* con un *break*. Cuando se alcanza una sentencia *break*, el *switch* termina y el flujo de control salta a la siguiente línea que sigue a la sentencia *switch*.

Fall through condition (condición de caída): esta condición se produce en la sentencia *switch* cuando no se utiliza *break* en un *case* y causa la ejecución de los siguientes *case* hasta que no se produce un *break* o se sale de la sentencia *switch*. Aunque a veces es necesario, se percibe como propenso a errores.

El bloque **default** sirve para ejecutar instrucciones para los casos en los que la expresión no se ajuste a ningún *case*. Técnicamente, el *break* del *default* no es necesario porque el flujo se sale de la sentencia *switch*, pero se recomienda utilizarlo para que la modificación del código sea más fácil y menos propenso a errores.

Funciona con los tipos de datos primitivos, con los tipos enumerados, con las cadenas (a partir de Java 7) y con los wrappers.

Ejemplo:

```
package tema2_1_SentenciasCondicionales;  
  
import java.util.Scanner;  
  
public class Switch1 {  
  
    @SuppressWarnings("resource")  
    public void show() {  
  
        Scanner keyboard = new Scanner(System.in);
```

```

    int weekday;

    System.out.print("Introduce un número del 1 al 7 correspondiente al día
de la semana: ");
    weekday = keyboard.nextInt();

    switch (weekday) {
    case 1:
        System.out.println("Lunes");
        break;
    case 2:
        System.out.println("Martes");
        break;
    case 3:
        System.out.println("Miércoles");
        break;
    case 4:
        System.out.println("Jueves");
        break;
    case 5:
        System.out.println("Viernes");
        break;
    case 6:
        System.out.println("Sábado");
        break;
    case 7:
        System.out.println("Domingo");
        break;
    default:
        System.out.println("Día incorrecto");
        break;
    }

}

public static void main(String[] args) {

    new Switch1().show();

}

}

```

Ejemplo:

```

package tema2_1_SentenciasCondicionales;

import java.util.Scanner;

public class Switch2 {

    @SuppressWarnings("resource")
    public void show() {

        Scanner keyboard = new Scanner(System.in);
    }
}

```

```

    int month, year, numDays = 0;

    System.out.print("Introduce un número del 1 al 12 correspondiente a un
mes: ");
    month = keyboard.nextInt();
    System.out.print("Introduce un año: ");
    year = keyboard.nextInt();

    switch (month) {
    case 1:
    case 3:
    case 5:
    case 7:
    case 8:
    case 10:
    case 12:
        numDays = 31;
        break;
    case 4:
    case 6:
    case 9:
    case 11:
        numDays = 30;
        break;
    case 2://Se calcula si es un año bisiesto
        if (((year % 4 == 0) && !(year % 100 == 0)) || (year % 400 == 0)) {
            numDays = 29;
        } else {
            numDays = 28;
        }
        break;
    default:
        System.out.println("Mes inválido");
        break;
    }
    if (month >= 1 && month <= 12) {
        System.out.printf("Número de días del mes %d del año %d: %d", month,
year, numDays);
    }

}

public static void main(String[] args) {

    new Switch2().show();

}

}

```

La decisión de utilizar sentencias *if-else-if* o una sentencia *switch* se basa en la legibilidad y en la expresión que la sentencia está probando. Una sentencia *if-else-if* puede comprobar expresiones basadas en rangos de valores o condiciones, mientras que una sentencia *switch* comprueba expresiones basadas solo en un único valor.

El switch clásico es una sentencia, no una expresión. En la versión 14 de java aparece una nueva sintaxis para el switch que puede ser utilizada como una expresión. Para ello se utiliza el operador `->` que indica que cada rama produce un valor. El operador `->` denota la ausencia de fall through.

```
package tema2_1_SentenciasCondicionales;

import java.util.Random;

public class Switch3 {

    public void show() {

        Random random = new Random();
        int seasonCode;
        String seasonName;

        seasonCode = random.nextInt(4);
        System.out.println(seasonCode);
        /* Al ser el switch una expresión, se puede
         * asignar el valor devuelto a una variable:
         */
        seasonName = switch (seasonCode) {
            case 0 -> "Spring";
            case 1 -> "Summer";
            case 2 -> "Fall";
            case 3 -> "Winter";
            default -> "???";
        };

        System.out.println(seasonName);

    }

    public static void main(String[] args) {

        new Switch3().show();

    }

}
```

Se pueden proporcionar varias etiquetas para cada caso separadas por comas:

```
package tema2_1_SentenciasCondicionales;

import java.util.Random;

public class Switch4 {

    public void show() {

        Random random = new Random();
        int seasonCode, numLetters;
```

```

String seasonName;

seasonCode = random.nextInt(4);
seasonName = switch (seasonCode) {
    case 0 -> "Spring";
    case 1 -> "Summer";
    case 2 -> "Fall";
    case 3 -> "Winter";
    default -> "???";
};

System.out.println(seasonName);

numLetters = switch (seasonName) {
    //Varias etiquetas separadas por comas:
    case "Spring", "Summer", "Winter" -> 6;
    case "Fall" -> 4;
    default -> -1;
};

System.out.println(numLetters);

}

public static void main(String[] args) {

    new Switch4().show();

}

}

```

También se pueden poner bloques (conjuntos de sentencias delimitadas por llaves) detrás del operador `->`. En este caso, el bloque debe salir a través de una sentencia `yield` que devuelva el valor del bloque. Al igual que `return` sale de un método con un valor, `yield` sale del bloque con un valor.

```

package tema2_1_SentenciasCondicionales;

import java.util.Random;

public class Switch5 {

    public void show() {

        Random random = new Random();
        int seasonCode;
        String seasonName;

        seasonCode = random.nextInt(4);
        seasonName = switch (seasonCode) {
            /* El bloque debe salir a través de una
             * sentencia yield que devuelva el valor
             * del bloque:
             */

```

```

        case 0 -> {
            System.out.println("spring time!");
            yield "Spring";
        }
        case 1 -> "Summer";
        case 2 -> "Fall";
        case 3 -> "Winter";
        default -> "???";
    };

    System.out.println(seasonName);

}

public static void main(String[] args) {

    new Switch5().show();

}

}

```

El operador `->` denota la ausencia de fall through. Por simetría, se proporciona una versión de las expresiones switch con fall through usando `:` en lugar de `->`:

```

package tema2_1_SentenciasCondicionales;

import java.util.Random;

public class Switch6 {

    public void show() {

        Random random = new Random();
        int seasonCode;
        String seasonName;

        seasonCode = random.nextInt(4);
        System.out.println(seasonCode);
        seasonName = switch (seasonCode) {
            case 0:
                System.out.println("spring time!");
            case 1:
                yield "Summer";
            case 2:
                yield "Fall";
            case 3:
                yield "Winter";
            default:
                yield "???";
        };

        System.out.println(seasonName);

    }

}

```



```
public static void main(String[] args) {  
  
    new Switch6().show();  
  
}  
  
}
```

En el caso de que el código de la estación salga 0, se produce fall through dando el siguiente resultado por consola:

```
0  
spring time!  
Summer
```

2.2 Bucles

2.2 Bucles

1. Introducción
2. Bucle while
3. Bucle do-while
4. Bucle for
5. Bucle for-each

1. Introducción

En programación, un **bucle** o *instrucción de control repetitiva* (o *iterativa*) permite ejecutar una o más instrucciones varias veces, es decir, permite ejecutar un bloque de instrucciones repetidamente, escribiéndolas una sola vez en el programa, reduciendo de este modo el código del mismo. Cada vuelta que da el bucle se conoce como **iteración**.

Las variables que se utilizan en los bucles pueden tener distintas funciones:

- **índice:** es una variable que apunta a un recurso concreto. En los bucles, apunta al número de iteración. Se suelen utilizar por convenio los identificadores de variables *i, j* y *k*.
- **contador:** cuenta cuántas veces ocurre algo.
- **acumulador:** acumula operaciones parciales.

Las instrucciones básicas que permiten construir este tipo de estructuras son while, do-while y for.

2. Bucle while

El bucle while agrupa instrucciones las cuales se ejecutan continuamente mientras una condición que se evalúa sea verdadera. La condición se evalúa antes de entrar dentro del while y cada vez que se termina de ejecutar las instrucciones del while.

```
while (expresiónLógica) {  
    instrucciones //se ejecutan si la expresion lógica es true  
}
```

El programa se ejecuta siguiendo estos pasos:

1. Se evalúa la expresión lógica.
2. Si la expresión es verdadera ejecuta las instrucciones, sino el programa abandona la sentencia while.
3. Tras ejecutar las instrucciones, volvemos al paso 1.

```
package tema2_2_Bucles;  
  
import java.util.Random;  
  
public class while1 {
```

```

public void show() {

    int number;
    Random random = new Random();

    /*
     * Se obtiene un número aleatorio entre 1 y 500.
     * Mientras dicho número sea par, continuamos
     * en el bucle while.
     */
    while ((number = random.nextInt(500) + 1) % 2 == 0) {
        System.out.println(number);
    }

}

public static void main(String[] args) {

    new while1().show();

}

}

```

3. Bucle do-while

```

do {
    instrucciones
} while (expresiónLógica);

```

La única diferencia respecto al while está en que la expresión lógica se evalúa después de haber ejecutado las instrucciones. Es decir, el bucle al menos se ejecuta una vez. Los pasos son los siguientes:

1. Ejecutar instrucciones.
2. Evaluar expresión lógica.
3. Si la expresión es verdadera, volver al paso 1. Si es falsa, salir del do-while.

Ejemplo que muestra números aleatorios entre 1 y 500 hasta que salga uno múltiplo de 7:

```

package tema2_2_Bucles;

import java.util.Random;

public class Dowhile1 {

    public void show() {

        boolean exit;
        int number;
        Random random = new Random();

        do {

```

```

        number = random.nextInt(500) + 1; //Se calcula un número aleatorio
entre 1 y 500
        System.out.println(number);
        exit = (number % 7 == 0); //exit se pone a true si el número es
múltiplo de 7
    } while (!exit); // Condición de repetición: que exit sea false

}

public static void main(String[] args) {

    new Dowhile1().show();

}

}

```

Si le añadimos al ejemplo anterior que solamente haya 5 posibilidades para encontrar el múltiplo de 7, eso significa que necesitamos una variable que apunte al número de iteración del bucle para poder controlar que solamente se ejecute 5 veces, es decir, necesitamos un índice:

```

package tema2_2_Bucles;

import java.util.Random;

public class Dowhile2 {

    public void show() {

        boolean exit;
        int number, i = 1; //i es un índice porque apunta al número de iteración
del bucle
        Random random = new Random();

        do {
            number = random.nextInt(500) + 1;
            System.out.println(number);
            exit = (number % 7 == 0);
            i++;
        } while (!exit && i <= 5); //Mientras exit sea false y además i sea menor
o igual a 5

        if (!exit) {
            System.out.println("No se encontró el múltiplo de 7");
        }

    }

    public static void main(String[] args) {

        new Dowhile2().show();

    }

}

```

```
}
```

El bucle do-while se utiliza cuando se sabe que las instrucciones del bucle se van a ejecutar al menos una vez. En el bucle while puede ser que no se ejecuten nunca si la condición es falsa desde un principio. Las peticiones de usuario se realizan con bucles do-while, ya que la petición del dato al usuario siempre se realiza, y si el usuario no introduce lo que le hemos pedido, entonces utilizamos el bucle para volvérselo a pedir.

```
package tema2_2_Bucles;

import java.util.Scanner;

public class Dowhile3 {

    @SuppressWarnings("resource")
    public void show() {

        Scanner keyboard = new Scanner(System.in);
        int number;
        do {
            System.out.print("Introduzca un número del 1 al 5: ");
            number = keyboard.nextInt();
            System.out.printf("Has introducido un %d\n", number);
        } while (number < 1 || number > 5);

    }

    public static void main(String[] args) {

        new Dowhile3().show();

    }

}
```

Veamos un ejemplo con índice, contador y acumulador:

```
package tema2_2_Bucles;

import java.util.Scanner;

public class Dowhile4 {

    @SuppressWarnings("resource")
    public void show() {

        Scanner keyboard = new Scanner(System.in);
        int number, counter = 0, sum = 0, index = 0;
        /*
         * index: es un índice porque apunta al número de iteración del bucle
         * counter: es un contador para contar el número de pares
         * sum: es un acumulador donde se acumula la suma de los números pares
         */
    }

}
```

```

do {
    System.out.print("Introduzca un número ó 0 para terminar: ");
    number = keyboard.nextInt();
    if (number != 0) {
        index++; //Se incrementa el número de iteración del bucle
        System.out.printf("El número introducido en la iteración %d es %d\n", index, number);
        if (number % 2 == 0) { //Si el número es par
            counter++; //Se incrementa el contador
            sum += number; //Se acumula la suma en el acumulador
        }
    }
} while (number != 0);

System.out.printf("En %d iteraciones se han introducido %d números pares cuya suma vale %d", index, counter, sum);

}

public static void main(String[] args) {

    new Dowhile4().show();

}

}

```

4. Bucle for

```

for(inicialización;condición;incremento){
    instrucciones
}

```

El funcionamiento es el siguiente:

1. Se ejecuta la instrucción de inicialización.
2. Se comprueba la condición.
3. Si la condición es cierta, entonces se ejecutan las instrucciones. Si la condición es falsa, se abandona el bloque for.
4. Tras ejecutar las instrucciones, se ejecuta la instrucción de incremento y se vuelve al paso 2.

Ejemplo para escribir números del 1 al 100:

```

package tema2_2_Bucles;

public class For1 {

    public void show() {

        int i;

        for (i = 1; i <= 100; i++) { //i es un índice porque apunta al número de iteración del bucle

```

```

        System.out.println(i);
    }

}

public static void main(String[] args) {

    new For1().show();

}

}

```

En el eclipse, si seleccionamos varias líneas de código, las podemos fácilmente insertar en un if o en un bucle: Menú *Source* → *Surround with*.

También es posible declarar la variable dentro del propio bucle for, pero su ámbito de vida se reduce exclusivamente al bucle for, es decir, no se conoce fuera del for ya que muere en cuanto el bucle finalice:

```

package tema2_2_Bucles;

public class For2 {

    public void show() {

        for (int i = 1; i <= 100; i++) {//se declara la variable i como int dentro del for
            System.out.println(i);
        }
        i = 1; //Error de compilación

    }

    public static void main(String[] args) {

        new For2().show();

    }

}

```

El bucle for también admite más de una variable, en cuyo caso, en las partes de inicialización e incremento, se utiliza como separador la coma ,:

```

package tema2_2_Bucles;

public class For3 {

    public void show() {

        for (int i = 0, j = 10; i <= j; i++, j--) {
            System.out.printf("i: %d j: %d\n", i, j);
        }

    }

}

```

```

    }

    public static void main(String[] args) {

        new For3().show();

    }

}

```

Los bucles for también se pueden hacer con while o do-while pero los for tienen una sintaxis más abreviada. Veamos el ejemplo *For1* hecho con un while:

```

package tema2_2_Bucles;

public class while2 {

    public void show() {

        int i = 1;

        while (i <= 100) {
            System.out.println(i);
            i++;
        }

    }

    public static void main(String[] args) {

        new while2().show();

    }

}

```

Entonces, si todos los bucles se pueden hacer con for, while y do-while, ¿cuál debemos usar? Es importante que el programador utilice la estructura más adecuada en cada caso:

- Cómo saber cuándo hay que utilizar un for: cuando se sepa el número de veces que se va a ejecutar el bucle, es decir, cuando se conozcan el número de iteraciones de antemano o lo tengamos almacenado en alguna variable. Por motivos de legibilidad de código, es muy importante que las variables del for aparezcan en las 3 partes del for, es decir, en la inicialización, en la condición y en el incremento, ya que el programador que quiera saber cuántas iteraciones realiza el bucle, solamente con fijarse en la línea de código donde se encuentra el for, pueda saberlo. Si las variables del for no aparecen en las 3 partes, eso nos indica que es más conveniente realizar un while o un do-while.
- Si no se sabe de antemano el número de iteraciones, entonces hay que utilizar while o do-while:
 - Cómo saber cuándo hay que utilizar un do-while: cuando las instrucciones del bucle se van a ejecutar al menos una vez.

- Cómo saber cuándo hay que utilizar un while: cuando pueda ocurrir que las instrucciones del bucle no se ejecuten nunca si la condición es falsa desde un principio.

Si las variables del for no aparecen en las 3 partes, eso nos indica que es más conveniente realizar un while o un do-while. Veamos el ejemplo que muestra números aleatorios entre 1 y 500 hasta que salga uno múltiplo de 7:

```
package tema2_2_Bucles;

import java.util.Random;

public class For4 {

    public void show() {

        boolean exit = false;
        int number = 0, i;
        Random random = new Random();

        for (i = 1; !exit; i++) { //Este for no es legible, se debe sustituir por
un do-while
            number = random.nextInt(500) + 1;
            System.out.printf("Iteración %d, número: %d\n", i, number);
            exit = (number % 7 == 0);
        }
        System.out.printf("El múltiplo de 7 con valor %d se ha encontrado en la
iteración %d", number, i - 1);

    }

    public static void main(String[] args) {

        new For4().show();

    }

}
```

Si le añadimos al ejemplo anterior que solamente haya 5 posibilidades para encontrar el múltiplo de 7:

```
package tema2_2_Bucles;

import java.util.Random;

public class For5 {

    public void show() {

        boolean exit = false;
        int number = 0, i;
        Random random = new Random();
        /*
        * Ahora el for sí es legible porque la variable i
        * aparece en las 3 partes: en la inicialización,
```

```

        * en la condición y en el incremento
        */
        for (i = 1; i <= 5 && !exit; i++) {
            number = random.nextInt(500) + 1;
            System.out.printf("Iteración %d, número: %d\n", i, number);
            exit = (number % 7 == 0);
        }
        if (exit) {
            System.out.printf("El múltiplo de 7 con valor %d se ha encontrado en
la iteración %d", number, i - 1);
        } else {
            System.out.println("El múltiplo de 7 no se ha encontrado");
        }
    }

    public static void main(String[] args) {

        new For5().show();

    }

}

```

También es muy importante por motivos de legibilidad del código, que el único sitio donde se modifiquen los valores de las variables del for sea en la zona del incremento ya que el programador que quiera saber cuántas iteraciones realiza el bucle, solamente con ver la línea de código del for, pueda saberlo. Por ejemplo, este código no es adecuado ya que para salirse del bucle del for, se está modificando la variable *i*.

```

package tema2_2_Bucles;

import java.util.Random;

public class For6 {

    public void show() {

        int number = 0, i;
        Random random = new Random();

        for (i = 1; i <= 5; i++) {
            number = random.nextInt(500) + 1;
            System.out.printf("Iteración %d, número: %d\n", i, number);
            if (number % 7 == 0) {
                i = 5; //No se pueden modificar las variables del for, solamente
en el incremento
            }
        }
        if (number % 7 == 0) {
            System.out.printf("El múltiplo de 7 encontrado vale %d", number);
        } else {
            System.out.println("El múltiplo de 7 no se ha encontrado");
        }
    }
}

```

```

    }

    public static void main(String[] args) {

        new For6().show();

    }

}

```

5. Bucle for-each

Una de las cosas que incorporó Java 5 fue el bucle for-each. Esta estructura nos permite recorrer una colección o un array de elementos de una forma sencilla, evitando el uso de iteradores o de un bucle for normal.

```

package tema2_2_Bucles;

public class ForEachArrays {

    public void show() {

        int[] grades = { 8, 7, 9 };

        for (int grade : grades) {
            System.out.printf("Nota: %d\n", grade);
        }

    }

    public static void main(String[] args) {

        new ForEachArrays().show();

    }

}

```

También se usa el for-each para recorrer los valores de un enum. Para ello, se utiliza el método *values()* de los enum que está implícitamente declarado por el compilador:

```

package tema2_2_Bucles;

public class ForEachEnum {

    public enum DayOfWeek {
        MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY
    }

    public void show() {

        for (DayOfWeek d : DayOfWeek.values()) {
            System.out.println(d);
        }

    }

}

```

```
}  
  
public static void main(String[] args) {  
  
    new ForEachEnum().show();  
  
}  
  
}
```

2.3 Funciones

2.3 Funciones

1. Introducción
2. Construcción
3. Llamada a la función
4. Ámbito de vida de los parámetros
5. Ejemplo de función: el factorial de un número
6. Ejemplo de procedimiento
7. Resultado de las funciones
8. Recursividad
9. La pila

1. Introducción

Una subrutina o subprograma, como idea general, se presenta como un subalgoritmo que forma parte del algoritmo principal, el cual permite resolver una tarea específica.

Podemos distinguir tres términos que poseen diferencias:

1. **Función**: conjunto de instrucciones que devuelven un resultado.
2. **Procedimiento**: conjunto de instrucciones que se ejecutan sin devolver ningún resultado.
3. **Método**: función o procedimiento que pertenece a un objeto.

Dichas funciones se pueden utilizar desde muchos sitios diferentes, por lo que de manera general, no se suelen poner mensajes en consola en las funciones ya que puede ser que dichos mensajes no interesen en todos los sitios donde se utilice dicha función. A no ser que la función se haya creado específicamente para dar mensajes informativos en consola.

2. Construcción

Una función se construye de la siguiente manera:

```
modificador_acceso tipo_resultado nombre_función (tipo_parámetro
nombre_parámetro, ... ) {
    instrucciones
    return expresión;
}
```

- **modificador_acceso**: es la visibilidad que posee la función (Lo veremos más adelante. De momento, lo utilizaremos como public).
- **tipo_resultado**: es el tipo del resultado que devuelve la función.
- **nombre_función**: es el nombre que identifica a la función. Utiliza la notación lowerCamelCase. Ejemplo: imprimirResultadoDecimal.

- **tipo_parámetro nombre_parámetro, ...:** puede ocurrir que la función necesite ciertos valores para efectuar la misión para la que ha sido creada. Por ejemplo, la función suma necesitaría los valores que tiene que sumar. En este caso, se deben indicar cada uno de dichos valores con sus tipos correspondientes. A estos valores se les conoce como parámetros de la función. Si la función no necesita parámetros, entonces solamente se ponen los paréntesis: `nombre_funcion( )`.

A todo esto se le conoce como la **firma** (signature) de la función:

```
modificador_acceso tipo_resultado nombre_función (tipo_parámetro
nombre_parámetro, ... )
```

- **instrucciones:** instrucciones que conforman el algoritmo de la función, para que realice la misión para la que ha sido creada.
- **return expresión :** con el return se termina la ejecución de la función y si va acompañado de una expresión, la función devuelve como resultado el valor de dicha expresión. Dicho valor tiene que ser del *tipo_resultado* indicado en la firma de la función.

Ejemplo de función con dos parámetros:

```
public int add(int sum1,int sum2) {
    return sum1+sum2;
}
```

Puede ser también que haya más de un return. En ese caso, el flujo de ejecución abandona la función en cuanto ejecute el primer return. Ejemplo:

```
public boolean isEven(int n){
    if(n%2==0){
        return true;
    }
    else{
        return false;
    }
}
```

En el caso anterior, como la expresión booleana ya devuelve *true* o *false*, también se podía haber devuelto la expresión directamente:

```
public boolean isEven(int n) {
    return n % 2 == 0;
}
```

En el caso de que estemos definiendo un **procedimiento**, no tendremos `return expresión` ya que no devuelve ningún resultado y el tipo_resultado es *void*. Como por ejemplo `system.out.println`, que escribe en pantalla lo que recibe por parámetro pero no devuelve nada.

3. Llamada a la función

Una función o procedimiento nos permite que reutilicemos un algoritmo ya que se puede utilizar cuando nos haga falta. Para ello, solamente tendremos que **llamar a la función o al procedimiento** por su nombre y pasarle los parámetros en el mismo orden que se han definido y pertenecientes al mismo tipo de dato o compatible. En la llamada, dichos parámetros se llaman **argumentos**, es decir, los argumentos son los valores iniciales de los parámetros.

```
package tema2_3_Funciones;

public class Functions1 {

    public void show() {

        boolean isAnEvenNumber;
        int result;

        isAnEvenNumber = isEven(5); //Se llama a la función isEven con un valor de
5 en el argumento
        showBoolean(isAnEvenNumber); //Se llama al procedimiento showBoolean con
la variable isAnEvenNumber como argumento
        isAnEvenNumber = isEven(4); //Se llama a la función isEven con un valor de
4 en el argumento
        showBoolean(isAnEvenNumber); //Se llama al procedimiento showBoolean con
la variable isAnEvenNumber como argumento

        result = add(5, 2); //Se llama a la función add con los valores 5 y 2 en
los argumentos
        showInt(result); //Se llama al procedimiento showInt con la variable
result como argumento

    }

    public boolean isEven(int n) { //Función que devuelve true si n es par

        return n % 2 == 0;

    }

    public int add(int sum1, int sum2) { //Función que suma sum1 y sum2

        return sum1 + sum2;

    }

    public void showBoolean(boolean b) { //Procedimiento que muestra por consola
el valor de b

        System.out.println(b);

    }

    public void showInt(int n) { //Procedimiento que muestra por consola el valor
de n
```

```

        System.out.println(n);

    }

    public static void main(String[] args) {

        new Functions1().show();

    }

}

```

Salida por consola:

```

false
true
7

```

4. Ámbito de vida de los parámetros

A nivel de visibilidad y de ámbito de vida, los parámetros funcionan como las variables locales (Ver [Tema 1.1 Introducción al lenguaje Java 8.3 Ámbito de vida de las variables](#)), por lo tanto el ámbito de vida de los parámetros es el bloque donde han sido definidos, es decir, la propia función. Cada vez que se llame a la función, los parámetros nacen, se ejecuta la función y una vez que la función ha terminado de ejecutarse, los parámetros mueren.

```

package tema2_3_Funciones;

public class Functions2 {

    public void show() {

        boolean isAnEvenNumber;
        int result;

        isAnEvenNumber = isEven(5); //Nace el parámetro n con el valor 5
        //Aquí n ya no existe porque la función isEven ya ha terminado de
        ejecutarse
        System.out.println(isAnEvenNumber);
        isAnEvenNumber = isEven(4); //Vuelve a nacer n pero esta vez con un valor
        de 4
        //Aquí n ya no existe porque la función isEven ya ha terminado de
        ejecutarse
        System.out.println(isAnEvenNumber);

        result = add(5, 2); //Nacen los parámetros sum1 y sum2 con los valores 5 y
        2 respectivamente
        //Aquí sum1 y sum2 ya no existen porque la función add ha terminado de
        ejecutarse
        System.out.println(result);

    }

    public boolean isEven(int n) { //Comienzo del ámbito de vida del parámetro n

```



```

        return n % 2 == 0;

    }//Fin del ámbito de vida del parámetro n

    public int add(int sum1, int sum2) { //Comienzo del ámbito de vida de los
    parámetros sum1 y sum2

        return sum1 + sum2;

    }//Fin del ámbito de vida de los parámetros sum1 y sum2

    public static void main(String[] args) {

        new Functions2().show();

    }

}

```

Salida por consola:

```

false
true
7

```

5. Ejemplo de función: el factorial de un número

El factorial de un entero positivo n , también indicado como $n!$, se define como el producto de todos los números enteros positivos desde 1 hasta n . Por ejemplo:

$$5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$$

La operación de factorial aparece en muchas áreas de las matemáticas, particularmente en combinatoria y análisis matemático. De manera fundamental, el factorial de n representa el número de formas distintas de ordenar n objetos distintos (elementos sin repetición). Este hecho ha sido conocido desde hace varios siglos, en el siglo XII, por los hindúes.

Veamos cómo se programaría dicha función factorial y las llamadas con distintos tipos de argumentos:

```

package tema2_3_Funciones;

import java.util.Scanner;

public class Factorial1 {

    @SuppressWarnings("resource")
    public void show() {

        Scanner keyboard = new Scanner(System.in);
        int n, variable;

        System.out.println("Llamada a la función con argumentos literales: ");
        System.out.printf("El factorial de 5 es %d\n", factorial(5));
    }
}

```

```

        System.out.println("Llamada a la función usando una variable como
argumento: ");
        variable = 5;
        System.out.printf("El factorial de %d es %d\n", variable,
factorial(variable));

        System.out.println("Llamada a la función usando una expresión como
argumento: ");
        variable = 3;
        System.out.printf("El factorial de %d es %d\n", variable + 2,
factorial(variable + 2));

        System.out.println("Llamada a la función con argumentos introducidos por
el usuario: ");
        do {
            System.out.println("Introduzca un número entero positivo: ");
            n = keyboard.nextInt();
        } while (n <= 0);

        System.out.printf("El factorial de %d es %d", n, factorial(n));

    }

    public int factorial(int n) {

        int result = 1;

        for (int i = 2; i <= n; i++) {
            result *= i;
        }

        return result;
    }

    public static void main(String[] args) {

        new Factorial1().show();

    }

}

```

6. Ejemplo de procedimiento

No tendremos `return expresión` ya que no devuelve ningún resultado y el tipo_resultado es `void`:

```

package tema2_3_Funciones;

import static tema1_4_EscrituraEnPantalla.colores.Colors.RED;
import static tema1_4_EscrituraEnPantalla.colores.Colors.RESET;

import java.util.Scanner;

```

```

public class Procedure {

    @SuppressWarnings("resource")
    public void show() {

        Scanner keyboard = new Scanner(System.in);
        String string;

        System.out.print("Introduce una cadena: ");
        string = keyboard.nextLine();
        paintRed(string);

    }

    public void paintRed(String string) {

        System.out.printf("La cadena que has introducido en rojo: %s", RED +
string + RESET);

    }

    public static void main(String[] args) {

        new Procedure().show();

    }

}

```

7. Resultado de las funciones

En las llamadas a funciones, no hay que obligatoriamente utilizar el valor devuelto:

```

package tema2_3_Funciones;

import static tema1_4_EscrituraEnPantalla.colores.Colors.RED;
import static tema1_4_EscrituraEnPantalla.colores.Colors.RESET;

import java.util.Scanner;

public class Result {

    @SuppressWarnings("resource")
    public void show() {

        Scanner keyboard = new Scanner(System.in);
        String string, stringRed;

        System.out.print("Introduce una cadena: ");
        string = keyboard.nextLine();
        /*
        * En la llamada a la función turnRed,
        * no estamos utilizando el valor devuelto:
        */
        turnRed(string);
    }

}

```

```

        //En la siguiente llamada sí lo vamos a utilizar:
        stringRed = turnRed(string);
        System.out.printf("La cadena \"%s\" convertida a rojo: %s", string,
stringRed);

    }

    public String turnRed(String string) {

        return String.format("%s", RED + string + RESET);

    }

    public static void main(String[] args) {

        new Result().show();

    }

}

```

Puede darse el caso de que las funciones reciban argumentos no válidos para la funcionalidad para la que se ha creado la función. En dicho caso, como tienen que devolver un valor, se puede devolver un resultado que no sea posible para su funcionalidad. Por ejemplo, si el rango del resultado de una función son los números positivos, en ese caso, se puede devolver un -1. Pero si el rango es un número tanto positivo como negativo, entonces la función tendrá que lanzar una excepción. Lo que nunca puede devolver una función es un resultado que no sea cierto para la funcionalidad para la que ha sido creada dicha función.

El concepto del factorial se aplica a los números enteros positivos, pero como los *int* admiten números negativos, se podría llamar a la función con un número negativo aunque no tenga sentido:

```

package tema2_3_Funciones;

public class Factorial2 {

    public void show() {

        int result;

        result = factorial(-5); //No tiene mucho sentido porque el factorial se
aplica a números positivos
        System.out.printf("El factorial de %d es %d\n", -5, result);

    }

    public int factorial(int n) {

        int result = 1;

        for (int i = 2; i <= n; i++) {
            result *= i;
        }

        return result;
    }
}

```

```

    }

    return result;
}

public static void main(String[] args) {

    new Factorial2().show();

}

}

```

Salida por consola (no tiene sentido el resultado):

```
El factorial de -5 es 1
```

Es decir, cuando programamos una función, no podemos dar por hecho que el programador que la vaya a utilizar lo haga de manera adecuada con la lógica que representa su funcionalidad. Así que siempre que programemos una función, debemos asegurarnos que va a funcionar correctamente para todos los valores posibles del argumento. En nuestro caso, no podemos dar un resultado coherente para los números negativos puesto que no tiene sentido matemáticamente el factorial de un número negativo, así que lo más conveniente es lanzar un error cuando llamen a la función con un número negativo.

```

package tema2_3_Funciones;

public class Factorial3 {

    public void show() {

        int result;

        result = factorial(-5); //Al que utilice la función con números negativos
        //le salta un error
        System.out.printf("El factorial de %d es %d\n", -5, result);

    }

    public int factorial(int n) {

        int result = 1;

        if (n < 0) {
            throw new IllegalArgumentException("El factorial se aplica a números
            positivos");
        }

        for (int i = 2; i <= n; i++) {
            result *= i;
        }

        return result;
    }
}

```

```

    public static void main(String[] args) {

        new Factorial3().show();

    }

}

```

Salida por consola:

```

Exception in thread "main" java.lang.IllegalArgumentException: El factorial se
aplica a números positivos
    at tema2_3_Funciones.Factorial3.factorial(Factorial3.java:19)
    at tema2_3_Funciones.Factorial3.main(Factorial3.java:9)

```

8. Recursividad

La recursividad es una técnica de escritura de funciones pensada para problemas complejos. La idea parte de que una función pueda llamarse a sí misma.

Esta técnica es peligrosa ya que se pueden generar fácilmente llamadas infinitas (la función se llama a sí misma, tras la llamada se vuelve a llamar a sí misma, y así sucesivamente sin freno ni control). Por lo tanto, es muy importante tener en cuenta cuándo la función debe dejar de llamarse.

Hay que ser muy cauteloso cuando se utiliza la recursividad, pero permite soluciones muy originales y abre la posibilidad de solucionar problemas muy complejos.

Veamos como ejemplo la versión recursiva del factorial:

```

package tema2_3_Funciones;

import java.util.Scanner;

public class RecursiveFactorial1 {

    @SuppressWarnings("resource")
    public void show() {

        Scanner keyboard = new Scanner(System.in);
        int n;

        do {
            System.out.println("Introduzca un número entero positivo: ");
            n = keyboard.nextInt();
        } while (n <= 0);

        System.out.printf("El factorial de %d es %d", n, factorial(n));

    }

    public int factorial(int n) {

        int result;
    }
}

```

```

        if (n == 1 || n == 0) { //Caso base: devuelve un 1
            result = 1;
        } else { //Caso recursivo
            result = n * factorial(n - 1);
        }

        return result;
    }

    public static void main(String[] args) {

        new RecursiveFactorial1().show();

    }
}

```

Para entenderlo mejor, vamos a imprimir los resultados intermedios añadiendo la siguiente línea de código en la función recursiva *factorial*:

```
System.out.printf("Factorial de %d
Resultado: %d%n",n,result);
```

```

package tema2_3_Funciones;

import java.util.Scanner;

public class RecursiveFactorial2 {

    @SuppressWarnings("resource")
    public void show() {

        Scanner keyboard = new Scanner(System.in);
        int n;

        do {
            System.out.println("Introduzca un número entero positivo: ");
            n = keyboard.nextInt();
        } while (n <= 0);

        System.out.printf("El factorial de %d es %d", n, factorial(n));

    }

    public int factorial(int n) {

        int result;

        if (n == 1 || n == 0) { //Caso base: devuelve un 1
            result = 1;
        } else { //Caso recursivo
            result = n * factorial(n - 1);
        }

        System.out.printf("Factorial de %d Resultado: %d%n", n, result);
    }
}

```

```

        return result;
    }

    public static void main(String[] args) {

        new RecursiveFactorial2().show();

    }

}

```

¿recursividad o iteración?

Hay otra versión del factorial resuelto mediante un bucle for (solución iterativa) en lugar de utilizar la recursividad. La cuestión es ¿cuál es mejor? Ambas implican sentencias repetitivas hasta llegar a una determinada condición, por lo que ambas pueden generar programas que no finalizan si la condición nunca se cumple. En el caso de la iteración es una condición la que permite determinar el final, la recursividad lo que hace es ir simplificando el problema hasta generar una llamada a la función que devuelva un valor y no se vuelva a llamar. Para un ordenador es más costosa la recursividad ya que implica realizar muchas llamadas a funciones, es decir, es más rápida la solución iterativa. Entonces, ¿por qué elegir recursividad? La recursividad se utiliza solo si:

- No encontramos la solución iterativa a un problema.
- El código es mucho más claro en su versión recursiva.

9. La pila

Una pila(**stack**) es una lista ordenada o estructura de datos que permite almacenar y recuperar datos, el modo de acceso a sus elementos es de tipo LIFO (del inglés *Last In, First Out*, «último en entrar, primero en salir»). Esta estructura se aplica en multitud de supuestos en el área de informática debido a su simplicidad y capacidad de dar respuesta a numerosos procesos.

Para el manejo de los datos cuenta con dos operaciones básicas: apilar (**push**), que coloca un objeto en la pila, y su operación inversa, desapilar (**pop**), que retira el último elemento apilado.

En cada momento sólo se tiene acceso a la parte superior de la pila, es decir, al último objeto apilado (denominado TOS, *Top of Stack*). La operación desapilar permite la obtención de este elemento, que es retirado de la pila permitiendo el acceso al anterior (apilado con anterioridad), que pasa a ser el último, el nuevo TOS.

Para las llamadas entre funciones, se utiliza una estructura de tipo pila: supongamos que se está procesando una función y en su interior llama a otra función. La función se abandona para procesar la función de la llamada, pero antes se almacena en una pila la dirección que apunta a la función. Ahora supongamos que esa nueva función llama a su vez a otra función. Igualmente, se almacena su dirección, se abandona y se atiende la petición. Así en tantos casos como existan peticiones. La ventaja de la pila es que no requiere definir ninguna estructura de control ni conocer las veces que el programa estará saltando entre funciones para después retomarlas, con la única limitación de la capacidad de almacenamiento de la pila. Conforme se van cerrando las funciones, se van rescatando las funciones precedentes mediante sus direcciones almacenadas en la pila y se va concluyendo su proceso, esto hasta llegar a la primera.

En el caso de una función recursiva, esto es posible implementarlo con sencillez mediante una pila. La función se llama a sí misma tantas veces como sea necesario hasta que el resultado de la función cumpla la condición de retorno; entonces, todas las funciones abiertas van completando su proceso en cascada. No se necesita saber cuántas veces se anidará y, por tanto, tampoco cuando se cumplirá la condición, con la única limitación de la capacidad de la pila. De sobrepasarse ese límite, normalmente porque se entra en un bucle sin final, se produce el «error de desbordamiento de la pila» (***stack overflow***).

3.1 Cadenas

3.1 Cadenas

1. Introducción
2. Inmutabilidad
3. Formatos de codificación de caracteres
4. Formateo de cadenas
5. Métodos de la clase `String`
6. La interfaz `CharSequence`
7. La clase `StringBuilder`

1. Introducción

El texto es uno de los tipos de datos más importantes y por ello Java lo trata de manera especial. Para Java las cadenas de texto son objetos especiales porque en determinados aspectos funcionan como los primitivos y en otros aspectos funcionan como los objetos.

Las cadenas deben manejarse creando objetos de tipo **String**. Se pueden inicializar de dos maneras:

- Usando el operador asignación como los primitivos: `String s="hola";`
- Usando el constructor como los objetos: `String s=new String("hola");`

Los **literales** cadena se escriben entre comillas dobles: `"Esto es un literal cadena"`.

En java existe también la **cadena vacía** (`""`), es decir, una cadena sin ningún carácter.

Ejemplo: `String s=""`; A la variable `s` se le está asignando la cadena vacía.

Algunas veces, para comparar que una variable de tipo `String` es equivalente a una determinada constante de cadena se usa la construcción `"Hello".equals(message)`, ya que dicho construcción no puede lanzar `NullPointerException` si `message` es `null`, sino que tan sólo retornará `false`, mientras que `message.equals("Hello")` lanzaría `NullPointerException` en ese caso.

Los *Text Blocks* (Bloques de Texto) permiten definir cadenas de texto multilínea de manera más sencilla y legible. Los bloques de texto comienzan y terminan con tres comillas dobles `"""`. En el ejemplo se pueden observar dos maneras de definir la misma cadena donde la segunda (`s2`) está utilizando *Text Blocks*:

```
String s1 = "Línea 1\nLínea 2\nLínea 3";
String s2 = """
    Línea 1
    Línea 2
    Línea 3
    """;
```

Veamos otro ejemplo:

```
String s1 = ""
    Línea 1
    Línea 2
    Línea 3
    ""
string s2 = ""
    Línea 1
    Línea 2
    Línea 3
    ""
System.out.println(s1);
System.out.println(s2);
```

Salida por consola:

```
Línea 1
Línea 2
Línea 3

    Línea 1
    Línea 2
    Línea 3
```

El compilador escapará e interpretará los caracteres especiales de modo que no tenemos que usar la barra invertida en los *Text Blocks* para escapar.

2. Inmutabilidad

Uno de los conceptos que suele venir asociado a la programación funcional es el de la inmutabilidad. Si bien es cierto que la inmutabilidad es una idea que no es exclusiva de la programación funcional, sí que cobra una importancia vital en este tipo de lenguajes. La programación funcional se asienta sobre muchos conceptos matemáticos que requieren de la inmutabilidad para seguir siendo válidos. Aún así, es un concepto que es interesante conocer independientemente del tipo de paradigma de programación que se utilice.

¿Qué es la inmutabilidad?

La idea es muy sencilla de entender: algo es inmutable cuando no se puede modificar. En el contexto de la programación, una variable es inmutable cuando su valor no se puede modificar. Y un objeto lo es cuando su estado no puede ser actualizado tras la creación del objeto. Es por tanto una forma de asegurar que los objetos no se modifican en lugares inesperados afectando con ello la ejecución de nuestro programa.

La inmutabilidad genera muchas ventajas en las aplicaciones multihilo, donde la inmutabilidad simplifica mucho el tratamiento de la concurrencia. Si algo no se puede modificar, da igual que se acceda a ello desde distintos hilos a la vez, así como el orden en que se haga.

La inmutabilidad hace que el código sea mucho más predecible y más fácil de testear, porque se acota mucho más los lugares donde se producen modificaciones de estado.

La inmutabilidad tiene el sobrecoste de la generación de objetos nuevos cada vez que cambia el estado, así que esto puede penalizar bastante en el rendimiento. Así como guía muy general, es recomendable usar objetos mutables en cualquier situación que requiera de un estado que se modifica a menudo y/o la duplicación de ese estado sea costosa.

La inmutabilidad, por el contrario, presenta ventajas en las siguientes situaciones:

- En objetos que no vayan a requerir modificaciones de estado.
- En objetos que sean simples de duplicar.
- En situaciones de concurrencia.

Así que hay que hacer inmutable todo el código que sea posible ya que es un concepto muy potente que nos puede ayudar a simplificar la complejidad de comprensión de nuestro código y a disminuir las probabilidades de que se produzcan errores inesperados.

En Java, los objetos String son inmutables.

3. Formatos de codificación de caracteres

En el Unicode, el Plano Multilingüe Básico (Basic Multilingual Plane, **BMP**) incluye los 65.536 caracteres cuyos códigos van desde U+0000 a U+FFFF, que son la mayoría de los caracteres utilizados más frecuentemente.

El número 65.536 es 2 elevado a 16, es decir, la cantidad máxima de combinaciones de bits que se pueden obtener en dos bytes.

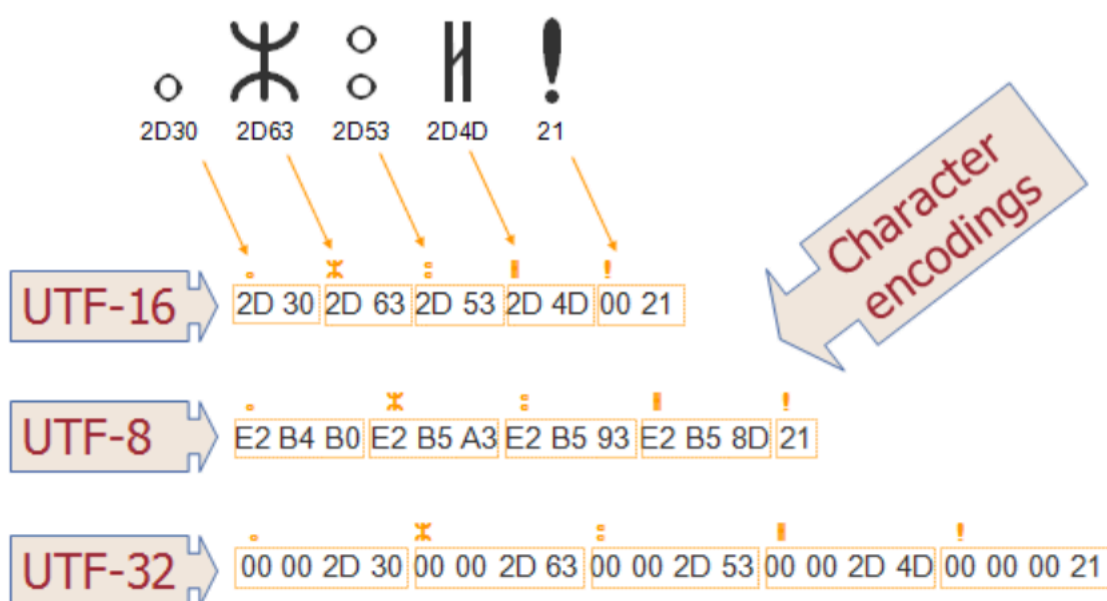
El resto de caracteres se denominan caracteres complementarios.

Los formatos de codificación que se pueden usar con Unicode se denominan UTF-8, UTF-16 y UTF-32:

UTF-8 utiliza 1 byte para representar caracteres ASCII, dos bytes para caracteres en otros bloques alfabéticos y tres bytes para el resto del BMP. Para los caracteres complementarios se utilizan 4 bytes.

UTF-16 utiliza 2 bytes para cualquier carácter en el BMP y 4 bytes para los caracteres complementarios.

UTF-32 emplea 4 bytes para todos los caracteres.



En Java, las cadenas utilizan el formato de codificación UTF-16.

4. Formateo de cadenas

Hay un método estático en la clase `String` que sirve para construir una cadena con cierto formato. Funciona de la misma manera que el método `printf` que vimos en el [Tema 1.4 Escritura en pantalla](#).

```
package tema3_1_Cadenas;

public class StringClass1 {

    public void show() {

        String formattedString;
        formattedString = String.format("Nombre: %s Edad: %d Sueldo: %.2f",
"Juan", 20, 1896.23);
        System.out.println(formattedString);

    }

    public static void main(String[] args) {

        new StringClass1().show();

    }

}
```

5. Métodos de la clase String

Si observamos en la API la clase `String`, incluye métodos para examinar los caracteres individuales de una cadena, para comparar cadenas, para buscar cadenas, para extraer subcadenas, para convertir cadenas a mayúsculas o minúsculas, etc.

Como vimos en el [Tema 1.1 Introducción al lenguaje Java](#), el operador concatenación `+` es un operador binario que devuelve una cadena resultado de concatenar las dos cadenas que actúan como operandos. Si sólo uno de los operandos es de tipo cadena, el otro operando se convierte implícitamente en tipo cadena.

Obsérvese en la API el método **valueOf**: es estático y está sobrecargado. Sirve para obtener la representación `String` de un valor u objeto.

```
package tema3_1_Cadenas;

public class StringClass2 {

    public void show() {

        int i = 100;
        String string1, string2, string3, string4;

        string1 = "Esto es un literal cadena"; //Se le da un valor inicial a la
cadena con el operador de asignación =
        System.out.println(string1);

    }

}
```

```

        System.out.println(string1 + " al cual le hemos concatenado este literal
cadena"); //Se concatena otra cadena con el operador +

        string2 = "hola";
        string3 = " que tal";
        string4 = string2 + string3;
        System.out.println(string4);

        System.out.println(i + 100); //Suma de enteros
        System.out.println(String.valueOf(i) + 100); //Concatenación de cadenas

    }

    public static void main(String[] args) {

        new StringClass2().show();

    }

}

```

Otros métodos de las cadenas muy útiles son:

- **charAt:** devuelve el carácter de la cadena del especificado índice. Dicho índice empieza en cero, es decir, con el cero se obtiene el primer carácter de la cadena.
- **length:** devuelve la longitud de la cadena.
- **equals:** compara si dos cadenas son iguales. Las cadenas se comparan con equals como los objetos y los primitivos se comparan con ==.
- **equalsIgnoreCase:** hace lo mismo que el anterior pero no tiene en cuenta las mayúsculas y minúsculas.

```

package tema3_1_cadenas;

public class StringClass3 {

    public void show() {

        String string = "hola";
        System.out.println(string.charAt(0)); //h
        System.out.println(string.charAt(1)); //o
        System.out.println(string.charAt(2)); //l
        System.out.println(string.charAt(3)); //a
        System.out.println(string.length()); //4
        System.out.println(string.equals("hola")); //true
        System.out.println(string.equals("Hola")); //false
        System.out.println(string.equals("adiós")); //false
        System.out.println(string.equalsIgnoreCase("HOLA")); //true
        System.out.println(string.equalsIgnoreCase("HOLA")); //true

        //También se le pueden aplicar métodos a un literal cadena:
        System.out.println("hola".equals("hola")); //true
        System.out.println("adios".equals("hola")); //false

    }

}

```

```

    public static void main(String[] args) {

        new StringClass3().show();

    }

}

```

Más métodos de cadenas interesantes:

- **compareTo**: compara dos cadenas lexicográficamente. La comparación se basa en el valor Unicode de cada carácter de las cadenas.
`s1.compareTo(s2)`: devuelve un número negativo si s1 es menor, un número positivo si s1 es mayor o cero si son iguales.
- **compareToIgnoreCase**: hace lo mismo que el anterior pero no tiene en cuenta las mayúsculas y minúsculas.
- **concat**: se utiliza para concatenar cadenas, como el operador +.
- **endsWith**: devuelve true si la cadena termina con un determinado texto.
- **indent**: ajusta la sangría de una cadena de texto, es decir, permite añadir o quitar espacios en blanco al principio de cada línea de una cadena.
- **startsWith**: devuelve true si la cadena empieza con un determinado texto.
- **indexOf**: devuelve la primera posición en la que aparece un determinado texto en la cadena. En el caso de que el texto buscado no se encuentre, devuelve -1. Este método está sobrecargado para que el texto a buscar pueda ser char o String.
- **lastIndexOf**: es como el anterior pero busca desde el final.
- **isEmpty**: devuelve true si la cadena está vacía, es decir, si su longitud es cero.
- **regionMatches**: compara una parte específica (una región) de una cadena con una parte específica de otra cadena. No necesariamente compara las cadenas enteras, sino solo las subcadenas seleccionadas por los índices y la longitud proporcionados. Se puede elegir si la comparación es sensible o no a mayúsculas y minúsculas.
- **repeat**: devuelve una cadena cuyo valor es la concatenación de la cadena repetida varias veces.
- **replace**: reemplaza todas las apariciones de un texto por otro texto. Este método está sobrecargado para que el texto pueda ser char o String.
- **substring**: obtiene una subcadena.
- **toLowerCase**: devuelve la cadena en minúsculas.
- **toUpperCase**: devuelve la cadena en mayúsculas.
- **trim**: elimina los espacios en blanco del principio y del final de la cadena.

```

package tema3_1_cadenas;

public class StringClass4 {

    public void show() {

        String string1 = "hola", string2 = "adios", string3 = "Hola";
    }

}

```

```

String string5 = "Estoy aprendiendo Programación";
String string6 = "Hace tiempo estuve aprendiendo Junit";
String string7 = ""
    Línea 1
    Línea 2
    Línea 3
    "";
String string8 = "    Línea 1\n    Línea 2\n    Línea 3";
String string9, string10;

//hola es mayor que adios
System.out.println(string1.compareTo(string2) > 0 ? String.format("%s es
mayor que %s", string1, string2)
    : String.format("%s es menor que %s", string1, string2));
//adios es menor que hola
System.out.println(string2.compareTo(string1) > 0 ? String.format("%s es
mayor que %s", string2, string1)
    : String.format("%s es menor que %s", string2, string1));
//En el Unicode, las mayúsculas están antes: Hola es menor que adios
System.out.println(string3.compareTo(string2) > 0 ? String.format("%s es
mayor que %s", string3, string2)
    : String.format("%s es menor que %s", string3, string2));
//Hola es mayor que adios si no tenemos en cuenta las mayúsculas y
minúsculas
System.out.println(string3.compareToIgnoreCase(string2) > 0
    ? String.format("%s es mayor que %s si no tenemos en cuenta las
mayúsculas y minúsculas", string3,
        string2)
    : String.format("%s es menor que %s si no tenemos en cuenta las
mayúsculas y minúsculas", string3,
        string2));

System.out.println(string4 = string1.concat(" que tal")); //string4="hola
que tal"
System.out.println(string4.endsWith("tal")); //true
System.out.println(string4.endsWith("hola")); //false
System.out.println(string4.startsWith("hola")); //true
System.out.println(string4.startsWith("tal")); //false

System.out.println(string4.indexOf("hola")); //0
System.out.println(string4.indexOf("tal")); //9
System.out.println(string4.indexOf("que")); //5
System.out.println(string4.indexOf(string2)); //-1
System.out.println(string4.indexOf(string3)); //-1
System.out.println(string4.indexOf('a')); //3
System.out.println("hola que tal hola que tal".lastIndexOf("tal")); //22

System.out.println(string4.isEmpty()); //false
System.out.println("").isEmpty(); //true

System.out.println("ole ".repeat(6)); //ole ole ole ole ole ole

System.out.println(string4.replace('a', '*')); //hol* que t*l
System.out.println("hola que tal hola que tal".replace("hola",
"buenas")); //buenas que tal buenas que tal

```



```

        //Para hacer desaparecer partes de una cadena, se reemplazan por cadena
        vacía
        System.out.println("hola que tal hola que tal".replace(" ",
        "")); //holaquetalholaquetal

        System.out.println(string4.substring(9)); //tal
        System.out.println(string4.substring(5, 8)); //que

        System.out.println("HOLA QUE TAL".toLowerCase()); //hola que tal
        System.out.println("hola que tal".toUpperCase()); //HOLA QUE TAL
        System.out.println("        Hola que tal        ".trim()); //Hola que tal
        System.out.printf("Las regiones %s son iguales\n",
        string5.regionMatches(true, 6, string6, 19, 11)? "sí": "no");

        //Se añaden 4 espacios al principio de cada línea
        string9 = string7.indent(4);
        System.out.printf("Con sangría añadida:\n%s", string9);

        //Se quitan 2 espacios al principio de cada línea
        string10 = string8.indent(-2);
        System.out.printf("Con sangría eliminada:\n%s", string10);

    }

    public static void main(String[] args) {

        new StringClass4().show();

    }

}

```

6. La interfaz CharSequence

Tal y como se explica en el [Tema 7. Herencia 12. Interfaces](#), una interfaz sirve para especificar un comportamiento que ciertas clases pueden seguir sin dictar cómo se debe implementar ese comportamiento. Pues la interfaz `CharSequence`, es una representación legible de una secuencia de caracteres. Está presente en el paquete `java.lang` y define un conjunto de métodos para trabajar con secuencias de caracteres de una manera uniforme. Sus métodos principales son `length`, `charAt` y `subSequence`. Las clases más comunes que implementan esta interfaz son `String`, `StringBuilder` y `StringBuffer`.

Ejemplo:

```

package tema3_1_Cadenas;

public class InterfaceCharSequence {

    public void show() {

        CharSequence cs = "Estoy aprendiendo a programar en Java";
        System.out.printf("Length: %d\n", cs.length());
        System.out.printf("Char at 1: %c\n", cs.charAt(1));
        System.out.printf("Subsequence (0, 5): %s", cs.subSequence(0, 5));
    }
}

```

```

    }

    public static void main(String[] args) {

        new InterfaceCharSequence().show();

    }

}

```

Salida por consola:

```

Length: 37
Char at 1: s
Subsequence (0, 5): Estoy

```

7. La clase StringBuilder

A diferencia de `String` que es inmutable (no puede cambiar una vez creada), `StringBuilder` permite modificar el contenido de la cadena sin crear nuevos objetos, lo que lo hace más eficiente en términos de memoria y rendimiento cuando se realizan múltiples modificaciones.

Características principales:

- **Mutable:** puede modificar el contenido sin crear un nuevo objeto.
- **Eficiente:** mejor rendimiento en operaciones como concatenación, inserción o borrado de caracteres.
- **Sincronización:** no es thread-safe, es decir, no es segura para su uso en entornos con múltiples hilos. Java dispone de la clase `StringBuffer` que sí es thread-safe, aunque a costa de un rendimiento ligeramente inferior en comparación con `StringBuilder`.

Métodos comunes:

- **append:** añade texto al final.
- **insert:** inserta texto en una posición específica.
- **delete:** elimina caracteres.
- **reverse:** invierte la secuencia de caracteres.

Ejemplo:

```

package tema3_1_cadenas;

public class StringBuilderClass {

    public void show() {

        stringBuilder sb = new stringBuilder("Estoy aprendiendo");
        System.out.println(sb);
        sb.append(" Java");
        System.out.printf("Después de append: %s\n", sb);
        sb.insert(17, " a programar en");
        System.out.printf("Después de insert: %s\n", sb);
    }
}

```

```
        sb.delete(17, 32);
        System.out.printf("Después de delete: %s%n", sb);
        sb.reverse();
        System.out.printf("Después de reverse: %s%n", sb);

    }

    public static void main(String[] args) {

        new StringBuilderClass().show();

    }

}
```

3.2 Expresiones regulares

3.2 Expresiones regulares

- 1. Expresiones regulares
- 2. La clase Pattern
- 3. La clase Matcher
- 4. Grupos
- 5. Quantifiers (cuantificadores)
- 6. Reemplazar partes de una cadena

1. Expresiones regulares

Una **expresión regular** es una secuencia de caracteres que forma un patrón de búsqueda proporcionando una manera muy flexible de buscar o reemplazar cadenas de texto.

En la clase String hay un método llamado **matches** que indica si la cadena coincide o no con la expresión regular que se le pasa por parámetro.

He aquí algunas de las construcciones de expresiones regulares que se encuentran en la API de Java en la clase *Pattern*:

- **Characters**

Construct	Matches
<code>x</code>	The character <code>x</code>
<code>\\</code>	The backslash character
<code>\\uhhhh</code>	The character with hexadecimal value <code>0xhhhh</code>
<code>\\t</code>	The tab character (' <code>\\u0009</code> ')
<code>\\n</code>	The newline (line feed) character (' <code>\\u000A</code> ')

```
package tema3_2_ExpresionesRegulares;

public class Characters {

    public void show() {

        System.out.println("a".matches("a")); //true
        System.out.println("b".matches("a")); //false
        System.out.println("A".matches("\\u0041")); //true
        System.out.println("\\n".matches("\\n")); //true
        System.out.println("\\t".matches("\\t")); //true
        System.out.println("\\\\".matches("\\\\\\")); //true

    }
}
```

```

    public static void main(String[] args) {

        new Characters().show();

    }

}

```

Tal y como vimos en el [Tema 1.1 Introducción al lenguaje Java 10 Tipos de datos primitivos 10.4 Caracteres](#), secuencias de escape es el conjunto de caracteres que en el código es interpretado con algún fin. En Java, la barra invertida `\` se denomina **carácter de escape**, el cual indica que el carácter puesto a continuación será convertido en carácter especial o, si ya es especial, dejará de ser especial. Por ejemplo, el carácter `n` no es especial pero con la `\` delante se convierte en especial ya que `\n` se interpreta como un salto de línea. La `\` es un carácter especial pero con otra `\` delante deja de ser especial y simplemente es una barra invertida.

En las expresiones regulares también se utiliza la barra invertida `\` como carácter de escape.

Veamos la última línea del código anterior `"\\".matches("\\\\")`: la cadena `"\\"` es una barra invertida, y el argumento del `matches` `"\\"` son dos barras invertidas ya que la expresión regular de la barra invertida son dos barras tal y como podemos observar en la tabla anterior.

- **Logical operators**

Construct	Matches
XY	X followed by Y
X Y	Either X or Y

```

package tema3_2_ExpresionesRegulares;

public class LogicalOperators {

    public void show() {

        System.out.println("hola".matches("hola")); //true
        System.out.println("hol".matches("hola")); //false

        System.out.println("hola".matches("hola|adios")); //true
        System.out.println("adios".matches("hola|adios")); //true
        System.out.println("hol".matches("hola|adios")); //false
        System.out.println("Adios".matches("hola|adios")); //false

    }

    public static void main(String[] args) {

        new LogicalOperators().show();

    }

}

```

- Character classes

Construct	Matches
[abc]	a, b, or c (simple class)
[^abc]	Any character except a, b, or c (negation)
[a-zA-Z]	a through z or A through Z, inclusive (range)
[a-d[m-p]]	a through d, or m through p: [a-dm-p] (union)
[a-z&&[def]]	d, e, or f (intersection)
[a-z&&[^bc]]	a through z, except for b and c: [ad-z] (subtraction)
[a-z&&[^m-p]]	a through z, and not m through p: [a-lq-z] (subtraction)

```
package tema3_2_ExpresionesRegulares;

public class CharactersClasses {

    public void show() {

        System.out.println("a".matches("[abc]")); //true
        System.out.println("d".matches("[abc]")); //false
        System.out.println("a".matches("[abc][abc]")); //false
        System.out.println("ac".matches("[abc][abc]")); //true
        System.out.println("ad".matches("[abc][abc]")); //false
        System.out.println(" a".matches("[abc ][abc]")); //true

        System.out.println("d".matches("[^abc]")); //true
        System.out.println("a".matches("[^abc]")); //false
        System.out.println("d".matches("[^abc][^abc]")); //false
        System.out.println("de".matches("[^abc][^abc]")); //true
        System.out.println("da".matches("[^abc][^abc]")); //false

        System.out.println("A".matches("[a-zA-Z]")); //true
        System.out.println("9".matches("[a-zA-Z]")); //false
        System.out.println("A".matches("[a-zA-Z][a-zA-Z]")); //false
        System.out.println("Az".matches("[a-zA-Z][a-zA-Z]")); //true
        System.out.println("A9".matches("[a-zA-Z][a-zA-Z]")); //false

        System.out.println("b".matches("[a-d[m-p]]")); //true
        System.out.println("n".matches("[a-d[m-p]]")); //true
        System.out.println("s".matches("[a-d[m-p]]")); //false

        System.out.println("d".matches("[a-z&&[def]]")); //true
        System.out.println("a".matches("[a-z&&[def]]")); //false

        System.out.println("d".matches("[a-z&&[^bc]]")); //true
        System.out.println("b".matches("[a-z&&[^bc]]")); //false

        System.out.println("d".matches("[a-z&&[^m-p]]")); //true
        System.out.println("n".matches("[a-z&&[^m-p]]")); //false
    }
}
```

```

    }

    public static void main(String[] args) {

        new CharactersClasses().show();

    }

}

```

- **Predefined character classes**

Construct	Matches
.	Any character (may or may not match line terminators)
\d	A digit: [0-9]
\D	A non-digit: [^0-9]
\s	A whitespace character: [\t\n\x0B\f\r]
\S	A non-whitespace character: [^\s]
\w	A word character: [a-zA-Z_0-9]
\W	A non-word character: [^\w]

Estas expresiones regulares van contenidas en una cadena. Entonces, aquellas expresiones regulares que lleven una barra invertida, como por ejemplo `\d`, tienen que llevar otra barra invertida `\\` delante ya que en las cadenas una barra invertida se expresa como `\\`.

```

package tema3_2_ExpresionesRegulares;

public class PredefinedCharacterClasses {

    public void show() {

        System.out.println("a".matches(".")); //true
        System.out.println("?".matches(".")); //true
        System.out.println("\n".matches(".")); //false

        System.out.println("2".matches("\\d")); //true
        System.out.println("a".matches("\\d")); //false

        System.out.println("2".matches("\\D")); //false
        System.out.println("a".matches("\\D")); //true
        System.out.println("a2".matches("\\D\\d")); //true

        System.out.println(" ".matches("\\s")); //true
        System.out.println("\t".matches("\\s")); //true
        System.out.println("\n".matches("\\s")); //true
        System.out.println("a".matches("\\s")); //false

        System.out.println(" ".matches("\\S")); //false
    }
}

```

```

        System.out.println("a".matches("\\s")); //true
        System.out.println(" a".matches("\\s\\s")); //true

        System.out.println("_".matches("\\w")); //true
        System.out.println("B".matches("\\w")); //true
        System.out.println("9".matches("\\w")); //true
        System.out.println("?".matches("\\w")); //false

        System.out.println("a".matches("\\w")); //false
        System.out.println("*".matches("\\w")); //true

    }

    public static void main(String[] args) {

        new PredefinedCharacterClasses().show();

    }

}

```

- **POSIX character classes (US-ASCII only)**

Construct	Matches
\p{Lower}	A lower-case alphabetic character: [a-z]
\p{Upper}	An upper-case alphabetic character: [A-Z]
\p{ASCII}	All ASCII: [\x00-\x7F]
\p{Alpha}	An alphabetic character: [\p{Lower}\p{Upper}]
\p{Digit}	A decimal digit: [0-9]
\p{Alnum}	An alphanumeric character: [\p{Alpha}\p{Digit}]
\p{Punct}	Punctuation: One of !"#\$%&'()*+,-./:;<=>?@[^_`{}~
\p{Graph}	A visible character: [\p{Alnum}\p{Punct}]
\p{Print}	A printable character: [\p{Graph}\x20]
\p{Blank}	A space or a tab: [\t]
\p{Cntrl}	A control character: [\x00-\x1F\x7F]
\p{XDigit}	A hexadecimal digit: [0-9a-fA-F]
\p{Space}	A whitespace character: [\t\n\x0B\f\r]

En los caracteres imprimibles (\p{Print}), \x20 es el carácter espacio.

```

package tema3_2_ExpresionesRegulares;

public class PosixCharacterClasses {

```



```

public void show() {

    System.out.println("a".matches("\\p{Lower}")); //true
    System.out.println("A".matches("\\p{Lower}")); //false

    System.out.println("A".matches("\\p{Upper}")); //true
    System.out.println("a".matches("\\p{Upper}")); //false
    System.out.println("aC".matches("\\p{Lower}\\p{Upper}")); //true

    System.out.println("a".matches("\\p{Alpha}")); //true
    System.out.println("A".matches("\\p{Alpha}")); //true
    System.out.println("8".matches("\\p{Alpha}")); //false

    System.out.println("8".matches("\\p{Digit}")); //true
    System.out.println("A".matches("\\p{Digit}")); //false
    System.out.println("8A".matches("\\p{Digit}\\p{Alpha}")); //true

    System.out.println("A".matches("\\p{Alnum}")); //true
    System.out.println("8".matches("\\p{Alnum}")); //true
    System.out.println("a".matches("\\p{Alnum}")); //true
    System.out.println("\n".matches("\\p{Alnum}")); //false

    System.out.println("?".matches("\\p{Punct}")); //true
    System.out.println("!".matches("\\p{Punct}")); //true
    System.out.println(";".matches("\\p{Punct}")); //true
    System.out.println("a".matches("\\p{Punct}")); //false

    System.out.println(" ".matches("\\p{Blank}")); //true
    System.out.println("\t".matches("\\p{Blank}")); //true
    System.out.println("\n".matches("\\p{Blank}")); //false

    System.out.println(" ".matches("\\p{Space}")); //true
    System.out.println("\t".matches("\\p{Space}")); //true
    System.out.println("\n".matches("\\p{Space}")); //true
    System.out.println("a".matches("\\p{Space}")); //false

}

public static void main(String[] args) {

    new PosixCharacterClasses().show();

}

}

```

- **Boundary matchers**

Construct	Matches
^	The beginning of a line
\$	The end of a line
\b	A word boundary

Construct	Matches
\B	A non-word boundary

```

package tema3_2_ExpresionesRegulares;

public class BoundaryMatchers {

    public void show() {

        System.out.println("hola".matches("^hola")); //true
        System.out.println("hola9".matches("^hola")); //false
        System.out.println("hola9".matches("^hola\\d")); //true
        System.out.println("ab".matches("[aA]\\p{Lower}")); //true
        System.out.println("bA".matches("[aA]\\p{Upper}")); //false

        System.out.println("hhola".matches(".hola$")); //true
        System.out.println("hola".matches(".hola$")); //false
        System.out.println("9hola".matches("\\dhola$")); //true
        System.out.println("hc".matches("[abc]$")); //true
        System.out.println("ch".matches("[abc]$")); //false

        /*
         * La expresión regular \\b se llama límite de palabra
         * ya que busca en los límites de una palabra, al
         * principio o al final:
         */
        System.out.println("hola".matches("\\bhol.")); //true, hol está al
principio de una palabra
        System.out.println("hola".matches(".ola\\b")); //true, ola está al final
de una palabra
        /*
         * Para realizar una búsqueda de palabras específicas
         * se coloca la palabra entre dos límites de palabra:
         */
        System.out.println("hola".matches("\\bhola\\b")); //true, hola está al
principio y al final

        // \B es justo lo contrario que \\b
        System.out.println("abc".matches(".\\Bb\\B.")); //true, b no está al
principio ni al final
        System.out.println("abc".matches("a\\B..")); //true, a no está al final

    }

    public static void main(String[] args) {

        new BoundaryMatchers().show();

    }

}

```

2. La clase Pattern

En Java disponemos de las clases `Pattern` y `Matcher` para poder hacer uso de las expresiones regulares. Ambas se encuentran en el paquete `java.util.regex`.

La clase **Pattern** nos permite definir el patrón, es decir, representa a la expresión regular. Veamos algunos métodos de esta clase:

- **compile**: crea un patrón a partir de una expresión regular.
- **pattern**: devuelve la expresión regular a partir de la cual se creó el patrón.
- **matches**: indica si la cadena coincide o no con la expresión regular.

```
package tema3_2_ExpresionesRegulares;

import java.util.regex.Pattern;

public class PatternClass {

    public void show() {

        Pattern pattern = Pattern.compile("\\p{Upper}\\p{Lower}");
        System.out.println(pattern.pattern()); // \p{Upper}\p{Lower}

        System.out.println(Pattern.matches("\\p{Upper}\\p{Lower}", "Ho")); // true
        System.out.println(Pattern.matches("\\p{Upper}\\p{Lower}", "ho")); // false

    }

    public static void main(String[] args) {

        new PatternClass().show();

    }

}
```

3. La clase Matcher

La clase **Matcher** realiza operaciones de coincidencia del patrón en una secuencia de caracteres.

Se puede crear un objeto de tipo `Matcher` mediante el método **matcher** de la clase `Pattern`. Una vez creado, un `matcher` puede ser utilizado para realizar tres tipos diferentes de operaciones:

- **matches**: intenta hacer coincidir toda la secuencia de entrada con el patrón.
- **lookingAt**: intenta hacer coincidir el principio de la secuencia de entrada con el patrón.
- **find**: intenta encontrar la próxima secuencia de entrada que coincide con el patrón. Si hay varias coincidencias con el patrón dentro del mismo texto, cada llamada a `find` devolverá la siguiente coincidencia.
- **start**: devuelve el índice de la secuencia de entrada donde empieza la coincidencia del último `find`.
- **end**: devuelve el índice de la secuencia de entrada del carácter que está justo después del último de la coincidencia del último `find`.

- **reset:** resetea el matcher.

```
package tema3_2_ExpresionesRegulares;

import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class MatcherClass {

    public void show() {

        int count;
        //Con Pattern.CASE_INSENSITIVE, no se distingue entre mayúsculas y
        minúsculas
        Pattern pattern = Pattern.compile("es", Pattern.CASE_INSENSITIVE);
        Matcher matcher = pattern.matcher("Estoy en España");
        System.out.println(matcher.matches()); //false
        System.out.println(matcher.lookingAt()); //true

        matcher.reset("Esto es un escrito en español");
        count = 0;
        while (matcher.find()) {
            count++;
            System.out.printf("Coincidencia número %d: empieza en %d y termina en
%d\n", count, matcher.start(),
                           matcher.end() - 1);
        }

    }

    public static void main(String[] args) {

        new MatcherClass().show();

    }

}
```

La expresión regular `.` coincide con cualquier carácter excepto un terminador de línea, a menos que se especifique la bandera `DOTALL`.

```
package tema3_2_ExpresionesRegulares;

import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class Point {

    public void show() {

        Pattern pattern = Pattern.compile(".", Pattern.DOTALL);
        Matcher matcher = pattern.matcher("\n");
        System.out.println(matcher.matches()); //true

        System.out.println("\n".matches(".")); //false

    }

}
```

```

    }

    public static void main(String[] args) {

        new Point().show();

    }

}

```

Veamos más ejemplos de *boundary matchers* utilizando objetos de tipo `Matcher`:

```

package tema3_2_ExpresionesRegulares;

import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class BoundaryMatchers2 {

    public void show() {

        String text = "Esto es un texto escrito en español";
        Pattern pattern;
        Matcher matcher;

        pattern = Pattern.compile("\\be", Pattern.CASE_INSENSITIVE);
        matcher = pattern.matcher(text);
        System.out.println("Las palabras que empiezan por e o por E se encuentran en las siguientes posiciones:");
        while (matcher.find()) {
            System.out.println(matcher.start());
        }

        pattern = Pattern.compile("\\Be");
        matcher = pattern.matcher(text);
        System.out.println("Las e que no son comienzos de palabra se encuentran en las siguientes posiciones:");
        while (matcher.find()) {
            System.out.println(matcher.start());
        }

        pattern = Pattern.compile("o\\b");
        matcher = pattern.matcher(text);
        System.out.println("Las o que son finales de palabras se encuentran en las siguientes posiciones:");
        while (matcher.find()) {
            System.out.println(matcher.start());
        }

        pattern = Pattern.compile("o\\B");
        matcher = pattern.matcher(text);
        System.out.println("Las o que no son finales de palabras se encuentran en las siguientes posiciones:");
        while (matcher.find()) {

```

```

        System.out.println(matcher.start());
    }
    /*
     * Para realizar una búsqueda de palabras específicas
     * se coloca la palabra entre dos límites de palabra:
     */
    pattern = Pattern.compile("\\btexto\\b");
    matcher = pattern.matcher(text);
    System.out.println("La palabra texto se encuentra en las siguientes
posiciones:");
    while (matcher.find()) {
        System.out.println(matcher.start());
    }

    pattern = Pattern.compile("^Lín.*", Pattern.MULTILINE);
    matcher = pattern.matcher("""
        Línea 1
        Línea 2
        Línea 3
        """);

    System.out.println("Las líneas que empiezan por Lín se encuentran en las
siguientes posiciones:");
    while (matcher.find()) {
        System.out.println(matcher.start());
    }

    pattern = Pattern.compile(".*nea$", Pattern.MULTILINE);
    matcher = pattern.matcher("""
        Primera Línea
        Segunda Línea
        Tercera Línea
        """);

    System.out.println("Las líneas que finalizan por nea se encuentran en las
siguientes posiciones:");
    while (matcher.find()) {
        System.out.println(matcher.start());
    }
}

public static void main(String[] args) {

    new BoundaryMatchers2().show();

}

}

```

En el ejemplo, la bandera **Pattern.MULTILINE** permite que `^` coincida con el inicio de cada línea, no solo con el inicio de toda la cadena. Por el mismo motivo, permite que `$` coincida con el final de cada línea y no solo con el final de toda la cadena.

4. Grupos

Los grupos sirven para extraer partes de una cadena y se marcan con un paréntesis en la expresión regular. Cuando se encuentra una coincidencia en un texto, se puede acceder a la parte que se encuentra dentro del grupo a través del método **group**. Una expresión regular puede tener más de un grupo, en cuyo caso cada uno lleva sus propios paréntesis. El grupo con número 0 es toda la expresión regular y los grupos marcados con paréntesis empiezan a numerarse a partir del 1. También puede haber grupos dentro de otros grupos, como en el siguiente ejemplo, donde el grupo 2 y el grupo 3 están dentro del grupo 1.

```
package tema3_2_ExpresionesRegulares;

import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class Group {

    public void show() {

        //Como veremos en el siguiente apartado, el + es 1 ó más veces
        Pattern pattern = Pattern.compile("((\\d+)\\+(\\d+))=(\\d+)");
        Matcher matcher = pattern.matcher("23+56=79;15+13=28;30+60=90");

        while (matcher.find()) {
            System.out.println(matcher.group(0));
            System.out.printf("Sumandos: %s\\n", matcher.group(1));
            System.out.printf("Sumando 1: %s\\n", matcher.group(2));
            System.out.printf("Sumando 2: %s\\n", matcher.group(3));
            System.out.printf("Resultado: %s\\n\\n", matcher.group(4));
        }

    }

    public static void main(String[] args) {

        new Group().show();

    }

}
```

5. Quantifiers (cuantificadores)

- Greedy quantifiers

Por defecto, los cuantificadores son greedy. Se llaman *greedy* (glotón) porque tratan de coger lo máximo posible de la cadena pero siempre intentando que el patrón completo se cumpla.

Construct	Matches
X?	X, once or not at all
X*	X, zero or more times

Construct	Matches
X ⁺	X, one or more times
X{n}	X, exactly n times
X{n,}	X, at least n times
X{n,m}	X, at least n but not more than m times

```
package tema3_2_ExpresionesRegulares;

public class GreedyQuantifiers {

    public void show() {

        /*
         * El símbolo ? sirve para indicar que la expresión que le
         * antecede puede aparecer una o ninguna vez
         */
        System.out.println("a".matches("a?")); //true
        System.out.println("").matches("a?")); //true
        System.out.println("b".matches("a?")); //false

        /*
         * El símbolo * indica que la expresión puede repetirse
         * ninguna o varias veces
         */
        System.out.println("a".matches("a*")); //true
        System.out.println("").matches("a*")); //true
        System.out.println("aaaa".matches("a*")); //true
        System.out.println("b".matches("a*")); //false

        /*
         * El símbolo + indica que la expresión puede repetirse
         * una o varias veces
         */
        System.out.println("a".matches("a+")); //true
        System.out.println("aaaa".matches("a+")); //true
        System.out.println("").matches("a+")); //false
        System.out.println("b".matches("a+")); //false

        //Un número entre llaves indica las veces que se repite la expresión
        System.out.println("aaa".matches("a{3}")); //true
        System.out.println("aaa".matches("a{4}")); //false

        /*
         * Un número entre llaves con una coma indica el mínimo
         * de veces que se repite la expresión
         */
        System.out.println("aaa".matches("a{3,}")); //true
        System.out.println("aaaaa".matches("a{3,}")); //true
        System.out.println("aa".matches("a{3,}")); //false

        /*
```



```

        * Como el anterior pero aparece un segundo número que
        * indica el máximo de veces que se repite la expresión
        */
        System.out.println("aaa".matches("a{3,6}")); //true
        System.out.println("aaaaaa".matches("a{3,6}")); //true
        System.out.println("aa".matches("a{3,6}")); //false
        System.out.println("aaaaaa".matches("a{3,6}")); //false

    }

    public static void main(String[] args) {

        new GreedyQuantifiers().show();

    }

}

```

- **Reluctant quantifiers**

Otro posible comportamiento es *reluctant* (reacio, reticente). Este comportamiento es el contrario de greedy, trata de coger lo menos posible pero siempre intentando que se cumpla el patrón.

Se escriben como los greedy pero añadiendo un ? detrás.

Construct	Matches
X??	X, once or not at all
X*?	X, zero or more times
X+?	X, one or more times
X{n}?	X, exactly n times
X{n,}??	X, at least n times
X{n,m}?	X, at least n but not more than m times

En el siguiente ejemplo se puede apreciar la diferencia entre greedy y reluctant:

```

package tema3_2_ExpresionesRegulares;

import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class ReluctantQuantifiers {

    public void show() {

        Pattern patternGreedy = Pattern.compile("a+");
        Pattern patternReluctant = Pattern.compile("a+?");
        Matcher matcherGreedy = patternGreedy.matcher("aaaa");
        Matcher matcherReluctant = patternReluctant.matcher("aaaa");

        while (matcherGreedy.find()) {

```

```

        System.out.printf("Greedy: coincidencia desde %d hasta %d\n",
matcherGreedy.start(),
        matcherGreedy.end() - 1);
    }
    while (matcherReluctant.find()) {
        System.out.printf("Reluctant: coincidencia desde %d hasta %d\n",
matcherReluctant.start(),
        matcherReluctant.end() - 1);
    }
}

public static void main(String[] args) {

    new ReluctantQuantifiers().show();

}
}

```

La salida por consola es la siguiente:

```

Greedy: coincidencia desde 0 hasta 3
Reluctant: coincidencia desde 0 hasta 0
Reluctant: coincidencia desde 1 hasta 1
Reluctant: coincidencia desde 2 hasta 2
Reluctant: coincidencia desde 3 hasta 3

```

- **Possessive quantifiers**

Tratan de coger lo máximo posible de la cadena pero no se preocupan de que se cumpla el patrón. Este comportamiento se usa únicamente por motivos de eficiencia. Si en la cadena hay un trozo que queramos quitar y que podamos distinguir con una expresión regular, podemos ponerlo con este modo possessive. De esta forma, el possessive se comerá directamente ese trozo de cadena y no perderá el tiempo tratando de hacer casar ese trozo con el patrón de alguna u otra forma.

Se escriben como los greedy pero añadiendo un + detrás.

Construct	Matches
X?+	X, once or not at all
X*+	X, zero or more times
X++	X, one or more times
X{n}+	X, exactly n times
X{n,}+	X, at least n times
X{n,m}+	X, at least n but not more than m times

En el siguiente ejemplo se puede apreciar la diferencia entre greedy y possessive:

```


```

```

package tema3_2_ExpresionesRegulares;

import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class PossessiveQuantifiers {

    public void show() {

        Pattern patternGreedy = Pattern.compile("a+a");
        Pattern patternPossessive = Pattern.compile("a++a");
        Matcher matcherGreedy = patternGreedy.matcher("aaaa");
        Matcher matcherPossessive = patternPossessive.matcher("aaaa");

        while (matcherGreedy.find()) {
            System.out.printf("Greedy: coincidencia desde %d hasta %d\n",
matcherGreedy.start(),
                matcherGreedy.end() - 1);
        }
        while (matcherPossessive.find()) {
            System.out.printf("Possessive: coincidencia desde %d hasta %d\n",
matcherPossessive.start(),
                matcherPossessive.end() - 1);
        }
    }

    public static void main(String[] args) {

        new PossessiveQuantifiers().show();

    }

}

```

La salida por consola es la siguiente:

```
Greedy: coincidencia desde 0 hasta 3
```

En el modo possessive, con la primera parte del patrón `a++` ya coge la cadena completa `"aaaa"`. Como no se preocupa de que el patrón se cumpla, entonces la última `a` del patrón ya no tiene coincidencia, con lo que el `find` devuelve false.

6. Reemplazar partes de una cadena

La clase `Matcher` tiene métodos para reemplazar partes de la cadena que cumplan con el patrón:

- **replaceAll**: reemplaza todas las coincidencias.
- **replaceFirst**: reemplaza solamente la primera coincidencia.

```

package tema3_2_ExpresionesRegulares;

import java.util.regex.Matcher;
import java.util.regex.Pattern;

```

```

public class Replace {

    public void show() {

        Pattern pattern = Pattern.compile("\\d");
        Matcher matcher = pattern.matcher("Tengo 20 años y vivo en la calle
Puerto Real 15");

        String replaceAll = matcher.replaceAll("*");
        String replaceFirst = matcher.replaceFirst("*");

        System.out.printf("replaceAll: %s\n", replaceAll);
        System.out.printf("replaceFirst: %s\n", replaceFirst);

    }

    public static void main(String[] args) {

        new Replace().show();

    }

}

```

Salida por consola:

```

replaceAll: Tengo ** años y vivo en la calle Puerto Real **
replaceFirst: Tengo *0 años y vivo en la calle Puerto Real 15

```

4 Programación Orientada a Objetos

4 Programación Orientada a Objetos

1. Introducción
2. Creación de clases
3. Creación de objetos
4. Acceso a los atributos y métodos del objeto
5. Modificadores de acceso
6. Sobrecarga de métodos
7. Atributos, métodos y bloques estáticos
8. JavaBean
9. Métodos get y set
10. Constructores
11. Visibilidad
12. instanceof
13. Referencias
 - 13.1 Representación en memoria de tipos primitivos
 - 13.2 Representación en memoria de objetos
 - 13.3 Garbage Collector
14. This
 - 14.1 Para acceder a un atributo del objeto actual
 - 14.2 Para invocar a un constructor
 - 14.3 Como referencia al objeto actual
15. Encadenamiento de llamadas a métodos
16. Sintaxis fluida
17. Invariante de una clase
18. Encapsulamiento
19. El método toString
20. Control de nulidad
21. Uso de las excepciones para el control del invariante de una clase
22. Records

1. Introducción

La Programación Orientada a Objetos (**POO**) es una técnica de programar aplicaciones basada en una serie de objetos independientes que se comunican entre sí.

A Java se le considera un lenguaje orientado a objetos ya que siempre que se crea un programa en Java, por simple que sea, se necesita declarar una clase, y el concepto de clase pertenece a la programación orientada a objetos.

Un **objeto** es un elemento del programa que integra sus propios datos y su propio funcionamiento. Es decir, un objeto está formado por datos (atributos o propiedades) y por las funciones que es capaz de realizar el objeto (métodos). Esta forma de programar se asemeja más al pensamiento humano. La cuestión es detectar adecuadamente los objetos necesarios para una aplicación. De hecho hay que detectar las distintas clases de objetos.

Una **clase** es lo que define a un tipo de objeto. Al definir una clase lo que se hace es indicar como funciona un determinado tipo de objeto. Luego, a partir de la clase, podremos crear objetos de esa clase, es decir, la clase es como un molde a partir del cual se crean los objetos que pertenecen a ella. Realmente la programación orientada a objetos es una programación orientada a clases. Es decir, lo que necesitamos programar es como funcionan las clases de objetos.

Por ejemplo, una clase podría ser la clase *Coche*. Cuando se defina esta clase, indicaremos los atributos o propiedades (como el color, modelo, marca, velocidad máxima,...) y los métodos (arrancar, parar, repostar, acelerar, frenar...). Todos los coches, es decir, todos los objetos de la clase *Coche*, tendrán esas propiedades y esos métodos. Para explicar la diferencia entre clase y objeto:

- la clase *Coche* representa a todos los coches.
- un coche concreto es un objeto, es decir, un ejemplar de una clase es un objeto. También se le llama a los objetos *instancias* de la clase. Este término procede del inglés, *instance*, que realmente significa ejemplar.

Por ejemplo, si quisiéramos crear el juego del parchís en Java, una clase sería la casilla, otra las fichas, otra el dado, etc. En el caso de la casilla, se definiría la clase para indicar su funcionamiento y sus propiedades, y luego se crearían tantos objetos casilla como casillas tenga el juego. Lo mismo ocurriría con las fichas, la clase ficha definiría las propiedades de la ficha (color y posición por ejemplo) y su funcionamiento mediante sus métodos (por ejemplo un método sería mover, otro llegar a la meta, etc), luego se crearían tantos objetos ficha como fichas tenga el juego.

2. Creación de clases

Una clase sirve para definir una serie de objetos con propiedades (atributos), comportamientos (métodos) y semántica comunes. Hay que pensar en una clase como un molde para crear objetos.

La definición de una clase incluye lo siguiente:

- El nombre o identificador de clase. Debe empezar con letra mayúscula y seguirle letras minúsculas, y si consta de varias palabras, se utiliza la notación UpperCamelCase. También pueden contener números pero no como primer carácter. Por ejemplo, `1Coche` no es un identificador válido de clase, pero `Coche1` sí sería válido. Veamos algunos consejos a la hora de elegir los identificadores de las clases:
 - Evitar abreviaturas a favor de la legibilidad del código. Es muy importante que el nombre de las clases sea claro y simbolice perfectamente al tipo de objetos que representa.
 - Evitar nombres excesivamente largos. Aunque parece que se contradice con la norma anterior, se trata de que los nombres sean concisos. No es conveniente que sean descripciones de clase, para eso ya están los comentarios javadoc.
 - Utilizar nombres ya reconocidos. Hay abreviaturas reconocidas como por ejemplo TCP, por eso el nombre de clase `ManejadorTCP` es mejor que `ManejadorProtocoloControlTransmission`.
- Los atributos, también llamados propiedades o campos. Los atributos son variables que poseerá cada objeto de la clase y por lo tanto marcarán el estado de los mismos. Por ejemplo, un coche puede estar parado, en marcha, estropeado, funcionando, sin gasolina, etc. El estado lo marca el valor que tengan los atributos del objeto.
- Los métodos. Son las acciones que pueden realizar los objetos de la clase, es decir, lo que determina el comportamiento de los objetos.

En Java, cada clase se define en un archivo. Además, el nombre de la clase y el del archivo tiene que ser el mismo. Es decir, si queremos definir la clase *Vehicle*, tendremos que hacerlo en un archivo llamado *Vehicle.java*. Dicho archivo contiene la definición de la clase *Vehicle*:

```
class Vehicle {
    int wheelCount; //Número de ruedas
    double speed;    //Velocidad
    String colour;   //Color del vehículo
    void accelerate(double amount){ //acelerar
        speed += amount;
    }
    void brake(double amount){ //frenar
        speed -= amount;
    }
}
```

3. Creación de objetos

Una vez definida la clase, ya se pueden crear objetos de la misma. Para crear un objeto, hay que declarar una variable cuyo tipo será la propia clase:

```
Vehicle car; //car es una variable de tipo Vehicle
```

Una vez definida la variable, se le crea el objeto llamando a un método que se llama constructor. Un constructor es un método que se invoca cuando se crea un objeto y que sirve para inicializar los atributos del objeto y para realizar las acciones pertinentes que requiera el mismo para ser creado. El constructor tiene el mismo nombre que la clase y para invocarlo se utiliza el operador **new**.

```
car = new Vehicle(); //Vehicle() es un método constructor
```

También se puede hacer todo en la misma línea:

```
Vehicle car = new Vehicle();
```

4. Acceso a los atributos y métodos del objeto

Una vez creado el objeto, se puede acceder a sus **atributos** de la siguiente manera:
`objeto.atributo`.

```
car.wheelCount = 4; //Se le asigna 4 al atributo número de ruedas de la variable car
```

Los **métodos** se utilizan de la misma forma que los atributos, a excepción de que los métodos poseen siempre paréntesis ya que son funciones que pertenecen a un objeto:

`objeto.método(argumentos)`.

```
car.accelerate(30);
/*El coche incrementa su velocidad en 30. Es decir, si iba a 90km/h, después de ejecutar el método el coche va a 120km/h */
```

5. Modificadores de acceso

Los modificadores de acceso son palabras reservadas del lenguaje Java que determinan ámbitos de visibilidad de los atributos y métodos de una clase.

Los modificadores de acceso son los siguientes:

- **Private:** el modificador `private` en Java es el más restrictivo de todos, cualquier elemento de una clase que sea privado puede ser accedido únicamente por la propia clase. Ninguna otra clase, sin importar la relación que tengan, podrá tener acceso a elementos privados.
- El modificador por defecto: Java nos da la opción de no usar un modificador de acceso y al no hacerlo, el elemento tendrá un acceso conocido como *default* o **friendly**, que permite que tanto la propia clase como las clases del mismo paquete accedan a dichos elementos.
- **Protected:** lo veremos en el [Tema 7 Herencia 2 Modificadores de acceso](#) donde se explica el funcionamiento de la herencia en Java.
- **Public:** el modificador de acceso `public` es el más permisivo de todos, nos permite acceso a los elementos desde cualquier clase incluso de otros paquetes.

Veamos una tabla que resume el funcionamiento de los modificadores de acceso en Java:

	Private	Friendly	Public
Misma clase	X	X	X
Mismo paquete		X	X
Otro paquete			X

El modificador se indica:

- Delante de `class` en la clase.
- Delante del tipo de datos en los atributos y métodos.

Veamos la clase `Vehicle` con los modificadores de acceso:

```
public class Vehicle { //Clase pública

    //Atributos privados:

    private int wheelCount;
    private double speed;
    private String colour;

    //Métodos públicos:

    public void accelerate(double amount){
        speed += amount;
    }
    public void brake(double amount){
        speed -= amount;
    }

}
```


6. Sobrecarga de métodos

Java admite sobrecargar los métodos, es decir, crear distintas variantes del mismo método con el mismo nombre pero que se diferencien en el orden, tipo o número de los parámetros.

Por ejemplo, tenemos el método para sumar `add(int x, int y)`:

- No podríamos definir otro método `add(int a, int b)` porque no varía el tipo ni el número de parámetros.
- Sí podríamos definir `add(int a)` y `add(int a, int b, int c)` porque el número de parámetros varía.
- También podríamos definir `add(int x, double y)` porque aunque no varíe el número de parámetros, sí varía uno de los tipos.

Otro ejemplo donde tenemos el método `add(int x, double y)`:

- Sí podríamos definir el método `add(double x, int y)` porque varía el orden de los parámetros.

El tipo resultado de los métodos sobrecargados puede ser igual o diferente.

7. Atributos, métodos y bloques estáticos

El concepto de estático es que pertenece a la clase, por lo tanto, puede ser accedido o invocado sin la necesidad de tener que instanciar un objeto de la clase. Se indica con la palabra clave **static**.

Los atributos y métodos asociados a los objetos se les conoce como dinámicos.

Los **atributos estáticos** son variables que pertenecen a la clase y son compartidos por todos los objetos de la clase. Son inicializados en el momento en que se carga la clase en memoria, respetando el orden de declaración. Para acceder a un atributo estático, hay que indicar el nombre de la clase: `Clase.AtributoEstático`

Los **métodos estáticos** son métodos que pertenecen a la clase y no al objeto por lo que pueden llamarse sin tener que crear un objeto de dicha clase. Para llamar a un método estático, hay que indicar el nombre de la clase: `Clase.MétodoEstático(Argumentos)`. Un método estático solo puede acceder a datos estáticos y llamar a métodos estáticos. No pueden utilizar el operador *this* ni *super* ya que son conceptos dinámicos. Los métodos estáticos es lo más parecido a lo que son las funciones en los lenguajes estructurados con la diferencia que se encuentran encapsulados en una clase.

Cada vez que creamos un programa en Java debemos especificar el método *main*:

```
public static void main(String[] args)
```

El método *main* es estático para que la máquina virtual de Java pueda llamarlo directamente sin tener que crear un objeto de la clase que lo contiene.

Ejemplos de llamadas a métodos dinámicos y estáticos:

- Llamada a método dinámico: `car.accelerate(30);`
- Llamada a método estático: `Math.pow(2, 3);`

El **bloque estático** es un bloque de instrucciones dentro de una clase Java que se ejecutará cuando una clase se cargue por primera vez en la JVM.

```

class MyClass{
    ...
    static{ //Comienzo del bloque estático
        ...
        //Instrucciones
        ...
    }//Fin del bloque estático
    ...
}

```

Ejemplo:

```

package tema4_POO.estatico;

public class MyClass {

    static int a;
    static int b;

    static {
        a = 10;
        b = 20;
    }

}

```

```

package tema4_POO.estatico;

public class Main {

    public void showStatic() {

        System.out.printf("valor de a: %d", MyClass.a);
        System.out.printf("\nvalor de b: %d", MyClass.b);

    }

    public static void main(String[] args) {

        new Main().showStatic();

    }

}

```

Salida por consola:

```

valor de a: 10
valor de b: 20

```

8. JavaBean

Los JavaBeans son un modelo de componentes para la construcción de aplicaciones en Java. Se usan para encapsular varios objetos en un único objeto (la vaina o Bean en inglés), para hacer uso de un solo objeto en lugar de varios más simples. La especificación de JavaBeans los define como "componentes de software reutilizables que se puedan manipular visualmente en una herramienta de construcción".

Para funcionar como una clase JavaBean, una clase debe obedecer ciertas convenciones sobre nomenclatura de métodos, construcción y comportamiento. Estas convenciones permiten tener herramientas que puedan utilizar, reutilizar, sustituir y conectar JavaBeans.

Las convenciones requeridas son:

- Debe tener un constructor sin argumentos.
- Sus atributos de clase deben ser privados.
- Sus propiedades deben ser accesibles mediante métodos *get* y *set* que siguen una convención de nomenclatura estándar.
- Debe ser serializable.

Dentro de un JavaBean podemos distinguir tres partes:

- Propiedades: Los atributos que contiene.
- Métodos: Se establecen los métodos *get* y *set* para acceder y modificar los atributos.
- Eventos: Permiten comunicar con otros JavaBeans.

9. Métodos get y set

Los métodos *get* y *set* son métodos de las clases para mostrar o modificar el valor de un atributo. Para mostrar se utiliza el método *get* y para modificar el método *set*.

Según las convenciones JavaBean, la nomenclatura de ambos debe ser la siguiente:

- **get:**
 - Debe ser declarado con el modificador de acceso *public*.
 - El nombre del método comienza con *get* y le sigue el nombre del atributo en UpperCamelCase. En el caso de los booleanos, el nombre del método comienza con *is*.
 - El tipo de retorno del método debe ser el mismo que el tipo del atributo.
- **set:**
 - Debe ser declarado con el modificador de acceso *public*.
 - El nombre del método comienza con *set* y le sigue el nombre del atributo en UpperCamelCase.
 - El tipo de retorno del método debe ser *void*.
 - El tipo del parámetro del método debe ser el mismo que el tipo del atributo.

Hacer uso de este convenio nos facilitará trabajar con el resto del mundo y nos permitirá ampliar las capacidades de nuestro código utilizando frameworks existentes que hacen uso del convenio y que si no seguimos no podremos utilizar.

Es muy importante no utilizar estas nomenclaturas para métodos que en realidad no sean métodos *get* ni *set*.

En el Eclipse se pueden generar ambos métodos automáticamente en el menú *Source* → *Generate Getters and Setters*.

Vamos a incluir en la clase *Vehicle* los *getters* y *setters*. Para ello, nos tenemos que plantear qué atributos vamos a permitir consultar y modificar. En cuanto a la consulta, vamos a permitir consultar todos los atributos por lo que tendremos que realizar 3 *getters*. En cuanto a las modificaciones, el número de ruedas de un vehículo no cambia a lo largo del tiempo por lo que no hace falta ponerle un método *set*. La velocidad se va modificando con los métodos de acelerar y frenar por lo que tampoco le vamos a poner un método *set*. En cuanto al color, vamos a permitir que un vehículo pueda cambiar de color por lo que sí vamos a hacerle un método *set* al color.

```
public class Vehicle {

    private int wheelCount;
    private double speed;
    private String colour;

    public int getWheelCount() { //Método para consultar el número de ruedas
        return wheelCount;
    }

    public double getSpeed() { //Método para consultar la velocidad
        return speed;
    }

    public String getColour() { //Método para consultar el color
        return colour;
    }

    public void setColour(String colour) { //Método para modificar el color
        this.colour = colour;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }

}
```

Fijémonos en el método *setColour*:

```
public void setColour(String colour) { //Método para modificar el color
    this.colour = colour;
}
```

Tenemos dos variables que se llaman igual: *colour*. Una es un parámetro y otra es un atributo. Para diferenciarlas, el atributo se utiliza con la palabra *this*. El uso de *this* se explica con detenimiento más adelante.

En el caso de los booleanos, el getter comienza por *is*. Si la clase *Vehicle* tuviera un atributo booleano llamado *empty* para detectar cuándo está vacío el depósito de gasolina, el getter se llamaría *isEmpty()*:

```
private boolean empty;
public boolean isEmpty() {
    return empty;
}
```

10. Constructores

Un **constructor** es un método que se invoca cuando se crea un objeto y que sirve para inicializar los atributos del objeto y para realizar las acciones pertinentes que requiera el mismo para ser creado. El constructor tiene el mismo nombre que la clase y para invocarlo se utiliza el operador **new**.

```
vehicle car = new vehicle(); //vehicle() es un constructor de la clase vehicle
```

En los constructores no se especifica tipo de retorno:

```
package tema4_POO.constructorPorDefecto;

public class Vehicle {

    private int wheelCount;
    private double speed;
    private String colour;

    public Vehicle() { //No se especifica tipo de retorno
        wheelCount = 4;
        speed = 0;
        colour = "blanco";
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public void accelerate(double amount) {
        speed += amount;
    }
}
```

```

    public void brake(double amount) {
        speed -= amount;
    }
}

```

```

package tema4_POO.constructorPorDefecto;

public class Main {

    public void showDefaultConstructor() {

        Vehicle car1, car2;
        car1 = new Vehicle();
        car2 = new Vehicle();
        System.out.printf("Coche1: %d ruedas y de color %s",
car1.getWheelCount(), car1.getColour());
        System.out.printf("\nCoche2: %d ruedas y de color %s",
car2.getWheelCount(), car2.getColour());
        System.out.printf("\nVelocidad del coche1: %.2f km/h", car1.getSpeed());
        System.out.println("\nAceleramos el coche1 90,50 km/h");
        car1.accelerate(90.5);
        System.out.printf("Velocidad del coche1: %.2f km/h", car1.getSpeed());
        System.out.println("\nFrenamos el coche1 20,30 km/h");
        car1.brake(20.30);
        System.out.printf("Velocidad del coche1: %.2f km/h", car1.getSpeed());

    }

    public static void main(String[] args) {

        new Main().showDefaultConstructor();

    }

}

```

La salida por consola es la siguiente:

```

Coche1: 4 ruedas y de color blanco
Coche2: 4 ruedas y de color blanco
velocidad del coche1: 0,00 km/h
Aceleramos el coche1 90,50 km/h
velocidad del coche1: 90,50 km/h
Frenamos el coche1 20,30 km/h
velocidad del coche1: 70,20 km/h

```

El constructor que no tiene parámetros se llama **constructor por defecto**, como por ejemplo `Vehicle()`. Si nos fijamos en la salida por consola, los dos coches tienen los mismos datos ya que el constructor por defecto crea todos los objetos de la misma manera. Para poder crear objetos diferentes, tendremos que usar un constructor con parámetros:

```

package tema4_POO.constructorConParametros;

public class Vehicle {

    private int wheelCount;
    private double speed;
    private String colour;

    public Vehicle(int wheelCount, String colour) { //Constructor con parámetros
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }

}

```

```

package tema4_POO.constructorConParametros;

public class Main {

    public void showParameterizedConstructor() {

        Vehicle car, moto;
        car = new Vehicle(4, "azul");
        moto = new Vehicle(2, "rojo");
        System.out.printf("Coche: %d ruedas y de color %s", car.getWheelCount(),
            car.getColour());
        System.out.printf("\nMoto: %d ruedas y de color %s",
            moto.getWheelCount(), moto.getColour());

    }

}

```

```

public static void main(String[] args) {

    new Main().showParameterizedConstructor();

}

}

```

Salida por consola:

```

Coche: 4 ruedas y de color azul
Moto: 2 ruedas y de color rojo

```

Al tener parámetros el constructor, esto nos permite poder crear objetos diferentes.

En el Eclipse, se puede generar el constructor con parámetros en Menú *Source* → *Generate Constructor using Fields*.

Los constructores se pueden sobrecargar, por lo tanto, una clase puede tener varios constructores:

```

package tema4_POO.constructores1;

public class Vehicle {

    private int wheelCount;
    private double speed;
    private String colour;

    public Vehicle() { //Constructor por defecto
        wheelCount = 4;
        speed = 0;
        colour = "blanco";
    }

    public Vehicle(int wheelCount, String colour) { //Constructor con parámetros
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {

```



```

        this.colour = colour;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }
}

```

```

package tema4_POO.constructores1;

public class Main {

    public void showConstructors1() {

        Vehicle car, moto;
        car = new Vehicle();
        moto = new Vehicle(2, "rojo");
        System.out.printf("Coche: %d ruedas y de color %s", car.getWheelCount(),
            car.getColour());
        System.out.printf("\nMoto: %d ruedas y de color %s",
            moto.getWheelCount(), moto.getColour());

    }

    public static void main(String[] args) {

        new Main().showConstructors1();

    }

}

```

Salida por consola:

```

Coche: 4 ruedas y de color blanco
Moto: 2 ruedas y de color rojo

```

Si una clase no tiene constructor, Java crea uno por defecto. Los atributos se inicializan a su valor por defecto en función del tipo que posean:

- Tipos numéricos enteros: 0
- Tipos numéricos decimales: 0.0
- Caracteres: carácter nulo (Código Unicode 0)
- Booleanos: false
- Referencias a objetos: null

Solo se inicializan los atributos, las variables locales de los métodos no son inicializadas por defecto.

```
package tema4_P00.javaConstructor;

public class Vehicle { //Clase sin constructor. Java crea un constructor por defecto

    private int wheelCount;
    private double speed;
    private String colour;

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }

}
```

```
package tema4_P00.javaConstructor;

public class Main {

    public void showJavaConstructor() {

        Vehicle car;
        car = new Vehicle();
        System.out.printf("Color: %s", car.getColour());
        System.out.printf("\n%d ruedas", car.getWheelCount());
        System.out.printf("\nVelocidad: %.2f km/h", car.getSpeed());

    }

    public static void main(String[] args) {

        new Main().showJavaConstructor();

    }

}
```

```
}
```

Salida por consola:

```
Color: null  
0 ruedas  
velocidad: 0,00 km/h
```

Java solamente crea un constructor cuando la clase no tiene ninguno, pero si la clase tiene un constructor, aunque sea con parámetros, Java ya no crea ninguno por defecto.

```
package tema4_POO.constructores2;  
  
public class Vehicle {  
  
    private int wheelCount;  
    private double speed;  
    private String colour;  
  
    public Vehicle(int wheelCount, String colour) { //Java no crea el constructor  
por defecto  
        this.wheelCount = wheelCount;  
        this.colour = colour;  
        speed = 0;  
    }  
  
    public int getWheelCount() {  
        return wheelCount;  
    }  
  
    public double getSpeed() {  
        return speed;  
    }  
  
    public String getColour() {  
        return colour;  
    }  
  
    public void setColour(String colour) {  
        this.colour = colour;  
    }  
  
    public void accelerate(double amount) {  
        speed += amount;  
    }  
  
    public void brake(double amount) {  
        speed -= amount;  
    }  
  
}
```

```
package tema4_POO.constructores2;
```

```

public class Main {

    public void showConstructors2() {

        vehicle moto;
        vehicle car = new Vehicle();//Error de compilación: el constructor
Vehicle() no está definido
        moto = new Vehicle(2, "rojo");
        System.out.printf("Coche: %d ruedas y de color %s", car.getWheelCount(),
car.getColour());
        System.out.printf("\nMoto: %d ruedas y de color %s",
moto.getWheelCount(), moto.getColour());

    }

    public static void main(String[] args) {

        new Main().showConstructors2();

    }

}

```

Cuando se intenta crear el coche con el constructor por defecto, el compilador nos da un mensaje de error de que el constructor *Vehicle()* no está definido.

11. Visibilidad

La visibilidad de los atributos y métodos de una clase se establece utilizando los modificadores de acceso.

Los modificadores de acceso permiten dar un nivel de seguridad mayor a nuestras aplicaciones restringiendo el acceso a diferentes atributos y métodos asegurándonos que el cliente no va a consultar el valor de un atributo que no debe, no va a modificar incorrectamente el valor de un atributo o que no va a utilizar un método que no le esté permitido. Cuando nos referimos al término cliente es cualquier programador que utilice nuestras clases.

Tal y como vimos en el apartado 5. *Modificadores de acceso*, se resume su funcionamiento en la siguiente tabla:

	Private	Friendly	Public
Misma clase	X	X	X
Mismo paquete		X	X
Otro paquete			X

Veamos un ejemplo de la clase *Vehicle* que está en el mismo paquete que una clase cliente *Main*. Vamos a comprobar la visibilidad que tiene la clase *Main* sobre la clase *Vehicle* en función de los modificadores de acceso. Tal y como podemos observar en la tabla anterior, tendrá acceso a *public* y a *friendly* pero no a *private*.

```

package tema4_POO.visibilidad;

public class Vehicle {

    private int wheelCount;
    private double speed;
    private String colour;

    public Vehicle(int wheelCount, String colour) {
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }

    int getWheelCount() { //Modificador de acceso friendly
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }

}

```

```

package tema4_POO.visibilidad;

public class Main {

    public void showVisibility() {

        Vehicle car;
        car = new Vehicle(4, "azul");
        //getWheelCount() es visible porque es friendly y la clase Main está en
        el mismo paquete que la clase Vehicle
        System.out.printf("El coche tiene %d ruedas", car.getWheelCount());
        //accelerate(100) es visible porque es public
        car.accelerate(100);
        //Error de compilación: speed no es visible porque es private
        System.out.printf("\nEl coche va a %.2f km/h", car.speed);
    }

}

```

```

    }

    public static void main(String[] args) {

        new Main().showVisibility();

    }

}

```

Podemos observar lo siguiente en el código anterior:

- El método `getWheelCount()` es visible porque es *friendly* y la clase *Main* está en el mismo paquete que la clase *Vehicle*.
- El método `accelerate(100)` es visible porque es *public*.
- El atributo `speed` no es visible porque es *private*.

12. instanceof

El operador *instanceof* permite comprobar si un determinado objeto pertenece a una clase concreta. Se utiliza de esta forma:

```
objeto instanceof clase
```

Devuelve *true* si el objeto pertenece a dicha clase.

```

package tema4_POO.instanceofObjetos;

public class Main {

    public void showInstanceof() {

        vehicle vehicle;
        vehicle = new Vehicle(2, "azul");
        System.out.println(vehicle instanceof Vehicle);//true

    }

    public static void main(String[] args) {

        new Main().showInstanceof();

    }

}

```

13. Referencias

Una **referencia** en cualquier lenguaje de programación hace alusión a la posición en memoria RAM que tiene una variable.

Una variable es conceptualmente un recipiente donde guardamos un dato. Java es un lenguaje fuertemente tipado. Las variables se definen siempre con un tipo asociado. No se puede meter en una variable ningún dato que no sea del tipo con que se definió.

Una variable es un lugar en la memoria donde se guarda un dato. Para ser exacto, este lugar en la memoria es la *Pila o Stack*. En el caso de los datos primitivos, como en `int i = 5;` hay cuatro bytes en la *Pila* donde se almacena el número 5. Pero cuando se crea un objeto en Java, el objeto se guarda en una parte de la memoria llamada *Heap*. Cuando asignamos el objeto a una variable como en `Vehicle car=new Vehicle();`, lo que guardamos en *car* es la dirección de memoria *Heap* donde está el objeto.

Stack en java es una sección de memoria que contiene métodos, variables locales y variables de referencia. La memoria de pila siempre se referencia en el orden de último en entrar primero en salir. Las variables locales se crean en la pila.

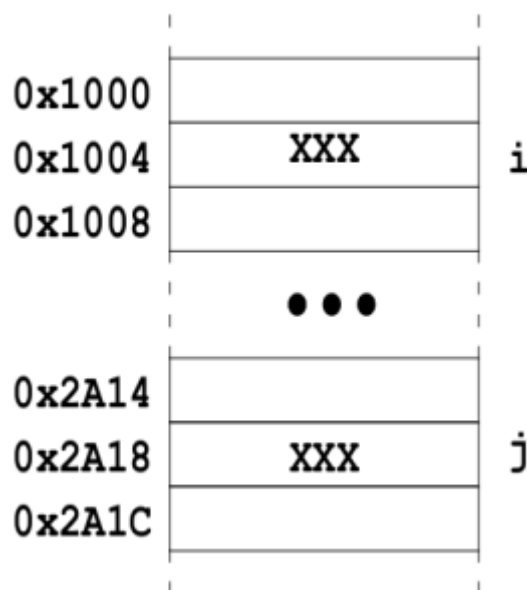
Heap es una sección de memoria que contiene objetos y también puede contener variables de referencia. Los atributos se crean en el *heap*.

13.1 Representación en memoria de tipos primitivos

Veámoslo paso a paso con un ejemplo:

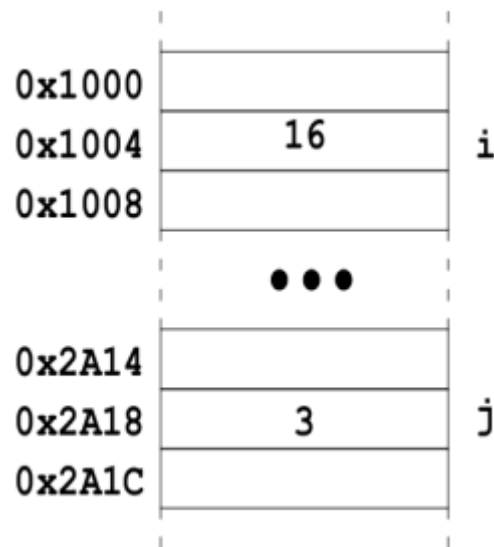
- Cuando se declara una variable de un tipo primitivo, el compilador reserva un área de memoria para ella:

```
int i , j;
```



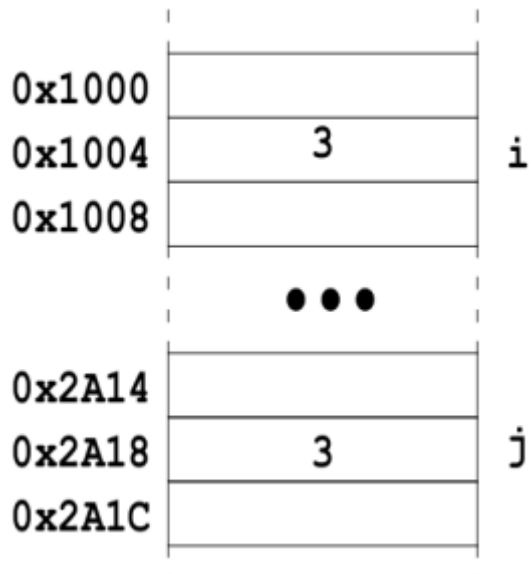
- Cuando se asigna un valor, éste es escrito en el área reservada:

```
i=16;  
j=3;
```



- La asignación entre variables significa copiar el contenido de una variable en la otra:

```
i = j;
```



- La comparación entre variables compara los contenidos de las mismas:

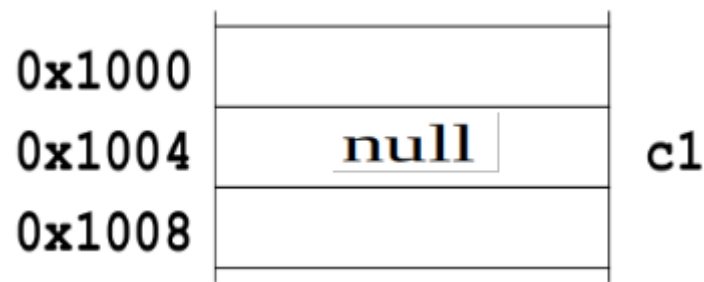
```
i == j; //true porque 3 es igual a 3
```

13.2 Representación en memoria de objetos

Supongamos que tenemos una clase *Complex* con dos atributos: *r*, *i*

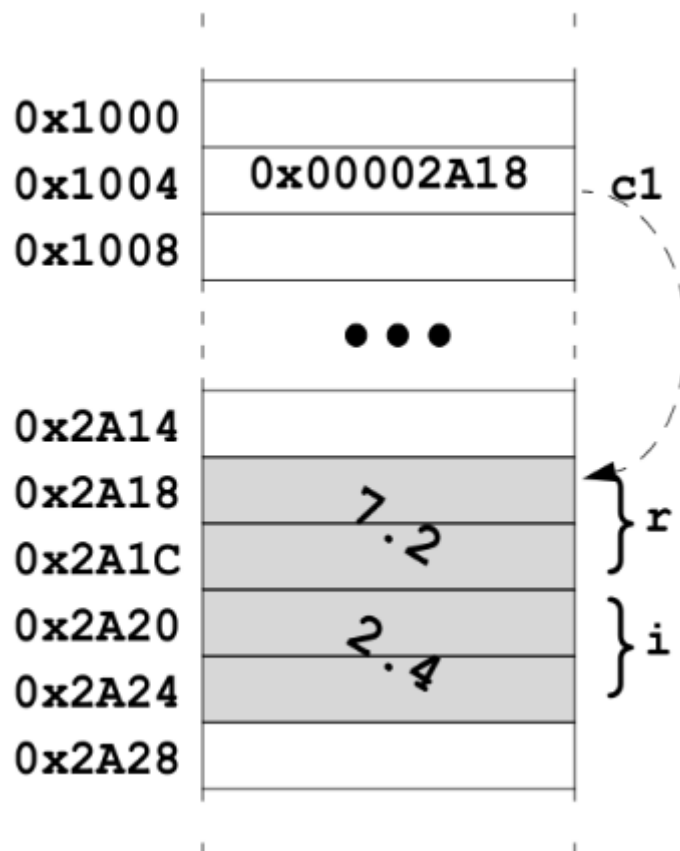
- Cuando se declara una variable de un objeto, el compilador reserva un área de memoria para ella. Mientras no se cree ningún objeto con el operador *new*, lo que contendrá será *null*. La palabra reservada *null* indica que una variable que referencia a un objeto se encuentra de momento sin referenciar a ninguno.

```
Complex c1;
```

- Cuando se crea el objeto llamando al constructor con el operador *new*, lo que se guarda en la variable *c1* es la dirección de memoria donde se ha almacenado el objeto en el *heap*. Es decir, la diferencia entre los tipos primitivos y los objetos es que en los primitivos la variable almacena el valor y en los objetos, la variable almacena la dirección de memoria donde se encuentra el objeto.

```
c1 = new Complex(7.2 , 2.4);
```



- La asignación entre variables objetos significa copiar la dirección de memoria donde se encuentra el objeto:

```
Complex c2 = c1;
```

En este caso, el valor de la variable *c2* es **0x2A18**, por lo tanto, ambas variables apuntan al mismo objeto. Si una de ellas modifica algún valor del objeto, también le afectará a la otra variable. Por ejemplo, si a *c2* le cambiamos el valor de *r* a 9.7, si consultamos el valor de *r* de *c1*, también valdrá 9.7, en lugar de 7.2.

- La comparación entre referencias no compara los contenidos de los objetos sino las direcciones de memoria, es decir, si apuntan al mismo sitio:

```
c2 == c1; //true porque apuntan al mismo sitio
Complex c3 = new Complex(7.2 , 2.4);
c3 == c1; //false porque aunque los objetos tengan los mismos valores de r e
i, c3 y c1 no apuntan al mismo sitio, es decir, no tienen la misma dirección
de memoria.
```

Para comparar los contenidos de los objetos, se utiliza el método *equals*:

```
c3.equals(c1); //true porque los contenidos de los objetos es el mismo
```

13.3 Garbage Collector

Garbage Collector (recolector de basura) es un programa que se ejecuta en el Java Virtual Machine que elimina objetos que ya no están siendo utilizados por una aplicación Java. Es una forma de gestión automática de la memoria.

Un objeto es **elegible para el recolector de basura** cuando deja de existir alguna referencia hacia él. Veamos algunos ejemplos:

```
Complex c1 = new Complex(7.2 , 2.4);
....
c1 = null;
....
```

Existe un objeto referenciado por la variable *c1*. Cuando la variable *c1* pierde la referencia porque se le asigna el *null*, se pierde cualquier forma de acceder al objeto, de modo que pasa a ser elegible para el recolector de basura.

```
Complex c1 = new Complex(7.2 , 2.4);
Complex c2 = c1;
....
c1 = null;
....
```

En este segundo ejemplo, el objeto no pasa a ser elegible para ser recolectado pues aunque la variable *c1* haya perdido la referencia porque se le asigna el *null*, todavía existe una referencia hacia el objeto por la variable *c2*.

```
Complex c1 = new Complex(7.2 , 2.4);
....
c1 = new Complex(8.3 , 2.7);
....
```

En este otro ejemplo, la variable *c1* es reasignada, es decir, se le asigna otro objeto mediante el operador *new*, por lo tanto se pierde cualquier referencia al objeto creado al principio `new Complex(7.2 , 2.4)` por lo que dicho objeto pasa a ser elegible para el recolector de basura.

14. This

Cuando se llama a un método, se pasa automáticamente un argumento implícito que es una referencia al objeto invocado, es decir, el objeto sobre el que se llama el método. Esta referencia se llama *this*, es decir, *this* es una variable que hace referencia al objeto actual.

Veamos los distintos usos que puede tener:

14.1 Para acceder a un atributo del objeto actual

Fijémonos de nuevo en la clase *Vehicle*:

```
public class Vehicle {  
  
    private int wheelCount;  
    private double speed;  
    private String colour;  
  
    public int getWheelCount() {  
        return wheelCount;  
    }  
  
    public double getSpeed() {  
        return speed;  
    }  
  
    public String getColour() {  
        return colour;  
    }  
  
    public void setColour(String colour) {  
        this.colour = colour;  
    }  
  
    public void accelerate(double amount) {  
        speed += amount;  
    }  
  
    public void brake(double amount) {  
        speed -= amount;  
    }  
  
}
```

Centrémonos en el método getter del color:

```
public String getColour() {  
    return colour;  
}
```

colour es un atributo de la clase *Vehicle*. El método *getColour()* de la clase *Vehicle* puede acceder perfectamente al atributo *colour* ya que cuando el método *getColour()* es llamado, recibe de forma implícita el objeto actual, es decir, el objeto sobre el que se llama el método. Esto le permite al método *getColour()* poder acceder a todos los atributos de la clase y llamar a cualquier método de

la misma.

Como *colour* es un atributo de la clase *Vehicle*, también podíamos haber accedido a él como *this.colour*. Pero en realidad, cuando desde un método se accede a un atributo, se usa *this* de forma implícita, es decir, que aunque no escribamos *this*, el compilador lo sobreentiende. Por eso en la práctica, solo se indica si es imprescindible. Veamos un ejemplo en el que sea imprescindible:

```
public void setColour(String colour) {  
    this.colour = colour;  
}
```

En este caso, nos encontramos con dos variables que se llaman *colour*: el atributo y el parámetro del método. El **shadow** (*sombra*) de variables se refiere a la práctica en programación de utilizar dos variables con el mismo nombre dentro de ámbitos que se superponen. La variable con el alcance de nivel superior se oculta porque la variable con el alcance de nivel inferior la anula. La variable de nivel superior se dice entonces que es "*sombreada*". En nuestro ejemplo, el atributo es "*sombreado*" por el parámetro, es decir, cuando en el método accedemos a *colour*, es el parámetro al que nos estamos refiriendo. Para poder acceder al atributo, necesitamos utilizar la palabra *this*: *this.colour*. ¿Qué es lo que estamos haciendo entonces con esta línea de código: *this.colour = colour* ? Pues le estamos asignando el valor del parámetro al atributo, es decir, estamos actualizando el color del objeto actual con el color pasado por parámetro al método *setColour*.

14.2 Para invocar a un constructor

Un constructor puede llamar a otro constructor de la clase utilizando *this*, pero esta llamada solamente puede estar en la primera línea de código.

```
package tema4_POO.thisConstructor;  
  
public class Vehicle {  
  
    private int wheelCount;  
    private double speed;  
    private String colour;  
  
    public Vehicle() { //Constructor por defecto  
  
        this(4, "blanco");  
        /*Llama al constructor con parámetros. Esta llamada se tiene  
        que hacer en la primera línea de código*/  
    }  
  
    public Vehicle(int wheelCount, String colour) { //Constructor con parámetros  
        this.wheelCount = wheelCount;  
        this.colour = colour;  
        speed = 0;  
    }  
  
    public int getWheelCount() {  
        return wheelCount;  
    }  
}
```

```

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }
}

```

```

package tema4_POO.thisConstructor;

public class Main {

    public void showThisConstructor() {

        Vehicle car, moto;
        car = new Vehicle();
        moto = new Vehicle(2, "rojo");
        System.out.printf("Coche: %d ruedas y de color %s", car.getWheelCount(),
            car.getColour());
        System.out.printf("\nMoto: %d ruedas y de color %s",
            moto.getWheelCount(), moto.getColour());

    }

    public static void main(String[] args) {

        new Main().showThisConstructor();

    }

}

```

Salida por consola:

```

Coche: 4 ruedas y de color blanco
Moto: 2 ruedas y de color rojo

```

14.3 Como referencia al objeto actual

Dentro de la clase, siempre que queramos obtener una referencia al objeto actual, podemos utilizar *this*.

Supongamos que queremos modificar el método acelerar de los vehículos para que devuelvan el objeto actual con la velocidad modificada por la aceleración:

```
public Vehicle accelerate(double amount) {  
    speed += amount; //Se incrementa la velocidad del objeto actual con la  
    cantidad pasada por parámetro  
    return this; //Se devuelve el objeto actual  
}
```

Veamos el ejemplo completo:

```
package tema4_POO.thisObjetoActual1;  
  
public class Vehicle {  
  
    private int wheelCount;  
    private double speed;  
    private String colour;  
  
    public Vehicle(int wheelCount, String colour) {  
        this.wheelCount = wheelCount;  
        this.colour = colour;  
        speed = 0;  
    }  
  
    public int getWheelCount() {  
        return wheelCount;  
    }  
  
    public double getSpeed() {  
        return speed;  
    }  
  
    public String getColour() {  
        return colour;  
    }  
  
    public void setColour(String colour) {  
        this.colour = colour;  
    }  
  
    public Vehicle accelerate(double amount) {  
        speed += amount;  
        return this; //Se devuelve una referencia al objeto actual  
    }  
  
    public void brake(double amount) {  
        speed -= amount;  
    }  
}
```

```
}
```

```
package tema4_POO.thisObjetoActual1;

public class Main {

    public void showthisCurrentObject1() {

        Vehicle car1, car2;
        car1 = new Vehicle(4, "azul");
        System.out.printf("Coche1: %d ruedas, color %s, velocidad %.2f",
            car1.getWheelCount(), car1.getColour(), car1.getSpeed());
        car2 = car1.accelerate(90);
        System.out.println("\nAceleramos el coche1 y se lo asignamos al coche2");
        System.out.printf("Coche1: %d ruedas, color %s, velocidad %.2f",
            car1.getWheelCount(), car1.getColour(), car1.getSpeed());
        System.out.printf("\nCoche2: %d ruedas, color %s, velocidad %.2f",
            car2.getWheelCount(), car2.getColour(), car2.getSpeed());
    }

    public static void main(String[] args) {

        new Main().showthisCurrentObject1();

    }

}
```

Salida por consola:

```
Coche1: 4 ruedas, color azul, velocidad 0,00
Aceleramos el coche1 y se lo asignamos al coche2
Coche1: 4 ruedas, color azul, velocidad 90,00
Coche2: 4 ruedas, color azul, velocidad 90,00
```

Veamos más ejemplos del uso de *this* como referencia al objeto actual. Vamos a añadir dos métodos nuevos a la clase *Vehicle*:

- *addSpeeds*: devuelve la suma de dos velocidades, la del objeto actual y la del vehículo pasado como parámetro.
- *doubleSpeed*: dobla la velocidad del objeto actual utilizando el método *addSpeeds*.

```
public double addSpeeds(Vehicle vehicle) { //Devuelve la suma de dos velocidades
    return speed + vehicle.speed;
}

public void doubleSpeed() {
    speed = addSpeeds(this); //Se le pasa al método addSpeeds la referencia al
    objeto actual
}
```

Para doblar la velocidad utilizando el método *addSpeeds*, tenemos que pasarle a este método la referencia al objeto actual para que sume la velocidad consigo mismo, doblando de esta forma su propia velocidad.

```
package tema4_POO.thisObjetoActual2;

public class Vehicle {

    private int wheelCount;
    private double speed;
    private String colour;

    public Vehicle(int wheelCount, String colour) {
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }

    public double addSpeeds(Vehicle vehicle) { //Devuelve la suma de dos
    velocidades
        return speed + vehicle.speed;
    }

    public void doubleSpeed() {
        speed = addSpeeds(this); //Se le pasa al método addSpeeds la referencia
    al objeto actual
    }

}
```



```

package tema4_POO.thisObjetoActual2;

public class Main {

    public void showthisCurrentObject2() {

        Vehicle car1, car2;
        car1 = new Vehicle(4, "azul");
        car2 = new Vehicle(4, "rojo");
        car1.accelerate(90.3);
        car2.accelerate(120.5);
        System.out.printf("Velocidad del coche1: %.2f", car1.getSpeed());
        System.out.printf("\nVelocidad del coche2: %.2f", car2.getSpeed());
        System.out.printf("\nSuma de las velocidades de los dos coches: %.2f",
car1.addSpeeds(car2));
        car1.doubleSpeed();
        System.out.printf("\nEl coche1 ha doblado su velocidad: %.2f",
car1.getSpeed());

    }

    public static void main(String[] args) {

        new Main().showthisCurrentObject2();

    }

}

```

Salida por consola:

```

Velocidad del coche1: 90,30
Velocidad del coche2: 120,50
Suma de las velocidades de los dos coches: 210,80
El coche1 ha doblado su velocidad: 180,60

```

Fijémonos de nuevo en el método de doblar la velocidad:

```

public void doubleSpeed() {
    speed = addSpeeds(this);
}

```

El método *doubleSpeed* de la clase *Vehicle* puede llamar perfectamente al método *addSpeeds* ya que cuando el método *doubleSpeed* es llamado, recibe de forma implícita el objeto actual, es decir, el objeto sobre el que se llama el método. Esto le permite al método *doubleSpeed* poder acceder a todos los atributos de la clase y llamar a cualquier método de la misma.

Como *addSpeeds* es un método de la clase *Vehicle*, el método *doubleSpeed* podía haberlo llamado también utilizando el *this*: *this.addSpeeds(this)*. Pero en realidad, cuando desde un método se llama a otro de la clase, se usa *this* de forma implícita, es decir, que aunque no escribamos *this*, el compilador lo sobreentiende. Por eso en la práctica, solo se indica si es imprescindible.

15. Encadenamiento de llamadas a métodos

Se emplea cuando invocamos a un método de un objeto que nos devuelve como resultado otro objeto al que podemos volver a invocar otro método y así encadenar varias operaciones.

```
package tema4_P00;

public class CallsToMethods {

    public void showCallsToMethods() {

        Boolean b;
        String string;

        string = "EntornosDeDesarrollo";
        System.out.println(string.substring(10).toUpperCase()); //DESARROLLO

        b = Boolean.TRUE;
        System.out.println(b.toString().charAt(2)); //u

    }

    public static void main(String[] args) {

        new CallsToMethods().showCallsToMethods();

    }

}
```

En este ejemplo, el método `substring(10)` está devolviendo una subcadena de la cadena `string` a partir del carácter 10 empezando en 0, es decir, "Desarrollo", al que se le invoca luego el método `toUpperCase` devolviendo como resultado la cadena "DESARROLLO".

Y el método `toString()` de la variable `b` de tipo `Boolean` está devolviendo la cadena "true" a la que se le encadena el método `charAt(2)` devolviendo el carácter 'u'.

16. Sintaxis fluida

Cuando un método modifica algún atributo del objeto, se puede devolver el objeto con un `return` para que dicho método pueda insertarse en una expresión. Esto permite encadenar otras llamadas de métodos consiguiendo que el código sea más corto, más legible y más fácil de manejar para los programadores.

Veamos un ejemplo de dos clases, una con sintaxis fluida y otra sin ella para ver la diferencia:

```
package tema4_P00.sintaxisFluida;

public class Vehicle1 { //Clase que no utiliza sintaxis fluida

    private int wheelCount;
    private double speed;
    private String colour;
```

```

public Vehicle1(int wheelCount, String colour) {
    this.wheelCount = wheelCount;
    this.colour = colour;
    speed = 0;
}

public int getWheelCount() {
    return wheelCount;
}

public double getSpeed() {
    return speed;
}

public String getColour() {
    return colour;
}

public void setColour(String colour) {
    this.colour = colour;
}

public void accelerate(double amount) {
    speed += amount;
}

public void brake(double amount) {
    speed -= amount;
}
}

```

```

package tema4_POO.sintaxisFluida;

public class Vehicle2 { //Clase que utiliza sintaxis fluida en los métodos
    accelerate y brake

    private int wheelCount;
    private double speed;
    private String colour;

    public Vehicle2(int wheelCount, String colour) {
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }
}

```

```

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public Vehicle2 accelerate(double amount) {
        speed += amount;
        return this; //Se devuelve una referencia al objeto actual para poder
        utilizar sintaxis fluida
    }

    public Vehicle2 brake(double amount) {
        speed -= amount;
        return this; //Se devuelve una referencia al objeto actual para poder
        utilizar sintaxis fluida
    }
}

```

```

package tema4_POO.sintaxisFluida;

public class Main {

    public void showfluidSintax() {

        Vehicle1 car1;
        Vehicle2 car2;
        car1 = new Vehicle1(4, "rojo");
        car2 = new Vehicle2(4, "azul");

        car1.accelerate(120.55);
        car1.brake(20.32);
        System.out.printf("Velocidad del coche1: %.2f", car1.getSpeed());

        /* car2 hace en una línea de código lo mismo que car1
        * hace en 3 líneas de código debido a que car2 utiliza
        * sintaxis fluida:
        */
        System.out.printf("\nVelocidad del coche2: %.2f",
        car2.accelerate(120.55).brake(20.32).getSpeed());

    }

    public static void main(String[] args) {

        new Main().showfluidSintax();

    }

}

```

Salida por consola:

```
velocidad del coche1: 100,23  
velocidad del coche2: 100,23
```

17. Invariante de una clase

El invariante de una clase es el conjunto de restricciones que deben cumplir todos los objetos que se instancien de dicha clase, como por ejemplo:

- Restricciones sobre los valores que pueden tomar los atributos de la clase. Ejemplo: el atributo *wheelCount* de la clase *Vehicle* tiene que ser mayor que 0.
- Restricciones que afecten a más de un atributo. Ejemplo: si una clase modela un rango de valores, el atributo que corresponda al límite superior del rango debe ser forzosamente mayor o igual que el atributo que corresponda al límite inferior del rango, dada la definición matemática de rango.
- Restricciones con respecto a los objetos con los que se relaciona. Ejemplo: un empleado tiene un atributo que es un objeto de la clase *Empresa* donde trabaja. Una restricción posible sería que dicha empresa no pueda ser nula.

Los métodos constructores de una clase deben respetar el invariante de la clase a la hora de construir los objetos.

Los métodos públicos de la clase también deben respetar el invariante. Un método puede no respetar el invariante en el transcurso de la ejecución, pero cuando el método finalice, el invariante se tiene que cumplir. Es decir, es perfectamente viable que un método para alcanzar su objetivo pueda perder de forma temporal el invariante pero siempre y cuando finalice con el invariante cumplido, es decir, antes de la llamada el invariante se debe cumplir y después de la llamada también, durante la ejecución del mismo puede no satisfacerse.

Únicamente se deben satisfacer los invariantes en las llamadas a métodos públicos, la ejecución de métodos privados de la misma clase pueden saltarse esta norma aunque no es aconsejable.

Definir invariantes de clase puede ayudar a los programadores y controladores de calidad a localizar más errores durante las pruebas de software.

18. Encapsulamiento

La encapsulación es un principio fundamental de la programación orientada a objetos que consiste en ocultar el estado o los atributos de un objeto y obligar a que toda interacción se realice a través de los métodos del objeto definidos en su clase para conservar su invariante.

El encapsulamiento se consigue utilizando los modificadores de acceso. Se recomienda que los atributos de una clase sean privados, por lo tanto, aquellos atributos que se permitan consultar deben tener sus propios métodos *get*, y los que se permitan modificar, deben tener sus propios métodos *set*. Hacer uso de este convenio nos facilitará trabajar con el resto del mundo y nos permitirá ampliar las capacidades de nuestro código utilizando frameworks existentes que hacen uso del convenio y que si no seguimos no podremos utilizar.

Veamos un ejemplo que no cumple el principio de encapsulamiento:

```
package tema4_POO.noEncapsulacion;  
  
public class Vehicle { //Clase que no utiliza encapsulación
```

```

public int wheelCount;
public double speed;
public String colour;

public Vehicle(int wheelCount, String colour) {
    this.wheelCount = wheelCount;
    this.colour = colour;
    speed = 0;
}

public void accelerate(double amount) {
    speed += amount;
}

public void brake(double amount) {
    speed -= amount;
}
}

```

```

package tema4_POO.noEncapsulacion;

public class Main {

    public void showNonEncapsulation() {

        Vehicle car;
        car = new Vehicle(4, "azul");
        car.accelerate(90.54);
        System.out.printf("velocidad: %.2f", car.speed);
        car.speed = 120;//El cliente está accediendo al atributo directamente no
        //cumpliendo el encapsulamiento
        System.out.printf("\nvelocidad: %.2f", car.speed);

    }

    public static void main(String[] args) {

        new Main().showNonEncapsulation();

    }

}

```

Salida por consola:

```

velocidad: 90,54
velocidad: 120,00

```

Podemos observar que no se está cumpliendo el principio de encapsulamiento ya que el cliente está accediendo directamente al atributo *speed* modificando incorrectamente su valor ya que la velocidad solamente se debe modificar acelerando o frenando el vehículo.

Vamos a modificar el ejemplo anterior para que sí cumpla con el principio de encapsulamiento, ocultando el estado o los atributos del objeto y obligando a que toda interacción se realice a través de los métodos del objeto definidos en su clase:

```
package tema4_POO.encapsulacion;

public class Vehicle { //Clase que utiliza encapsulación

    private int wheelCount;
    private double speed;
    private String colour;

    public Vehicle(int wheelCount, String colour) {
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }

}
```

```
package tema4_POO.encapsulacion;

public class Main {

    public void showEncapsulation() {

        Vehicle car;
        car = new Vehicle(4, "azul");
        car.accelerate(90.54);
        System.out.printf("Velocidad: %.2f", car.getSpeed());
    }
}
```

```

        car.speed = 120; //Error de compilación: el atributo speed no es visible
    }

    public static void main(String[] args) {

        new Main().showEncapsulation();

    }

}

```

Al cliente no se le permite modificar directamente el atributo *speed* ya que éste tiene como modificador de acceso *private*. Si quiere modificar la velocidad tiene que hacerlo a través de los métodos *accelerate* y *brake*, cumpliendo de esta forma con el principio de encapsulamiento.

19. El método toString

El método `toString()` se utiliza para obtener una cadena de texto que represente al objeto. Lo veremos en más profundidad en el [Tema 7 Herencia 6 El método toString](#). Veamos su uso:

```

package tema4_POO.toString;

public class Vehicle {

    private int wheelCount;
    private double speed;
    private String colour;

    public Vehicle(int wheelCount, String colour) {
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public void accelerate(double amount) {
        speed += amount;
    }
}

```



```

    public void brake(double amount) {
        speed -= amount;
    }

    //Método toString para obtener una cadena de texto que represente al objeto
    @Override
    public String toString() {
        return String.format("Número de ruedas: %d, color: %s, velocidad: %.2f",
            wheelCount, colour, speed);
    }
}

```

```

package tema4_P00.toString;

public class Main {

    public void showToString() {

        Vehicle vehicle;

        vehicle = new Vehicle(2, "azul");
        System.out.println(vehicle.toString());
        System.out.println(vehicle); //hace lo mismo que vehicle.toString()

    }

    public static void main(String[] args) {

        new Main().showToString();

    }

}

```

Salida por consola:

```

Número de ruedas: 2, color: azul, velocidad: 0,00
Número de ruedas: 2, color: azul, velocidad: 0,00

```

El método `toString()` también se puede generar con el Eclipse en el menú *Source* → *Generate toString()*.

Cuando se saca por consola un objeto y solo se pone el nombre de la variable del objeto, se llama al `toString` del objeto tal y como podemos observar en la línea de código:

```

System.out.println(vehicle); //hace lo mismo que vehicle.toString()

```

20. Control de nulidad

El método `Objects.requireNonNull` es una utilidad estándar en Java que pertenece a la clase `java.util.Objects`. Su principal función es validar que un objeto no sea `null`. Si el objeto es `null`, lanza una excepción de tipo `NullPointerException`. Esto lo hace útil para asegurar que un parámetro, variable o referencia no pueda ser `null` en tiempo de ejecución. Si el objeto no es `null`, simplemente lo retorna, lo que permite que se siga utilizando en el código de manera segura. Ejemplo:

```
import java.util.Objects;

public class Vehicle {

    private int wheelCount;
    private double speed;
    private String colour;

    public Vehicle(int wheelCount, String colour) {
        this.wheelCount = wheelCount;
        this.colour = Objects.requireNonNull(colour, "El color no puede ser nulo");
        speed = 0;
    }
}
```

¿Cuándo utilizar `Objects.requireNonNull`?:

- Validación de parámetros: es muy común usarlo en los constructores y en los métodos que requieren que ciertos parámetros no sean nulos.
- Asegurar inmutabilidad: en clases que siguen el patrón de objetos inmutables, se puede asegurar que los atributos esenciales no sean `null` desde el inicio.
- Seguridad en tiempo de ejecución: es una manera de asegurarse en tiempo de ejecución de que efectivamente un valor no es `null`.

21. Uso de las excepciones para el control del invariante de una clase

Veamos un ejemplo de invariante en la clase *Vehicle* con respecto al color: un vehículo en todo momento tiene que tener información sobre el color, no puede estar vacío dicho campo. En la clase *Vehicle* hay dos sitios que afectan a la modificación del color: en el constructor y en el método *setColour*. En este caso, controlamos el invariante en el método *setColour* y el constructor llama al *setColour*. En el caso de un atributo que no tuviera método *set*, el control del invariante se haría en el constructor.

```
package tema4_POO.invariante;

public class Vehicle {

    private int wheelCount;
    private double speed;
    private String colour;
```

```

public Vehicle(int wheelCount, String colour) {
    this.wheelCount = wheelCount;
    setColour(colour); //El constructor llama al método setColour
    speed = 0;
}

public int getWheelCount() {
    return wheelCount;
}

public double getSpeed() {
    return speed;
}

public String getColour() {
    return colour;
}

public void setColour(String colour) {
    if(colour.isEmpty()) { //Control del invariante
        throw new IllegalArgumentException();
    }
    this.colour = colour;
}

public void accelerate(double amount) {
    speed += amount;
}

public void brake(double amount) {
    speed -= amount;
}

@Override
public String toString() {
    return String.format("Número de ruedas: %d, color: %s, velocidad: %.2f",
wheelCount, colour, speed);
}
}

```

```

package tema4_POO.invariante;

public class Invariancel {

    public void show() {

        Vehicle vehicle = new Vehicle(4, "");
        System.out.println(vehicle);

    }

    public static void main(String[] args) {

```

```

        new Invariance1().show();

    }

}

```

En este código estamos intentando crear un vehículo con un color vacío por lo que abortará:

```

Exception in thread "main" java.lang.IllegalArgumentException
    at tema4_POO.invariante.Vehicle.setColour(Vehicle.java:29)
    at tema4_POO.invariante.Vehicle.<init>(Vehicle.java:11)
    at tema4_POO.invariante.Invariance1.show(Invariance1.java:7)
    at tema4_POO.invariante.Invariance1.main(Invariance1.java:14)

```

De esta manera controlamos que no se va a crear un vehículo sin color. Veamos un ejemplo donde el vehículo lo creamos con un color y luego intentamos modificarlo con un color vacío:

```

package tema4_POO.invariante;

public class Invariance2 {

    public void show() {

        vehicle vehicle = new Vehicle(4, "azul");
        System.out.println(vehicle);
        vehicle.setColour("");

    }

    public static void main(String[] args) {

        new Invariance2().show();

    }

}

```

En este caso también abortará porque el método *setColour* controla que no se modifique un vehículo con un color vacío:

```

Número de ruedas: 4, color: azul, velocidad: 0,00
Exception in thread "main" java.lang.IllegalArgumentException
    at tema4_POO.invariante.Vehicle.setColour(Vehicle.java:29)
    at tema4_POO.invariante.Invariance2.show(Invariance2.java:9)
    at tema4_POO.invariante.Invariance2.main(Invariance2.java:15)

```

En ambos casos, el cliente puede capturar la excepción y dar un mensaje al usuario:

```

package tema4_POO.invariante;

import java.util.Scanner;

public class Invariance3 {

```

```

@SuppressWarnings("resource")
public void show() {

    vehicle vehicle;
    String colour;
    Scanner keyboard = new Scanner(System.in);

    try {
        System.out.println("Introduzca un color para el vehículo: ");
        colour = keyboard.nextLine();
        vehicle = new vehicle(4, colour);
        System.out.println(vehicle);
        System.out.println("Introduzca el nuevo color del vehículo: ");
        colour = keyboard.nextLine();
        vehicle.setColour(colour);
        System.out.println(vehicle);
    } catch (IllegalArgumentException e) {
        System.err.println("El color del vehículo no puede estar vacío");
    }

}

public static void main(String[] args) {

    new Invariance3().show();

}

}

```

22. Records

Los registros (*records*) son una nueva clase especial en Java que simplifica la creación de clases que son principalmente "contenedores de datos", es decir, clases cuyos principales propósitos son almacenar valores. Los records contienen datos inmutables.

Un ejemplo típico de un record es un `Point` que representa una clase inmutable utilizada para almacenar coordenadas en un espacio bidimensional (con `x` e `y` como componentes). La definición de dicho record sería la siguiente:

```
public record Point(int x, int y) {}
```

Este código genera automáticamente:

- Un constructor con `x` e `y` como parámetros llamado *constructor canónico*.
- Métodos getters para `x` e `y`. El nombre de los getters de los records no sigue las convenciones JavaBean `getX()` y `getY()` sino mediante métodos con el mismo nombre que el campo: `x()` e `y()`.
- Implementaciones de los métodos *equals*, *hashCode* y *toString* basados en `x` e `y`.

Estos métodos también se pueden anular si hiciera falta una implementación diferente.

Cualquier registro extiende implícitamente de `java.lang.Record`, al igual que cualquier tipo enumerado extiende implícitamente de `java.lang.Enum`. La superclase `Record` no tiene estado y sólo tiene métodos abstractos *equals*, *hashCode* y *toString*.

Veamos un ejemplo de utilización del record `Point`:

```
package tema4_POO.records;

public record Point(int x, int y) {}
```

```
package tema4_POO.records;

public class showRecords {

    public void show() {

        Point p = new Point(3,4);
        System.out.println(p); //Llama por defecto al toString
        System.out.println(p.x());
        System.out.println(p.y());
        System.out.println(p.equals(new Point(3,4)));
        System.out.println(p.hashCode());

    }

    public static void main(String[] args) {

        new showRecords().show();

    }

}
```

Salida por consola:

```
Point[x=3, y=4]
3
4
true
97
```

Se puede añadir lógica personalizada en el constructor canónico de un record, por ejemplo, validar que los valores de `x` e `y` sean positivos en el momento de la creación de un `Point`:

```

package tema4_P00.recordsConstructorCanónico;

public record Point(int x, int y) {

    public Point {
        if (x < 0 || y < 0) {
            throw new IllegalArgumentException("Coordinates must be positive");
        }
    }
}

```

```

package tema4_P00.recordsConstructorCanónico;

public class showRecordsCanonicalConstructor {

    public void show() {

        Point p1,p2;
        p1 = new Point(3,4);
        System.out.println(p1);
        p2 = new Point(5,-2);//Lanza la excepción IllegalArgumentException
        System.out.println(p2);

    }

    public static void main(String[] args) {

        new showRecordsCanonicalConstructor().show();

    }

}

```

Además del constructor canónico, se pueden añadir otros constructores cuya primera sentencia debe invocar a otro constructor, de forma que en última instancia se invoque al constructor canónico. Veamos el siguiente ejemplo donde hay dos constructores: el constructor canónico y otro constructor que devuelve el origen:

```

package tema4_P00.recordsConstructores;

public record Point(int x, int y) {

    public Point {
        if (x < 0 || y < 0) {
            throw new IllegalArgumentException("Coordinates must be positive");
        }
    }

    public Point() {
        this(0, 0);
    }
}

```

```
}
```

```
package tema4_POO.recordsConstructores;

public class showRecordsConstructors {

    public void show() {

        Point p1,p2;
        p1 = new Point(3,4);
        System.out.println(p1);
        p2 = new Point();
        System.out.println(p2);

    }

    public static void main(String[] args) {

        new showRecordsConstructors().show();

    }

}
```

Salida por consola:

```
Point[x=3, y=4]
Point[x=0, y=0]
```

El *constructor canónico* asigna los parámetros `x` e `y` a los atributos `this.x` y `this.y`. Esta asignación se produce al final del constructor canónico, es decir, si añadimos código a este constructor, después de dicho código se realizará dicha asignación:

```
this.x = x;
this.y = y;
```

Esto se hace automáticamente, es decir, el programador no tiene que hacerlo, de hecho, el programador no puede leer ni modificar los atributos en el cuerpo del constructor canónico. Pero lo que sí puede modificar es los parámetros del constructor canónico antes de asignarlos a los atributos. Por ejemplo, verificamos si los parámetros `x` o `y` son negativos y en dicho caso se ajustan a `0`. Esta modificación ocurre antes de la asignación a los atributos. Después de modificar los valores de los parámetros, el compilador se encarga de asignar `x` e `y` a los atributos correspondientes, lo que garantiza que las modificaciones se reflejen en el objeto creado:

```
package tema4_POO.recordsParametrosConstructor;

public record Point(int x, int y) {

    public Point {

        if (x < 0) {
            x = 0;
        }
    }
}
```



```

    }
    if (y < 0) {
        y = 0;
    }

}

}

```

```

package tema4_POO.recordsParametrosConstructor;

public class showRecordsConstructorParameters {

    public void show() {

        Point p1 = new Point(3, 4);
        Point p2 = new Point(-5, 10);
        Point p3 = new Point(7, -8);

        System.out.println(p1);
        System.out.println(p2);
        System.out.println(p3);

    }

    public static void main(String[] args) {

        new showRecordsConstructorParameters().show();

    }

}

```

Salida por consola:

```

Point[x=3, y=4]
Point[x=0, y=10]
Point[x=7, y=0]

```

Veamos aspectos que **sí** se pueden hacer con los records:

- Añadir métodos adicionales igual que en una clase normal.
- Implementar interfaces.
- Se pueden declarar tanto en su propio archivo .java como dentro de otra clase o de otro record. La elección de cómo declararlo depende del contexto y del propósito para el que lo estás utilizando.
- Pueden contener atributos y métodos estáticos.
- Pueden definirse localmente dentro de un método.
- Pueden ser genéricos.

Veamos aspectos que **no** se pueden hacer con los records:

- No pueden heredar de otra clase ni de otro record.

- No pueden tener más atributos que las variables declaradas con el constructor canónico, es decir, en el caso de `record Point(int x, int y) {}`, el record Point no puede tener más atributos aparte de `x` e `y`.
- No existen los *inner records* (registros internos), es decir, un registro que se define dentro de otra clase o método es automáticamente estático, es decir, no tiene una referencia a la clase que lo contiene.
- El constructor canónico no puede lanzar excepciones checked:

```
record SleepyPoint(double x, double y) {  
    public SleepyPoint throws InterruptedException { // Error  
        Thread.sleep(1000);  
    }  
}
```

5.1 Excepciones

5.1 Excepciones

1. Introducción
2. Try-catch
3. Manejo de excepciones
4. Métodos de las excepciones
5. Cuándo se debe capturar una excepción

1. Introducción

Uno de los problemas más importantes al escribir aplicaciones es el tratamiento de los errores. Los errores detienen la ejecución del programa e impiden su desarrollo normal y, lo peor, además provocan que el usuario esté desinformado. Los programadores tienen que reconocer las situaciones que pueden provocar el fin de la ejecución normal del programa por un error no controlado. Dicho de otra forma, todos los posibles errores en un programa deben de estar controlados. A veces es imposible evitarlos (por ejemplo no hay papel en la impresora, o falla el disco duro), pero sí reaccionar de forma lógica para que el usuario reconozca lo que está ocurriendo.

Java nos echa una mano para ello a través de las excepciones. Se denomina **excepción** a un hecho que podría provocar la detención del programa; es decir una condición de error en tiempo de ejecución pero que puede ser controlable (a través de los mecanismos adecuados). En Java sin embargo se denomina error a una condición de error incontrolable (ejemplos son el error que ocurre cuando no se dispone de más memoria o errores de sintaxis).

Los errores de sintaxis son detectados durante la compilación, pero los errores de ejecución pueden provocar situaciones irreversibles, su control debe hacerse también en tiempo de ejecución y eso siempre ha sido problemático para la programación de aplicaciones.

En Java se puede preparar el código susceptible a provocar excepciones de modo que si ocurre una excepción, el código es lanzado (throw) a una determinada rutina previamente preparada por el programador, que permite manipular esa excepción. Si la excepción no fuera capturada, la ejecución del programa se detendría irremediablemente (en muchas ocasiones la propia sintaxis de Java impide que la excepción no sea controlada; es decir, obliga a controlarla).

2. Try-catch

El control de las excepciones se realiza mediante las sentencias **try** y **catch**.

```

try {
    instrucciones susceptibles de provocar una excepción
}
catch (ClaseExcepción1 objetoQueCapturaLaExcepción) {
    instrucciones que se ejecutan si hay un error de tipo ClaseExcepción1
}
catch (ClaseExcepción2 objetoQueCapturaLaExcepción) {
    instrucciones que se ejecutan si hay un error de tipo ClaseExcepción2
}
...

```

Como se puede observar, puede haber más de una sentencia catch para un mismo bloque try debido a que un bloque de código puede ser susceptible a provocar varios tipos diferentes de excepciones.

Dentro del bloque try se colocan las instrucciones susceptibles de provocar una excepción, el bloque catch sirve para capturar esa excepción y evitar el fin de la ejecución del programa. Desde el bloque catch se maneja, en definitiva, la excepción.

Cada catch maneja un tipo de excepción. Cuando se produce una excepción, se busca el catch que posea el manejador de excepción adecuado, será el que utilice el mismo tipo de excepción que se ha producido. La búsqueda del catch se realiza en el orden en que se han escrito. Si se produce una excepción, primero se mira si cuadra en el primer catch. Si no cuadra, se pasa al siguiente, y así sucesivamente. Por este motivo, es importante el orden en que se coloquen los bloques catch.

Cuando acaba la ejecución del bloque catch, el programa continúa con la ejecución del código que le sigue al bloque del try-catch.

Ejemplo:

```

package tema5_1_Excepciones;

import java.util.InputMismatchException;
import java.util.Scanner;

public class TryCatch {

    @SuppressWarnings("resource")
    public void show() {

        final String FIN = "fin";
        int base, exponent;
        String baseString;
        Scanner keyboard = new Scanner(System.in);

        try {
            System.out.println("Bienvenido al programa para calcular una
potencia.");
            System.out.print("Introduce la base o fin para terminar: ");
            baseString = keyboard.nextLine();
            if (!baseString.toLowerCase().equals(FIN)) {
                base = Integer.parseInt(baseString);
                System.out.print("Introduce el exponente: ");
                exponent = keyboard.nextInt();
            }
        }
    }
}

```

```

        System.out.printf("%d elevado a %d es igual a %d", base,
exponent, (int) Math.pow(base, exponent));
    }
    } catch (NumberFormatException e) {
        System.err.println("Error en la base");
    } catch (InputMismatchException e) {
        System.err.println("Error en el exponente");
    }
}

public static void main(String[] args) {

    new TryCatch().show();

}
}

```

Para la base, si el usuario no introduce *fin* y tampoco introduce un número entero, el `Integer.parseInt` al intentar convertir la cadena a número lanzará una excepción de tipo `NumberFormatException` y será manejada por el catch correspondiente. Para el exponente, si se introduce algo que no sea un número (una letra, un símbolo...), se producirá una excepción de tipo `InputMismatchException` y se manejará por su correspondiente catch. Si se produce un error de otro tipo, el programa se detendrá.

En este ejemplo se está utilizando *encadenamiento de llamadas a métodos*:

`if(!baseString.toLowerCase().equals(FIN))` ya que el método `toLowerCase` devuelve la cadena convertida en minúsculas a la cual se le aplica después el método `equals`. En este caso nos sirve para que el usuario pueda finalizar utilizando *fin* tanto en minúsculas como en mayúsculas.

Puede ser que el programador quiera mostrar el mismo mensaje de error para ambas excepciones. En este caso se utiliza el **multi-catch**, incorporado en Java desde la versión 7:

```

package tema5_1_Excepciones;

import java.util.InputMismatchException;
import java.util.Scanner;

public class MultiCatch {

    @SuppressWarnings("resource")
    public void show() {

        final String FIN = "fin";
        int base, exponent;
        String baseString;
        Scanner keyboard = new Scanner(System.in);

        try {
            System.out.println("Bienvenido al programa para calcular una
potencia.");
            System.out.print("Introduce la base o fin para terminar: ");

```

```

        baseString = keyboard.nextLine();
        if (!baseString.toLowerCase().equals(FIN)) {
            base = Integer.parseInt(baseString);
            System.out.print("Introduce el exponente: ");
            exponent = keyboard.nextInt();
            System.out.printf("%d elevado a %d es igual a %d", base,
exponent, (int) Math.pow(base, exponent));
        }
    } catch (NumberFormatException | InputMismatchException e) {
        System.err.println("Error en la base o en el exponente");
    }

}

public static void main(String[] args) {

    new MultiCatch().show();

}

}

```

En el eclipse, si seleccionamos varias líneas de código, las podemos fácilmente insertar en un bloque try-catch: Menú *Source* → *Surround with*.

3. Manejo de excepciones

En la programación de aplicaciones en general siempre ha habido dos formas de manejar una excepción:

- Interrupción: en este caso se asume que el programa ha encontrado un error irrecuperable. La operación que dio lugar a la excepción se anula y se entiende que no hay manera de regresar al código que provocó la excepción.
- Reanudación: se puede manejar el error y regresar de nuevo al código que provocó el error.

La filosofía de Java es del tipo interrupción, pero se puede intentar emular la reanudación encerrando el bloque try-catch en un bucle que se repetirá hasta que el error deje de existir.

Ejemplo:

```

package tema5_1_Excepciones;

import java.util.InputMismatchException;
import java.util.Scanner;

public class Resumption {

    @SuppressWarnings("resource")
    public void show() {

        byte number = 0;
        Scanner keyboard = new Scanner(System.in);
        boolean error = false;

        do {
            try {

```

```

        System.out.print("Introduce un número de tipo byte, es decir,
entre -128 y 127: ");
        number = keyboard.nextByte();
        System.out.printf("Valor del número: %d\n", number);
        error = false; //Si se ha entrado antes en el catch, error está a
true
    } catch (InputMismatchException e) {
        System.err.println("Error");
        error = true;
        keyboard.nextLine(); //Limpieza del buffer
    }
} while (error);

}

public static void main(String[] args) {

    new Resumption().show();

}

}

```

Si el usuario introduce cualquier cosa que no sea un número, el método `nextByte()` lanza la excepción `InputMismatchException` y lo que ha metido el usuario se queda en el buffer. Para solucionarlo, se realiza una limpieza del buffer con `keyboard.nextLine()`.

4. Métodos de las excepciones

- `getMessage`: obtiene el mensaje descriptivo de la excepción.
- `toString`: devuelve una cadena sobre la situación de la excepción. Suele indicar la clase de excepción y el texto de `getMessage`.
- `printStackTrace`: escribe el método y mensaje de la excepción (la llamada información de pila). El resultado es el mismo mensaje que muestra el ejecutor (la máquina virtual de Java) cuando no se controla la excepción.

```

package tema5_1_Excepciones;

import java.util.Scanner;
import java.util.InputMismatchException;
import static tema1_11_EscrituraEnPantalla.colores.Colors.*;

public class MethodsOfExceptions {

    @SuppressWarnings("resource")
    public void show() {

        final String FIN = "fin";
        int base, exponent;
        String baseString;
        Scanner keyboard = new Scanner(System.in);

        try {

```

```

        System.out.println("Bienvenido al programa para calcular una
potencia.");
        System.out.print("Introduce la base o fin para terminar: ");
        baseString = keyboard.nextLine();
        if (!baseString.toLowerCase().equals(FIN)) {
            base = Integer.parseInt(baseString);
            System.out.print("Introduce el exponente: ");
            exponent = keyboard.nextInt();
            System.out.printf("%d elevado a %d es igual a %d", base,
exponent, (int) Math.pow(base, exponent));
        }
    } catch (NumberFormatException e) {
        System.err.println(GREEN + e.getMessage() + RESET);
        System.err.println(CYAN + e.toString() + RESET);
        e.printStackTrace();
    } catch (InputMismatchException e) {
        System.err.println(GREEN + e.getMessage() + RESET);
        System.err.println(CYAN + e.toString() + RESET);
        e.printStackTrace();
    }
}

public static void main(String[] args) {

    new MethodsOfExceptions().show();
}
}

```

5. Cuándo se debe capturar una excepción

Una excepción se debe capturar cuando no sea un error de programación. Es decir, el programador debe distinguir si esa excepción es un error suyo de programación o no, porque si es un error de programación, no debe capturar el error sino solucionarlo. Veamos un ejemplo con la excepción *StringIndexOutOfBoundsException*.

Cuando se intenta acceder a una parte de la cadena que no existe, por ejemplo, si a la cadena "hola" le hacemos un *charAt(4)*, java nos lanza la excepción *StringIndexOutOfBoundsException*. El programador no debe capturar con un try-catch la excepción sino que debe corregir el error de programación que ha producido dicha excepción. Veamos un ejemplo:

```

package tema5_1_Excepciones;

public class StringIndexOutOfBoundsException1 {

    public void show() {

        String string = "hola";
        for (int i = 0; i <= string.length(); i++) {
            System.out.println(string.charAt(i));
        }
    }
}

```



```

public static void main(String[] args) {

    new StringIndexOutOfBoundsException1().show();

}

}

```

La salida por consola es la siguiente:

```

h
o
l
a
Exception in thread "main" java.lang.StringIndexOutOfBoundsException: String
index out of range: 4
    at java.base/java.lang.StringLatin1.charAt(StringLatin1.java:47)
    at java.base/java.lang.String.charAt(String.java:702)
    at
tema5_1_Excepciones.StringIndexOutOfBoundsException1.show(StringIndexOutOfBoundsE
xception1.java:9)
    at
tema5_1_Excepciones.StringIndexOutOfBoundsException1.main(StringIndexOutOfBoundsE
xception1.java:16)

```

El programador no debe solucionarlo con un try-catch:

```

package tema5_1_Excepciones;

public class StringIndexOutOfBoundsException2 {

    public void show() {

        String string = "hola";
        try {
            for (int i = 0; i <= string.length(); i++) {
                System.out.println(string.charAt(i));
            }
        } catch (StringIndexOutOfBoundsException e) { //Esto no se debe hacer
            System.err.println("Esto no se debe hacer");
        }

    }

    public static void main(String[] args) {

        new StringIndexOutOfBoundsException2().show();

    }

}

```

Sino que debe corregir el error, es decir, quitar el = de la condición del for, ya que en la última iteración, *i* toma el valor de *string.length*, que es 4, cuando el *charAt(4)* no es un carácter válido ya que los caracteres válidos van de 0 a 3. Por lo tanto, el programador debe corregir el error:

```
package tema5_1_Excepciones;

public class StringIndexOutOfBoundsException3 {

    public void show() {

        String string = "hola";
        for (int i = 0; i < string.length(); i++) {
            System.out.println(string.charAt(i));
        }

    }

    public static void main(String[] args) {

        new StringIndexOutOfBoundsException3().show();

    }

}
```

5.2 Throws, throw y finally

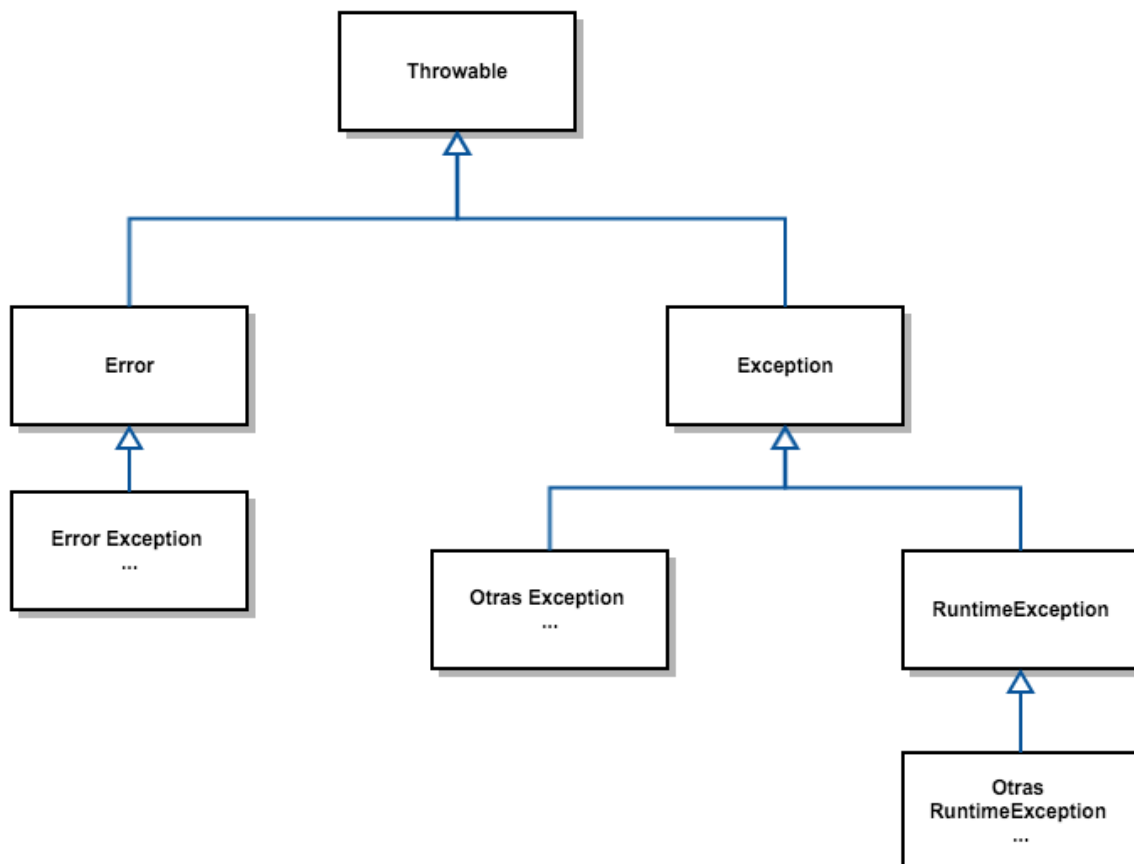
5.2 Throws, throw y finally

1. Tipos de excepciones
2. Throws
3. Orden de los catch
4. Throw
5. Finally

1. Tipos de excepciones

En Java, la **herencia** define una relación entre clases en la cual una clase posee características (métodos y propiedades) que proceden de otra. A la clase que posee las características a heredar se la llama **superclase** o **clase padre** y la clase que las hereda se llama **subclase** o **clase hija**. Lo beneficioso de este tipo de relaciones es que los hijos pueden heredar las características de los padres y luego aportar características propias convirtiéndose en una manera de reutilizar código muy eficiente.

Java utiliza muchísimo la herencia en las clases que conforman su API. En cuanto a las excepciones se refiere, existen tres tipos de excepciones que Java maneja a través de la herencia: errores, comprobadas (*checked*) y no comprobadas (*unchecked*). El gráfico que se muestra a continuación muestra el árbol de herencia de las excepciones en Java. El paquete de todas es java.lang:



La clase principal de la cual heredan todas las excepciones Java es `Throwable`. De ella nacen dos ramas: `Error` y `Exception`. `Error` representa errores de una magnitud tal que una aplicación nunca debería intentar realizar nada con ellos (como errores de la JVM, desbordamientos de buffer, etc). La segunda rama, encabezada por `Exception`, representa aquellos errores que normalmente sí solemos gestionar, y a los que comúnmente solemos llamar *excepciones*.

De `Exception` nacen múltiples ramas: `ClassNotFoundException`, `IOException`, `ParseException`, `SQLException` y otras muchas, todas ellas de tipo *checked*. La única excepción que es de tipo *unchecked* es `RuntimeException` y encabeza todas las de este tipo.

Excepciones checked

Una excepción de tipo *checked* representa un error del cual técnicamente podemos recuperarnos. En ciertos momentos, a pesar de la promesa de recuperabilidad, nuestro código no estará preparado para gestionar la situación de error, o simplemente no será su responsabilidad. En estos casos lo más razonable es relanzar la excepción y confiar en que un método superior en la cadena de llamadas sepa gestionarla.

Por tanto, todas las excepciones de tipo *checked* deben ser capturadas o relanzadas. En el primer caso, utilizamos el bloque `try-catch`. En caso de querer relanzar la excepción, debemos declarar dicha intención en la firma de la función que contiene las sentencias que lanzan la excepción, y lo hacemos mediante la cláusula `throws`.

Hay que tener presente que cuando se relanza una excepción estamos forzando al código cliente de nuestro método a capturarla o relanzarla. Una excepción que sea relanzada una y otra vez hacia arriba terminará llegando al método primigenio y, en caso de no ser capturada por éste, producirá la finalización de su hilo de ejecución (thread).

La dos preguntas que debemos hacernos en este momento es: ¿Cuándo capturar una excepción? ¿Cuándo relanzarla? La respuesta es muy simple. Capturamos una excepción cuando:

- Podemos recuperarnos del error y continuar con la ejecución.
- Queremos registrar el error.
- Queremos relanzar el error con un tipo de excepción distinto.

En definitiva, cuando tenemos que realizar algún tratamiento del propio error. Por contra, relanzamos una excepción cuando:

- No es competencia nuestra ningún tratamiento de ningún tipo sobre el error que se ha producido.
- Para centralizar el control de excepciones y facilitar el mantenimiento del código.

Excepciones unchecked

Son excepciones de tipo `RuntimeException` o de cualquiera de sus subclases. El aspecto más destacado de las excepciones de tipo *unchecked* es que no deben ser forzosamente declaradas ni capturadas (en otras palabras, no son comprobadas). Por ello no son necesarios bloques `try-catch` ni declarar formalmente en la firma de la función el lanzamiento de excepciones de este tipo. Ésto, por supuesto, también afecta a funciones y/o clases más hacia arriba en la cadena invocante.

2. Throws

En caso de querer relanzar la excepción, debemos declarar dicha intención en la firma de la función que contiene las sentencias que lanzan la excepción, y lo hacemos mediante la cláusula throws.

¿Tiene obligación de tener un try-catch el que llame a la función? Si la excepción es de tipo RuntimeException o cualquiera de sus hijas, no existe la obligación de poner un try-catch. En cualquier otro caso, sí existe dicha obligación, de hecho si no se hace, Java da un error de compilación.

Ejemplo:

```
package tema5_2_ThrowsThrowFinally;

import java.util.Scanner;

public class Throws {

    @SuppressWarnings("resource")
    public void show() {

        final String FIN = "fin", SIGUIENTE = "siguiente";
        int number;
        String string;
        Scanner keyboard = new Scanner(System.in);

        System.out.print("Introduce un número o siguiente para pasar al siguiente número: ");
        string = keyboard.nextLine();
        if (!string.toLowerCase().equals(SIGUIENTE)) {
            /*
             * Aquí no estamos obligados a poner un try-catch porque
             * NumberFormatException es hija de RuntimeException:
             */
            number = convertirNumero(string);
            System.out.printf("Valor del número introducido: %d\n", number);
        }

        try {
            System.out.print("Introduce un número o fin para terminar: ");
            string = keyboard.nextLine();
            if (!string.toLowerCase().equals(FIN)) {
                /*
                 * Aquí sí estamos obligados a poner un try-catch
                 * porque Exception no es hija de RuntimeException:
                 */
                number = convertirNumero2(string);
                System.out.printf("Valor del número introducido: %d\n", number);
            }
        } catch (Exception e) {
            System.err.println("Error en el número");
        }
    }
}
```

```

    public int convertirNumero(String string) throws NumberFormatException {

        return Integer.parseInt(string);

    }

    public int convertirNumero2(String string) throws Exception {

        return Integer.parseInt(string);

    }

    public static void main(String[] args) {

        new Throws().show();

    }

}

```

3. Orden de los catch

Cuando se tienen varios catch para el mismo try, hay que tener la precaución de poner los catch de las subclases antes que los de las superclases porque si no, los catch de las subclases nunca se capturarían. Ejemplo:

```

package tema5_2_ThrowsThrowFinally;

import java.util.InputMismatchException;
import java.util.Scanner;

public class CatchOrder1 {

    @SuppressWarnings("resource")
    public void show() {

        final String FIN = "fin";
        String string;
        byte number = 0;
        Scanner keyboard = new Scanner(System.in);

        try {
            System.out.print("Introduce un número de tipo byte, es decir, entre
-128 y 127: ");
            number = keyboard.nextByte();
            keyboard.nextLine(); //Limpieza del buffer
            System.out.printf("Valor del número introducido: %d\n", number);
            System.out.print("Introduzca otro número de tipo byte o fin para
terminar: ");
            string = keyboard.nextLine();
            if (!string.toLowerCase().equals(FIN)) {
                number = Byte.parseByte(string);
                System.out.printf("Valor del otro número introducido: %d\n",
number);
            }
        }
    }
}

```

```

    }
    } catch (Exception e) {
        System.err.println("Error: no ha introducido un número entre -128 y
127");
    } catch (InputMismatchException e) { //Error de compilación: inalcanzable
bloque catch
        System.err.println("Error");
    }
}

}

public static void main(String[] args) {

    new CatchOrder1().show();

}

}

```

En este caso, Java da un error de compilación ya que `InputMismatchException` es hija de `Exception`. Si el usuario no introduce un número de tipo `byte`, se lanza una excepción de tipo `InputMismatchException`. Dicho objeto también es un objeto de tipo `Exception`, ya que los objetos de tipo hijo, también son objetos de tipo padre. Por lo tanto, nunca entraría en el `catch` del `InputMismatchException`. La solución es cambiar el orden de los `catch`:

```

package tema5_2_ThrowsThrowFinally;

import java.util.InputMismatchException;
import java.util.Scanner;

public class CatchOrder2 {

    @SuppressWarnings("resource")
    public void show() {

        final String FIN = "fin";
        String string;
        byte number = 0;
        Scanner keyboard = new Scanner(System.in);

        try {
            System.out.print("Introduce un número de tipo byte, es decir, entre
-128 y 127: ");
            number = keyboard.nextByte();
            keyboard.nextLine(); //Limpieza del buffer
            System.out.printf("Valor del número introducido: %d\n", number);
            System.out.print("Introduzca otro número de tipo byte o fin para
terminar: ");
            string = keyboard.nextLine();
            if (!string.toLowerCase().equals(FIN)) {
                number = Byte.parseByte(string);
                System.out.printf("Valor del otro número introducido: %d\n",
number);
            }
        } catch (InputMismatchException e) {

```

```

        System.err.println("Error: no ha introducido un número entre -128 y
127");
    } catch (Exception e) {
        System.err.println("Error");
    }

}

public static void main(String[] args) {

    new CatchOrder2().show();

}

}

```

4. Throw

Esta instrucción le permite al programador lanzar una excepción. El flujo del programa se dirigirá a la instrucción try-catch más cercana. Ejemplos:

- `throw new Exception();`
- `throw new Exception("Error grave");` Construye una excepción con el mensaje indicado. Ese mensaje se puede obtener en el catch con el método `getMessage` del objeto que contiene la excepción. Ejemplo:

```

package tema5_2_ThrowsThrowFinally;

import java.util.Scanner;

public class Throw {

    @SuppressWarnings("resource")
    public void show() {

        double n;
        Scanner keyboard = new Scanner(System.in);

        try {
            System.out.println("Introduce un número: ");
            n = keyboard.nextDouble();
            division(n);
        } catch (ArithmeticException e) {
            System.err.println(e.getMessage());
        }

    }

    public void division(double n) {
        /*
        * float y double admiten Infinity por lo que no se lanza
        * una excepción cuando se divide entre cero.
        * Podemos lanzar la excepción usando throw
        */
    }
}

```



```

        if (n == 0) {
            throw new ArithmeticException("Error: no se puede dividir entre
cero");
        }
        System.out.printf("500 entre %.2f: %.2f\n", n, 500 / n);

    }

    public static void main(String[] args) {

        new Throw().show();

    }

}

```

5. Finally

Las instrucciones que se encuentran en el **finally** se ejecutan independientemente de si hubo o no excepción. Es decir, esas sentencias se ejecutan siempre. Si el código del try ha ido bien y no se ha lanzado ninguna excepción, después se ejecutan las instrucciones del *finally* y luego se continúa por el código que le sigue al bloque del try-catch. Pero si se ha lanzado alguna excepción, se ejecutan las instrucciones del catch correspondiente, luego se ejecutan las instrucciones del *finally* y luego se continúa por el código que le sigue al bloque del try-catch. Por lo tanto, se coloca en el bloque *finally* código común para todas las excepciones y también para cuando no hay excepciones. Normalmente se suelen poner instrucciones de limpieza. Ejemplo:

```

package tema5_2_ThrowsThrowFinally;

import java.util.InputMismatchException;
import java.util.Scanner;

public class Finally {

    @SuppressWarnings("resource")
    public void show() {

        int number = 0;
        String string;
        boolean error = false;
        Scanner keyboard = new Scanner(System.in);

        try {
            System.out.print("Introduce un número: ");
            number = keyboard.nextInt();
        } catch (InputMismatchException e) {
            System.err.println("Error");
            error = true;
        } finally {
            keyboard.nextLine();//Limpieza del buffer
        }

        System.out.print("Introduce una cadena: ");
        string = keyboard.nextLine();
    }
}

```

```
        System.out.printf("El número introducido ha sido: %s\n", error ? "error"
: number);
        System.out.printf("La cadena introducida ha sido: %s", string);

    }

    public static void main(String[] args) {

        new Finally().show();

    }

}
```

En el ejemplo, hay que limpiar el buffer haya o no una excepción, por lo tanto, se limpia en el *finally*.

Pero ¿qué diferencia hay entre poner código en el *finally* o ponerlo después del bloque try-catch? Pues que en el *finally* siempre se va a ejecutar aunque se rompa el flujo de ejecución, es decir, aunque exista por ejemplo un `return` en el *try* o un `throw` en el *catch*. El código que se encuentre después del bloque try-catch, no se ejecutaría en esos casos.

6.1 Arrays unidimensionales

6.1 Arrays unidimensionales

1. Introducción
2. Declaración
3. Creación
4. Referencia a los elementos del array
5. Asignación de valores
6. El atributo length
7. Utilización del bucle for
8. Método split
9. Los arrays se implementan como objetos
10. Qué puede contener un array
 - 10.1 Arrays de tipos primitivos
 - 10.2 Arrays de objetos
11. Asignación de arrays
12. Comparación de arrays
13. Arrays como parámetros
14. Cómo retornar un array
15. Excepción `ArrayIndexOutOfBoundsException`

1. Introducción

Los tipos de datos que conocemos hasta ahora no permiten solucionar problemas que requieren gestionar muchos datos a la vez. Por ejemplo, imaginemos que deseamos almacenar las notas de una clase de 25 alumnos, para ello, con lo que conocemos hasta ahora, no habrá más remedio que declarar 25 variables. Eso es tremendamente pesado de programar. Manejar esos datos significaría estar continuamente manejando 25 variables. Por ello, en casi todos los lenguajes se pueden agrupar una serie de variables del mismo tipo en una misma estructura que comúnmente se conoce como array. Esa estructura permite referirnos a todos los elementos pero también nos permite acceder individualmente a cada uno de ellos.

Los **arrays** son una colección de datos del mismo tipo al que se le pone un nombre. Para acceder a un dato individual de la colección hay que utilizar su posición. La posición es un número entero, normalmente se le llama **índice**. Hay que tener en cuenta que en los arrays el primer elemento tiene como índice el número cero, el segundo el uno y así sucesivamente.

Un array unidimensional es aquel que tiene una única dimensión, es decir, un único índice. También se le suele llamar *vector*.

2. Declaración

Existen dos formas de declarar un array unidimensional:

- `tipo nombre_array[];`
- `tipo[] nombre_array;`

tipo declara el tipo de elemento del array, es decir, el tipo de datos de cada elemento que comprende el array. Dicho tipo de datos puede ser un tipo primitivo o un objeto.

Esta declaración le dice al compilador que dicha variable va a contener un array con elementos de dicho tipo pero todavía no se reserva espacio en la memoria RAM ya que no se conoce el tamaño del mismo.

3. Creación

Se realiza con el operador **new**, que es el que realmente crea el array indicando un tamaño. Cuando se usa new es cuando se reserva el espacio necesario en memoria para el array.

Se crea un array de la siguiente manera: `nombre_array = new tipo[tamaño];`

Ejemplo:

```
int[] grades;//declaración del array de enteros llamado grades
grades = new int[3];//creación del array grades reservando en memoria espacio
para 3 enteros
```

Se puede realizar lo mismo en una única línea de código:

```
int[] grades = new int[3];
```

4. Referencia a los elementos del array

Para referenciar los elementos del array se utiliza el índice de los mismos entre corchetes:

`grades[0]` : es el primer elemento del array, es decir, la primera nota.

`grades[1]` : es el segundo elemento del array, es decir, la segunda nota.

`grades[2]` : es el tercer elemento del array, es decir, la tercera nota.

5. Asignación de valores

Se pueden asignar valores a los elementos del array utilizando el signo `=`:

```
grades[2] = 8;//Se asigna un 8 a la tercera nota
```

También se pueden asignar valores a todos los elementos del array utilizando **literales array**:

```
int[] grades = new int[] {8, 7, 9};://{8, 7, 9} es un literal array
```

No es necesario escribir **new int[]** en las últimas versiones de Java:

```
int[] grades = {8, 7, 9};
```

En este caso, se está asignando un 8 a la primera nota, un 7 a la segunda y un 9 a la tercera.

6. El atributo length

Los arrays poseen el atributo **length** que contiene el tamaño del array:

```
System.out.println(grades.length); //Muestra 3
```

7. Utilización del bucle for

La ventaja de usar arrays es que gracias a un simple bucle for se pueden recorrer fácilmente todos los elementos de un array:

```
package tema6_1_ArraysUnidimensionales;

public class ArraysFor {

    public void show() {

        int[] grades = { 8, 7, 9 };

        for (int i = 0; i < grades.length; i++) {
            System.out.printf("Nota en el índice %d: %d\n", i, grades[i]);
        }

    }

    public static void main(String[] args) {

        new ArraysFor().show();

    }

}
```

Los índices del array van desde 0 hasta `length-1`. En el ejemplo, el array es de tamaño 3 y sus índices son 0, 1 y 2. Por eso en la condición del for se itera mientras `i` sea menor que `grades.length`, es decir, el último índice que se procesa es el 2. Si pusiéramos la condición del for como menor o igual, nos lanzaría la excepción ***ArrayIndexOutOfBoundsException*** ya que estaríamos accediendo al array con el índice 3, el cual es un índice no válido:

```
for(int i=0;i<=grades.length;i++) { //Se lanza la excepción
ArrayIndexOutOfBoundsException
```

También se pueden utilizar con los arrays los bucles for-each:

```
package tema6_1_ArraysUnidimensionales;

public class ArraysForEach {

    public void show() {

        int[] grades = { 8, 7, 9 };

        for (int grade : grades) {
            System.out.printf("Nota: %d\n", grade);
        }

    }

}
```

```

    public static void main(String[] args) {

        new ArraysForEach().show();

    }

}

```

8. Método split

Las clases *String* y *Pattern* poseen el método **split** que divide una cadena en subcadenas en función de una expresión regular y devuelve un array con las subcadenas resultantes.

```

package tema6_1_ArraysUnidimensionales;

import java.util.Arrays;

public class Split {

    public void show() {

        String string = "Esto:es:una:cadena:dividida:por:split";
        String[] stringArray = string.split(":");
        System.out.println(Arrays.toString(stringArray)); //[Esto, es, una,
cadena, dividida, por, split]

    }

    public static void main(String[] args) {

        new Split().show();

    }

}

```

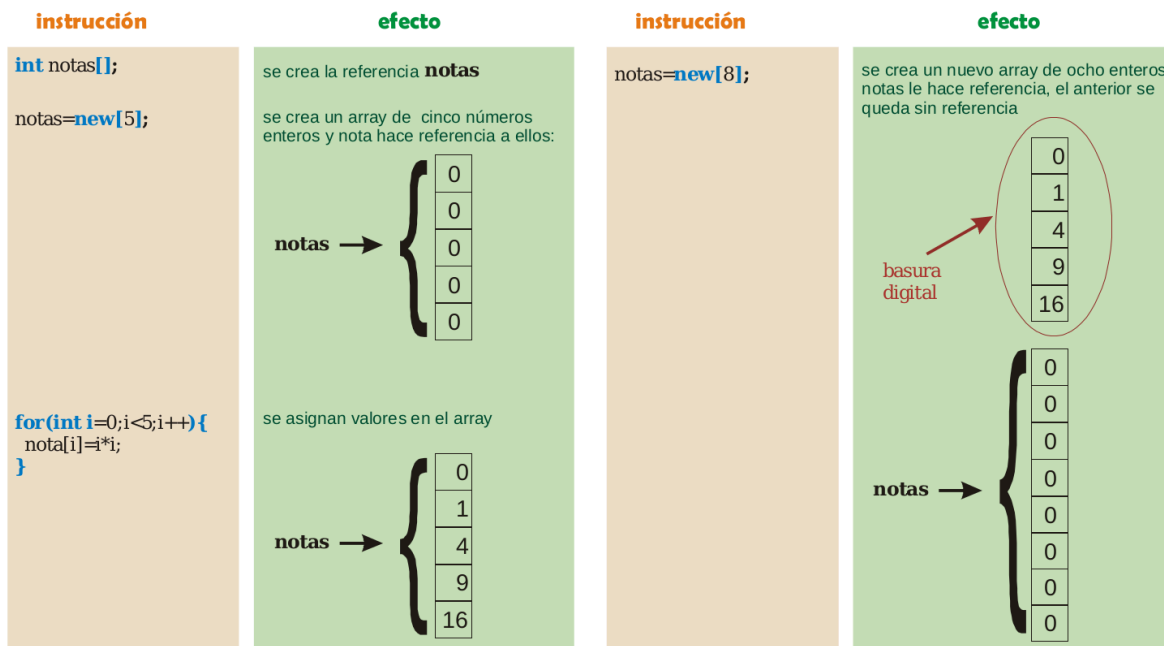
Fíjense en el método estático *toString* de la clase *Arrays*. Devuelve una cadena con los elementos del array entre corchetes y separados por comas.

9. Los arrays se implementan como objetos

En Java, los arrays se implementan como objetos. Una de las ventajas que tiene esto es que los arrays que pierden la referencia pueden ser recolectados como basura. (Ver [Tema 4 Programación Orientada a Objetos 13 Referencias](#)).

En la perspectiva de Java, un array es una referencia a una serie de valores que se almacenan en la memoria. El operador *new* en realidad lo que hace es devolver dicha referencia para poder leer y escribir en esos valores.

Veamos el efecto del uso del operador *new* en los arrays y cómo afectan en la memoria:



Vemos como el anterior array se ha quedado sin referencia y se marca como elegible para el recolector de basura.

10. Qué puede contener un array

El array puede contener objetos o tipos de datos primitivos. En el caso de los tipos de datos primitivos, los valores reales se almacenan en ubicaciones de memoria contigua. En el caso de los objetos de una clase, los objetos reales se almacenan en *heap*.

10.1 Arrays de tipos primitivos

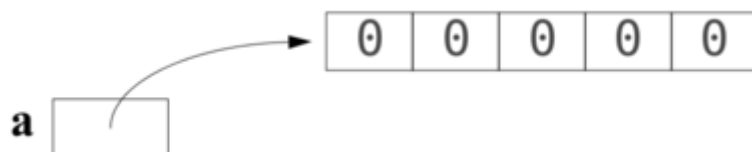
- Crear una variable de tipo array sólo crea una referencia:

```
int[] a;
```

a X

- El espacio en memoria para el array se crea con *new*. Cuando se trata de un array de un tipo primitivo, se crean los elementos del array y se inicializan a sus valores por defecto:

```
a = new int[5];
```



10.2 Arrays de objetos

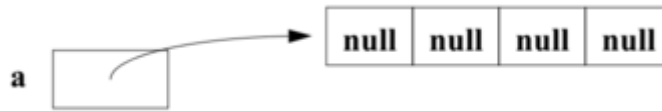
- Crear una variable de tipo array de objetos sólo crea la referencia:

```
Complejo[] a;
```

a X

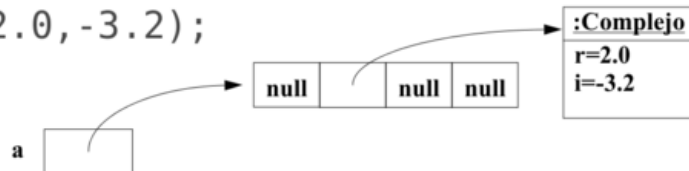
- Con new se reserva el espacio para el array de referencias pero no los objetos a los que apuntarán las referencias. Las referencias se inicializan a su valor por defecto (*null*):

```
a = new Complejo[4];
```



- Los objetos hay que crearlos posteriormente:

```
a[1]=new Complejo(2.0, -3.2);
```

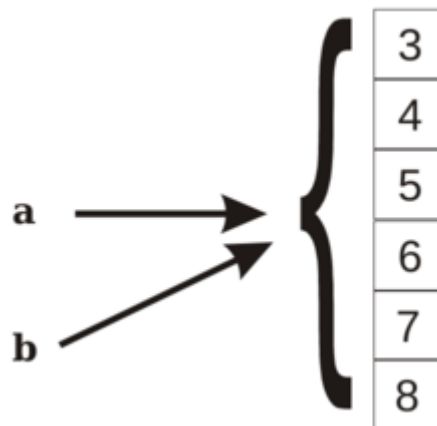


11. Asignación de arrays

Como en Java los arrays son objetos, la asignación se realiza como los objetos (Ver [Tema 4 Programación Orientada a Objetos 13 Referencias](#)).

Un array se puede asignar a otro array (si son del mismo tipo). La asignación solo copia referencias, es decir, la dirección de memoria. Por lo tanto, ambos arrays apuntarán al mismo sitio:

```
int[] a;  
int[] b={3,4,5,6,7,8};  
a=b;
```



Esta asignación provoca que cualquier cambio en *a* también cambie el array *b* ya que de hecho, son el mismo array:

```
package tema6_1_ArraysUnidimensionales;  
  
import java.util.Arrays;  
  
public class Assignment {
```



```

public void show() {

    int[] a;
    int[] b = { 3, 4, 5, 6, 7, 8 };
    a = b;
    System.out.println(Arrays.toString(b)); //[3, 4, 5, 6, 7, 8]
    a[0] = 1;
    System.out.println(Arrays.toString(b)); //[1, 4, 5, 6, 7, 8]

}

public static void main(String[] args) {

    new Assignment().show();

}

}

```

12. Comparación de arrays

Como en Java los arrays son objetos, la comparación se realiza como los objetos (Ver [Tema 4 Programación Orientada a Objetos 13 Referencias](#)).

El operador de igualdad == cuando se utiliza con arrays, no compara el contenido de los arrays sino sus direcciones de memoria o referencias, es decir, si apuntan al mismo array. Lo mismo ocurre con el método *equals* de los arrays, que compara las direcciones de memoria. Si queremos comparar el contenido de los arrays, tendremos que utilizar el método estático *equals* de la clase *Arrays*:

```

package tema6_1_ArraysUnidimensionales;

import java.util.Arrays;

public class Comparison {

    public void show() {

        int[] array1;
        int[] array2 = { 3, 4, 5, 6, 7, 8 };
        int[] array3 = { 3, 4, 5, 6, 7, 8 };
        array1 = array2;
        System.out.println(array1 == array2); //true porque apuntan al mismo array
        System.out.println(array2 == array3); //false porque no apuntan al mismo array
        System.out.println(array1.equals(array2)); //true porque apuntan al mismo array
        System.out.println(array2.equals(array3)); //false porque no apuntan al mismo array
        System.out.println(Arrays.equals(array1, array2)); //true porque el contenido es el mismo ya que apuntan al mismo array
        System.out.println(Arrays.equals(array2, array3)); //true porque el contenido es el mismo

    }

}

```

```

    public static void main(String[] args) {

        new Comparison().show();

    }

}

```

13. Arrays como parámetros

Al igual que las variables, también podemos pasar arrays a los métodos, es decir, se pueden definir parámetros de tipo array. En la llamada al método, lo que el método recibe como argumento es la dirección de memoria del array, por lo que si el método modifica el array, el array del método que efectúa la llamada también se ve afectado por dichos cambios.

```

package tema6_1_ArraysUnidimensionales;

import java.util.Arrays;

public class ArraysAsParameters {

    public void show() {

        int[] array = { 3, 4, 5, 6, 7, 8 };
        System.out.println(Arrays.toString(array)); //[3, 4, 5, 6, 7, 8]
        method(array);
        System.out.println(Arrays.toString(array)); //[6, 8, 10, 12, 14, 16]

    }

    public void method(int[] array) {

        for (int i = 0; i < array.length; i++) {
            array[i] *= 2;
        }

    }

    public static void main(String[] args) {

        new ArraysAsParameters().show();

    }

}

```

14. Cómo retornar un array

Un método también puede devolver un array, en cuyo caso, lo que retorna es la dirección de memoria de dicho array.

```

package tema6_1_ArraysUnidimensionales;

```

```

import java.util.Arrays;

public class ReturningAnArray {

    public void show() {

        int[] a;
        a = method();
        System.out.println(Arrays.toString(a)); //[3, 4, 5, 6, 7, 8]

    }

    public int[] method() {

        int[] array = { 3, 4, 5, 6, 7, 8 };

        return array;

    }

    public static void main(String[] args) {

        new ReturningAnArray().show();

    }

}

```

En este caso, el método *show* define una variable *a* de tipo array de enteros pero no hace el *new* del array, sino que recibe la dirección de memoria que le devuelve el método *method*. Sin embargo, el método *method* sí que hace el *new* del array porque su responsabilidad es crear el array, darle valores y devolverlo.

15. Excepción `ArrayIndexOutOfBoundsException`

Cuando se intenta acceder a un elemento del array que no existe, java nos lanza la excepción `ArrayIndexOutOfBoundsException`. El programador no debe capturar con un try-catch la excepción sino que debe corregir el error de programación que ha producido dicha excepción. Veamos un ejemplo:

```

package tema6_1_ArraysUnidimensionales;

public class ArrayIndexOutOfBoundsException1 {

    public void show() {

        int[] grades = { 8, 7, 9 };

        for (int i = 0; i <= grades.length; i++) {
            System.out.printf("Nota en el índice %d: %d\n", i, grades[i]);
        }

    }

}

```

```

public static void main(String[] args) {

    new ArrayIndexOutOfBoundsException1().show();

}

}

```

La salida por consola es la siguiente:

```

Nota en el índice 0: 8
Nota en el índice 1: 7
Nota en el índice 2: 9
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out
of bounds for length 3
    at
tema6_1_ArraysUnidimensionales.ArrayIndexOutOfBoundsException1.show(ArrayIndexOut
OfBoundsException1.java:10)
    at
tema6_1_ArraysUnidimensionales.ArrayIndexOutOfBoundsException1.main(ArrayIndexOut
OfBoundsException1.java:17)

```

El programador no debe hacer esto:

```

package tema6_1_ArraysUnidimensionales;

public class ArrayIndexOutOfBoundsException2 {

    public void show() {

        int[] grades = { 8, 7, 9 };

        try {
            for (int i = 0; i <= grades.length; i++) {
                System.out.printf("Nota en el índice %d: %d\n", i, grades[i]);
            }
        } catch (ArrayIndexOutOfBoundsException e) { //Esto no se debe hacer
            System.err.println("Esto no se debe hacer");
        }

    }

    public static void main(String[] args) {

        new ArrayIndexOutOfBoundsException2().show();

    }

}

```

Sino que debe corregir el error, es decir, quitar el = de la condición del for, ya que en la última iteración, *i* toma el valor de *grades.length* que es 3, cuando *grades[3]* no es un elemento válido ya que los elementos válidos van de 0 a 2. Por lo tanto, el programador debe corregir el error:

```
package tema6_1_ArraysUnidimensionales;

public class ArrayIndexOutOfBoundsException3 {

    public void show() {

        int[] grades = { 8, 7, 9 };

        for (int i = 0; i < grades.length; i++) {
            System.out.printf("Nota en el índice %d: %d\n", i, grades[i]);
        }

    }

    public static void main(String[] args) {

        new ArrayIndexOutOfBoundsException3().show();

    }

}
```

6.2 Arrays multidimensionales

6.2 Arrays multidimensionales

1. Introducción
2. Declaración
3. Creación
4. Referencia a los elementos del array
5. Asignación de valores
6. El atributo length
7. Utilización del bucle for
8. Comparación de arrays
9. Arrays irregulares

1. Introducción

Los arrays multidimensionales son aquellos que tienen más de una dimensión, es decir, son arrays que contienen otros arrays. Tienen más de un índice, uno por cada dimensión.

Un array con dos dimensiones es un array bidimensional y también se le suele llamar *matriz*. Si además, la matriz tiene el mismo número de filas que de columnas, entonces se llama *matriz cuadrada*.

2. Declaración

Se declaran como los arrays unidimensionales añadiendo tantos corchetes `[]` como dimensiones haya. Por ejemplo, veamos las dos formas de declarar un array bidimensional:

- `tipo nombre_array[][];`
- `tipo[][] nombre_array;`

tipo declara el tipo de elemento del array, es decir, el tipo de datos de cada elemento que comprende el array. Dicho tipo de datos puede ser un tipo primitivo o un objeto.

Esta declaración le dice al compilador que dicha variable va a contener un array con elementos de dicho tipo pero todavía no se reserva espacio en la memoria RAM ya que no se conoce el tamaño del mismo.

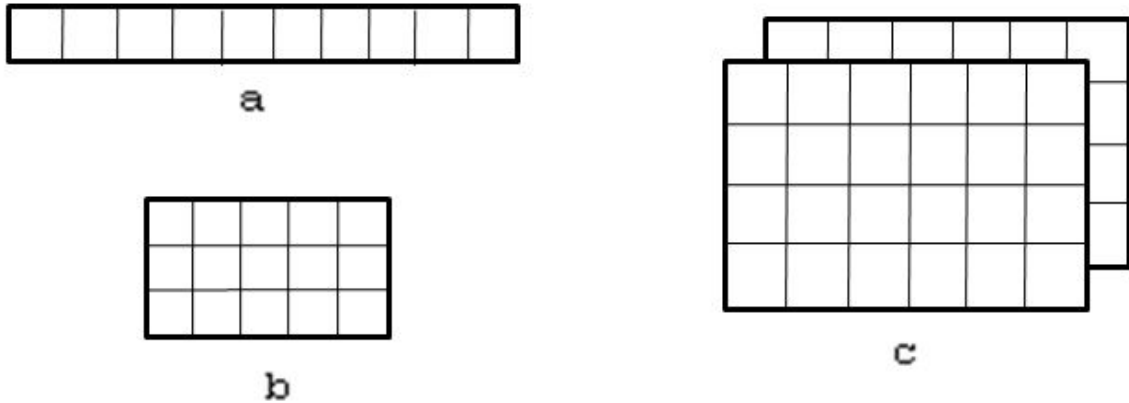
3. Creación

Se realiza con el operador **new**, que es el que realmente crea el array indicando un tamaño. Cuando se usa new es cuando se reserva el espacio necesario en memoria para el array.

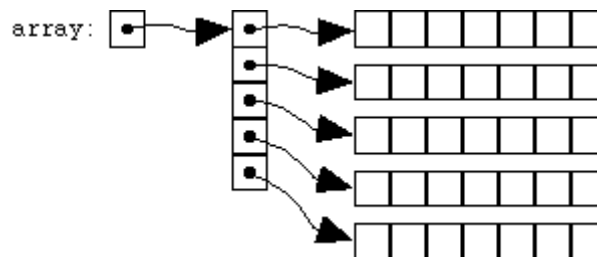
Se crea un array bidimensional de la siguiente manera: `nombre_array = new tipo[tamaño][tamaño];`

Ejemplos:

```
int[] a= new int[10]; //Array de una dimensión
int[][] b= new int[3][5]; //Array bidimensional (dos dimensiones)
int[][][] c= new int[4][6][2]; //Array tridimensional (tres dimensiones)
```



Los arrays multidimensionales son arrays que contienen arrays, por lo tanto un array bidimensional se puede también representar de la siguiente manera:



4. Referencia a los elementos del array

Para referenciar los elementos del array se utiliza el índice de los mismos entre corchetes:

`a[9]` : es el último elemento del array unidimensional *a*.

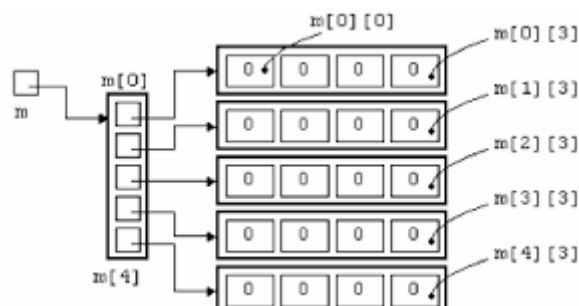
`b[1][2]` : es el elemento que está justo en medio del array bidimensional *b*.

`c[0][0][0]` : es el primer elemento del array tridimensional *c*.

Supongamos que tenemos el siguiente array bidimensional:

```
int[][] m= new int[5][4];
```

Es un array que contiene 5 arrays unidimensionales de 4 posiciones cada uno. Veamos en la imagen la referencia de sus elementos:



- *m*: contiene la referencia al array completo
- *m[0]*: contiene la referencia del primer array unidimensional.

- `m[4]`: contiene la referencia del quinto array unidimensional.
- `m[0][0]`: contiene el primer elemento del primer array unidimensional.
- `m[0][3]`: contiene el último elemento del primer array unidimensional.
- `m[4][3]`: contiene el último elemento del último array unidimensional.

5. Asignación de valores

Se pueden asignar valores a los elementos del array utilizando el signo `=`:

```
a[9] = 8; //Se asigna un 8 al último elemento del array unidimensional a
b[1][2] = 9; //Se asigna un 9 al elemento que está justo en medio del array
bidimensional b
c[0][0][0] = 1; //Se asigna un 1 al primer elemento del array tridimensional c
```

También se pueden asignar valores a todos los elementos del array utilizando **literales array**:

```
//Array de dos dimensiones [3][5]
int[][] twoDimensions= { {0, 1, 2, 3, 4},
                        {5, 6, 7, 8, 9},
                        {10, 11, 12, 13, 14} };
//Array de tres dimensiones [2][3][2]
int[][][] threeDimensions={ { {0, 1}, {2, 3}, {4, 5} },
                             { {6, 7}, {8, 9}, {10, 11} } };
```

6. El atributo length

Los arrays poseen el atributo **length** que contiene el tamaño del array. Como los arrays multidimensionales son arrays de arrays, se puede aplicar a cualquier posición que contenga una referencia. Si se aplica a una posición que contenga un elemento, entonces da error de compilación.

```
package tema6_2_ArraysMultidimensionales;

public class LengthMultidimensional {

    public void show() {

        //Array de dos dimensiones [3][5]
        int[][] twoDimensions = { { 0, 1, 2, 3, 4 }, { 5, 6, 7, 8, 9 }, { 10, 11,
12, 13, 14 } };

        //Array de tres dimensiones [2][3][2]
        int[][][] threeDimensions = { { { 0, 1 }, { 2, 3 }, { 4, 5 } }, { { 6, 7
}, { 8, 9 }, { 10, 11 } } };

        System.out.println(twoDimensions.length); //3
        System.out.println(twoDimensions[0].length); //5
        System.out.println(twoDimensions[1].length); //5
        System.out.println(twoDimensions[2].length); //5

        System.out.println(threeDimensions.length); //2
        System.out.println(threeDimensions[0].length); //3
```



```

        System.out.println(threeDimensions[1].length); //3
        System.out.println(threeDimensions[0][0].length); //2
        System.out.println(threeDimensions[1][2].length); //2
        System.out.println(threeDimensions[1][2][0].length); //Error de
compilación

    }

    public static void main(String[] args) {

        new LengthMultidimensional1().show();

    }

}

```

7. Utilización del bucle for

La ventaja de usar arrays es que gracias a un simple bucle for se pueden recorrer fácilmente todos los elementos de un array multidimensional:

```

package tema6_2_ArraysMultidimensionales;

public class ForMultidimensional {

    public void show() {

        //Array de dos dimensiones [3][5]
        int[][] twoDimensions = { { 0, 1, 2, 3, 4 }, { 5, 6, 7, 8, 9 }, { 10, 11,
12, 13, 14 } };

        //Array de tres dimensiones [2][3][2]
        int[][][] threeDimensions = { { { 0, 1 }, { 2, 3 }, { 4, 5 } }, { { 6, 7
}, { 8, 9 }, { 10, 11 } } };

        for (int i = 0; i < twoDimensions.length; i++) { //Recorrido de arrays
bidimensionales
            for (int j = 0; j < twoDimensions[i].length; j++) {
                System.out.printf("%2d ", twoDimensions[i][j]);
            }
            System.out.println();
        }

        System.out.println();

        for (int i = 0; i < threeDimensions.length; i++) { //Recorrido de arrays
tridimensionales
            for (int j = 0; j < threeDimensions[i].length; j++) {
                for (int k = 0; k < threeDimensions[i][j].length; k++) {
                    System.out.printf("%2d ", threeDimensions[i][j][k]);
                }
                System.out.println();
            }
            System.out.println();
        }
    }
}

```

```

    }

    public static void main(String[] args) {

        new ForMultidimensional().show();
    }

}

```

Salida por consola:

```

0  1  2  3  4
5  6  7  8  9
10 11 12 13 14

0  1
2  3
4  5

6  7
8  9
10 11

```

También se pueden utilizar los bucles for-each. En un array multidimensional, se puede utilizar un for-each en cualquiera de los arrays unidimensionales que forman parte del array multidimensional.

```

package tema6_2_ArraysMultidimensionales;

public class ArraysForEachTwoDimensions {

    public void show() {

        //Array de dos dimensiones [3][5]
        int[][] twoDimensions = { { 0, 1, 2, 3, 4 }, { 5, 6, 7, 8, 9 }, { 10, 11, 12, 13, 14 } };
        //Recorrido del segundo array unidimensional del array bidimensional:
        for (int number : twoDimensions[1]) {
            System.out.printf("%d ", number); //5 6 7 8 9
        }
        System.out.println();

        // Bucle for-each para recorrer el array bidimensional
        for (int[] oneDimensions : twoDimensions) {
            // Bucle for-each para recorrer cada subarray unidimensional
            for (int number : oneDimensions) {
                System.out.printf("%d ", number);
            }
            System.out.println();
        }
    }

}

```

```

    public static void main(String[] args) {

        new ArraysForEachTwoDimensions().show();

    }

}

```

Salida por consola:

```

5 6 7 8 9
0 1 2 3 4
5 6 7 8 9
10 11 12 13 14

```

```

package tema6_2_ArraysMultidimensionales;

public class ArraysForEachThreeDimensions {

    public void show() {

        //Array de tres dimensiones [2][3][2]

        int[][][] threeDimensions = { { { 0, 1 }, { 2, 3 }, { 4, 5 } }, { { 6, 7 }, { 8, 9 }, { 10, 11 } } };

        for (int[][] twoDimensions : threeDimensions) {
            for (int[] oneDimensions : twoDimensions) {
                for(int number: oneDimensions) {
                    System.out.printf("%d ", number);
                }
                System.out.println();
            }
            System.out.println();
        }

    }

    public static void main(String[] args) {

        new ArraysForEachThreeDimensions().show();

    }

}

```

Salida por consola:

```
0 1
2 3
4 5

6 7
8 9
10 11
```

Para mostrar un array multidimensional, también se puede utilizar el método estático *deepToString* de la clase *Arrays*. Devuelve una cadena con los elementos del array entre corchetes y separados por comas. Se utiliza con arrays multidimensionales. Para los arrays unidimensionales se usa el método *Arrays.toString*.

```
package tema6_2_ArraysMultidimensionales;

import java.util.Arrays;

public class DeepToString {

    public void show() {

        int[][] twoDimensions = { { 0, 1, 2, 3, 4 }, { 5, 6, 7, 8, 9 }, { 10, 11, 12, 13, 14 } };

        System.out.println(Arrays.deepToString(twoDimensions));
        //[0, 1, 2, 3, 4], [5, 6, 7, 8, 9], [10, 11, 12, 13, 14]]

    }

    public static void main(String[] args) {

        new DeepToString().show();

    }

}
```

8. Comparación de arrays

Como en Java los arrays son objetos, la comparación se realiza como los objetos (Ver [Tema 4 Programación Orientada a Objetos 13 Referencias](#)).

El operador de igualdad `==` cuando se utiliza con arrays, no compara el contenido de los arrays sino sus direcciones de memoria o referencias, es decir, si apuntan al mismo array. Lo mismo ocurre con el método *equals* de los arrays, que compara las direcciones de memoria. Si queremos comparar el contenido de los arrays, tendremos que utilizar el método estático *equals* de la clase *Arrays* para los unidimensionales y el método ***deepEquals*** para los multidimensionales:

```
package tema6_2_ArraysMultidimensionales;

import java.util.Arrays;

public class ComparisonMultidimensional {
```

```

public void show() {

    int[][] twoDimensions1;

    int[][] twoDimensions2 = { { 0, 1, 2, 3, 4 }, { 5, 6, 7, 8, 9 }, { 10,
11, 12, 13, 14 } };

    int[][] twoDimensions3 = { { 0, 1, 2, 3, 4 }, { 5, 6, 7, 8, 9 }, { 10,
11, 12, 13, 14 } };

    twoDimensions1 = twoDimensions2;

    System.out.println(twoDimensions1 == twoDimensions2);//true porque
apuntan al mismo array
    System.out.println(twoDimensions2 == twoDimensions3);//false porque no
apuntan al mismo array
    System.out.println(twoDimensions1.equals(twoDimensions2));//true porque
apuntan al mismo array
    System.out.println(twoDimensions2.equals(twoDimensions3));//false porque
no apuntan al mismo array
    System.out.println(Arrays.deepEquals(twoDimensions1,
twoDimensions2));//true porque el contenido es el mismo ya que apuntan al mismo
array
    System.out.println(Arrays.deepEquals(twoDimensions2,
twoDimensions3));//true porque el contenido es el mismo
    System.out.println(Arrays.equals(twoDimensions2, twoDimensions3));//false
porque para arrays multidimensionales es deepEquals en lugar de equals

}

public static void main(String[] args) {

    new ComparisonMultidimensional().show();

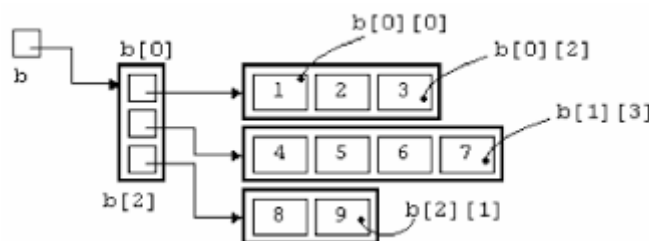
}

}

```

9. Arrays irregulares

Los arrays unidimensionales que forman un array multidimensional no tienen por qué tener todos el mismo tamaño.



```

package tema6_2_ArraysMultidimensionales;

public class IrregularArrays {

```

```
public void show() {  
  
    int[][] b = { { 1, 2, 3 }, { 4, 5, 6, 7 }, { 8, 9 } };  
  
    for (int i = 0; i < b.length; i++) {  
        for (int j = 0; j < b[i].length; j++) {  
            System.out.printf("%2d ", b[i][j]);  
        }  
        System.out.println();  
    }  
  
}  
  
public static void main(String[] args) {  
  
    new IrregularArrays().show();  
  
}  
  
}
```

6.3 Argumentos variables

6.3 Argumentos variables

1. Introducción
2. Sintaxis de Varargs
3. Métodos con parámetros normales y parámetros variables

1. Introducción

Se puede dar el caso de que se necesite crear un método que tome una cantidad variable de argumentos, según su uso preciso. Por ello, Java en su versión 5 incorporó los llamados **varargs** que es la abreviatura de argumentos de longitud variable (*Variable-Length Arguments*). Un método que toma una cantidad variable de argumentos se denomina **método varargs**.

2. Sintaxis de Varargs

Un parámetro de longitud variable se especifica por tres puntos (...).

```
public static int add(int... nums)
```

Esta sintaxis le dice al compilador que se puede llamar al método *add* con cero o más argumentos. Además, hace que *nums* se declare implícitamente como un array de tipo `int[]`. Por lo tanto, dentro del método *add* se accede a *nums* usando la sintaxis de array normal:

```
package tema6_3_ArgumentosVariables;

public class Varargs1 {

    public void show() {

        System.out.println(add());
        System.out.println(add(1));
        System.out.println(add(new int[] { 1 })); // También se le puede pasar un
array
        System.out.println(add(1, 2));
        System.out.println(add(1, 2, 3));
        System.out.println(add(1, 2, 3, 4));
        System.out.println(add(new int[] { 1, 2, 3, 4 }));

    }

    public int add(int... nums) {

        int sum = 0;

        for (int i = 0; i < nums.length; i++) {
            sum += nums[i];
        }

    }

}
```

```

        return sum;

    }

    public static void main(String[] args) {

        new Varargs1().show();

    }

}

```

Si nos fijamos en el ejemplo, el método *add* se llama con diferentes números de argumentos, incluyendo ningún argumento. Los argumentos se colocan automáticamente en un array y se pasan a *nums*. En el caso de que no haya argumentos, la longitud del array es cero.

3. Métodos con parámetros normales y parámetros variables

Un método puede tener parámetros normales junto con un parámetro de longitud variable. Sin embargo, el parámetro de longitud variable debe ser el último parámetro declarado por el método y solamente se acepta un *varargs* por método. Por ejemplo, esta declaración de método es perfectamente aceptable:

```
int compute(int x,int y,double z,int ... values)
```

En este caso, los tres primeros argumentos utilizados en una llamada al método *compute* se corresponden con los primeros tres parámetros. Entonces, se supone que los argumentos restantes pertenecen a *values*.

Aquí hay un ejemplo que toma un argumento normal y un argumento de longitud variable:

```

package tema6_3_ArgumentosVariables;

public class Varargs2 {

    public void show() {

        System.out.println(operate(2));
        System.out.println(operate(2, 1));
        System.out.println(operate(2, 1, 2));
        System.out.println(operate(2, 1, 2, 3));
        System.out.println(operate(2, 1, 2, 3, 4));

    }

    public int operate(int mult, int... summands) {

        int sum = 0;

        for (int i = 0; i < summands.length; i++) {
            sum += summands[i];
        }

    }

}

```



```
        return sum * mult;
    }

    public static void main(String[] args) {

        new Varargs2().show();

    }

}
```

Recuerda, el parámetro varargs debe ser el último. Por ejemplo, la siguiente declaración es incorrecta:

```
int compute(int x,int y,double z,int ... values,boolean exit)//error
```

Además, solamente se acepta un varargs por método. Por ejemplo, esta declaración tampoco es válida:

```
int compute(int x,int y,double z,int ... values,double ... nums)//error
```

6.4 La clase Arrays

6.4 La clase Arrays

1. Introducción
2. Contenido de un array
3. Rellenar un array
4. Búsquedas
5. Comparaciones
6. Copias
7. Ordenaciones

1. Introducción

En el paquete *java.util* se encuentra la clase *Arrays*. Esta clase contiene métodos estáticos para manipular arrays, como por ejemplo, búsquedas, comparaciones, copias y ordenaciones.

2. Contenido de un array

- Método **toString**: devuelve una representación en cadena del contenido del array especificado. Dicha cadena está formada por los elementos del array entre corchetes y separados por comas. Es muy útil para sacar por consola la información sobre el array. Se utiliza con arrays unidimensionales.

```
package tema6_4_ArraysClass;

import java.util.Arrays;

public class ToString {

    public void show() {

        int[] array = { 3, 4, 5, 6, 7, 8 };
        System.out.printf("La información del array es: %s",
            Arrays.toString(array));

    }

    public static void main(String[] args) {

        new ToString().show();

    }

}
```

La salida por consola es:

La información del array es: [3, 4, 5, 6, 7, 8]

- Método **deepToString**: igual que el anterior pero para arrays multidimensionales.

```
package tema6_4_ArraysClass;

import java.util.Arrays;

public class DeepToString {

    public void show() {

        int[][] array = { { 0, 1, 2, 3, 4 }, { 5, 6, 7, 8, 9 }, { 10, 11, 12, 13, 14 } };

        System.out.printf("La información del array es: %s",
Arrays.deepToString(array));

    }

    public static void main(String[] args) {

        new DeepToString().show();

    }

}
```

La salida por consola es:

La información del array es: [[0, 1, 2, 3, 4], [5, 6, 7, 8, 9], [10, 11, 12, 13, 14]]

3. Rellenar un array

Método **fill**: permite rellenar todo un array unidimensional con un determinado valor. También tiene una versión sobrecargada para especificar un rango.

```
package tema6_4_ArraysClass;

import java.util.Arrays;

public class Fill {

    public void show() {

        int[] array = new int[6];
        Arrays.fill(array, 1);
        System.out.println(Arrays.toString(array)); //[1, 1, 1, 1, 1, 1]
        Arrays.fill(array, 2, 5, 0);
        System.out.println(Arrays.toString(array)); //[1, 1, 0, 0, 0, 1]

    }

}
```

```

    public static void main(String[] args) {

        new Fill().show();

    }

}

```

4. Búsquedas

Método **binarySearch**:

- permite buscar un elemento de forma ultrarrápida en un array ordenado (en un array desordenado, sus resultados son impredecibles).
- Devuelve el índice en el que está colocado el elemento.
- Si el elemento no se encuentra, devuelve un número negativo.
- Si el array contiene múltiples elementos con el valor especificado, no hay garantía de cuál será encontrado.
- Está sobrecargado para buscar en un determinado rango del array. Se proporcionan dos parámetros más para especificar el rango:
 - *fromIndex*: el índice (incluido) de la parte inferior del rango.
 - *toIndex*: el índice (excluido) de la parte superior del rango.

```

package tema6_4_ArraysClass;

import java.util.Arrays;

public class BinarySearch {

    public void show() {

        int lowerRange, higherRange;
        int[] array = { 3, 4, 5, 6, 7, 8 };
        int number = 9;
        int index = Arrays.binarySearch(array, number);
        if (index >= 0) {
            System.out.printf("El número %d se encuentra en el índice %d\n",
number, index);
        } else {
            System.out.printf("El número %d no se encuentra en el array\n",
number);
        }
        lowerRange = 3;
        higherRange = 6; // Como está excluido, se busca hasta el índice 5
        index = Arrays.binarySearch(array, lowerRange, higherRange, number);
        if (index >= 0) {
            System.out.printf("El número %d se encuentra en el índice %d",
number, index);
        } else {
            System.out.printf("El número %d no se encuentra en el array en el
rango %d-%d", number, lowerRange,

```

```

        higherRange - 1);
    }

}

public static void main(String[] args) {

    new BinarySearch().show();

}

}

```

Si ejecutamos dicho código con *number=5*, tendremos la siguiente salida por consola:

```

El número 5 se encuentra en el índice 2
El número 5 no se encuentra en el array en el rango 3-5

```

Si ejecutamos dicho código con *number=9*, entonces obtendremos:

```

El número 9 no se encuentra en el array
El número 9 no se encuentra en el array en el rango 3-5

```

5. Comparaciones

- Método **equals**: compara dos arrays y devuelve true si son iguales. Se consideran iguales si son del mismo tipo, tamaño y contienen los mismos valores. También tiene una versión sobrecargada para especificar un rango.

```

package tema6_4_ArraysClass;

import java.util.Arrays;

public class Equals {

    public void show() {

        int[] array1 = { 3, 4, 5, 6, 7, 8 };
        int[] array2 = { 3, 4, 5, 6, 7, 8 };
        int[] array3 = { 3, 4, 5, 6, 7, 8, 9 };

        System.out.printf("%s y %s %s iguales\n", Arrays.toString(array1),
Arrays.toString(array2),
        Arrays.equals(array1, array2) ? "son" : "no son");
        System.out.printf("%s y %s %s iguales\n", Arrays.toString(array1),
Arrays.toString(array3),
        Arrays.equals(array1, array3) ? "son" : "no son");
        System.out.printf("%s y %s %s iguales en el rango 1-5",
Arrays.toString(array1), Arrays.toString(array3),
        Arrays.equals(array1, 1, 6, array3, 1, 6) ? "son" : "no son");

    }

}

```

```

    public static void main(String[] args) {

        new Equals().show();

    }

}

```

La salida por consola es la siguiente:

```

[3, 4, 5, 6, 7, 8] y [3, 4, 5, 6, 7, 8] son iguales
[3, 4, 5, 6, 7, 8] y [3, 4, 5, 6, 7, 8, 9] no son iguales
[3, 4, 5, 6, 7, 8] y [3, 4, 5, 6, 7, 8, 9] son iguales en el rango 1-5

```

- Método **deepEquals**: compara arrays multidimensionales y devuelve true si son iguales.

```

package tema6_4_ArraysClass;

import java.util.Arrays;

public class DeepEquals {

    public void show() {

        int[][] array1 = { { 0, 1, 2, 3, 4 }, { 5, 6, 7, 8, 9 }, { 10, 11, 12, 13, 14 } };

        int[][] array2 = { { 0, 1, 2, 3, 4 }, { 5, 6, 7, 8, 9 }, { 10, 11, 12, 13, 14 } };

        int[][] array3 = { { 0, 1, 2, 3, 4 }, { 5, 6, 7, 8, 9 }, { 10, 11, 12, 13, 15 } };

        System.out.printf("%s\n y\n%s\n%s iguales\n\n",
            Arrays.deepToString(array1), Arrays.deepToString(array2),
            Arrays.deepEquals(array1, array2) ? "son" : "no son");
        System.out.printf("%s\n y\n%s\n%s iguales\n",
            Arrays.deepToString(array1), Arrays.deepToString(array3),
            Arrays.deepEquals(array1, array3) ? "son" : "no son");

    }

    public static void main(String[] args) {

        new DeepEquals().show();

    }

}

```

La salida por consola es la siguiente:

```
[0, 1, 2, 3, 4], [5, 6, 7, 8, 9], [10, 11, 12, 13, 14]]  
y  
[0, 1, 2, 3, 4], [5, 6, 7, 8, 9], [10, 11, 12, 13, 14]]  
son iguales
```

```
[0, 1, 2, 3, 4], [5, 6, 7, 8, 9], [10, 11, 12, 13, 14]]  
y  
[0, 1, 2, 3, 4], [5, 6, 7, 8, 9], [10, 11, 12, 13, 15]]  
no son iguales
```

- Método **compare**: compara dos arrays lexicográficamente. También tiene una versión sobrecargada para especificar un rango. Devuelve:
 - el valor 0: si los dos arrays son iguales y contienen los mismos elementos en el mismo orden.
 - un número negativo: si el primer array es lexicográficamente inferior al segundo.
 - un número positivo: si el primer array es lexicográficamente superior al segundo.

```
package tema6_4_ArraysClass;  
  
import java.util.Arrays;  
  
public class Compare {  
  
    public void show() {  
  
        int result;  
        boolean[] arrayBoolean1 = { true, true, false };  
        boolean[] arrayBoolean2 = { true, true, true };  
        char[] arrayChar1 = { 'm', 'n', 't' };  
        char[] arrayChar2 = { 'm', 'n', 'a' };  
        char[] arrayChar3 = { 'M', 'n', 'a' };  
        int[] arrayInt1 = { 3, 4, 5 };  
        int[] arrayInt2 = { 3, 4, 5 };  
        int[] arrayInt3 = { 3, 4, 5, 6 };  
  
        result = Arrays.compare(arrayBoolean1, arrayBoolean2);  
        System.out.printf("El resultado de comparar %s y %s es: ",  
Arrays.toString(arrayBoolean1),  
        Arrays.toString(arrayBoolean2));  
        comparison(result);  
  
        result = Arrays.compare(arrayChar1, arrayChar2);  
        System.out.printf("El resultado de comparar %s y %s es: ",  
Arrays.toString(arrayChar1),  
        Arrays.toString(arrayChar2));  
        comparison(result);  
  
        result = Arrays.compare(arrayChar3, arrayChar2);  
        System.out.printf("El resultado de comparar %s y %s es: ",  
Arrays.toString(arrayChar3),  
        Arrays.toString(arrayChar2));  
        comparison(result);  
    }  
}
```

```

        result = Arrays.compare(arrayInt1, arrayInt2);
        System.out.printf("El resultado de comparar %s y %s es: ",
Arrays.toString(arrayInt1),
        Arrays.toString(arrayInt2));
        comparison(result);

        result = Arrays.compare(arrayInt1, arrayInt3);
        System.out.printf("El resultado de comparar %s y %s es: ",
Arrays.toString(arrayInt1),
        Arrays.toString(arrayInt3));
        comparison(result);

        result = Arrays.compare(arrayInt1, 0, 3, arrayInt3, 0, 3);
        System.out.printf("El resultado de comparar %s y %s en el rango 0-2 es:
", Arrays.toString(arrayInt1),
        Arrays.toString(arrayInt3));
        comparison(result);

    }

    public void comparison(int result) {
        if (result == 0) {
            System.out.println("los arrays son iguales");
        } else if (result < 0) {
            System.out.println("el primer array es menor");
        } else {
            System.out.println("el primer array es mayor");
        }
    }

    public static void main(String[] args) {

        new Compare().show();

    }

}

```

La salida por consola es la siguiente:

```

El resultado de comparar [true, true, false] y [true, true, true] es: el primer
array es menor
El resultado de comparar [m, n, t] y [m, n, a] es: el primer array es mayor
El resultado de comparar [M, n, a] y [m, n, a] es: el primer array es menor
El resultado de comparar [3, 4, 5] y [3, 4, 5] es: los arrays son iguales
El resultado de comparar [3, 4, 5] y [3, 4, 5, 6] es: el primer array es menor
El resultado de comparar [3, 4, 5] y [3, 4, 5, 6] en el rango 0-2 es: los arrays
son iguales

```

- Método **mismatch**: encuentra y devuelve el índice de la primera diferencia entre dos arrays. Si no encuentra ninguna diferencia, devuelve -1. También tiene una versión sobrecargada para especificar un rango.

```
package tema6_4_ArraysClass;
```



```

import java.util.Arrays;

public class Mismatch {

    public void show() {

        int[] array1 = { 3, 4, 5, 6, 7, 8 };
        int[] array2 = { 3, 4, 5, 6, 7, 8 };
        int[] array3 = { 3, 4, 5, 6, 7, 8, 9, 10, 11 };
        int indexMismatch;

        indexMismatch = Arrays.mismatch(array1, array2);

        if (indexMismatch == -1) {
            System.out.printf("%s y %s son iguales\n", Arrays.toString(array1),
Arrays.toString(array2));
        } else {
            System.out.printf("%s y %s tienen su primera diferencia en el
elemento %d\n", Arrays.toString(array1),
Arrays.toString(array2), indexMismatch);
        }

        indexMismatch = Arrays.mismatch(array1, array3);

        if (indexMismatch == -1) {
            System.out.printf("%s y %s son iguales\n", Arrays.toString(array1),
Arrays.toString(array3));
        } else {
            System.out.printf("%s y %s tienen su primera diferencia en el
elemento %d\n", Arrays.toString(array1),
Arrays.toString(array3), indexMismatch);
        }

        indexMismatch = Arrays.mismatch(array1, 2, 6, array3, 2, 6);

        if (indexMismatch == -1) {
            System.out.printf("%s y %s son iguales en el rango 2-5\n",
Arrays.toString(array1),
Arrays.toString(array3));
        } else {
            System.out.printf("%s y %s tienen su primera diferencia en el
elemento %d en el rango 2-5\n",
Arrays.toString(array1), Arrays.toString(array3),
indexMismatch);
        }

    }

    public static void main(String[] args) {

        new Mismatch().show();

    }

}

```

La salida por consola es la siguiente:

```
[3, 4, 5, 6, 7, 8] y [3, 4, 5, 6, 7, 8] son iguales
[3, 4, 5, 6, 7, 8] y [3, 4, 5, 6, 7, 8, 9, 10, 11] tienen su primera diferencia
en el elemento 6
[3, 4, 5, 6, 7, 8] y [3, 4, 5, 6, 7, 8, 9, 10, 11] son iguales en el rango 2-5
```

6. Copias

- Método **copyOf**: obtiene una copia de un array. Recibe dos parámetros, el array a copiar y el tamaño del array resultante. De modo que:
 - si el tamaño del array resultante es **menor** que el tamaño del array original: sólo obtiene copia de los primeros elementos, tantos como indique el tamaño.
 - si el tamaño del array resultante es **mayor** que el tamaño del array original: devuelve un array en el que los elementos que superan al original se rellenan con el valor por defecto según el tipo de datos del array.

```
package tema6_4_ArraysClass;

import java.util.Arrays;

public class CopyOf {

    public void show() {

        int[] array1 = { 3, 4, 5, 6, 7, 8, 9, 10 };
        int[] array2 = Arrays.copyOf(array1, 5);
        int[] array3 = Arrays.copyOf(array1, 10);

        System.out.println(Arrays.toString(array2)); //[3, 4, 5, 6, 7]
        System.out.println(Arrays.toString(array3)); //[3, 4, 5, 6, 7, 8, 9, 10,
0, 0]

    }

    public static void main(String[] args) {

        new CopyOf().show();

    }

}
```

- Método **copyOfRange**: obtiene una copia de un array especificando el rango, es decir, de qué elemento a qué elemento se hace la copia.

```
package tema6_4_ArraysClass;

import java.util.Arrays;

public class CopyOfRange {
```

```

public void show() {

    int[] array1 = { 3, 4, 5, 6, 7, 8, 9, 10 };
    int[] array2 = Arrays.copyOfRange(array1, 2, 6);
    System.out.println(Arrays.toString(array2)); //[5, 6, 7, 8]

}

public static void main(String[] args) {

    new CopyOfRange().show();

}

}

```

7. Ordenaciones

Método **sort**: ordena un array en orden ascendente. También tiene una versión sobrecargada para especificar un rango.

```

package tema6_4_ArraysClass;

import java.util.Arrays;

public class Sort {

    public void show() {

        int[] array1 = { 8, 4, 3, 7, 5, 6 };
        int[] array2 = { 8, 4, 3, 7, 5, 6 };

        Arrays.sort(array1);
        System.out.println(Arrays.toString(array1)); //[3, 4, 5, 6, 7, 8]

        Arrays.sort(array2, 1, 5);
        System.out.println(Arrays.toString(array2)); //[8, 3, 4, 5, 7, 6]

    }

    public static void main(String[] args) {

        new Sort().show();

    }

}

```

7 Herencia

7 Herencia

1. Introducción
2. Modificadores de acceso
3. Sobrecarga y anulación de métodos
4. Super
5. Constructores
6. El método toString
7. Casting de objetos
8. instanceof
9. Modificador final
10. Polimorfismo
11. Clases abstractas
12. Interfaces
 - 12.1 Herencia entre interfaces
 - 12.2 Métodos por defecto
 - 12.3 Métodos estáticos
 - 12.4 Métodos privados
 - 12.5 Diferencias entre una interfaz y una clase abstracta
 - 12.6 Utilización de una interfaz como un tipo de datos
13. Clases anidadas
 - 13.1 Clases anidadas estáticas
 - 13.2 Clases anidadas no estáticas o clases internas
 - 13.2.1 Clases internas miembro
 - 13.2.2 Clases internas dentro de un método
 - 13.2.3 Clases internas dentro de un bloque
 - 13.2.4 Clases inline anónimas
14. Clases selladas
15. Interfaces selladas

1. Introducción

La herencia permite definir una clase tomando como base a otra clase ya existente. Dicha clase base se conoce como **superclase** o **clase padre** y la clase que hereda se denomina **subclase** o **clase hija**. Por lo tanto, una subclase es una versión especializada de una superclase ya que hereda tanto los atributos como los métodos definidos por la superclase y además añade los suyos propios.

Esto es una de las bases de la **reutilización** de código ya que cuando se quiere crear una clase nueva y ya existe una clase que incluye parte del código que queremos, podemos heredar nuestra nueva clase de la clase existente reutilizando los atributos y métodos de la misma. La herencia facilita el trabajo del programador porque permite crear clases estándar y a partir de ellas crear nuestras propias clases personales. Esto es más cómodo que tener que crear todas las clases desde cero.

Por ejemplo, si quisiéramos realizar una aplicación de vehículos, definiríamos una superclase o clase padre con lo común a todos los vehículos y luego definiríamos una subclase o clase hija para cada tipo de vehículo donde se añadiría lo particular de cada uno.

En java, la herencia se especifica en la subclase añadiendo la palabra **extends** seguida del nombre de la superclase. Por ejemplo, así sería para indicar que *Coche* es hija de *Vehiculo*:

```
public class Coche extends vehiculo { //Coche es hija de vehiculo
...
}
```

En Java, solamente se puede tener un padre pero puede haber varios niveles de herencia, es decir, clases hijas que a su vez son padres de otras clases. Por ejemplo, el *Coche* es hija de *Vehiculo* pero puede ser padre de otra clase, como por ejemplo de *Todoterreno*:

```
public class Todoterreno extends Coche { //Coche es hija de vehiculo y padre de
Todoterreno
...
}
```

Si el padre tiene algún atributo estático, también lo pueden usar los hijos.

`ClasePadre.AtributoEstático` y `ClaseHijo.AtributoEstático` acceden a la misma variable porque es el mismo atributo estático.

2. Modificadores de acceso

En el [Tema 4. Programación Orientada a Objetos 5 Modificadores de acceso](#) vimos los modificadores de acceso y cómo afectaban a la visibilidad. En este tema vamos a incorporar el modificador de acceso *protected* que es el que está pensado para la herencia.

Los modificadores de acceso afectan a la visibilidad y también afectan a la herencia. Visibilidad es lo que una clase puede ver de otra clase y herencia es lo que una clase hereda de otra clase. He aquí dos tablas con los modificadores de acceso, una para la visibilidad y otra para la herencia:

Tabla de visibilidad:

	Private	Friendly	Protected	Public
Misma clase	X	X	X	X
Mismo paquete		X	X	X
Otro paquete				X
Subclase en el mismo paquete		X	X	X
Subclase en distinto paquete				X

Tabla de herencia:

	Private	Friendly	Protected	Public
Subclase en el mismo paquete		X	X	X

	Private	Friendly	Protected	Public
Subclase en distinto paquete			X	X

Si las clases están en un subpaquete, a efectos de visibilidad y herencia se considera que están en otro paquete.

Las conclusiones que se pueden obtener a partir de las dos tablas son las siguientes:

- La visibilidad es la misma independientemente de que la clase sea hija o no.
- En herencia, siempre se hereda el *protected* independientemente del paquete donde se encuentre la clase hija. Por lo tanto, cuando diseñemos una clase que vaya a tener descendientes, es conveniente declarar sus atributos como *protected*.

Un atributo *private* no se hereda pero si los *getters* y *setters* del padre tienen un modificador distinto de *private*, sí puede el hijo utilizar dicho atributo a través de dichos métodos. Pero no es conveniente programar de esta manera, es más adecuado utilizar el modificador *protected*.

Veamos un ejemplo de herencia con el uso del modificador *protected*:

```
package tema7_Herencia.modificadorProtected;

public class Vehicle { //Superclase o clase padre

    protected int wheelCount; //Atributos protected
    protected double speed;
    protected String colour;

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }

}
```

```

package tema7_Herencia.modificadorProtected;

public class Car extends vehicle { //Car es hijo de vehicle

    private double gasoline;//Atributo propio de Car

    //Car hereda de vehicle los atributos wheelCount,speed y colour ya que son
    protected
    //También hereda todos los métodos de vehicle ya que son public

    public double getGasoline() { //Método propio de Car
        return gasoline;
    }

    public void refuel(double liters) { //Método propio de Car
        gasoline += liters;
    }

}

```

```

package tema7_Herencia.modificadorProtected;

public class Main {

    public void showProtected() {

        Car car;
        car = new Car();

        car.accelerate(100); //Método heredado
        System.out.printf("La velocidad del coche es %.2f km/h", car.getSpeed());

        car.refuel(40.35); //Método propio
        System.out.printf("\nEl coche tiene %.2f litros de gasolina",
        car.getGasoline());

    }

    public static void main(String[] args) {

        new Main().showProtected();

    }

}

```

Salida por consola:

```

La velocidad del coche es 100,00 km/h
El coche tiene 40,35 litros de gasolina

```

3. Sobrecarga y anulación de métodos

Se puede **sobrecargar** un método heredado para proporcionar una versión del mismo adaptado a las necesidades de la subclase. Ejemplo:

```
package tema7_Herencia.sobrecarga;

public class Car extends Vehicle {

    private double gasoline;

    public double getGasoline() {
        return gasoline;
    }

    public void refuel(double liters) {
        gasoline += liters;
    }

    public void accelerate() { //Sobrecarga del método heredado accelerate
        speed += 10;
    }

}
```

```
package tema7_Herencia.sobrecarga;

public class Main {

    public void showOverload() {

        Car car;
        car = new Car();

        car.accelerate(100); //Método heredado
        System.out.printf("La velocidad del coche es %.2f km/h", car.getSpeed());

        car.accelerate(); //Método sobrecargado
        System.out.printf("\nLa velocidad del coche es %.2f km/h",
car.getSpeed());

    }

    public static void main(String[] args) {

        new Main().showOverload();

    }

}
```

Salida por consola:

La velocidad del coche es 100,00 km/h
La velocidad del coche es 110,00 km/h

Si la subclase define un método con la misma firma que un método heredado, entonces **anula** o **sobrescribe** el método de la superclase. Veamos un ejemplo donde el coche ha anulado el método heredado de acelerar para añadirle el consumo de gasolina producido por la aceleración. Vamos a considerar que se gasta de gasolina un 1% de la cantidad que se acelera:

```
package tema7_Herencia.anulacionMetodos;

public class Car extends vehicle {

    private double gasoline;

    public double getGasoline() {
        return gasoline;
    }

    public void refuel(double liters) {
        gasoline += liters;
    }

    @Override
    public void accelerate(double amount) { //Anula el método heredado accelerate
        speed += amount;
        gasoline -= amount * 0.01; //Se consume de gasolina un 1% de amount
    }

}
```

```
package tema7_Herencia.anulacionMetodos;

public class Main {

    public void showOverride() {

        Car car;
        car = new Car();

        car.refuel(40.35);
        System.out.printf("El coche tiene %.2f litros de gasolina",
car.getGasoline());
        car.accelerate(100); //Método que ha anulado al heredado
        System.out.printf("\nEl coche tiene %.2f litros de gasolina",
car.getGasoline());
    }

    public static void main(String[] args) {

        new Main().showOverride();
    }

}
```

```
}
```

Salida por consola:

```
El coche tiene 40,35 litros de gasolina  
El coche tiene 39,35 litros de gasolina
```

En la clase *Car*, se puede observar la anotación `@Override` antes de la firma del método *accelerate*.

Las anotaciones de Java comienzan con `@` y permiten incrustar información suplementaria en un programa para que pueda ser utilizada por varias herramientas.

La anotación `@Override` le indica al compilador que el método debe sobrescribir un método de la superclase. Si no lo hace, el compilador generará un error. Se utiliza para asegurar que un método de superclase esté anulado, y no simplemente sobrecargado. Es una manera de comprobar en tiempo de compilación que se está anulando correctamente un método, y de este modo evitar errores en tiempo de ejecución los cuales serían mucho más difíciles de detectar.

La visibilidad de lo que se hereda es con respecto al paquete de la superclase, no con respecto al paquete de la subclase. Si no interesa, la subclase tendrá que sobrescribir lo heredado aunque no haga ningún cambio para que la visibilidad sea con respecto al paquete de la subclase. Veamos un ejemplo donde *Vehicle* está en un paquete distinto que *Car*. *Vehicle* tiene el método *accelerate* como *protected*, por lo tanto *Car* lo hereda aunque esté en otro paquete como podemos observar en la tabla de herencia. La clase *Main* se encuentra en el mismo paquete que *Car* y quiere acceder al método *accelerate* del mismo. Pero la visibilidad de *accelerate* es con respecto al paquete de *Vehicle* ya que la visibilidad de lo que se hereda es con respecto al paquete de la superclase, no con respecto al paquete de la subclase. Dicho método es *protected* y si nos fijamos en la tabla de visibilidad, un *protected* no es visible desde otro paquete, por lo que no se le va a permitir dando un error de compilación:

```
package tema7_Herencia.visibilidad1;  
  
public class Vehicle {  
  
    protected int wheelCount;  
    protected double speed;  
    protected String colour;  
  
    public int getWheelCount() {  
        return wheelCount;  
    }  
  
    public double getSpeed() {  
        return speed;  
    }  
  
    public String getColour() {  
        return colour;  
    }  
  
    public void setColour(String colour) {  
        this.colour = colour;  
    }  
}
```

```

        protected void accelerate(double amount) { //Método protected
            speed += amount;
        }

        public void brake(double amount) {
            speed -= amount;
        }
    }
}

```

```

package tema7_Herencia.visibilidad2;

import tema7_Herencia.visibilidad1.vehicle;

public class Car extends vehicle {

    private double gasoline;

    public double getGasoline() {
        return gasoline;
    }

    public void refuel(double liters) {
        gasoline += liters;
    }
}

```

```

package tema7_Herencia.visibilidad2;

public class Main {

    public void showVisibility() {

        Car car;
        car = new Car();

        car.accelerate(100); //Error de compilación: el método accelerate de
        vehicle no es visible
        System.out.printf("La velocidad del coche es %.2f km/h", car.getSpeed());

        car.refuel(40.35);
        System.out.printf("\nEl coche tiene %.2f litros de gasolina",
        car.getGasoline());

    }

    public static void main(String[] args) {

        new Main().showVisibility();

    }
}

```

La solución es que *Car* anule el método *accelerate* heredado de *Vehicle* para que la visibilidad de dicho método sea con respecto al paquete de *Car*. *Car* no va a realizar ningún cambio en dicho método, es decir, lo va a anular para dejarlo exactamente igual, pero de esta forma modifica el paquete para la visibilidad:

```
package tema7_Herencia.visibilidad3;

import tema7_Herencia.visibilidad1.Vehicle;

public class Car extends Vehicle {

    private double gasoline;

    public double getGasoline() {
        return gasoline;
    }

    public void refuel(double liters) {
        gasoline += liters;
    }

    @Override
    protected void accelerate(double amount) { //Anula el método heredado y lo
deja exactamente igual
        speed += amount;
    }

}
```

```
package tema7_Herencia.visibilidad3;

public class Main {

    public void showVisibility() {

        Car car;
        car = new Car();

        car.accelerate(100);//No hay error de compilación, el método ya es
visible
        System.out.printf("La velocidad del coche es %.2f km/h", car.getSpeed());

        car.refuel(40.35);
        System.out.printf("\nEl coche tiene %.2f litros de gasolina",
car.getGasoline());
    }

    public static void main(String[] args) {

        new Main().showVisibility();
    }

}
```

```
}
```

Salida por consola:

```
La velocidad del coche es 100,00 km/h
El coche tiene 40,35 litros de gasolina
```

Pero, ¿qué ocurriría si la clase *Main* estuviera en un paquete distinto a *Car*? Daría un error de compilación porque un *protected* no es visible desde otro paquete:

```
package tema7_Herencia.visibilidad4;

import tema7_Herencia.visibilidad1.vehicle;

public class Car extends vehicle {

    private double gasoline;

    public double getGasoline() {
        return gasoline;
    }

    public void refuel(double liters) {
        gasoline += liters;
    }

    @Override
    protected void accelerate(double amount) { //Anula el método heredado y lo
deja exactamente igual
        speed += amount;
    }

}
```

```
package tema7_Herencia.visibilidad5;

import tema7_Herencia.visibilidad4.Car;

public class Main {

    public void showVisibility() {

        Car car;
        car = new Car();

        car.accelerate(100); //Error de compilación: el método accelerate de Car
no es visible
        System.out.printf("La velocidad del coche es %.2f km/h", car.getSpeed());

        car.refuel(40.35);
        System.out.printf("\nEl coche tiene %.2f litros de gasolina",
car.getGasoline());

    }

}
```

```

    public static void main(String[] args) {

        new Main().showVisibility();

    }

}

```

La solución sería que *Car* cambiara la visibilidad del método. Si una subclase quiere anular algún método de la superclase para cambiar la visibilidad, se permite únicamente si amplía la visibilidad, no si la reduce. La escala de valores de más restrictivo a menos es: *private*, *friendly*, *protected* y *public*. Por ejemplo, no se puede cambiar de *protected* a *friendly* pero sí al revés.

Solucionemos el ejemplo anterior para que la clase *Main* pueda acceder al método *accelerate* de *Car*. Si nos fijamos en la tabla de visibilidad, el único modificador que nos permite visibilidad desde otro paquete es *public*. Entonces, tendríamos que cambiar el *protected* a *public* y se permite porque se amplía la visibilidad, no se reduce:

```

package tema7_Herencia.visibilidad6;

import tema7_Herencia.visibilidad1.Vehicle;

public class Car extends Vehicle {

    private double gasoline;

    public double getGasoline() {
        return gasoline;
    }

    public void refuel(double liters) {
        gasoline += liters;
    }

    @Override
    //Anula el método heredado cambiando el modificador de acceso de protected a
    public
    public void accelerate(double amount) {
        speed += amount;
    }

}

```

```

package tema7_Herencia.visibilidad7;

import tema7_Herencia.visibilidad6.Car;

public class Main {

    public void showVisibility() {

        Car car;
        car = new Car();
    }
}

```

```

        car.accelerate(100);//No hay error de compilación, el método ya es
visible
        System.out.printf("La velocidad del coche es %.2f km/h", car.getSpeed());

        car.refuel(40.35);
        System.out.printf("\nEl coche tiene %.2f litros de gasolina",
car.getGasoline());
    }

    public static void main(String[] args) {

        new Main().showVisibility();
    }
}

```

Salida por consola:

```

La velocidad del coche es 100,00 km/h
El coche tiene 40,35 litros de gasolina

```

4. Super

A veces se requiere llamar a un método de la superclase. Eso se realiza con la palabra reservada **super**. En el [Tema 4. Programación Orientada a Objetos 14 This](#) vimos que *this* es una variable que hace referencia al objeto actual, pues *super* es una variable que hace referencia a la superclase del objeto actual, por lo tanto es un método imprescindible para poder acceder a métodos anulados por herencia.

```

package tema7_Herencia.usoSuper;

public class Car extends vehicle {

    private double gasoline;

    public double getGasoline() {
        return gasoline;
    }

    public void refuel(double liters) {
        gasoline += liters;
    }

    @Override
    public void accelerate(double amount) {
        super.accelerate(amount);//Uso de super para llamar al método del padre
        gasoline -= amount * 0.01;
    }
}

```

```

package tema7_Herencia.usoSuper;

public class Main {

    public void showUsoSuper() {

        Car car;
        car = new Car();

        car.refuel(40.35);
        System.out.printf("El coche tiene %.2f litros de gasolina",
car.getGasoline());
        System.out.printf(" y va a %.2f km/h", car.getSpeed());
        car.accelerate(100);
        System.out.printf("\nEl coche tiene %.2f litros de gasolina",
car.getGasoline());
        System.out.printf(" y va a %.2f km/h", car.getSpeed());

    }

    public static void main(String[] args) {

        new Main().showUsoSuper();

    }

}

```

Salida por consola:

```

El coche tiene 40,35 litros de gasolina y va a 0,00 km/h
El coche tiene 39,35 litros de gasolina y va a 100,00 km/h

```

En el ejemplo anterior, *super.accelerate(amount)* llama al método *accelerate* de la clase *Vehicle* el cual acelerará la marcha. Es necesario redefinir el método *accelerate* en la clase *Car* ya que aunque la velocidad varía igual que en la superclase, hay que tener en cuenta el consumo de gasolina.

5. Constructores

Los constructores no se heredan de la superclase a las subclases pero sí se pueden invocar los constructores de la superclase desde los constructores de las subclases mediante *super*.

```

package tema7_Herencia.constructores1;

public class Vehicle {

    protected int wheelCount;
    protected double speed;
    protected String colour;

    public Vehicle(int wheelCount, String colour) {
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }
}

```



```

    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }
}

```

```

package tema7_Herencia.constructores1;

public class Car extends Vehicle {

    private double gasoline;

    public Car(int wheelCount, String colour) {
        super(wheelCount, colour); //Uso de super para llamar al constructor de la
superclase
        gasoline = 0;
    }

    public double getGasoline() {
        return gasoline;
    }

    public void refuel(double liters) {
        gasoline += liters;
    }

    @Override
    public void accelerate(double amount) {
        super.accelerate(amount);
        gasoline -= amount * 0.01;
    }

}

```

```

package tema7_Herencia.constructores1;

public class Main {

    public void showConstructors1() {

        Vehicle vehicle;
        Car car;

        vehicle = new Vehicle(2, "azul");
        System.out.printf("Vehículo: %d ruedas y de color %s",
            vehicle.getWheelCount(), vehicle.getColour());

        car = new Car(4, "rojo");
        System.out.printf("\nCoche: %d ruedas y de color %s",
            car.getWheelCount(), car.getColour());

    }

    public static void main(String[] args) {

        new Main().showConstructors1();

    }

}

```

Salida por consola:

```

Vehículo: 2 ruedas y de color azul
Coche: 4 ruedas y de color rojo

```

Si una clase no tiene constructor, Java crea uno por defecto. Pero en el caso de que sea una subclase, Java lo crea con la línea de código *super()*, es decir, con una llamada al constructor de la superclase.

```

package tema7_Herencia.constructores2;

public class Vehicle {

    protected int wheelCount;
    protected double speed;
    protected String colour;

    //No tiene constructor, por lo que Java lo crea por defecto: Vehicle()

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

}

```

```

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }
}

```

```

package tema7_Herencia.estructuras2;

public class Car extends Vehicle {

    private double gasoline;
    /*
     * No tiene constructor. Java lo crea por defecto con la línea de
     * código super(), es decir, con una llamada al constructor por
     * defecto de la superclase. Por lo tanto, está llamando a vehicle().
     */

    public double getGasoline() {
        return gasoline;
    }

    public void refuel(double liters) {
        gasoline += liters;
    }

    @Override
    public void accelerate(double amount) {
        super.accelerate(amount);
        gasoline -= amount * 0.01;
    }
}

```

```

package tema7_Herencia.estructuras2;

public class Main {

    public void showConstructors2() {

        Vehicle vehicle;
        Car car;
    }
}

```

```

        vehicle = new Vehicle();//Constructor creado por Java
        car = new Car();//Constructor creado por Java que contiene super()
        vehicle.accelerate(3);
        car.accelerate(100);
        System.out.printf("El vehículo va a %.2f km/h", vehicle.getSpeed());
        System.out.printf("\nEl coche va a %.2f km/h", car.getSpeed());

    }

    public static void main(String[] args) {

        new Main().showConstructors2();

    }

}

```

Salida por consola:

```

El vehículo va a 3,00 km/h
El coche va a 100,00 km/h

```

Pero si la superclase tuviera un constructor con parámetros, Java ya no crearía el constructor por defecto *Vehicle()* y a las subclases les daría un error de compilación con la llamada del *super()*:

```

package tema7_Herencia.constructores3;

public class Vehicle {

    protected int wheelCount;
    protected double speed;
    protected String colour;

    /*
     * Tiene un constructor con parámetros, por lo tanto, Java no crea
     * el constructor por defecto vehicle()
     */
    public Vehicle(int wheelCount, String colour) {
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

}

```

```

    public void setColour(String colour) {
        this.colour = colour;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }
}

```

```

package tema7_Herencia.constructores3;

public class Car extends Vehicle {

    private double gasoline;

    /*
     * No tiene constructor. Java lo crea por defecto con la línea de
     * código super(), es decir, con una llamada al constructor por
     * defecto de la superclase. Por lo tanto, está llamando a Vehicle(),
     * dando un error de compilación ya que no se ha creado.
     */

    public double getGasoline() {
        return gasoline;
    }

    public void refuel(double liters) {
        gasoline += liters;
    }

    @Override
    public void accelerate(double amount) {
        super.accelerate(amount);
        gasoline -= amount * 0.01;
    }

}

```

Incluso aunque se defina un constructor con parámetros en la clase *Car*, Java sigue añadiendo de manera implícita la llamada *super()* en el constructor y como no hay constructor por defecto en la clase *Vehicle*, continúa dando error de compilación.

```

package tema7_Herencia.constructores4;

public class Car extends Vehicle {

    private double gasoline;

```

```

/*
 * Java sigue añadiendo de manera implícita la llamada super()
 * en el constructor y como no hay constructor por defecto en
 * la clase Vehicle, da un error de compilación
 */
public Car(int wheelCount, String colour) { //Error de compilación
    this.wheelCount = wheelCount;
    this.colour = colour;
    speed = 0;
    gasoline = 0;
}

public double getGasoline() {
    return gasoline;
}

public void refuel(double liters) {
    gasoline += liters;
}

@Override
public void accelerate(double amount) {
    super.accelerate(amount);
    gasoline -= amount * 0.01;
}
}

```

Se puede arreglar realizando una llamada explícita al constructor con parámetros de la superclase:

```

package tema7_Herencia.constructores5;

public class Car extends Vehicle {

    private double gasoline;

    public Car(int wheelCount, String colour) {
        super(wheelCount, colour); //Está llamando al constructor con parámetros
de Vehicle
        gasoline = 0;
    }

    public double getGasoline() {
        return gasoline;
    }

    public void refuel(double liters) {
        gasoline += liters;
    }

    @Override
    public void accelerate(double amount) {
        super.accelerate(amount);
        gasoline -= amount * 0.01;
    }
}

```

```
}  
  
}
```

Si una subclase tiene más constructores, este problema habría que arreglarlo en cada uno de ellos:

```
package tema7_Herencia.constructores6;  
  
public class Car extends vehicle {  
  
    private double gasoline;  
  
    public Car(int wheelCount, String colour) {  
        super(wheelCount, colour);  
        gasoline = 0;  
    }  
  
    /*  
     * Java sigue añadiendo de manera implícita la llamada super()  
     * en el constructor y como no hay constructor por defecto en  
     * la clase vehicle, da un error de compilación  
     */  
    public Car(int wheelCount) { //Error de compilación  
        this.wheelCount = wheelCount;  
        this.colour = "blanco";  
        speed = 0;  
        gasoline = 0;  
    }  
  
    public double getGasoline() {  
        return gasoline;  
    }  
  
    public void refuel(double liters) {  
        gasoline += liters;  
    }  
  
    @Override  
    public void accelerate(double amount) {  
        super.accelerate(amount);  
        gasoline -= amount * 0.01;  
    }  
  
}
```

Se puede solucionar de dos maneras:

1. Haciendo una llamada a un constructor de la superclase mediante super.
2. Haciendo una llamada a un constructor de la propia clase mediante this.

En ambos casos, tienen que ser la primera instrucción del constructor, por lo que el uso de super y this no puede ser simultáneo, lo que significa que hay que elegir entre ambas.

Veamos en código las dos soluciones:

1. Haciendo una llamada a un constructor de la superclase mediante super:

```
package tema7_Herencia.constructores7;

public class Car extends Vehicle {

    private double gasoline;

    public Car(int wheelCount, String colour) {
        super(wheelCount, colour);
        gasoline = 0;
    }

    public Car(int wheelCount) {
        super(wheelCount, "blanco");//Está llamando al constructor de Vehicle
mediante super
        gasoline = 0;
    }

    public double getGasoline() {
        return gasoline;
    }

    public void refuel(double liters) {
        gasoline += liters;
    }

    @Override
    public void accelerate(double amount) {
        super.accelerate(amount);
        gasoline -= amount * 0.01;
    }

}
```

2. Haciendo una llamada a un constructor de la propia clase mediante this.

```
package tema7_Herencia.constructores8;

public class Car extends Vehicle {

    private double gasoline;

    public Car(int wheelCount, String colour) {
        super(wheelCount, colour);
        gasoline = 0;
    }

    public Car(int wheelCount) {
        this(wheelCount, "blanco");//Está llamando al otro constructor de Car
mediante this
    }

    public double getGasoline() {
        return gasoline;
    }

}
```



```

    }

    public void refuel(double liters) {
        gasoline += liters;
    }

    @Override
    public void accelerate(double amount) {
        super.accelerate(amount);
        gasoline -= amount * 0.01;
    }
}

```

6. El método toString

La clase *Object* es la clase raíz de todo el árbol de la jerarquía de clases Java, es decir, es una superclase implícita de todas las demás clases. En otras palabras, todas las demás clases son subclases de *Object*. Esto significa que una variable de referencia de tipo *Object* puede referirse a un objeto de cualquier otra clase.

La clase *Object* proporciona un cierto número de métodos de utilidad general que pueden utilizar todos los objetos ya que los heredan. Pero normalmente hay que sobrescribirlos para que funcionen adecuadamente adaptándolos a la clase correspondiente. Esto se hace con la idea de que todas las clases utilicen el mismo nombre y prototipo de método para hacer operaciones comunes. Como por ejemplo, `toString()` que se utiliza para obtener una cadena de texto que represente al objeto. El método `toString()` de la clase *Object* devuelve una cadena que consiste en el nombre de la clase del objeto, el carácter arroba '@' y la representación hexadecimal sin signo del código hash del objeto. Siempre se recomienda sobrescribir el método `toString()` para obtener nuestra propia representación del objeto.

```

package tema7_Herencia.toString;

public class Vehicle {

    protected int wheelCount;
    protected double speed;
    protected String colour;

    public Vehicle(int wheelCount, String colour) {
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {

```

```

        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }

    @Override
    public String toString() { //Se anula el toString de Object
        return String.format("Número de ruedas: %d, color: %s, velocidad: %.2f",
            wheelCount, colour, speed);
    }
}

```

```

package tema7_Herencia.toString;

public class Car extends Vehicle {

    private double gasoline;

    public Car(int wheelCount, String colour) {
        super(wheelCount, colour);
        gasoline = 0;
    }

    public double getGasoline() {
        return gasoline;
    }

    public void refuel(double liters) {
        gasoline += liters;
    }

    @Override
    public void accelerate(double amount) {
        super.accelerate(amount);
        gasoline -= amount * 0.01;
    }

    @Override
    public String toString() { //Se anula el toString de Vehicle y se llama al
        super.toString
        return String.format("%s, gasolina: %.2f", super.toString(), gasoline);
    }
}

```

```

package tema7_Herencia.toString;

public class Main1 {

    public void showToString() {

        Vehicle vehicle;
        Car car;
        vehicle = new Vehicle(2, "azul");
        vehicle.accelerate(200);
        System.out.println(vehicle.toString());
        car = new Car(4, "rojo");
        car.refuel(100);
        car.accelerate(45);
        System.out.println(car.toString());

    }

    public static void main(String[] args) {

        new Main1().showToString();

    }

}

```

Salida por consola:

```

Número de ruedas: 2, color: azul, velocidad: 200,00
Número de ruedas: 4, color: rojo, velocidad: 45,00, gasolina: 99,55

```

Cuando se muestra por consola un objeto y solo se pone el nombre de la variable del objeto, se llama al toString del objeto:

```

package tema7_Herencia.toString;

public class Main2 {

    public void showToString() {

        Vehicle vehicle;
        Car car;
        vehicle = new Vehicle(2, "azul");
        vehicle.accelerate(200);
        System.out.println(vehicle); //Se llama al toString del objeto
        car = new Car(4, "rojo");
        car.refuel(100);
        car.accelerate(45);
        System.out.println(car); //Se llama al toString del objeto

    }

    public static void main(String[] args) {

```

```
new Main2().showToString();

}

}
```

7. Casting de objetos

Como ocurre con los tipos primitivos, también es posible realizar casting entre objetos siempre y cuando esté dentro de la estructura jerárquica en su herencia.

Para crear un objeto, hay que declarar una variable cuyo tipo es una clase:

```
vehicle vehicle; //vehicle es una variable de tipo vehicle
```

Eso significa que *vehicle* es de tipo *Vehicle* y que nunca va a cambiar de tipo, siempre va a ser de tipo *Vehicle*.

Creemos ahora una variable de tipo *Car*:

```
car car; //car es una variable de tipo Car
```

Lo mismo que antes, *car* es una variable de tipo *Car* y nunca va a cambiar de tipo, siempre va a ser de tipo *Car*.

A la variable *vehicle* se le puede asignar un objeto de tipo *Vehicle* y también se le pueden asignar objetos cuyo tipo pertenezca a cualquiera de sus descendientes (hijos, nietos, bisnietos, tataranietos, etc):

```
vehicle = car; //A vehicle se le está asignando un objeto de tipo Car, que es una
subclase de vehicle
```

La variable *vehicle* contiene un objeto de tipo *Car* pero su tipo es *Vehicle*, entonces solamente podrá acceder a los atributos y métodos de *Vehicle*:

```
vehicle.refuel(50); //Error de compilación: el método refuel no está definido para
el tipo vehicle
```

La variable *vehicle* no puede acceder al método *refuel* porque dicho método está definido en la clase *Car*. Es decir, una variable padre puede contener un objeto de tipo hijo pero solamente podrá acceder a los atributos y métodos definidos en el padre.

¿y si la asignación la hiciéramos al revés, es decir, a *car* le asignamos *vehicle*?

```
car = vehicle; //Error de compilación: falta de coincidencia de tipos, no puede
convertir de vehicle a Car
```

Daríamos un error de coincidencia de tipos, pero se podría solucionar con un casting:

```
car = (Car) vehicle; //Solucionado con un casting ya que vehicle contiene un Car
```

Hay que tener en cuenta que para que el casting funcione, la variable *vehicle* debe contener un objeto de tipo *Car*, porque si no, dará un error de ejecución *ClassCastException*.

```
vehicle = new Vehicle(2, "blanco");
car = (Car) vehicle; //Error de ejecución: ClassCastException ya que vehicle no
contiene un objeto de tipo Car
```

Veamos en el siguiente código todos los ejemplos anteriores:

```
package tema7_Herencia.castingObjetos;

public class Main {

    public void showObjectCasting() {

        Vehicle vehicle;
        Car car;

        car = new Car(4, "rojo");
        System.out.printf("Coche: %d ruedas y de color %s", car.getWheelCount(),
car.getColour());

        vehicle = car; //A vehicle se le está asignando un objeto de tipo Car, que
es una subclase de vehicle
        System.out.printf("\nCoche: %d ruedas y de color %s",
vehicle.getWheelCount(), vehicle.getColour());
        vehicle.refuel(50); //Error de compilación: el método refuel no está
definido para el tipo Vehicle

        car = vehicle; //Error de compilación: falta de coincidencia de tipos, no
puede convertir de vehicle a Car
        car = (Car) vehicle; //Solucionado con un casting ya que vehicle contiene
un Car

        vehicle = new Vehicle(2, "blanco");
        car = (Car) vehicle; //Error de ejecución: ClassCastException ya que
vehicle no contiene un objeto de tipo Car
    }

    public static void main(String[] args) {

        new Main().showObjectCasting();

    }

}
```

8. instanceof

El operador *instanceof* permite comprobar si un determinado objeto pertenece a una clase concreta. Se utiliza de esta forma:

```
objeto instanceof clase
```

Devuelve *true* si el objeto pertenece a dicha clase.

```
package tema7_Herencia.instanceofObjetos;

public class Main {

    public void showInstanceof() {

        Vehicle vehicle;
        Car car;

        vehicle = new Vehicle(2, "azul");
        car = new Car(4, "rojo");
        System.out.println(vehicle instanceof Vehicle);//true
        System.out.println(vehicle instanceof Car);//false
        System.out.println(car instanceof Car);//true
        System.out.println(car instanceof Vehicle);//true
        vehicle = car;
        System.out.println(vehicle instanceof Car);//true

    }

    public static void main(String[] args) {

        new Main().showInstanceof();

    }

}
```

Podemos observar en el ejemplo lo siguiente:

- *car* también devuelve *true* con *Vehicle* ya que los objetos de las subclases también devuelven *true* con la superclase.
- `vehicle instanceof Car` devuelve *true* solamente si el objeto que contiene es un *Car*.

No es aconsejable programar en una superclase de esta forma:

```
public class Vehicle {

    ...

    public void accelerate(double amount) {

        if(this instanceof Car){//No es aconsejable esta manera de programar
            ...
        }
        else if(this instanceof Bike){
            ...
        }
    }

    ...

}
```

```
}
```

El motivo es porque una superclase puede tener muchísima descendencia (hijos, nietos, etc). Si en dicha superclase se hace referencia a la descendencia, cada vez que se cree un descendiente nuevo o que haya algún tipo de modificación en alguno, hay que estar modificando el padre. Es decir, en el padre no debe existir ningún tipo de referencia hacia su posible descendencia para que su código sea independiente.

En Java 14 se introduce una mejora en el `instanceof` para evitar el casting explícito después de comprobar el tipo de una variable. Con este cambio, Java automáticamente realiza el casting si la comprobación es verdadera. Esto se logra utilizando lo que se llama **Pattern Matching** para `instanceof`. Veamos un ejemplo de comparación de cómo se hacía antes de Java 14 y cómo se puede hacer a partir de Java 14:

```
package tema7_Herencia.instanceofObjetos;

public class Main2 {

    public void showInstanceof() {

        Vehicle vehicle = new Car(4, "rojo");
        Car car;

        if(vehicle instanceof Car) { //Antes de Java 14
            car = (Car) vehicle;
            car.refuel(50);
            System.out.println(car);
        }

        if(vehicle instanceof Car car2) { //A partir de Java 14
            car2.refuel(100);
            System.out.println(car2);
        }

    }

    public static void main(String[] args) {

        new Main2().showInstanceof();

    }

}
```

Salida por consola:

```
Número de ruedas: 4, color: rojo, velocidad: 0,00, gasolina: 50,00
Número de ruedas: 4, color: rojo, velocidad: 0,00, gasolina: 150,00
```

La variable `car3` solo existe dentro del bloque `if`, ni siquiera se puede usar en un bloque `else`:

```
package tema7_Herencia.instanceofObjetos;
```

```

public class Main3 {

    public void showInstanceof() {

        vehicle vehicle = new Car(4, "rojo");

        if(vehicle instanceof Car car) {
            car.refuel(100);
            System.out.println(car);
        }else {
            car.refuel(50); //Error de compilación
        }

    }

    public static void main(String[] args) {

        new Main3().showInstanceof();

    }

}

```

Se puede combinar con operadores lógicos:

```

package tema7_Herencia.instanceofObjetos;

public class Main4 {

    public void showInstanceof() {

        vehicle vehicle = new Car(4, "rojo");

        if(vehicle instanceof Car car && car.getGasoline()==0) {
            car.refuel(100);
            System.out.println(car);
        }

    }

    public static void main(String[] args) {

        new Main4().showInstanceof();

    }

}

```

9. Modificador final

El modificador *final* tiene varios usos en función de dónde se utilice:

- Delante de una variable en su declaración, crea una constante. La constante puede recibir el valor en tiempo de compilación o en tiempo de ejecución.

Ejemplo de constante en tiempo de compilación:

```
final double PI=3.141591;
```

Ejemplo de constante en tiempo de ejecución:

```
public class MyClass {  
  
    private final int NUMBER;  
  
    public MyClass(int n) {  
        NUMBER = n;  
    }  
  
}
```

La constante *NUMBER* recibe el valor en la construcción del objeto.

- Delante de una variable que referencia a un objeto: dicha variable no puede referenciar a otro objeto.

```
final Car car = new Car(4, "rojo");  
car = new Car(4, "blanco");//Error: la variable car no puede referenciar a  
otro objeto
```

- En los parámetros de un método: el valor del parámetro no puede cambiar dentro del método.

```
public void refuel(final double liters) {  
    liters++;//Error: no se puede modificar liters  
    gasoline += liters;  
}
```

- En la declaración de un método: dicho método no se puede anular por las subclases:

```
public class Vehicle {  
    ...  
    final public void accelerate(double amount) {//Este método no puede ser  
anulado por las subclases  
        speed += amount;  
    }  
    ...  
}
```

```

public class Car extends vehicle {
    ...
    @Override
    public void accelerate(double amount) { //No se puede anular el método
        accelerate de vehicle
        super.accelerate(amount);
        gasoline -= amount * 0.01;
    }
    ...
}

```

- En la definición de una clase: significa que esa clase no puede tener descendencia.

Veamos en el siguiente código algunos de los ejemplos anteriores:

```

package tema7_Herencia.usosFinal;

public class Vehicle {

    protected int wheelCount;
    protected double speed;
    protected String colour;

    public Vehicle(int wheelCount, String colour) {
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    final public void accelerate(double amount) { //Este método no puede ser
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }

}

```

```

package tema7_Herencia.usosFinal;

public class Car extends Vehicle {

    private double gasoline;

    public Car(int wheelCount, String colour) {
        super(wheelCount, colour);
        gasoline = 0;
    }

    public double getGasoline() {
        return gasoline;
    }

    public void refuel(final double liters) {
        liters++; //Error: no se puede modificar liters
        gasoline += liters;
    }

    @Override
    public void accelerate(double amount) { //No se puede anular el método
        // accelerate de Vehicle
        super.accelerate(amount);
        gasoline -= amount * 0.01;
    }

}

```

```

package tema7_Herencia.usosFinal;

public class Main {

    public void showUsosFinal() {

        final Car car = new Car(4, "rojo");
        car = new Car(4, "blanco"); //Error: la variable car no puede referenciar
        // a otro objeto

    }

    public static void main(String[] args) {

        new Main().showUsosFinal();

    }

}

```

10. Polimorfismo

Polimorfismo es la capacidad de un objeto de adquirir varias formas.

La sobrecarga de métodos es un tipo de **polimorfismo estático** porque se resuelve en tiempo de compilación el método apropiado a ser llamado basado en la lista de argumentos.

La anulación de métodos es un tipo de **polimorfismo dinámico** porque se resuelve en tiempo de ejecución atendiendo al tipo del objeto. Con una variable de tipo padre, si se utiliza un objeto del padre para invocar al método, entonces se ejecutará el método de la clase padre, pero si se utiliza un objeto de la clase hija para invocar al método, entonces se ejecutará el método de la clase hija.

Veamos un ejemplo de polimorfismo dinámico:

```
package tema7_Herencia.polimorfismoDinamico;

public class Main {

    public void showDynamicPolymorphism() {

        vehicle vehicle;//Variable de tipo padre

        vehicle = new Vehicle(2, "azul");//Objeto del padre
        vehicle.accelerate(100.39);//Método del padre
        System.out.printf("La velocidad del vehículo es %.2f km/h",
        vehicle.getSpeed());

        vehicle = new Car(4, "rojo");//Objeto del hijo
        vehicle.accelerate(50.89);//Método del hijo
        System.out.printf("\nLa velocidad del coche es %.2f km/h",
        vehicle.getSpeed());

    }

    public static void main(String[] args) {

        new Main().showDynamicPolymorphism();

    }

}
```

11. Clases abstractas

Cuando se crea una estructura con herencia, puede darse el caso de que algún método del padre no se pueda implementar porque los detalles de la implementación dependan de cada uno de los hijos. Entonces, dicho método se declara como abstracto en el padre y solamente se define su firma, no se implementa código en él. Los hijos pueden hacer dos cosas:

1. Implementar el código de dicho método.
2. Declararlo también como abstracto.

Si la clase contiene algún método abstracto, se convierte en una clase abstracta. Una clase abstracta puede contener métodos no abstractos pero al menos uno de los métodos debe ser abstracto.

Para indicar en Java que un método o una clase son abstractos, se utiliza la palabra reservada **abstract**.

Veamos un ejemplo donde el hijo implementa el código del método abstracto del padre:

```
package tema7_Herencia.clasesAbstractas;

public abstract class Vehicle { //Clase abstracta

    protected int wheelCount;
    protected double speed;
    protected String colour;

    public Vehicle(int wheelCount, String colour) {
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    public abstract void accelerate(double amount); //Método abstracto

    public void brake(double amount) {
        speed -= amount;
    }

}
```

```
package tema7_Herencia.clasesAbstractas;

public class Car extends Vehicle {

    private double gasoline;

    public Car(int wheelCount, String colour) {
        super(wheelCount, colour);
        gasoline = 0;
    }

    public double getGasoline() {
        return gasoline;
    }

}
```

```

    public void refuel(double liters) {
        gasoline += liters;
    }

    @Override
    public void accelerate(double amount) { //Implementación del método abstracto
        speed += amount;
        gasoline -= amount * 0.01;
    }
}

```

```

package tema7_Herencia.clasesAbstractas;

public class Main {

    public void showAbstractClass() {

        Car car;

        car = new Car(4, "rojo");

        System.out.printf("La velocidad del coche es %.2f km/h", car.getSpeed());
        car.refuel(40.35);
        System.out.printf("\nEl coche tiene %.2f litros de gasolina",
            car.getGasoline());

        car.accelerate(100);
        System.out.printf("\nLa velocidad del coche es %.2f km/h",
            car.getSpeed());
        System.out.printf("\nEl coche tiene %.2f litros de gasolina",
            car.getGasoline());

    }

    public static void main(String[] args) {

        new Main().showAbstractClass();

    }

}

```

Salida por consola:

```

La velocidad del coche es 0,00 km/h
El coche tiene 40,35 litros de gasolina
La velocidad del coche es 100,00 km/h
El coche tiene 39,35 litros de gasolina

```

Una clase abstracta no se puede instanciar, es decir, no se pueden crear objetos de ella haciendo uso del `new`:

```

package tema7_Herencia.clasesAbstractas;

```

```

public class Main2 {

    public void showAbstractClass() {

        vehicle vehicle;
        vehicle = new Vehicle(2, "azul");//Error de compilación: no se puede
instanciar el tipo vehicle
    }

    public static void main(String[] args) {

        new Main2().showAbstractClass();

    }

}

```

Haciendo uso del polimorfismo, se puede declarar una variable de una clase abstracta que referencie a un objeto hijo, de tal forma que si se invoca el método abstracto, se ejecutará la implementación realizada por el hijo de dicho método abstracto:

```

package tema7_Herencia.clasesAbstractas;

public class Main3 {

    public void showAbstractClass() {

        Vehicle vehicle;

        vehicle = new Car(4, "rojo");
        vehicle.accelerate(50.89);//Se ejecuta la implementación realizada en Car
del método abstracto accelerate
        System.out.printf("La velocidad del coche es %.2f km/h",
vehicle.getSpeed());

    }

    public static void main(String[] args) {

        new Main3().showAbstractClass();

    }

}

```

12. Interfaces

Una interfaz en Java es una colección de métodos abstractos, es decir, en una interfaz se especifica qué se debe hacer pero no cómo hacerlo. Serán las clases que implementen estas interfaces las que describen la lógica del comportamiento de los métodos.

Sirven para especificar un comportamiento que ciertas clases pueden seguir sin dictar cómo se debe implementar ese comportamiento.

A estos métodos abstractos no hace falta ponerles la palabra reservada *abstract*. Los métodos de una interfaz son públicos por defecto así que tampoco hace falta ponerles la palabra reservada *public*.

Una clase puede implementar más de una interfaz, lo que implica que debe realizar todos los métodos de cada una de ellas. Si algún método lo deja como abstracto, entonces se convierte en una clase abstracta.

En una interfaz también se pueden declarar constantes que luego puedan ser utilizadas por las clases que implementen dicha interfaz. A estas constantes de la interfaz no hace falta ponerles las palabras reservadas *static* ni *final*.

Una interfaz se define en un archivo con el mismo nombre de la interfaz y con extensión .java. Las clases que quieran implementarla, tienen que añadir la palabra reservada *implements* detrás del nombre de la clase.

Para crear una interface en el Eclipse, nos situamos en el paquete donde queramos crearla y en el botón derecho pulsamos *new* → *interface*.

Veamos un ejemplo de una interfaz que va a contener acciones que pueda realizar un vehículo, como por ejemplo, acelerar y frenar:

```
package tema7_Herencia.interfaces;

public interface ActionsVehicle {

    void accelerate(double amount);

    void brake(double amount);

}
```

La clase *Vehicle* implementa dicha interfaz por lo que el compilador le va a obligar a implementar ambos métodos:

```
package tema7_Herencia.interfaces;

public class Vehicle implements ActionsVehicle { //La clase Vehicle implementa la
    interfaz ActionsVehicle

    protected int wheelCount;
    protected double speed;
    protected String colour;

    public Vehicle(int wheelCount, String colour) {
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
```



```

        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    @Override
    public void accelerate(double amount) { //Método a implementar debido a la
    interfaz ActionsVehicle
        speed += amount;
    }

    @Override
    public void brake(double amount) { //Método a implementar debido a la interfaz
    ActionsVehicle
        speed -= amount;
    }
}

```

El operador *instanceof* también se puede usar con interfaces:

```
objeto instanceof interfaz
```

Devuelve *true* si el objeto pertenece a una clase que implemente la interfaz:

```
vehicle instanceof ActionsVehicle //true
```

12.1 Herencia entre interfaces

Las interfaces también pueden heredar de otras interfaces. En este caso, la clase que implemente la interfaz hija tendrá que realizar los métodos de la interfaz hija y los métodos de la interfaz padre.

En este caso, no ocurre como las clases que solamente pueden tener un único padre. En el caso de las interfaces, pueden tener más de un padre.

Veamos un ejemplo de interfaz que contenga métodos de un vehículo con motor de gasolina y que herede de la interfaz *ActionsVehicle*:

```

package tema7_Herencia.interfaces;

public interface GasolineMotor extends ActionsVehicle { //GasolineMotor hereda de
la interfaz ActionsVehicle

    double getGasoline();

    void refuel(double liters);

}

```

La clase *Car* va a implementar dicha interfaz por lo que tendrá que realizar los métodos de *GasolineMotor* y los métodos de *ActionsVehicle*:

```

package tema7_Herencia.interfaces;

public class Car implements GasolineMotor { //La clase Car implementa la interfaz
GasolineMotor

    private int wheelCount;
    private double speed;
    private String colour;
    private double gasoline;

    public Car(int wheelCount, String colour) {
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
        gasoline = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }

    @Override
    public void accelerate(double amount) { //Método a implementar debido a la
interfaz padre ActionsVehicle
        speed += amount;
        gasoline -= amount * 0.01;
    }
}

```

```

@Override
public void brake(double amount) { //Método a implementar debido a la interfaz
padre ActionsVehicle
    speed -= amount;
}

@Override
public double getGasoline() { //Método a implementar debido a la interfaz hija
GasolineMotor
    return gasoline;
}

@Override
public void refuel(double liters) { //Método a implementar debido a la
interfaz hija GasolineMotor
    gasoline += liters;
}
}

```

12.2 Métodos por defecto

A partir de Java 8, se pueden definir métodos con una implementación por defecto dentro de las interfaces. Las clases que implementan la interfaz heredan dicho método. También pueden anularlo o incluso convertirlo en abstracto.

Para indicar que un método es por defecto se utiliza la palabra reservada *default*.

Por ejemplo, si definimos la siguiente interfaz con el siguiente método por defecto:

```

public interface Interfaz1 {
    default void metodo1(String str) {
        // Implementación por defecto del metodo1.
    }
}

```

y ahora queremos crear una clase que implemente la interfaz, dicha clase no estará obligada a implementar el método `metodo1()`. Por ejemplo, el siguiente código es totalmente válido:

```

public class Clase1 implements Interfaz1 {
    /* No estamos obligados a implementar metodo1(), aunque
    tenemos la posibilidad de anularlo.*/
}

```

Pero ¿qué ocurre si una clase implementa dos interfaces que tienen el mismo método con implementación por defecto? En ese caso la clase estará obligada a anular dicho método, porque no se puede decidir qué implementación por defecto usar. Por ejemplo:

```

public interface Interfaz1 {
    default void metodo1(String str) {
        // Implementación por defecto de metodo1.
    }
}

```

```

public interface Interfaz2 {
    default void metodo1(String str) {
        // Implementación por defecto de metodo1.
    }
}

public class Clase1 implements Interfaz1, Interfaz2 {
    // Estamos obligados a deshacer la ambigüedad
    @Override
    void metodo1(String str) {
        // Implementación específica de metodo1 en Clase1
    }
}

```

Otro aspecto interesante es que un método con implementación por defecto no puede anular a un método de la clase `java.lang.Object`, ya que es la clase base de todas las clases.

12.3 Métodos estáticos

A partir de Java 8, las interfaces también puede contener métodos estáticos con implementación por defecto. Se usan sobre todo para métodos de utilidad. Veamos un ejemplo:

```

public interface Interfaz3 {
    default void print(String str) {
        // Se llama al método estático de la interfaz desde otro
        // método default de la interfaz.
        if (!isNull(str))
            System.out.println("Cadena: " + str);
    }
    static boolean isNull(String str) {
        System.out.println("Interface Null Check");
        return str == null ? true : "".equals(str) ? true : false;
    }
}

```

Un método estático con implementación por defecto de una interfaz, como el método `isNull()` anterior, puede ser llamado desde otro método de la propia interfaz, por ejemplo desde el método `print()`.

También puede ser llamado estáticamente usando el nombre de la interfaz:

```

boolean resultado = Interfaz3.isNull("abc");

```

Se puede llamar desde cualquier clase que tenga visibilidad, aunque no implemente la interfaz. Pero no puede ser llamado a través de una instancia de una clase que implemente la interfaz, por ejemplo, el siguiente código da un error de compilación:

```

Clase miObjeto = new Clase();
miObjeto.isNull("hola");//Error de compilación

```

No se pueden definir en una interfaz métodos estáticos con implementación que tengan la misma firma que métodos de la clase `java.lang.Object`, ya que es la clase base de todas las clases.

12.4 Métodos privados

A partir de Java 9, las interfaces pueden contener métodos privados, que sólo pueden ser llamados desde métodos *default* de dicha interfaz u otros métodos privados de la misma. Sirven, básicamente, para poder separar el código de métodos con implementación por defecto. Veamos un ejemplo:

```
public interface Interfaz4 {

    // Este método privado sólo puede ser llamado por métodos default
    // de la misma interfaz.
    private int randomNumberGenerator() {
        return (new Random()).nextInt(100);
    }

    default String method(String s) {
        // Un método default puede llamar a un método privado de la interfaz.
        return s + randomNumberGenerator();
    }

}
```

A partir de Java 9 también podemos definir en una interfaz métodos estáticos privados, que sólo podrán ser llamados desde otros métodos estáticos de la interfaz. Sirven, básicamente, para poder separar el código de métodos estáticos de la interfaz. Por ejemplo:

```
public interface Interfaz5 {

    private static String prefix(String p) {
        return p.equals("male")? "Mr. " : "Ms. ";
    }

    static String name(String n, String p) {
        return prefix(p) + n;
    }

}
```

12.5 Diferencias entre una interfaz y una clase abstracta

Pero entonces, si las interfaces pueden tener métodos con implementación por defecto y métodos privados ¿qué diferencia hay entre una interfaz con métodos por defecto y una clase abstracta? La diferencia principal es que una interfaz no tiene estado, es decir, no podemos almacenar atributos en ella, mientras que una clase abstracta sí.

12.6 Utilización de una interfaz como un tipo de datos

Al declarar una interfaz, se declara un nuevo tipo de datos, lo que significa que se puede declarar una variable cuyo tipo es una interfaz. Pero, ¿qué va a contener dicha variable? Puede contener un objeto de cualquier clase que implemente dicha interfaz. Con dicha variable, las acciones que se pueden realizar son los métodos de la interfaz.

```
package tema7_Herencia.interfaces;
```

```

public class Main1 {

    public void showInterfaces() {

        Vehicle vehicle;
        ActionsVehicle actionsVehicle;//El tipo de la variable es la interfaz
        ActionsVehicle

        vehicle = new Vehicle(2, "azul");
        System.out.printf("La velocidad del vehículo es %.2f km/h",
        vehicle.getSpeed());

        actionsVehicle = vehicle;//Se le asigna un objeto de la clase Vehicle,
        que implementa la interfaz ActionsVehicle
        actionsVehicle.accelerate(100.39);//Ejecuta un método de la interfaz
        System.out.printf("\nLa velocidad del vehículo es %.2f km/h",
        vehicle.getSpeed());

    }

    public static void main(String[] args) {

        new Main1().showInterfaces();

    }

}

```

Salida por consola:

```

La velocidad del vehículo es 0,00 km/h
La velocidad del vehículo es 100,39 km/h

```

Con una variable de tipo interfaz, las acciones que se pueden realizar son los métodos de la interfaz, es decir, con una variable de tipo *ActionsVehicle* lo que se puede realizar son los métodos *accelerate* y *brake* que son los métodos que pertenecen a la interfaz. Si intentáramos realizar un *getSpeed* que pertenece a la clase *Vehicle* y no se encuentra entre los métodos de la interfaz, nos daría un error de compilación:

```

package tema7_Herencia.interfaces;

public class Main2 {

    public void showInterfaces() {

        Vehicle vehicle;
        ActionsVehicle actionsVehicle;

        vehicle = new Vehicle(2, "azul");
        System.out.printf("La velocidad del vehículo es %.2f km/h",
        vehicle.getSpeed());

        actionsVehicle = vehicle;
    }
}

```

```

        actionsVehicle.accelerate(100.39);
        System.out.printf("\nLa velocidad del vehículo es %.2f km/h",
actionsVehicle.getSpeed());//Error de compilación: el método getSpeed no está
definido para el tipo ActionsVehicle

    }

    public static void main(String[] args) {

        new Main2().showInterfaces();

    }

}

```

El error de compilación se puede solucionar con un casting:

```

package tema7_Herencia.interfaces;

public class Main3 {

    public void showInterfaces() {

        Vehicle vehicle;
        ActionsVehicle actionsVehicle;

        vehicle = new Vehicle(2, "azul");
        System.out.printf("La velocidad del vehículo es %.2f km/h",
vehicle.getSpeed());

        actionsVehicle = vehicle;
        actionsVehicle.accelerate(100.39);
        System.out.printf("\nLa velocidad del vehículo es %.2f km/h", ((Vehicle)
actionsVehicle).getSpeed());//Casting para solucionar el error

    }

    public static void main(String[] args) {

        new Main3().showInterfaces();

    }

}

```

Salida por consola:

```

La velocidad del vehículo es 0,00 km/h
La velocidad del vehículo es 100,39 km/h

```

También podemos usar el casting para asignar una variable de tipo interfaz a una variable de tipo clase, siempre y cuando la variable de tipo interfaz contenga un objeto de dicha clase:

```
vehicle vehicle = (Vehicle) actionsVehicle;
```

El casting también se puede utilizar para asignar un array de tipo clase a un array de tipo interfaz, como por ejemplo `actionsVehicle = (ActionsVehicle[]) vehicle;`:

```
package tema7_Herencia.interfaces;

public class Main4 {

    public void showInterfaces() {

        vehicle vehicle[] = new Vehicle[3];
        ActionsVehicle actionsVehicle[];

        vehicle[0] = new Vehicle(2, "azul");
        vehicle[1] = new Vehicle(4, "rojo");
        vehicle[2] = new Vehicle(4, "blanco");
        for(int i=0; i<vehicle.length; i++) {
            System.out.printf("La velocidad del vehículo es %.2f km/h\n",
vehicle[i].getSpeed());
        }
        //Casting entre arrays clase-interfaz:
        actionsVehicle = (ActionsVehicle[]) vehicle;
        for(int i=0; i<vehicle.length; i++) {
            actionsVehicle[i].accelerate(i*40);
        }
        for(int i=0; i<vehicle.length; i++) {
            System.out.printf("La velocidad del vehículo es %.2f km/h\n",
vehicle[i].getSpeed());
        }

    }

    public static void main(String[] args) {

        new Main4().showInterfaces();

    }

}
```

Sin embargo al revés no se puede, es decir, no se puede asignar un array de tipo interfaz a un array de tipo clase, porque da un error `ClassCastException`:

```
vehicle = (Vehicle[]) actionsVehicle;//Error de ejecución ClassCastException
```

En Java, aunque Vehicle implemente ActionsVehicle, los arrays no son covariantes de esta manera. Si necesitáramos realizar algo así, tendríamos que hacer un casting en un bucle para cada elemento individual.

13. Clases anidadas

En Java se permite escribir una clase dentro de otra clase. La clase de dentro se llama clase **anidada** y la que la contiene, clase **contenedora o externa**.

Estas clases se utilizan con los siguientes propósitos:

- Agrupación de clases relacionadas.
- Control de visibilidad de las clases.
- Proximidad entre la definición y el uso de las clases.
- Definición de clases simples de ayuda o adaptación.
- Código más claro que evita el exceso de clases muy pequeñas que no necesitan conocer los usuarios de un paquete.

Las clases anidadas se dividen en dos categorías: las clases anidadas estáticas y las clases anidadas no estáticas o clases internas.

13.1 Clases anidadas estáticas

Son clases declaradas de tipo *static* que se comportan como una clase normal de Java pero que se encuentran dentro de otra clase. También se pueden usar dentro de una interfaz.

Desde la clase anidada estática solamente se pueden acceder a los atributos estáticos de la clase contenedora.

Se pueden crear objetos sin crear ningún objeto de la clase contenedora.

Para hacer referencia a una clase anidada estática hay que indicar también la clase contenedora: `ClaseContenedora.ClaseAnidada`.

Se usan principalmente con excepciones y enumerados.

```
package tema7_Herencia.clasesAnidadas;

public class ContainerClass {

    public static class StaticNestedClass {

        public void staticNestedMethod() {
            System.out.println("Clase anidada estática");
        }

    }

    public void containerMethod() {
        System.out.println("Clase externa o contenedora");
    }

}
```

```
package tema7_Herencia.clasesAnidadasEstaticas;

public class Main {
```

```

    public void showStaticNestedClass() {

        ContainerClass.StaticNestedClass nested = new
ContainerClass.StaticNestedClass();
        nested.staticNestedMethod();

        ContainerClass container = new ContainerClass();
        container.containerMethod();

    }

    public static void main(String[] args) {

        new Main().showStaticNestedClass();

    }

}

```

Salida por consola:

```

Clase anidada estática
Clase externa o contenedora

```

13.2 Clases anidadas no estáticas o clases internas

Las clases anidadas no estáticas o clases internas tienen acceso a todos los atributos y métodos de la clase contenedora, por lo tanto, para que exista un objeto de una clase interna es necesario que exista un objeto de la clase contenedora.

```

package tema7_Herencia.clasesInternas;

public class ContainerClass {

    private int numContainer = 10;

    public class InnerClass {

        public void innerMethod() {
            System.out.printf("Clase interna. Puede acceder a numContainer: %d",
numContainer);
        }

    }

    public void containerMethod() {
        InnerClass inner = new InnerClass();
        inner.innerMethod();
    }

}

```

```

package tema7_Herencia.clasesInternas;

```

```

public class Main {

    public void showInnerClass() {

        ContainerClass container = new ContainerClass();
        container.containerMethod();

    }

    public static void main(String[] args) {

        new Main().showInnerClass();

    }

}

```

Salida por consola:

```
Clase interna. Puede acceder a numContainer: 10
```

También se puede crear un objeto de la clase interna desde fuera de la clase externa siempre y cuando la clase interna sea visible:

```

package tema7_Herencia.clasesInternas;

public class Main2 {

    public void showInnerClass() {

        ContainerClass container = new ContainerClass();
        ContainerClass.InnerClass inner = container.new InnerClass();
        inner.innerMethod();

    }

    public static void main(String[] args) {

        new Main2().showInnerClass();

    }

}

```

Salida por consola:

```
Clase interna. Puede acceder a numContainer: 10
```

Se puede hacer una clasificación de las clases internas en función de dónde y cómo se utilicen:

- Clases miembro
- Clases dentro de un método.

- Clases dentro de un bloque.
- Clases anónimas.

13.2.1 Clases internas miembro

Se utilizan como atributos de la clase contenedora. Si se declaran como privadas, la clase contenedora es la única que conoce su existencia.

```
package tema7_Herencia.clasesInternasMiembro;

public class ContainerClass {

    private int num = 10;
    private InnerClass inner = new InnerClass();

    private class InnerClass {

        private int num = 20;

        public void innerMethod() {
            System.out.printf("Número de la contenedora: %d. Número de la
interna: %d", ContainerClass.this.num, num);
        }

    }

    public void containerMethod() {
        inner.innerMethod();
    }

}
```

```
package tema7_Herencia.clasesInternasMiembro;

public class Main {

    public void showInnerMemberClass() {

        ContainerClass container = new ContainerClass();
        container.containerMethod();

    }

    public static void main(String[] args) {

        new Main().showInnerMemberClass();

    }

}
```

Salida por consola:

Número de la contenedora: 10. Número de la interna: 20

Al usar *this* dentro de una clase interna, éste se refiere al objeto de la clase interna. Para poder referirse al objeto de la clase contenedora, hay que anteponerle al *this* el nombre de dicha clase, tal y como se puede observar en el ejemplo: `ContainerClass.this.num`.

13.2.2 Clases internas dentro de un método

Se definen dentro de un método de la clase contenedora por lo que solamente se pueden utilizar dentro de dicho método.

Se utilizan cuando el método intenta solucionar un problema y necesita apoyarse en una clase pero no se necesita que esta clase esté disponible fuera, por lo tanto, son clases que quedan fuera del diseño.

La clase interna tiene acceso a los métodos y atributos de la clase contenedora y a las variables locales y parámetros del método donde se la declara.

```
package tema7_Herencia.clasesInternasMetodo;

public class ContainerClass {

    private int attribute = 10;

    public void containerMethod(int parameter) {

        int localVariable = 20;

        class InnerClass {

            public void innerMethod() {
                System.out.printf("Clase interna a método--->\nAtributo de la
contenedora: %d", attribute);
                System.out.printf("\nVariable local: %d", localVariable);
                System.out.printf("\nParámetro: %d", parameter);
            }

        }

        InnerClass inner = new InnerClass();
        inner.innerMethod();
    }

}
```

```
package tema7_Herencia.clasesInternasMetodo;

public class Main {

    public void showInnerMethodClass() {

        ContainerClass container = new ContainerClass();
        container.containerMethod(30);
    }

}
```

```

    }

    public static void main(String[] args) {

        new Main().showInnerMethodClass();

    }

}

```

Salida por consola:

```

Clase interna a método--->
Atributo de la contenedora: 10
Variable local: 20
Parámetro: 30

```

13.2.3 Clases internas dentro de un bloque

Sólo son visibles y utilizables dentro del bloque de código en el que se encuentran definidas.

```

package tema7_Herencia.clasesInternasBloque;

public class ContainerClass {

    private int attribute = 10;

    public void containerMethod(int parameter) {

        int localVariable = 20;

        if (parameter > localVariable) {
            class InnerClass {

                public void innerMethod() {
                    System.out.printf("Clase interna a bloque--->\nAtributo de la
contenedora: %d", attribute);
                    System.out.printf("\nVariable local: %d", localVariable);
                    System.out.printf("\nParámetro: %d", parameter);
                }

            }
            InnerClass inner = new InnerClass();
            inner.innerMethod();
        }

    }

}

```

```

package tema7_Herencia.clasesInternasBloque;

public class Main {

```

```

    public void showInnerBlockClass() {

        ContainerClass container = new ContainerClass();
        container.containerMethod(30);

    }

    public static void main(String[] args) {

        new Main().showInnerBlockClass();

    }

}

```

Salida por consola:

```

Clase interna a bloque--->
Atributo de la contenedora: 10
variable local: 20
Parámetro: 30

```

Si se intenta utilizar la clase fuera del bloque, da un error de compilación informando que la clase no puede ser resuelta como un tipo:

```

package tema7_Herencia.clasesInternasBloque;

public class ContainerClass2 {

    private int attribute = 10;

    public void containerMethod(int parameter) {

        int localVariable = 20;

        if (parameter > localVariable) {
            class InnerClass {

                public void innerMethod() {
                    System.out.printf("Clase interna a bloque--->\nAtributo de la
contenedora: %d", attribute);
                    System.out.printf("\nvariable local: %d", localVariable);
                    System.out.printf("\nParámetro: %d", parameter);
                }

            }

            InnerClass inner = new InnerClass();//Error de compilación: InnerClass no
puede ser resuelto como un tipo
            inner.innerMethod();
        }

    }

}

```

```
}
```

13.2.4 Clases inline anónimas

Son clases sin nombre que se definen e instancian en una sola operación. Este tipo de clases se utiliza cuando se quiere anular el método de una clase o implementar una interfaz solamente para un momento puntual evitando crear una clase nueva para un solo uso. El término **inline** se debe a que en el cuerpo de un método se puede escribir una clase ahí mismo, en la línea, es decir, sin necesidad de hacerlo en otro archivo.

Por ejemplo, tenemos la siguiente clase *MyClass*:

```
package tema7_Herencia.clasesInlineAnonimas;

public class MyClass {

    protected String message = "Clases inline anónimas";

    public void showMessage() {
        System.out.println(message);
    }

}
```

Supongamos que queremos anular el método `showMessage()` para mostrar el mensaje en rojo en un momento puntual y creamos una subclase o clase hija de *MyClass* que sobrescribiera el método:

```
package tema7_Herencia.clasesInlineAnonimas;

import static tema1_11_EscrituraEnPantalla.colores.Colors.*;

public class Subclass extends MyClass {

    @Override
    public void showMessage() {
        System.out.println(RED + message + RESET);
    }

}
```

Entonces estaríamos creando una clase para algo que vamos a hacer solamente una vez. Y si quisiéramos en otro momento hacer lo mismo pero en azul, tendríamos que crear otra subclase para hacerlo. Entonces, la solución es crear una clase inline anónima. Se llama anónima porque en ningún momento aparece el nombre `Subclass`, es decir, estamos haciendo lo mismo que antes pero sin crear la subclase y lo estamos haciendo sobre la marcha:

```
package tema7_Herencia.clasesInlineAnonimas;

import static tema1_4_EscrituraEnPantalla.colores.Colors.RED;
import static tema1_4_EscrituraEnPantalla.colores.Colors.RESET;
```



```

public class Main {

    public void showAnonymousInnerClass() {

        MyClass anonymousInner = new MyClass() {
            @Override
            public void showMessage() {
                System.out.println(RED + message + RESET);
            }
        };
        anonymousInner.showMessage();

        new MyClass() { //Lo mismo pero sin crear la variable anonymousInner
            @Override
            public void showMessage() {
                System.out.println(RED + message + RESET);
            }
        }.showMessage();

    }

    public static void main(String[] args) {

        new Main().showAnonymousInnerClass();

    }

}

```

Salida por consola:

Clases inline anónimas

Clases inline anónimas

Veamos otro ejemplo pero con interfaces:

```

package tema7_Herencia.clasesInlineAnonimas;

public interface Message {

    void showMessage();

}

```

Creamos una clase que implemente la interfaz y que escriba el mensaje en rojo:

```

package tema7_Herencia.clasesInlineAnonimas;

import static tema1_11_EscrituraEnPantalla.colores.Colors.*;

public class MyClass2 implements Message {

    @Override
    public void showMessage() {
        System.out.println(RED + "Clases inline anónimas" + RESET);
    }

}

```

Pero igual que antes, si solamente vamos a hacer ésto una vez, estamos creando una clase para un solo uso. Incluso si quisiéramos hacer también lo mismo en azul, tendríamos que crear otra clase para ello.

Entonces, lo podemos hacer mediante una clase inline anónima:

```

package tema7_Herencia.clasesInlineAnonimas;

import static tema1_4_EscrituraEnPantalla.colores.Colors.BLUE;
import static tema1_4_EscrituraEnPantalla.colores.Colors.RED;
import static tema1_4_EscrituraEnPantalla.colores.Colors.RESET;

public class Main2 {

    public void showAnonymousInnerClass() {

        new Message() {
            @Override
            public void showMessage() {
                System.out.println(RED + "Clases inline anónimas" + RESET);
            }
        }.showMessage();

        new Message() {
            @Override
            public void showMessage() {
                System.out.println(BLUE + "Clases inline anónimas" + RESET);
            }
        }.showMessage();

    }

    public static void main(String[] args) {

        new Main2().showAnonymousInnerClass();

    }

}

```

Salida por consola:

14. Clases selladas

Las clases selladas (en inglés, *sealed classes*) permiten restringir qué clases pueden heredar de una clase. Son útiles para definir jerarquías de clases más controladas y seguras, mejorando el diseño de tipos y ayudando a evitar errores.

Esto se hace utilizando la palabra clave `sealed` en la declaración de la clase y `permits` para listar las clases que tienen permitido extenderla. Las clases que extienden una clase sellada deben ser marcadas como una de estas opciones:

- `final`: la clase no puede ser extendida más.
- `sealed`: la clase sigue siendo sellada y permite que otras clases la extiendan, pero también deben ser especificadas con `permits`.
- `non-sealed`: la clase puede ser extendida libremente sin restricciones.

Todas las subclases permitidas deben ser conocidas en tiempo de compilación y deben estar ubicadas en el mismo módulo o paquete.

Veamos un ejemplo de una clase sellada `Vehicle` que solamente permite que hereden de ella las clases `Car` y `Bike`:

```
package tema7_Herencia.clasesSelladas;

public sealed class Vehicle permits Car, Bike {

    protected int wheelCount;
    protected double speed;
    protected String colour;

    public Vehicle(int wheelCount, String colour) {
        this.wheelCount = wheelCount;
        this.colour = colour;
        speed = 0;
    }

    public int getWheelCount() {
        return wheelCount;
    }

    public double getSpeed() {
        return speed;
    }

    public String getColour() {
        return colour;
    }

    public void setColour(String colour) {
        this.colour = colour;
    }
}
```

```

    public void accelerate(double amount) {
        speed += amount;
    }

    public void brake(double amount) {
        speed -= amount;
    }

    @Override
    public String toString() {
        return String.format("Número de ruedas: %d, color: %s, velocidad: %.2f",
wheelCount, colour, speed);
    }
}

```

```

package tema7_Herencia.clasesSelladas;

public final class Car extends Vehicle {

    private double gasoline;

    public Car(int wheelCount, String colour) {
        super(wheelCount, colour);
        gasoline = 0;
    }

    public double getGasoline() {
        return gasoline;
    }

    public void refuel(double liters) {
        gasoline += liters;
    }

    @Override
    public void accelerate(double amount) {
        super.accelerate(amount);
        gasoline -= amount * 0.01;
    }

    @Override
    public String toString() {
        return String.format("Coche -> %s, gasolina: %.2f", super.toString(),
gasoline);
    }
}

```

```

package tema7_Herencia.clasesSelladas;

public final class Bike extends Vehicle {

```

```

public Bike(int wheelCount, String colour) {
    super(wheelCount, colour);
}

@Override
public String toString() {
    return String.format("Bicicleta -> %s", super.toString());
}

}

```

Si se intenta crear una motocicleta que herede de `Vehicle`, nos daría error de compilación:

```

package tema7_Herencia.clasesSelladas;

public final class Motorbike extends Vehicle { //Error de compilación indicando
que no es una subclase permitida

    private double gasoline;

    public Motorbike(int wheelCount, String colour) {
        super(wheelCount, colour);
        gasoline = 0;
    }

    public double getGasoline() {
        return gasoline;
    }

    public void refuel(double liters) {
        gasoline += liters;
    }

    @Override
    public void accelerate(double amount) {
        super.accelerate(amount);
        gasoline -= amount * 0.05;
    }

    @Override
    public String toString() {
        return String.format("Moto -> %s, gasolina: %.2f", super.toString(),
gasoline);
    }

}

```

```

package tema7_Herencia.clasesSelladas;

public class Main {

    public void showSealedClasses() {

        Vehicle v;
    }
}

```

```

        v = new Car(4, "rojo");
        System.out.println(v);
        v = new Bike(2, "verde");
        System.out.println(v);
        v = new Motorbike(2, "azul");//Error de ejecución
        System.out.println(v);

    }

    public static void main(String[] args) {

        new Main().showSealedClasses();

    }

}

```

Salida por consola que aborta con un error de ejecución:

```

Coche -> Número de ruedas: 4, color: rojo, velocidad: 0,00, gasolina: 0,00
Bicicleta -> Número de ruedas: 2, color: verde, velocidad: 0,00
Exception in thread "main" java.lang.IncompatibleClassChangeError: class
tema7_Herencia.clasesSelladas.Motorbike cannot inherit from sealed class
tema7_Herencia.clasesSelladas.Vehicle
    at java.base/java.lang.ClassLoader.defineClass1(Native Method)
    at java.base/java.lang.ClassLoader.defineClass(ClassLoader.java:1027)
    at
java.base/java.security.SecureClassLoader.defineClass(SecureClassLoader.java:150)
    at
java.base/jdk.internal.loader.BuiltinClassLoader.defineClass(BuiltinClassLoader.j
ava:862)
    at
java.base/jdk.internal.loader.BuiltinClassLoader.findClassOnClassPathOrNull(Built
inClassLoader.java:760)
    at
java.base/jdk.internal.loader.BuiltinClassLoader.loadClassOrNull(BuiltinClassLoad
er.java:681)
    at
java.base/jdk.internal.loader.BuiltinClassLoader.loadClass(BuiltinClassLoader.jav
a:639)
    at
java.base/jdk.internal.loader.ClassLoaders$AppClassLoader.loadClass(ClassLoaders.
java:188)
    at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:526)
    at tema7_Herencia.clasesSelladas.Main.showSealedClasses(Main.java:13)
    at tema7_Herencia.clasesSelladas.Main.main(Main.java:20)

```

15. Interfaces selladas

Las interfaces selladas (en inglés, *sealed interfaces*) permiten restringir qué clases pueden implementarla y qué interfaces pueden extenderla. Son útiles para definir jerarquías de comportamientos más controladas y seguras, mejorando el diseño de tipos y ayudando a evitar errores.

Para declarar una interfaz sellada, se utiliza la palabra clave `sealed` en la declaración de la interfaz y `permits` para listar las clases y/o interfaces que están permitidas implementarla o extenderla. Ejemplo:

```
public sealed interface ActionsVehicle permits Vehicle, GasolineMotor {  
  
    void accelerate(double amount);  
  
    void brake(double amount);  
  
}
```

Las clases que implementan una interfaz sellada deben ser marcadas como final, sealed o non-sealed. Las interfaces que heredan de una interfaz sellada deben ser marcadas como sealed o non-sealed.

8 Tipos enumerados

8 Tipos enumerados

- 1. Introducción
- 2. Definición
- 3. Constructores
- 4. Comparaciones
- 5. Enums en switch
- 6. Atributos
- 7. Métodos
 - 7.1 Método toString
 - 7.2 Método valueOf
 - 7.3 Método values
 - 7.4 Método ordinal
- 8. Métodos abstractos
- 9. Implementación de interfaces

1. Introducción

Un tipo enumerado (***enum type***) es un tipo cuyos únicos valores legales consisten en un conjunto fijo de constantes, como por ejemplo las estaciones del año o los palos en una baraja de cartas.

Antes de que los lenguajes nos permitieran definir tipos enumerados, éstos eran simulados declarando un grupo de constantes enteras, una para cada valor posible del tipo. Por ejemplo, para definir los tipos de manzanas y los tipos de naranjas podríamos hacer:

```
// Apple types.  
public static final int APPLE_FUJI = 0;  
public static final int APPLE_PIPPIN = 1;  
public static final int APPLE_GRANNY_SMITH = 2;  
  
// Orange types.  
public static final int ORANGE_NAVEL = 0;  
public static final int ORANGE_TEMPLE = 1;
```

Esta técnica, conocida como **el patrón de enumeración entera**, tiene muchos defectos, por lo que no se recomienda su uso.

Para empezar, al no definir realmente un tipo, el compilador no puede proporcionarnos seguridad real respecto al tipo. Por ejemplo, podemos pasar un tipo de naranja como argumento de un método que espera recibir un tipo de manzana, o almacenar un tipo de naranja en una variable que debe tener un tipo de manzana.

```
// Para el compilador esto es correcto.  
int appleType = ORANGE_TEMPLE;
```

No solo eso, podemos comparar erróneamente tipos de manzanas y tipos de naranjas.


```
// Para el compilador esta comparación es válida.
int appleType = APPLE_PIPPIN;
int orangeType = ORANGE_TEMPLE;
if (appleType == orangeType) {
    // ...
}
```

Como Java no proporciona espacios de nombre para grupos de constantes enteras, debemos usar prefijos (como *APPLE_* y *ORANGE_*) para evitar conflictos con los nombres de las constantes correspondientes a distintos grupos.

Otro problema es que si se cambia el valor asociado a una constante, todas las clases que la usen seguirán compilando, pero su comportamiento puede llegar a ser incorrecto.

Finalmente, el patrón de enumeración entera tiene el inconveniente de que no hay una forma fácil de traducir las constantes enteras a cadenas imprimibles, es decir, la descripción textual del valor que desea representar.

En otras ocasiones se usa una variante del patrón anterior, pero usando el tipo `String` como tipo para las constantes en lugar de `int`. Por ejemplo:

```
// Apple types.
public static final String APPLE_FUJI = "FUJI";
public static final String APPLE_PIPPIN = "PIPPIN";
public static final String APPLE_GRANNY_SMITH = "SMITH";

// Orange types.
public static final String ORANGE_NAVEL = "NAVEL";
public static final String ORANGE_TEMPLE = "TEMPLE";
```

Esta variación se conoce como **patrón de enumeración con cadenas, y es aún menos deseable que la anterior**, porque aunque proporciona cadenas imprimibles para sus constantes, los desarrolladores pueden tener la tentación de usar en otras clases los valores de cadena directamente en vez de las constantes, por lo que un simple error tipográfico puede conllevar fallos en tiempo de ejecución. Además, la comparación de cadenas es mucho más costosa que la de enteros.

La alternativa adecuada a los patrones de enumeración anteriores es que el lenguaje de programación nos permita **definir tipos enumerados**, es decir, que nos permita definir un nuevo tipo indicando cuáles son los valores legales para dicho tipo y con qué nombre queremos referirnos a dichos valores.

En algunos lenguajes de programación, estos tipos enumerados son internamente tipos enteros. Sin embargo, en Java cuando definimos un tipo enumerado, estamos definiendo una clase completa, lo que nos proporciona una mayor funcionalidad a la de otros lenguajes de programación. En la versión 5 de Java, se incorporaron al lenguaje los tipos de datos enumerados.

Así, siguiendo el ejemplo anterior, definiríamos las siguientes clases `enum`:

```
public enum Apple {  
    FUJI, PIPPIN, GRANNY_SMITH  
}  
  
public enum Orange {  
    NAVEL, TEMPLE  
}
```

Los *enums* presentan numerosas ventajas frente al uso de los patrones de enumeración entera y de enumeración con cadenas. La más importante es que la clase *enum* corresponderá a un tipo, por lo que el compilador va a ser capaz de proporcionar comprobación de tipos en tiempo de compilación. Por ejemplo, si se declara que un parámetro es del tipo `Apple`, el compilador garantiza que cualquier referencia de objeto no nulo pasada al parámetro debe ser alguno de las instancias de `Apple` válidas que hemos definido en el *enum*. En caso contrario, se producirá un error de compilación. Ocurre exactamente lo mismo al asignar una expresión de un tipo de enumeración a una variable de un tipo *enum*, o al usar el operador `==` para comparar valores de diferentes tipos *enum*.

Además, los tipos *enum* definen su propio espacio de nombres, por lo que no hay ningún problema en usar constantes con el mismo nombre en distintos *enums*.

Otra ventaja es que si cambiamos el orden de las constantes en el *enum*, esto no afecta al código de los clientes del *enum*, dado que las constantes no son compiladas en el código cliente, a diferencia de como ocurría en los patrones anteriores.

2. Definición

Los enumerados se definen con la palabra reservada **enum**, el nombre del enumerado y luego el conjunto de las constantes, que por las convenciones del lenguaje se escriben en mayúscula. Las constantes de la enumeración son *public* y *static* de forma implícita. Ejemplo:

```
public enum DayOfWeek {  
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY  
}
```

En el *Package Explorer* del *Eclipse*, hay que colocarse en el paquete donde se quiera crear el *enum*. Luego se pulsa el botón derecho del ratón y *New* → *Enum*.

Por otra parte, ¿dónde debemos definir un *enum*? Si va a ser usado desde varios lugares, lo lógico es definir el *enum* como una clase en su propio fichero. Si su uso está restringido a una única clase cliente, es más razonable definir el *enum* como una clase interna miembro de dicha clase.

Una vez definido el *enum*, se pueden crear variables de ese tipo:

```
DayOfWeek day; //day es una variable del tipo de enumeración DayOfWeek
```

Como *day* es de tipo *DayOfWeek*, los únicos valores que se le pueden asignar son los definidos por la enumeración:

```
day = DayOfWeek.MONDAY; //Se asigna a day el valor MONDAY
```

El compilador nos da un error si intentamos asignar a una variable *enum* un tipo que no le corresponde:

```
DayOfWeek day = Apple.FUJI; //Error de compilación: falta de correspondencia entre los tipos: no se puede convertir de Apple a DayOfWeek
```

3. Constructores

Los *enum* se implementan internamente como clases. El *enum* *DayOfWeek* internamente se convierte en:

```
class DayOfWeek {  
    public static final DayOfWeek MONDAY = new DayOfWeek();  
    public static final DayOfWeek TUESDAY = new DayOfWeek();  
    public static final DayOfWeek WEDNESDAY = new DayOfWeek();  
    ...  
}
```

Los *enum* de Java son clases que exportan una instancia (objeto) para cada constante de enumeración a través de un campo final estático público. Estas instancias son creadas automáticamente al hacer referencia al *enum* en nuestro código, es decir, el constructor se ejecuta para cada constante en el momento de la carga de la clase *enum*. No podemos crear objetos *enum* explícitamente y, por lo tanto, no podemos invocar al constructor directamente:

```
DayOfWeek day = new DayOfWeek(); //Error de compilación: no se puede instanciar el tipo DayOfWeek
```

Los únicos modificadores de acceso permitidos para los constructores de los *enum* son *friendly* y *private*. Los modificadores de acceso *protected* y *public* dan un error de compilación.

Como vemos, los `enum` de Java son una generación del patrón de diseño *singleton* permitiendo una determinada serie de instancias, cada una de las cuales es accesible a través de una constante. Visto de forma inversa, podríamos decir que un *singleton* no es más que un `enum` con una sola instancia.

4. Comparaciones

Los *enum* se pueden comparar utilizando el operador relacional `==`:

```
if(day == DayOfWeek.MONDAY) { //Si day contiene MONDAY, se ejecuta el bloque if  
    ...  
}
```

```
DayOfWeek day = DayOfWeek.MONDAY;  
System.out.println(day == DayOfWeek.MONDAY ? true : false); //Imprime true
```

El compilador nos da un error si intentamos comparar valores de diferentes tipos *enum*:

```
DayOfWeek day = DayOfWeek.MONDAY;
Apple apple = Apple.FUJI;
if(day == apple) {//Error de compilación: tipos de operandos incompatibles
    ...
}
```

5. Enums en switch

También podemos utilizar **switch** para comprobar en los **case** los distintos valores del *enum*:

```
switch (day) {
    case MONDAY -> System.out.println("Lunes");
    case TUESDAY -> System.out.println("Martes");
    ...
}
```

No es necesario calificar las constantes en las declaraciones de *case* con su nombre de tipo *enum*. De hecho, intentar hacerlo provocará un error de compilación:

```
switch (day) {
    case DayOfWeek.MONDAY -> System.out.println("Lunes");//Error de compilación
    case TUESDAY -> System.out.println("Martes");
    ...
}
```

6. Atributos

Los *enums* no dejan de ser clases, por lo que podemos declararles atributos, que deben ser **final**, ya que los *enums* son inmutables por naturaleza. Aunque estos atributos pueden ser *public*, se recomienda definirlos como **private** y definir los *getters* correspondientes si es necesario acceder a los atributos desde fuera del *enum*. En los *enums* se pueden definir métodos *setters* pero no es aconsejable.

Para establecer el valor de estos atributos, debemos definir en la clase *enum* el constructor adecuado y al establecer las instancias pasaremos como argumento los valores deseados que se pasarán al constructor. En el ejemplo, cuando definimos *PLUS*, le ponemos entre paréntesis el carácter '+'. Cuando se cree el objeto para *PLUS*, se ejecuta el constructor `Operation(char symbol)` y en *symbol* se pasa '+':

```
package tema8_TiposEnumerados;

public enum Operation {

    PLUS('+'), MINUS('-'), TIMES('*'), DIVIDE('/');//El ; es necesario cuando se definen atributos

    private final char symbol;

    Operation(char symbol) {
        this.symbol = symbol;
    }
}
```

```

        public char getSymbol() {
            return symbol;
        }
    }
}

```

```

package tema8_TiposEnumerados;

public class EnumAttributes {

    public void show() {

        Operation operation1 = Operation.PLUS;
        Operation operation2 = Operation.MINUS;
        System.out.printf("El atributo de PLUS es una cadena con el valor %c",
            operation1.getSymbol());
        System.out.printf("\nEl atributo de MINUS es una cadena con el valor %c",
            operation2.getSymbol());

    }

    public static void main(String[] args) {

        new EnumAttributes().show();

    }

}

```

Salida por consola:

```

El atributo de PLUS es una cadena con el valor +
El atributo de MINUS es una cadena con el valor -

```

7. Métodos

Una enumeración no puede heredar de otra clase ni puede actuar como superclase de otra clase, pero por defecto, hereda de la clase `java.lang.Enum<E>` y por tanto tiene una serie de métodos heredados, como por ejemplo los métodos `toString()` y `ordinal()`.

Además, todas las enumeraciones tienen automáticamente dos métodos estáticos predefinidos: `values()` y `valueOf(String str)`.

7.1 Método toString

Las clases *enum* proporcionan de serie implementaciones de alta calidad de todos los métodos de la clase `Object` (la clase `Enum` hereda de `Object`), como el método `public String toString()`, que retornará el nombre de la constante asociada a la instancia correspondiente. Por ejemplo `Apple.FUJI.toString()` retorna la cadena `FUJI`, aunque si no estamos contentos con dicha implementación, podemos sobrescribir nosotros dicho método. Además, las clases *enum* implementan de serie las interfaces `Comparable` y `Serializable`.

```

package tema8_TiposEnumerados;

public class EnumToString {

    public void show() {

        Operation operation1 = Operation.PLUS;
        Operation operation2 = Operation.MINUS;
        System.out.println(operation1.toString()); //PLUS
        System.out.println(operation2.toString()); //MINUS
        System.out.println(Operation.PLUS.toString()); //PLUS
        System.out.println(Operation.MINUS.toString()); //MINUS
    }

    public static void main(String[] args) {

        new EnumToString().show();

    }

}

```

De hecho, por defecto se llama al método `toString` cuando se imprime un *enum* o una variable de tipo *enum*:

```

package tema8_TiposEnumerados;

public class EnumPrinting {

    public void show() {

        Operation operation1 = Operation.PLUS;
        Operation operation2 = Operation.MINUS;
        System.out.println(operation1); //PLUS
        System.out.println(operation2); //MINUS
        System.out.println(Operation.PLUS); //PLUS
        System.out.println(Operation.MINUS); //MINUS
    }

    public static void main(String[] args) {

        new EnumPrinting().show();

    }

}

```

7.2 Método valueOf

`public static E valueOf(String name)` es un método declarado implícitamente que devuelve la instancia del *enum* que posee asociado el nombre de constante recibida.

```

package tema8_TiposEnumerados;

```

```

public class EnumValueOf1 {

    public void show() {

        Operation operation;
        operation = Operation.valueOf("PLUS");//operation se asigna con la
instancia correspondiente a la constante de enumeración PLUS
        System.out.printf("La variable operation es de tipo enum %s y su símbolo
es %c", operation,
            operation.getSymbol());

    }

    public static void main(String[] args) {

        new EnumValueOf1().show();

    }

}

```

Salida por consola:

```
La variable operation es de tipo enum PLUS y su símbolo es +
```

Si el argumento cadena que recibe el método *valueOf* no corresponde a ninguna constante de enumeración, el método lanza la excepción `IllegalArgumentException`:

```

package tema8_TiposEnumerados;

public class EnumValueOf2 {

    public void show() {

        Operation operation;
        operation = Operation.valueOf("PLUS1");//PLUS1 no se corresponde con
ninguna constante de enumeración
        System.out.printf("La variable operation es de tipo enum %s y su símbolo
es %c", operation,
            operation.getSymbol());

    }

    public static void main(String[] args) {

        new EnumValueOf2().show();

    }

}

```

Salida por consola:

```
Exception in thread "main" java.lang.IllegalArgumentException: No enum constant
tema8_TiposEnumerados.Operation.PLUS1
    at java.base/java.lang.Enum.valueOf(Enum.java:264)
    at tema8_TiposEnumerados.Operation.valueOf(Operation.java:1)
    at tema8_TiposEnumerados.EnumValueOf2.show(EnumValueOf2.java:8)
    at tema8_TiposEnumerados.EnumValueOf2.main(EnumValueOf2.java:16)
```

7.3 Método values

`public static E[] values()` es un método declarado implícitamente que devuelve un array con todas las instancias de la clase *enum* en el orden en que fueron declarados.

```
package tema8_TiposEnumerados;

public class EnumValues {

    public void show() {

        // Uso de values() en un for-each
        for (Operation operation : Operation.values()) {
            System.out.println(operation);
        }

    }

    public static void main(String[] args) {

        new EnumValues().show();

    }

}
```

Salida por consola:

```
PLUS
MINUS
TIMES
DIVIDE
```

7.4 Método ordinal

Una enumeración hereda de la clase `java.lang.Enum<E>` y por tanto tiene una serie de métodos heredados, como por ejemplo `ordinal()`.

`public final int ordinal()` devuelve un entero que indica la posición de la constante dentro de la enumeración. La primera constante tiene el valor 0.

```
package tema8_TiposEnumerados;

public class EnumOrdinal {

    public void show() {
```



```

        System.out.println(Operation.PLUS.ordinal()); //0
        System.out.println(Operation.MINUS.ordinal()); //1
        System.out.println(Operation.TIMES.ordinal()); //2
        System.out.println(Operation.DIVIDE.ordinal()); //3
    }

    public static void main(String[] args) {

        new EnumOrdinal().show();

    }
}

```

El problema de usar el método `ordinal()` es que el funcionamiento del código cliente depende completamente del orden en el que se definen las constantes en el *enum*, lo que puede resultar problemático para el mantenimiento del código si en un futuro se cambia dicho orden. En realidad, este método está diseñado para ser utilizado por estructuras de datos sofisticadas basadas en enumeraciones como `EnumSet` y `EnumMap`. (Se explican en el [Tema 11. Colecciones](#)).

Por lo tanto se recomienda no hacer uso del método `ordinal()`, sino definir un atributo y asignar un valor para dicho atributo en cada instancia:

```

package tema8_TiposEnumerados.ordinal;

public enum Operation {

    PLUS(1, '+'), MINUS(2, '-'), TIMES(3, '*'), DIVIDE(4, '/');

    private final int optionNumber;
    private final char symbol;

    Operation(int optionNumber, char symbol) {
        this.optionNumber = optionNumber;
        this.symbol = symbol;
    }

    public char getSymbol() {
        return symbol;
    }

    public int getOptionNumber() {
        return optionNumber;
    }

}

```

```

package tema8_TiposEnumerados.ordinal;

public class EnumAttributeInsteadOfOrdinal {

    public void show() {

```

```

        System.out.println(Operation.PLUS.getOptionNumber()); //1
        System.out.println(Operation.MINUS.getOptionNumber()); //2
        System.out.println(Operation.TIMES.getOptionNumber()); //3
        System.out.println(Operation.DIVIDE.getOptionNumber()); //4
    }

    public static void main(String[] args) {

        new EnumAttributeInsteadOfOrdinal().show();

    }

}

```

8. Métodos abstractos

En algunas ocasiones es necesario asociar un comportamiento ligeramente diferente en cada instancia del *enum*. En estos casos, debemos declarar un método abstracto en el *enum* e incluir la implementación concreta de dicho método abstracto en la definición de cada instancia. Estas implementaciones reciben el nombre de *constant-specific methods*. Por ejemplo:

```

package tema8_TiposEnumerados.metodosAbstractos;

public enum Operation {

    PLUS('+') {
        @Override
        public double apply(double x, double y) {
            return x + y;
        }
    },
    MINUS('-') {
        @Override
        public double apply(double x, double y) {
            return x - y;
        }
    },
    TIMES('*') {
        @Override
        public double apply(double x, double y) {
            return x * y;
        }
    },
    DIVIDE('/') {
        @Override
        public double apply(double x, double y) {
            return x / y;
        }
    };

    private final char symbol;
}

```

```

    Operation(char symbol) {
        this.symbol = symbol;
    }

    public char getSymbol() {
        return symbol;
    }

    public abstract double apply(double x, double y);
}

```

```

package tema8_TiposEnumerados.metodosAbstractos;

public class EnumAbstractMethods {

    public void show() {

        for (Operation operation : Operation.values()) {
            System.out.printf("%-8s  %.2f %c %.2f = %.2f\n", operation + ":", 5f,
operation.getSymbol(), 3f,
                        operation.apply(5, 3));
        }

    }

    public static void main(String[] args) {

        new EnumAbstractMethods().show();

    }

}

```

Salida por consola:

```

PLUS:      5,00 + 3,00 = 8,00
MINUS:     5,00 - 3,00 = 2,00
TIMES:     5,00 * 3,00 = 15,00
DIVIDE:    5,00 / 3,00 = 1,67

```

9. Implementación de interfaces

Como ya hemos comentado, un *enum* no puede heredar de otro *enum*. Pero hay ocasiones donde tiene sentido que otros programadores puedan "extender" un *enum* que hayamos proporcionado en nuestra API. Por ejemplo cuando el *enum* corresponda a códigos de operación sobre una determinada máquina, como por ejemplo el *enum Operation*, sería recomendable que quien use nuestra API puede "añadir" nuevas operaciones. Para estos casos podemos simular la herencia entre *enums* definiendo una interfaz con el método de la operación y hacer que nuestro *enum* implemente dicha interfaz. Por ejemplo:

```
package tema8_TiposEnumerados.implementacionInterfaces;

public interface Operation {

    double apply(double x, double y);

    char getSymbol();

}
```

```
package tema8_TiposEnumerados.implementacionInterfaces;

public enum BasicOperation implements Operation {

    PLUS('+') {
        @Override
        public double apply(double x, double y) {
            return x + y;
        }
    },
    MINUS('-') {
        @Override
        public double apply(double x, double y) {
            return x - y;
        }
    },
    TIMES('*') {
        @Override
        public double apply(double x, double y) {
            return x * y;
        }
    },
    DIVIDE('/') {
        @Override
        public double apply(double x, double y) {
            return x / y;
        }
    };

    private final char symbol;

    BasicOperation(char symbol) {
        this.symbol = symbol;
    }

    @Override
    public char getSymbol() {
        return symbol;
    }

}
```

De esta manera, si otro equipo de desarrolladores quiere "extender" nuestro *enum* simplemente debe crear otro *enum* distinto que implemente la misma interfaz:

```

package tema8_TiposEnumerados.implementacionInterfaces;

public enum ExtendedOperation implements Operation {

    EXP('^') {
        @Override
        public double apply(double x, double y) {
            return Math.pow(x, y);
        }
    },
    REMAINDER('%') {
        @Override
        public double apply(double x, double y) {
            return x % y;
        }
    };

    private final char symbol;

    ExtendedOperation(char symbol) {
        this.symbol = symbol;
    }

    @Override
    public char getSymbol() {
        return symbol;
    }

}

```

De esta manera, el código cliente que quiera trabajar con una operación, tendrá que definir la variable correspondiente de tipo interfaz:

```

Operation operation = ExtendedOperation.EXP;

```

El problema principal de esta técnica es que dado que no se trata de herencia real, no podemos reutilizar código entre los *enums*.

Veamos un ejemplo de uso de ambos *enums*:

```

package tema8_TiposEnumerados.implementacionInterfaces;

public class InterfacesImplementation {

    public void show() {

        for (Operation operation : BasicOperation.values()) {
            show(operation);
        }
        for (Operation operation : ExtendedOperation.values()) {
            show(operation);
        }
    }

}

```

```

    public void show(Operation operation) {

        System.out.printf("%-10s  %.2f %c %.2f = %.2f\n", operation + ":", 5f,
operation.getSymbol(), 3f,
        operation.apply(5, 3));

    }

    public static void main(String[] args) {

        new InterfacesImplementation().show();

    }

}

```

Salida por consola:

```

PLUS:      5,00 + 3,00 = 8,00
MINUS:     5,00 - 3,00 = 2,00
TIMES:     5,00 * 3,00 = 15,00
DIVIDE:    5,00 / 3,00 = 1,67
EXP:       5,00 ^ 3,00 = 125,00
REMAINDER: 5,00 % 3,00 = 2,00

```

9 Fechas

9 Fechas

1. Introducción
2. Clase Locale
3. Métodos de la API de fechas
4. Enum DayOfWeek
5. Enum Month
6. Clase LocalDate
7. Clase YearMonth
8. Clase MonthDay
9. Clase Year
10. Clase LocalTime
11. Clase LocalDateTime
12. Clase DateTimeFormatter
13. Enum ChronoUnit
14. Clase Period
15. Clase Duration
16. Enum TimeUnit

1. Introducción

Java 8 introduce una nueva API para fechas y horas que es más fácil de leer, más amplia que la API anterior y thread-safe, es decir, los objetos se pueden utilizar con seguridad desde varios hilos de ejecución.

Esta API se introduce para cubrir los siguientes inconvenientes de la anterior:

1. La API anterior no es thread-safe por lo que los desarrolladores tienen que hacer frente a problemas de concurrencia durante el uso de la fecha. La nueva API de fecha y hora es inmutable y no tiene métodos setter. Al ser inmutables, no necesitan ser clonadas.
2. Dificultad para manejar zona-horaria en la API anterior.

En java 8 se crea un nuevo paquete para el manejo de fechas, se trata del paquete `java.time`. Las clases definidas en este paquete representan los principales conceptos de fecha - hora, incluyendo instantes, fechas, horas, períodos, zonas de tiempo, etc. Están basados en el sistema de calendario ISO, el cual es el calendario mundial *de-facto* que sigue las reglas del calendario Gregoriano.

2. Clase Locale

Hay que destacar la importancia y el valor de respetar el idioma y la región geográfica del usuario al desarrollar aplicaciones de software. Permitir que el usuario se comunice con el software en su propio idioma es un gran impulso para las ventas del software. Cuando se trata de Java, es el concepto de configuración regional de Java lo que explica el proceso de internacionalización.

La configuración regional de Java consta de tres elementos principales: idioma, país y variantes. Las variantes la utilizan los proveedores de software, ya sean sistemas operativos o navegadores, para funcionalidades adicionales.

En Java se utiliza la clase `Locale` del paquete `java.util` como una localización, es decir, representa una región geográfica, política o cultural específica. Una operación que requiere una localización para realizar su tarea se denomina sensible a la localización y utiliza la localización para adaptar la información al usuario. Por ejemplo, mostrar un número es una operación sensible a la configuración regional: el número debe formatearse de acuerdo con las costumbres y convenciones del país, región o cultura del usuario. Estas clases utilizan el objeto `Locale` para conocer qué configuración regional se está utilizando en la aplicación y se personalizan en cuanto a cómo formatear y presentar los datos al usuario. Por ejemplo, la clase `NumberFormat`, que puede devolver un número como 302 400 para Francia, 302 .400 para Alemania y 302, 400 para los Estados Unidos. También son clases sensibles a la configuración regional `DecimalFormat` y `DateTimeFormatter`.

Hay tres constructores disponibles para la creación de objetos `Locale`:

- `Locale(String language)`: utiliza solamente el idioma. Ejemplo: `Locale locale = new Locale("en");` En este caso, al no indicar un país, la aplicación no podrá adaptarse a las diferencias regionales del idioma.
- `Locale(String language, String country)`: utiliza tanto el idioma como el país. Ejemplo: `Locale locale = new Locale("en", "US");`
- `Locale(String language, String country, String variant)`: utiliza el idioma, el país y la variante. Ejemplo: `Locale locale = new Locale("en", "US", "Siliconvalley");`

Los códigos se pueden encontrar en <https://www.iana.org/assignments/language-subtag-registry/language-subtag-registry>:

- *Type: language*: son los códigos de idioma.
- *Type: region*: son los códigos de país.
- *Type: variant*: son los códigos de la variante.

Los códigos de variante no conforman un estándar sino que son arbitrarios y específicos de la aplicación. Si existen variaciones en el idioma utilizado dentro del mismo país, se puede especificar una variante. Por ejemplo, en el Sur de Estados Unidos, la gente suele decir "y'all," cuando en el Norte dicen "you all." Se podría crear diferentes objetos `Locale` de esta forma:

```
Locale localeNorth = new Locale("en", "US", "NORTH");
Locale localesouth = new Locale("en", "US", "SOUTH");
```

Si se crean objetos `Locale` con los códigos de variante `NORTH` y `SOUTH`, como en el ejemplo anterior, solo nuestra aplicación sabrá cómo tratarlos.

Normalmente, se especifican códigos de variante para identificar diferencias causadas por la plataforma de ordenador. Por ejemplo, las diferencias de fuentes podría forzar a utilizar caracteres diferentes en Windows y en UNIX. Se podría definir los objetos **Locale** con estos códigos de variante:

```
Locale localeUnix = new Locale("de", "DE", "UNIX");
Locale localewindows = new Locale("de", "DE", "WINDOWS");
```


Tal y como podemos observar en la API, la clase *Locale* tiene constantes predefinidas para algunos idiomas y países. Por ejemplo, **JAPAN** es la constante del país Japón mientras que **JAPANESE** es la constante del idioma japonés. Se puede crear un objeto *Locale* utilizando una constante:

```
Locale l1 = Locale.JAPAN; // Se crea un objeto Locale con el idioma japonés y país Japón
Locale l2 = Locale.JAPANESE; // Se crea un objeto Locale con el idioma japonés
```

Si no se asigna un objeto *Locale* a un objeto sensible a la localidad, entonces se utiliza la localización por defecto, la cual se puede conocer con el método `getDefault()`, que obtiene el valor actual de la configuración regional por defecto para esa instancia de la Máquina Virtual de Java. La máquina virtual de Java establece la configuración regional por defecto durante el inicio basándose en el entorno del host. Se puede cambiar utilizando el método `setDefault`.

```
package tema9_Fechas;

import java.util.Locale;

public class ClassLocale {

    public void show() {

        Locale l = Locale.getDefault();
        System.out.println(l.toString());

    }

    public static void main(String[] args) {

        new ClassLocale().show();

    }

}
```

Salida por consola:

```
es_ES
```

3. Métodos de la API de fechas

Muchas de las clases de la API de fechas contienen los siguientes métodos:

- Métodos estáticos `of`: se utilizan para crear instancias de objetos de fecha y hora con literales y constantes enum sin la necesidad de utilizar constructores. Ejemplo:

```
LocalDate localDate = LocalDate.of(2024, Month.AUGUST, 15);
```

- Métodos `plus` y `minus`: se utilizan para realizar operaciones de suma y resta respectivamente sobre objetos de fecha y hora. Ejemplos:

```
Month month = Month.APRIL;
Month month1 = month.plus(3); //JULY
```

```
LocalDate localDate = LocalDate.of(2024, Month.AUGUST, 15); //2024-08-15
LocalDate localDate1 = localDate.plusDays(5); //2024-08-20
```

- Métodos `now`: se utilizan para obtener la fecha y la hora actuales del sistema basándose en el reloj del sistema del entorno en el que se ejecuta la aplicación. Ejemplos:

```
LocalDate localDate = LocalDate.now();
Year year = Year.now();
```

- Métodos `parse`: se utilizan para crear objetos de fecha y hora a partir de cadenas. Ejemplos:

```
LocalDate localDate = LocalDate.parse("2024-03-22");
Year year = Year.parse("2024");
```

- Métodos `get`: se utilizan para obtener componentes específicos de un objeto de fecha y hora. Ejemplos:

```
LocalDate date = LocalDate.of(2024, 2, 8);
int year = date.getYear(); //Obtiene el año 2024
int month = date.getMonthValue(); //Obtiene el mes 2
int day = date.getDayOfMonth(); //Obtiene el día 8
LocalTime time = LocalTime.of(15, 30, 25);
int hour = time.getHour(); //Obtiene la hora 15
int minute = time.getMinute(); //Obtiene el minuto 30
int second = time.getSecond(); //Obtiene el segundo 25
```

- Métodos `between`: se utilizan para calcular la diferencia entre dos objetos de fecha y hora. Estos métodos permiten calcular la cantidad de tiempo transcurrido entre dos puntos en el tiempo, expresados en unidades específicas como días, meses, años, horas, minutos, etc. Ejemplos:

```
long days = ChronoUnit.DAYS.between(localDate1, localDate2);
Period period = Period.between(localDate1, localDate2);
```

4. Enum DayOfWeek

En el enum `DayOfWeek` del paquete `java.time` se definen los días de la semana con el siguiente conjunto de constantes:

```
MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY
```

Podemos crear variables de tipo `DayOfWeek` y asignarle una instancia del enum a través de dichas constantes:

```
DayOfWeek day1; //day1 es una variable del tipo de enumeración DayOfWeek
day1 = DayOfWeek.MONDAY; //Se le asigna la instancia del lunes a través de la
constante MONDAY
```

También podemos crear una instancia del enum con el método estático `static DayOfWeek of(int dayOfWeek)`, que obtiene una instancia de *DayOfWeek* a partir de un valor *int* del 1 (lunes) al 7 (domingo).

```
DayOfWeek day2; // day2 es una variable del tipo de enumeración DayOfWeek
day2 = DayOfWeek.of(1); // Se le asigna la instancia del lunes a través del método of
```

Este enum contiene métodos que permiten manipular días hacia adelante y hacia atrás:

- `DayOfWeek plus(long days)`: devuelve el día de la semana que es el número especificado de días después de éste.
- `DayOfWeek minus(long days)`: devuelve el día de la semana que es el número especificado de días antes de éste.

```
DayOfWeek day3, day4;
day3 = day1.plus(4); // Viernes
day4 = day2.minus(2); // Sábado
```

Este enum contiene también el método `String getDisplayName(TextStyle style, Locale locale)` que devuelve la representación textual utilizada para identificar el día de la semana adecuada para su presentación al usuario. Los parámetros controlan el estilo del texto devuelto y la configuración regional. El estilo del texto se representa con el enum `TextStyle` donde se definen tres tamaños para el texto formateado: `full`, `short` y `narrow`.

```
package tema9_Fechas;

import java.time.DayOfWeek;
import java.time.format.TextStyle;
import java.util.Locale;

public class EnumDayOfWeek {

    public void show() {

        DayOfWeek day1, day2, day3, day4; // Variables del tipo de enumeración DayOfWeek
        day1 = DayOfWeek.MONDAY; // Se le asigna la instancia del lunes a través de la constante MONDAY
        day2 = DayOfWeek.of(1); // Se le asigna la instancia del lunes a través del método of
        day3 = day1.plus(4); // Viernes
        day4 = day2.minus(2); // Sábado
        System.out.printf("day1: %s, day2: %s, day3: %s, day4: %s", day1, day2, day3, day4);

        Locale l = new Locale("es", "ES"); // idioma español castellano, país España
        System.out.printf("\nTextStyle.FULL:%s\n", day1.getDisplayName(TextStyle.FULL, l));
        System.out.printf("TextStyle.NARROW:%s\n", day1.getDisplayName(TextStyle.NARROW, l));
        System.out.printf("TextStyle.SHORT:%s\n", day1.getDisplayName(TextStyle.SHORT, l));
    }
}
```

```

    }

    public static void main(String[] args) {

        new EnumDayOfWeek().show();

    }

}

```

Salida por consola:

```

day1: MONDAY, day2: MONDAY, day3: FRIDAY, day4: SATURDAY
TextStyle.FULL: lunes
TextStyle.NARROW: L
TextStyle.SHORT: lun

```

5. Enum Month

En el enum `Month` del paquete `java.time` se definen los meses con el siguiente conjunto de constantes:

```

JANUARY, FEBRUARY, MARCH, APRIL, MAY, JUNE, JULY, AUGUST, SEPTEMBER, OCTOBER,
NOVEMBER, DECEMBER

```

También tiene los métodos `of`, `plus`, `minus` y `getDisplayName` como el enumerado de días de la semana. Además, contiene métodos para conocer el número de días máximos y mínimos de cada mes:

- `int minLength()`: obtiene la duración mínima de este mes en días. Febrero tiene una duración mínima de 28 días. Abril, junio, septiembre y noviembre tienen 30 días. Todos los demás meses tienen 31 días.
- `int maxLength()`: obtiene la duración máxima de un mes en días. Febrero tiene una duración máxima de 29 días. Abril, junio, septiembre y noviembre tienen 30 días. Todos los demás meses tienen 31 días.

```

package tema9_Fechas;

import java.time.Month;
import java.time.format.TextStyle;
import java.util.Locale;

public class EnumMonth {

    public void show() {

        Month month1, month2, month3, month4; //Variables del tipo de enumeración Month
        month1 = Month.APRIL; //Se le asigna la instancia del mes de abril a través de la constante APRIL
        month2 = Month.of(2); //Se le asigna la instancia del mes de febrero a través del método of
        month3 = month1.plus(4); //Agosto
    }
}

```

```

        month4 = month2.minus(3);//Noviembre
        System.out.printf("month1: %s, month2: %s, month3: %s, month4:
%s",month1,month2,month3,month4);
        System.out.printf("\n%s tiene como mínimo %d días y como máximo %d
días",month2,month2.minLength(),month2.maxLength());
        System.out.printf("\n%s tiene como mínimo %d días y como máximo %d
días",month1,month1.minLength(),month1.maxLength());
        System.out.printf("\n%s tiene como mínimo %d días y como máximo %d
días",month3,month3.minLength(),month3.maxLength());
        Locale l = new Locale("es","ES");//idioma español castellano, país España
        System.out.printf("\nTextStyle.FULL: %s\n",
month1.getDisplayName(TextStyle.FULL, l));
        System.out.printf("TextStyle.NARROW: %s\n",
month1.getDisplayName(TextStyle.NARROW, l));
        System.out.printf("TextStyle.SHORT: %s\n",
month1.getDisplayName(TextStyle.SHORT, l));

    }

    public static void main(String[] args) {

        new EnumMonth().show();

    }

}

```

Salida por consola:

```

month1: APRIL, month2: FEBRUARY, month3: AUGUST, month4: NOVEMBER
FEBRUARY tiene como mínimo 28 días y como máximo 29 días
APRIL tiene como mínimo 30 días y como máximo 30 días
AUGUST tiene como mínimo 31 días y como máximo 31 días
TextStyle.FULL: abril
TextStyle.NARROW: A
TextStyle.SHORT: abr

```

6. Clase LocalDate

La clase `LocalDate` del paquete `java.time` nos permite registrar fechas específicas inmutables.

```

package tema9_Fechas;

import java.time.LocalDate;

public class ClassLocalDate {

    public void show() {

        LocalDate currentDate, concertDate, parseDate;
        currentDate = LocalDate.now();//Se obtiene la fecha actual
        System.out.printf("Fecha actual: %s", currentDate);
        System.out.printf(", es decir, día %s del mes %s del año %s.",
currentDate.getDayOfMonth(),

```

```

        currentDate.getMonthValue(), currentDate.getYear());
        concertDate = LocalDate.of(2022, 8, 13); //Obtiene una instancia de
LocalDate a partir de un año, mes y día
        System.out.printf("\n\nEl concierto es el %s.", concertDate);
        System.out.println(" Las entradas se podrán comprar como máximo hasta");
        System.out.printf("5 días antes, es decir, hasta el %s.",
concertDate.minusDays(5));
        parseDate = LocalDate.parse("2022-02-22"); //Obtiene un LocalDate a partir
de una cadena
        System.out.printf("\n\nFecha obtenida a partir de una cadena: %s",
parseDate);

    }

    public static void main(String[] args) {

        new ClassLocalDate().show();

    }

}

```

Salida por consola:

```

Fecha actual: 2022-02-03, es decir, día 3 del mes 2 del año 2022.

El concierto es el 2022-08-13. Las entradas se podrán comprar como máximo hasta
5 días antes, es decir, hasta el 2022-08-08.

Fecha obtenida a partir de una cadena: 2022-02-22

```

7. Clase YearMonth

Con la clase `YearMonth` del paquete `java.time` podemos generar objetos inmutables que representan la combinación de un año y un mes.

Tiene el método `int lengthOfMonth()` que devuelve la duración del mes en días.

```

package tema9_Fechas;

import java.time.Month;
import java.time.YearMonth;

public class ClassYearMonth {

    public void show() {

        YearMonth currentYearMonth, ofYearMonth, parseYearMonth;
        currentYearMonth = YearMonth.now(); //Se obtiene el YearMonth actual
        System.out.printf("Este año-mes es %s y tiene %d días", currentYearMonth,
currentYearMonth.lengthOfMonth());
        ofYearMonth = YearMonth.of(2004, Month.FEBRUARY); //Obtiene una instancia
YearMonth a partir de un año y un mes
    }
}

```

```

        System.out.printf("\n\nEl año-mes %s tuvo %d días", ofYearMonth,
ofYearMonth.lengthOfMonth());
        ofYearMonth = YearMonth.of(2010, Month.FEBRUARY);
        System.out.printf("\n\nEl año-mes %s tuvo %d días", ofYearMonth,
ofYearMonth.lengthOfMonth());
        ofYearMonth = YearMonth.of(2000, Month.FEBRUARY);
        System.out.printf("\n\nEl año-mes %s tuvo %d días", ofYearMonth,
ofYearMonth.lengthOfMonth());
        parseYearMonth = YearMonth.parse("2022-12");//Obtiene un YearMonth a
partir de una cadena
        System.out.printf("\n\nAño-mes obtenido a partir de una cadena: %s",
parseYearMonth);
    }

    public static void main(String[] args) {

        new ClassYearMonth().show();

    }

}

```

Salida por consola:

```

Este año-mes es 2022-02 y tiene 28 días

El año-mes 2004-02 tuvo 29 días
El año-mes 2010-02 tuvo 28 días
El año-mes 2000-02 tuvo 29 días

Año-mes obtenido a partir de una cadena: 2022-12

```

8. Clase MonthDay

Con la clase `MonthDay` del paquete `java.time` podemos generar objetos inmutables que representan la combinación de un día y un mes.

```

package tema9_Fechas;

import java.time.Month;
import java.time.MonthDay;

public class ClassMonthDay {

    public void show() {

        MonthDay currentMonthDay, ofMonthDay, parseMonthDay;
        currentMonthDay = MonthDay.now();//Se obtiene el MonthDay actual
        System.out.printf("Este día es %s, es decir, día %d del mes %d",
currentMonthDay,
            currentMonthDay.getDayOfMonth(),
currentMonthDay.getMonthValue());
        ofMonthDay = MonthDay.of(Month.FEBRUARY, 29);//Obtiene una instancia
MonthDay a partir de un mes y un día
    }
}

```

```

        System.out.printf("\n\nEl día %s del mes %s es válido para el año
2004", ofMonthDay.getDayOfMonth(),
        ofMonthDay.getMonthValue(), ofMonthDay.isValidYear(2004) ? "" : "
no");

        System.out.printf("\n\nEl día %s del mes %s es válido para el año 2010",
ofMonthDay.getDayOfMonth(),
        ofMonthDay.getMonthValue(), ofMonthDay.isValidYear(2010) ? "" : "
no");

        parseMonthDay = MonthDay.parse("--12-03");//Obtiene un MonthDay a partir
de una cadena
        System.out.printf("\n\nMes-día obtenido a partir de una cadena: %s",
parseMonthDay);

    }

    public static void main(String[] args) {

        new ClassMonthDay().show();

    }

}

```

Salida por consola:

```

Este día es --02-07, es decir, día 7 del mes 2

El día 29 del mes 2 es válido para el año 2004
El día 29 del mes 2 no es válido para el año 2010

Mes-día obtenido a partir de una cadena: --12-03

```

9. Clase Year

Con la clase `Year` del paquete `java.time` podemos generar objetos inmutables que representan un año concreto.

Tiene métodos como:

- `boolean isLeap()`: devuelve un booleano indicando si el año es bisiesto.
- `boolean isBefore(Year other)`: comprueba si el año es anterior al año del parámetro.
- `boolean isAfter(Year other)`: comprueba si el año es posterior al año del parámetro.

```

package tema9_Fechas;

import java.time.Year;

public class ClassYear {

    public void show() {

        Year currentYear, ofYear, parseYear;
        currentYear = Year.now();//Obtiene el año actual
    }
}

```



```

        System.out.printf("Este año es %s y %s es bisiesto\n", currentYear,
currentYear.isLeap() ? "sí" : "no");
        ofYear = Year.of(2012); //Obtiene una instancia Year a partir de un año
        System.out.printf("\nEl año %s es %s al año actual", ofYear,
            ofYear.isBefore(currentYear) ? "anterior" : "posterior");
        parseYear = Year.parse("2021"); //Obtiene un Year a partir de una cadena
        System.out.printf("\n\nAño obtenido a partir de una cadena: %s",
parseYear);

    }

    public static void main(String[] args) {

        new ClassYear().show();

    }

}

```

Salida por consola:

```

Este año es 2022 y no es bisiesto

El año 2012 es anterior al año actual

Año obtenido a partir de una cadena: 2021

```

10. Clase LocalTime

Con la clase `LocalTime` del paquete `java.time` podemos generar objetos inmutables que representan una hora concreta.

```

package tema9_Fechas;

import java.time.LocalTime;

public class ClassLocalTime {

    public void show() {

        LocalTime currentTime, meetingTime, parseTime;
        currentTime = LocalTime.now(); //Se obtiene la hora actual
        System.out.printf("Hora actual: %s", currentTime);
        System.out.printf("\nHora: %s Minutos: %s Segundos: %s Nanosegundos: %s",
currentTime.getHour(),
            currentTime.getMinute(), currentTime.getSecond(),
currentTime.getNano());
        meetingTime = LocalTime.of(10, 15); //Obtiene una instancia de LocalTime
indicando hora y minutos
        System.out.printf("\n\nHora de la reunión: %s", meetingTime);
        parseTime = LocalTime.parse("11:45:30"); //Obtiene un LocalTime a partir
de una cadena
        System.out.printf("\n\nHora obtenida a partir de una cadena: %s",
parseTime);
    }
}

```

```

    }

    public static void main(String[] args) {

        new ClassLocalTime().show();

    }

}

```

Salida por consola:

```

Hora actual: 08:56:53.868514995
Hora: 8 Minutos: 56 Segundos: 53 Nanosegundos: 868514995

Hora de la reunión: 10:15

Hora obtenida a partir de una cadena: 11:45:30

```

11. Clase LocalDateTime

Con la clase `LocalDateTime` del paquete `java.time` podemos generar objetos inmutables que representan una fecha y hora concretas.

```

package tema9_Fechas;

import java.time.LocalDateTime;

public class ClassLocalDateTime {

    public void show() {

        LocalDateTime currentDateTime, meetingDateTime, parseDateTime;
        currentDateTime = LocalDateTime.now();//Se obtiene la fecha y hora actual
        System.out.printf("Fecha y hora actual: %s", currentDateTime);
        System.out.printf("\nDía del año: %s", currentDateTime.getDayOfYear());
        System.out.printf("\nDía de la semana: %s",
currentDateTime.getDayOfWeek());
        System.out.printf("\nHora: %s", currentDateTime.getHour());
        System.out.printf("\nNanosegundos: %s", currentDateTime.getNano());
        meetingDateTime = LocalDateTime.of(2022, 11, 16, 9, 30, 45);//Obtiene una
instancia de LocalDateTime a partir de un año,mes,día,hora,minutos y segundos
        System.out.printf("\n\nLa reunión es %s", meetingDateTime);
        System.out.printf("\nFecha de la reunión: %s",
meetingDateTime.toLocalDate());
        System.out.printf("\nHora de la reunión: %s",
meetingDateTime.toLocalTime());
        parseDateTime = LocalDateTime.parse("2022-12-03T10:15:30");//Obtiene un
LocalDateTime a partir de una cadena
        System.out.printf("\n\nFecha y hora obtenida a partir de una cadena: %s",
parseDateTime);

    }

}

```

```
public static void main(String[] args) {  
  
    new ClassLocalDateTime().show();  
  
}  
  
}
```

Salida por consola:

```
Fecha y hora actual: 2022-02-04T13:48:19.851351656  
Día del año: 35  
Día de la semana: FRIDAY  
Hora: 13  
Nanosegundos: 851351656  
  
La reunión es 2022-11-16T09:30:45  
Fecha de la reunión: 2022-11-16  
Hora de la reunión: 09:30:45  
  
Fecha y hora obtenida a partir de una cadena: 2022-12-03T10:15:30
```

12. Clase DateTimeFormatter

La clase `DateTimeFormatter` del paquete `java.time.format` se utiliza para dar formato a las fechas.

Se puede indicar el formato con:

- formatos predefinidos.
- creando patrones que se basan en una secuencia de letras y símbolos.

En la API podemos encontrar dichos formatos predefinidos y el significado de las letras y símbolos para crear un patrón. Veamos algunos:

- d: día del mes. Posibilidades:
 - `d`: si el día tiene un dígito, se muestra solamente ese dígito. Ejemplo: `7`. Si tiene dos dígitos, se muestran los dos. Ejemplo: `27`.
 - `dd`: si el día tiene un dígito, se muestra el dígito con un cero a la izquierda. Ejemplo: `07`. Si tiene dos dígitos, se muestran los dos. Ejemplo: `27`.
 - Más de dos: salta la excepción `IllegalArgumentException`.
- M: mes del año. Posibilidades:
 - `M`: si el mes tiene un dígito, se muestra solamente ese dígito. Ejemplo: `7`. Si tiene dos dígitos, se muestran los dos. Ejemplo: `11`.
 - `MM`: si el mes tiene un dígito, se muestra el dígito con un cero a la izquierda. Ejemplo: `07`. Si tiene dos dígitos, se muestran los dos. Ejemplo: `11`.
 - `MMM`: se muestra en formato `short`. Ejemplo: `jul`.
 - `MMMM`: se muestra en formato `full`. Ejemplo: `julio`.

- `MMMM`: se muestra en formato `narrow`. Ejemplo: `J`. Hay que tener en cuenta que los meses que empiecen con la misma letra, les va a salir la misma letra en el formato `narrow`. Por ejemplo, junio y julio van a tener la letra `J` así como marzo y mayo, que van a tener la letra `M`.
- Más de 5: salta la excepción `IllegalArgumentException`.
- G: era. Hay dos eras:
 - Current Era (CE): era actual, es decir, después de Cristo.
 - Before Current Era (BCE): antes de la era actual, es decir, antes de Cristo.

Posibilidades:

- `G`, `GG`, `GGG`: *anno Dómini*(CE) y *antes de Cristo*(BCE).
- `GGGG`: *después de Cristo*(CE) y *antes de Cristo*(BCE).
- `GGGGG`: *d. C.*(CE) y *a. C.*(BCE).
- Más de 5: salta la excepción `IllegalArgumentException`.
- u: año proléptico. Posibilidades:
 - `u`, `uuu`, `uuuu`: muestra el año con 4 dígitos.
 - `uu`: muestra el año con 2 dígitos.
 - Más de 4: se rellena con ceros a la izquierda.
- y: año de la era. Posibilidades:
 - `y`, `yyy`, `yyyy`: muestra el año con 4 dígitos.
 - `yy`: muestra el año con 2 dígitos.
 - Más de 4: se rellena con ceros a la izquierda.

¿qué diferencia existe entre `y` y `u`?:

En la era actual CE, `y` y `u` funcionan igual. La diferencia existe en los años de la era BCE, donde el año proléptico puede ser cero o negativo pero el año de la era no:

- Año proléptico: el año proléptico 0 es el 1 a.C., el año proléptico -1 es el 2 a.C., el año proléptico -2 es el 3 a.C., y así sucesivamente.
- Año de la era: el año aumenta en 1 a medida que el tiempo retrocede. El año de la era 1 es el 1 a.C., el año de la era 2 es el 2 a.C., el año de la era 3 es el 3 a.C., y así sucesivamente.

Los métodos `parse` que hemos visto en clases anteriores están sobrecargados para utilizarlos con un formato concreto.

Dispone del método `format` que devuelve una cadena con el formato especificado.

```
package tema9_Fechas;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class ClassDateTimeFormatter {

    public void show() {
```

```

LocalDateTime localDateTime;
LocalDate localDate = LocalDate.of(2023, 7, 9);
LocalDate localDateEra = LocalDate.of(1, 1, 1);//1 de enero del año 1

//Formato predefinido ISO_ORDINAL_DATE: año y día del año
DateTimeFormatter formatter = DateTimeFormatter.ISO_ORDINAL_DATE;
System.out.printf("Formato predefinido ISO_ORDINAL_DATE: %s",
formatter.format(localDate));

//Creación de patrones basados en secuencia de letras y símbolos
formatter = DateTimeFormatter.ofPattern("G d M y");
System.out.printf("\n\nPatrón \"%s\": %s", "G d M y",
formatter.format(localDate));
formatter = DateTimeFormatter.ofPattern("GG d/MM/yy");
System.out.printf("\nPatrón \"%s\": %s", "GG d/MM/yy",
formatter.format(localDate));
formatter = DateTimeFormatter.ofPattern("GGG d MMM yyy");
System.out.printf("\nPatrón \"%s\": %s", "GGG d MMM yyy",
formatter.format(localDate));
formatter = DateTimeFormatter.ofPattern("GGGG dd-MMMM-yyyy");
System.out.printf("\nPatrón \"%s\": %s", "GGGG dd-MMMM-yyyy",
formatter.format(localDate));
formatter = DateTimeFormatter.ofPattern("GGGGG dd-MMMMM-yyyyy");
System.out.printf("\nPatrón \"%s\": %s", "GGGGG dd-MMMMM-yyyyy",
formatter.format(localDate));

//Era CE
System.out.print("\n\nAño 1 d.C.");
formatter = DateTimeFormatter.ofPattern("GGGG dd/MM/uuuu");
System.out.printf("\nPatrón \"%s\": %s", "GGGG dd/MM/uuuu",
formatter.format(localDateEra));
formatter = DateTimeFormatter.ofPattern("GGGG dd/MM/yyyy");
System.out.printf("\nPatrón \"%s\": %s", "GGGG dd/MM/yyyy",
formatter.format(localDateEra));

//Era BCE
localDateEra = localDateEra.minusYears(1);
System.out.print("\n\nAño 1 a.C.");
formatter = DateTimeFormatter.ofPattern("GGGG dd/MM/uuuu");
System.out.printf("\nPatrón \"%s\": %s", "GGGG dd/MM/uuuu",
formatter.format(localDateEra));
formatter = DateTimeFormatter.ofPattern("GGGG dd/MM/yyyy");
System.out.printf("\nPatrón \"%s\": %s", "GGGG dd/MM/yyyy",
formatter.format(localDateEra));
localDateEra = localDateEra.minusYears(1);
System.out.print("\n\nAño 2 a.C.");
formatter = DateTimeFormatter.ofPattern("GGGG dd/MM/uuuu");
System.out.printf("\nPatrón \"%s\": %s", "GGGG dd/MM/uuuu",
formatter.format(localDateEra));
formatter = DateTimeFormatter.ofPattern("GGGG dd/MM/yyyy");
System.out.printf("\nPatrón \"%s\": %s", "GGGG dd/MM/yyyy",
formatter.format(localDateEra));
localDateEra = localDateEra.minusYears(1);
System.out.print("\n\nAño 3 a.C.");
formatter = DateTimeFormatter.ofPattern("GGGG dd/MM/uuuu");

```

```

        System.out.printf("\nPatrón \"%s\": %s", "GGGG dd/MM/uuuu",
formatter.format(localDateEra));
        formatter = DateTimeFormatter.ofPattern("GGGG dd/MM/yyyy");
        System.out.printf("\nPatrón \"%s\": %s", "GGGG dd/MM/yyyy",
formatter.format(localDateEra));

        //Uso de método parse con formato
        formatter = DateTimeFormatter.ofPattern("d MMMM yy HH:mm:ss");
        LocalDateTime = LocalDateTime.parse("7 febrero 22 10:01:30", formatter);
        System.out.printf("\n\nFecha y hora obtenida a partir de una cadena y un
formato: %s", LocalDateTime);
    }

    public static void main(String[] args) {

        new ClassDateTimeFormatter().show();

    }
}

```

Salida por consola:

```

Formato predefinido ISO_ORDINAL_DATE: 2023-190

Patrón "G d M y": anno Dómini 9 7 2023
Patrón "GG d/MM/yy": anno Dómini 9/07/23
Patrón "GGG d MMM yy": anno Dómini 9 jul 2023
Patrón "GGGG dd-MMMM-yyyy": después de Cristo 09-julio-2023
Patrón "GGGGG dd-MMMMM-yyyyy": d. C. 09-J-02023

Año 1 d.C:
Patrón "GGGG dd/MM/uuuu": después de Cristo 01/01/0001
Patrón "GGGG dd/MM/yyyy": después de Cristo 01/01/0001

Año 1 a.C:
Patrón "GGGG dd/MM/uuuu": antes de Cristo 01/01/0000
Patrón "GGGG dd/MM/yyyy": antes de Cristo 01/01/0001

Año 2 a.C:
Patrón "GGGG dd/MM/uuuu": antes de Cristo 01/01/-0001
Patrón "GGGG dd/MM/yyyy": antes de Cristo 01/01/0002

Año 3 a.C:
Patrón "GGGG dd/MM/uuuu": antes de Cristo 01/01/-0002
Patrón "GGGG dd/MM/yyyy": antes de Cristo 01/01/0003

Fecha y hora obtenida a partir de una cadena y un formato: 2022-02-07T10:01:30

```

13. Enum ChronoUnit

En este enum se definen un conjunto estándar de unidades de períodos de fecha, como por ejemplo, *DAYS*, *MONTHS*, *WEEKS*, *YEARS*, *HOURS*, *MINUTES*, *SECONDS*, etc.

```

package tema9_Fechas;

import java.time.LocalDateTime;
import java.time.Month;
import java.time.temporal.ChronoUnit;

public class EnumChronoUnit {

    public void show() {

        LocalDateTime currentDateTime, concertDateTime;
        currentDateTime = LocalDateTime.of(2023, Month.JANUARY, 22, 17, 30);
        concertDateTime = LocalDateTime.of(2023, Month.AUGUST, 13, 21, 30);
        System.out.printf("Quedan para el concierto: %d meses o %d días o %d
horas o %d minutos o %d segundos",
            ChronoUnit.MONTHS.between(currentDateTime, concertDateTime),
            ChronoUnit.DAYS.between(currentDateTime, concertDateTime),
            ChronoUnit.HOURS.between(currentDateTime, concertDateTime),
            ChronoUnit.MINUTES.between(currentDateTime, concertDateTime),
            ChronoUnit.SECONDS.between(currentDateTime, concertDateTime));

    }

    public static void main(String[] args) {

        new EnumChronoUnit().show();

    }

}

```

Salida por consola:

```

Quedan para el concierto: 6 meses o 203 días o 4876 horas o 292560 minutos o
17553600 segundos

```

14. Clase Period

Esta clase modela un período de tiempo en términos de años, meses y días.

```

package tema9_Fechas;

import java.time.LocalDate;
import java.time.Period;

public class ClassPeriod {

    public void show() {

        Period period;
        LocalDate oldDate, currentDate;
        oldDate = LocalDate.of(1988, 9, 28);
        currentDate = LocalDate.now();
    }

}

```

```

        period = Period.between(olddate, currentDate); //Obtiene una instancia de
        Period a partir de dos fechas
        System.out.printf("Período entre %s y %s: son %d años, %d meses y %d
        días. ", olddate, currentDate,
            period.getYears(), period.getMonths(), period.getDays());
        System.out.printf("En total son %d meses.", period.toTotalMonths());
        period = Period.ofWeeks(50); //Obtiene una instancia de Period de 50
        semanas
        System.out.printf("\n\n50 semanas son %d días.", period.getDays());

    }

    public static void main(String[] args) {

        new ClassPeriod().show();

    }

}

```

Salida por consola:

```

Período entre 1988-09-28 y 2022-02-07: son 33 años, 4 meses y 10 días. En total
son 400 meses.

50 semanas son 350 días.

```

15. Clase Duration

Esta clase modela un período de tiempo en términos de unidades de tiempo, como años, meses, días, horas, minutos, segundos y nanosegundos.

```

package tema9_Fechas;

import java.time.Duration;

public class ClassDuration {

    public void show() {

        Duration duration;
        duration = Duration.ofMinutes(10);
        System.out.printf("Minutos: %d", duration.toMinutes());
        System.out.printf("\nSegundos: %d", duration.toSeconds());
        System.out.printf("\nNanosegundos: %d", duration.toNanos());

    }

    public static void main(String[] args) {

        new ClassDuration().show();

    }

}

```



```
}
```

Salida por consola:

```
Minutos: 10
Segundos: 600
Nanosegundos: 600000000000
```

16. Enum TimeUnit

Con este enum se representan duraciones de tiempo en una determinada unidad. Proporciona métodos de utilidad para convertir entre unidades y para realizar operaciones de temporización y retraso.

```
package tema9_Fechas;

import java.util.concurrent.TimeUnit;

public class EnumTimeUnit {

    public void show() {

        try {
            for (int i = 0; i < 5; i++) {
                System.out.printf("Iteración %d\n", i);
                TimeUnit.SECONDS.sleep(2); //Se duerme durante 2 segundos
            }
        } catch (InterruptedException e) {
            System.err.println("Interrupción mientras se duerme");
        }
    }

    public static void main(String[] args) {

        new EnumTimeUnit().show();

    }

}
```

Resumen programacion funcional v2

1. Introducción a la Programación Funcional

La programación funcional es un estilo de programación en el que usamos funciones como bloques fundamentales. En lugar de centrarnos en "cómo" hacer algo (como en la programación imperativa), nos enfocamos en "qué" queremos hacer.

- ♦ **Ejemplo simple:** En lugar de usar un `for` para filtrar números pares, podemos usar una función que los seleccione.

💡 Se basa en **funciones puras**, **inmutabilidad** y **expresiones lambda**.

2. Interfaces Funcionales

Una interfaz funcional es una interfaz que tiene un solo método abstracto. Esto permite que se usen con expresiones lambda.

Ejemplo:

```
@FunctionalInterface interface Operacion { int calcular(int a, int b); }
```

💡 **Regla clave:** Solo debe tener un **método abstracto**.

3. Implementación de Interfaces Funcionales con Clases Inline Anónimas

Antes de las expresiones lambda, usábamos **clases anónimas** para definir una interfaz funcional.

Ejemplo:

```
Operacion suma = new Operacion() { @Override public int calcular(int a, int b) { return a + b; } };  
System.out.println(suma.calcular(5, 3)); // 8
```

💡 Esto es **muy verboso**, por eso usamos expresiones lambda.

4. Interfaces Funcionales Puras Predefinidas

Java ya tiene **interfaces funcionales listas para usar**, en el paquete `java.util.function`.

Ejemplo de `Function`:

```
Function<Integer, Integer> cuadrado = x -> x * x; System.out.println(cuadrado.apply(4)); // 16
```

💡 Nos evitan tener que **crear nuestras propias interfaces funcionales**.

5. Interfaces Funcionales para Tipos Primitivos

Las interfaces funcionales estándar trabajan con objetos (`Integer`, `Double`...), lo que puede ser **ineficiente**. Por eso existen versiones optimizadas como `IntPredicate`, `DoubleFunction`, etc.

Ejemplo con `IntPredicate`:

```
IntPredicate esPar = num -> num % 2 == 0; System.out.println(esPar.test(4)); // true
```

💡 Usamos estas versiones para evitar el **auto-boxing** y mejorar el **rendimiento**.

6. Expresiones Lambda

Las expresiones lambda permiten escribir **código más limpio y conciso**.

```
Operacion suma = new Operacion() { public int calcular(int a, int b) { return a + b; } };
```

Ejemplo con lambda:

```
Operacion suma = (a, b) -> a + b;
```

💡 **Sintaxis:** `(parámetros) -> { cuerpo }`

6.1 Sintaxis de una Expresión Lambda

```
// Si el cuerpo tiene solo una línea, las llaves no son necesarias Operacion multiplicar = (a, b) -> a * b;
```

💡 Si solo hay un **parámetro**, se pueden **omitir los paréntesis**:

```
Consumer<String> imprimir = mensaje -> System.out.println(mensaje);
```

6.2 Ámbito de una Expresión Lambda

Dentro de una lambda puedes acceder a:

- ✓ **Parámetros de la función**
- ✓ **Variables locales** (pero deben ser `final` o **efectivamente finales**)

Ejemplo:

```
String saludo = "Hola"; // saludo = "Adios"; // Esto no compila Consumer<String> imprimirSaludo = nombre -> System.out.println(saludo + ", " + nombre);
```

6.3 Limitaciones de las Expresiones Lambda

- ✗ No pueden **modificar** variables externas que no sean `final` o **efectivamente final**.
- ✗ No pueden **definir múltiples métodos**.
- ✗ No pueden acceder a `this` en el mismo sentido que una clase.

7. Referencias a Método

En lugar de escribir una lambda, a veces podemos usar una **referencia a método**.

Ejemplo:

```
List<String> nombres = Arrays.asList("Ana", "Juan", "Luis"); nombres.forEach(System.out::println);
```

// En lugar de (nombre -> System.out.println(nombre))

💡 Usamos `::` para referenciar un método ya existente.

8. Consumer y BiConsumer

- `Consumer<T>` acepta un valor y **no devuelve nada**.
- `BiConsumer<T, U>` acepta **dos valores** y no devuelve nada.

Ejemplo:

```
Consumer<String> imprimir = mensaje -> System.out.println(mensaje); imprimir.accept("Hola"); // Hola
```

9. Predicate y BiPredicate

- `Predicate<T>` evalúa una condición y devuelve **true o false**.
- `BiPredicate<T, U>` hace lo mismo con **dos parámetros**.

Ejemplo:

```
Predicate<Integer> esPar = num -> num % 2 == 0; System.out.println(esPar.test(4)); // true
```

10. Function

Convierte un tipo en otro.

Ejemplo:

```
Function<String, Integer> obtenerLongitud = str -> str.length();  
System.out.println(obtenerLongitud.apply("Hola")); // 4
```

11. Comparator

Sirve para **comparar y ordenar elementos**.

Ejemplo:

```
Comparator<String> compararPorLongitud = (s1, s2) -> Integer.compare(s1.length(), s2.length());
```

💡 Usamos `Comparator.comparing()` para hacerlo más limpio.

12. Optional

`Optional` evita errores `NullPointerException` al trabajar con valores que pueden ser `null`.

Ejemplo:

```
Optional<String> nombre = Optional.ofNullable(null); System.out.println(nombre.orElse("Desconocido")); //
"Desconocido"
```

12.1 Construcción

```
Optional<String> nombre = Optional.of("Juan"); // No puede ser null Optional<String> nombreOpcional =
Optional.ofNullable(null); // Puede ser null
```

12.2 Igualdad

Podemos comparar con `equals()`.

12.3 Procesamiento

Podemos transformar con `map()` y filtrar con `filter()`.

Ejemplo:

```
Optional<String> nombre = Optional.of("Juan"); nombre.map(String::toUpperCase).ifPresent(System.out::println); //
JUAN
```

12.4 Dónde NO usar **Optional**

- ✗ No usar **Optional** en **atributos de clases**.
- ✗ No usar **Optional** en **parámetros de métodos**.
- ✓ Usarlo solo para **retornos de métodos**.

Resumen de Interfaces Funcionales en Java

Interfaz Funcional	Descripción	Ejemplo
<code>Consumer<T></code>	Acepta un argumento y no devuelve nada	<pre>Consumer<String> imprimir = mensaje -> System.out.println(mensaje);</pre>
<code>Predicate<T></code>	Evalúa una condición y devuelve <code>true</code> o <code>false</code>	<pre>Predicate<Integer> esPar = num -> num % 2 == 0;</pre>
<code>Function<T, R></code>	Convierte un tipo en otro	<pre>Function<String, Integer> obtenerLongitud = str -> str.length();</pre>
<code>Supplier<T></code>	No acepta argumentos y devuelve un resultado	<pre>Supplier<Double> aleatorio = () -> Math.random();</pre>

Resumen de Interfaces Funcionales en Java

1. **Consumer**: Acepta un argumento y no devuelve nada.

- Ejemplo: `Consumer<String> imprimir = mensaje -> System.out.println(mensaje);`

2. **BiConsumer<T, U>**: Acepta dos argumentos y no devuelve nada.

- Ejemplo: `BiConsumer<String, Integer> imprimirNombreEdad = (nombre, edad) -> System.out.println(nombre + " tiene " + edad + " años");`

3. **Predicate**: Acepta un argumento y devuelve un booleano.

- Ejemplo: `Predicate<Integer> esPar = num -> num % 2 == 0;`

4. **BiPredicate<T, U>**: Acepta dos argumentos y devuelve un booleano.

- Ejemplo: `BiPredicate<Integer, Integer> sonIguales = (a, b) -> a.equals(b);`

5. **Function<T, R>**: Acepta un argumento y devuelve un resultado.

- Ejemplo: `Function<String, Integer> obtenerLongitud = str -> str.length();`

6. **BiFunction<T, U, R>**: Acepta dos argumentos y devuelve un resultado.

- Ejemplo: `BiFunction<Integer, Integer, Integer> suma = (a, b) -> a + b;`

7. **Supplier**: No acepta argumentos y devuelve un resultado.

- Ejemplo: `Supplier<Double> aleatorio = () -> Math.random();`

8. **UnaryOperator**: Es una `Function<T, T>` (mismo tipo de entrada y salida).

- Ejemplo: `UnaryOperator<Integer> cuadrado = x -> x * x;`

9. **BinaryOperator**: Es una `BiFunction<T, T, T>` (mismo tipo de entrada y salida).

- Ejemplo: `BinaryOperator<Integer> multiplicar = (a, b) -> a * b;`

Última modificación: jueves, 13 de marzo de 2025, 09:54

MINISTERIO
DE HACIENDA

 **Junta de Andalucía**
Consejería de Desarrollo Educativo
y Formación Profesional

Reiniciar tour para usuario en esta página

Resumen Streams

Resumen sobre Streams en Java

1. Introducción

Los **Streams** en Java permiten procesar colecciones de datos de manera funcional y eficiente, sin modificar la estructura original.

```
List<String> nombres = List.of("Ana", "Juan", "Pedro");
nombres.stream().forEach(System.out::println);
```

Salida:

```
Ana
Juan
Pedro
```

2. Pipeline y modelo MapReduce

Un **pipeline** de Streams sigue tres fases:

1. **Creación del Stream**
2. **Operaciones intermedias** (*filter, map, sorted*)
3. **Operaciones terminales** (*collect, reduce, forEach*)

Ejemplo de **MapReduce**:

```
List<Integer> numeros = List.of(1, 2, 3, 4, 5);
int suma = numeros.stream().mapToInt(n -> n).sum();
System.out.println(suma);
```

Salida: 15

3. Creación de un Stream a partir de una fuente de datos

Desde listas:

```
List<String> frutas = List.of("Manzana", "Banana", "Cereza");
frutas.stream().forEach(System.out::println);
```

Salida:

```
Manzana
Banana
Cereza
```

Desde arrays:

```
String[] colores = {"Rojo", "Verde", "Azul"};
Stream<String> streamColores = Arrays.stream(colores);
```

4. Tratamiento individual de elementos

```
List<String> palabras = List.of("hola", "mundo");
palabras.stream().map(String::toUpperCase).forEach(System.out::println);
```

Salida:

```
HOLA  
MUNDO
```

5. Filtrado

```
List<Integer> numeros = List.of(1, 2, 3, 4, 5, 6);  
numeros.stream().filter(n -> n % 2 == 0).forEach(System.out::println);
```

Salida: 2 4 6

6. Ordenación

Ordenación natural:

```
List<String> nombres = List.of("Juan", "Ana", "Pedro");  
nombres.stream().sorted().forEach(System.out::println);
```

Salida:

```
Ana  
Juan  
Pedro
```

Ordenación inversa:

```
List<Integer> numeros = List.of(3, 1, 4, 1, 5);  
numeros.stream().sorted(Comparator.reverseOrder()).forEach(System.out::println);
```

Salida: 5 4 3 1 1

7. Transformación

```
List<String> palabras = List.of("Java", "Stream", "API");  
palabras.stream().map(String::length).forEach(System.out::println);
```

Salida: 4 6 3

8. Reducción pura

Métodos estándar de reducción:

```
List<Integer> numeros = List.of(1, 2, 3, 4, 5);  
int suma = numeros.stream().mapToInt(Integer::intValue).sum();  
System.out.println(suma);
```

Salida: 15

Método reduce:

```
int producto = numeros.stream().reduce(1, (a, b) -> a * b);  
System.out.println(producto);
```

Salida: 120

9. Operaciones terminales de consulta

```
boolean hayMayores = numeros.stream().anyMatch(n -> n > 4);  
System.out.println(hayMayores);
```

Salida: true

10. Reducción mutable

Recolectores a estructuras de datos clásicas

```
List<String> lista = Stream.of("A", "B", "C").collect(Collectors.toList());  
System.out.println(lista);
```

Salida: [A, B, C]

Recolectores de reducción básicas

```
String resultado = Stream.of("Java", "Streams").collect(Collectors.joining(", "));  
System.out.println(resultado);
```

Salida: Java, Streams

Recolectores de transformación

```
Map<Integer, String> mapa = Stream.of("uno", "dos", "tres")  
    .collect(Collectors.toMap(String::length, Function.identity(), (a, b) -> a));  
System.out.println(mapa);
```

Salida: {3=uno, 4=cuatro, 5=cinco}

Recolectores de agrupación

```
Map<Integer, List<String>> agrupado = Stream.of("uno", "dos", "cuatro", "cinco")  
    .collect(Collectors.groupingBy(String::length));  
System.out.println(agrupado);
```

Salida: {3=[uno, dos], 4=[cuatro], 5=[cinco]}

Recolectores de particionado

```
Map<Boolean, List<Integer>> particionado = List.of(1, 2, 3, 4, 5)  
    .stream()  
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));  
System.out.println(particionado);
```

Salida: {false=[1, 3, 5], true=[2, 4]}

Combinación de dos recolectores

```
Map<Boolean, Long> cuenta = List.of("Java", "Python", "C")  
    .stream()  
    .collect(Collectors.partitioningBy(s -> s.length() > 3, Collectors.counting()));  
System.out.println(cuenta);
```

Salida: {false=1, true=2}

Este formato es **compatible con Moodle**, asegurando una correcta visualización con bloques de código bien formateados. ¡Listo para copiar y pegar! 🚀

Última modificación: jueves, 13 de marzo de 2025, 09:22